

C und C++

Einführung in die Programmiersprachen

Dokumentversion: 820
Datum/Uhrzeit: 04.10.2026 15:31:57
Anzahl: 406407/59349/4418/227
Vorlage: Dokumentation

Diese Dokumentation und die dazugehörige Software wurde mit größter Sorgfalt entwickelt. Dennoch kann nicht ausgeschlossen werden, dass der eine oder andere Fehler enthalten ist. Wir sind jedoch stets bemüht, jeden Fehler auszuschließen.

Wenn Sie bei der Benutzung des Programms oder in diesem Handbuch ein Fehler finden sollten, so wenden Sie sich bitte direkt an den Autor.

Kontakt:

Hofmannsthalweg 14
A-4030 Linz

Http: www.gaeckler.at

E-Mail: martin@gaeckler.at

Inhaltsverzeichnis

1. EINLEITUNG.....	7
2. GRUNDLAGEN DER PROGRAMMIERUNG.....	8
2.1. Grundfunktion eines Computersystems.....	8
2.2. Kodierung.....	11
2.3. Zahlensysteme.....	12
2.4. Negative Zahlen.....	12
2.5. Konstanten.....	13
2.6. Handle.....	13
2.7. Syntax/Semantik.....	14
2.8. Programmierstile.....	14
2.9. Compiler/Interpreter.....	15
3. DER ÜBERSETZUNGSVORGANG.....	16
4. C/C ++ BEGRIFFE.....	18
4.1. Anweisung - Statement.....	18
4.2. Ausdruck - Expression.....	18
4.3. RValue - LValue.....	19
4.4. Bezeichner - Identifier.....	19
4.5. Schlüsselwort - Keyword.....	19
4.6. Deklaration - Declaration.....	19
4.7. Definition.....	19
4.8. Initialisierung.....	19
4.9. White space.....	19
4.10. Ressource.....	20
5. DER SPRACHKERN.....	20
5.1. Präprozessor.....	21
5.2. Einfache Typen.....	25
5.3. Typqualifizierer.....	27
5.4. Zeiger, Adressen und Referenzen.....	28
5.5. Arrays.....	31
5.6. Aufzählungstypen.....	32
5.7. Konstante - Literale.....	33
5.8. Strukturen.....	36
5.9. Funktionen.....	38
5.10. C++: Funktionsüberladung.....	42
5.11. Variablen.....	43
5.12. Bezeichner.....	45
5.13. C++ Namensräume.....	46
5.14. Operatoren.....	49
5.15. C++: Operatorüberladung.....	55
5.16. C++11 Benutzerdefinierte Literale.....	56
5.17. Kontrollstrukturen.....	56
5.18. C++: Schablonen.....	61
6. DAS ERSTE PROGRAMM.....	61
7. C++ KLASSEN BASISWISSEN.....	63
7.1. Klassen.....	63
7.2. member function.....	64

7.3. this pointer.....	64
7.4. Modularisierung.....	65
7.5. Const Funktionen.....	66
7.6. Statische Elemente.....	67
7.7. Konstruktoren.....	69
7.8. Destruktoren.....	72
7.9. Vererbung.....	73
7.10. Zugriffsmodifikator (Access Specifier).....	75
7.11. Zeiger und Referenzen.....	79
7.12. Virtuelle Funktionen.....	80
7.13. Elementtypen (Member Types).....	84
8. EIN- UND AUSGABE VON DATEN.....	85
8.1. Ein- und Ausgabe in C.....	85
8.2. Ein- und Ausgabe in C++ (iostream).....	108
8.3. Ergänzungen.....	114
9. C++ TEMPLATES.....	115
9.1. Definition.....	115
9.2. Funktionstemplates.....	116
9.3. Skalare Templateparameter.....	117
9.4. Klassentemplates.....	117
9.5. Klassenfunktionstemplates.....	119
9.6. Spezialisierung.....	120
9.7. Ableitungen.....	122
9.8. Template-Template-Parameter.....	123
9.9. Templates und Elementtypen (member types).....	124
9.10. Templatequellen.....	125
9.11. Neue Standards.....	126
10. ZEICHENKETTEN.....	126
10.1. C-Zeichenketten.....	126
10.2. C++ Zeichenketten.....	129
11. TYPUMWANDLUNGEN.....	133
11.1. Implizite Typumwandlung.....	133
11.2. Explizite Typumwandlung in C.....	134
11.3. Explizite Typumwandlung in C++.....	134
11.4. Klassenzeiger- und -referenzumwandlung in C++.....	141
11.5. Umwandlungsmethoden.....	142
12. FEHLERBEHANDLUNG.....	145
13. SPEICHERVERWALTUNG.....	153
13.1. Konzepte.....	153
13.2. C Speicherverwaltung.....	156
13.3. C++ Speicherverwaltung.....	157

14. NÜTZLICHES.....	159
14.1. Variable Anzahl Argumente.....	159
14.2. C++ Ratio.....	160
14.3. C++ Hash.....	161
14.4. C++: Funktionsobjekte, Lambdaausdrücke.....	162
15. C++ CONTAINER.....	164
15.1. Container Templates.....	164
15.2. Iterationen.....	167
16. DATUM, ZEITSTEMPEL UND STOPPUHREN.....	173
16.1. Funktionen und Typen in C <time.h>.....	174
16.2. Funktionen und Klassen in C++.....	176
17. THREADS.....	178
17.1. Wissenswertes.....	178
17.2. Threads mit C.....	180
17.3. Threads mit C++.....	188
18. C STANDARD BIBLIOTHEK.....	196
18.1. Standardfunktionen <stdlib.h>.....	196
18.2. Zeichenfunktionen <ctype.h>.....	198
18.3. Nützliche Makrodefinitionen <stddef.h>.....	198
18.4. Wertebereiche <limits.h>.....	199
18.5. Mathematische Funktionen <math.h>.....	200
18.6. Andere Funktionen.....	203
19. C++ STANDARD BIBLIOTHEK.....	204
19.1. Allgemeines (General Utilities).....	204
19.2. Zeichenfunktionen.....	204
19.3. Zeichenketten.....	204
19.4. Mathematische Funktionen.....	205
19.5. Sprachunterstützung (Language Support).....	205
19.6. Wertebereiche.....	206
19.7. Sonstiges.....	209
20. WEITERFÜHRENDE INFORMATIONEN.....	210
20.1. Lebensdauer.....	210
20.2. Virtuelle Destruktoren.....	211
20.3. Zeiger auf Memberfunktionen.....	212
20.4. Die Regel der goldenen Drei (Fünf).....	213
20.5. C und C++ mischen.....	213
20.6. Programmierguidelines.....	215
20.7. Sonstiges.....	218
21. INDEX.....	218

1. Einleitung

Meine erste Berührung mit der Programmiersprache C war der Compiler Megamax auf einem Atari ST etwa 1987. Vorher lernte ich an der Schule Basic, privat Assembler für den 6502 und an der FH Pascal, Fortran und Cobol. Megamax war noch ein alter K&R C-compiler, so war es eine enorme Verbesserung, als ich 1988 bei Heimsoeth & Borland anfang und am Turbo C für Atari mitwirkte, der den damals neuen ANSI-Standard verwendete.

Ende 1990 kam ich das erste Mal mit C++ in Kontakt und zwar mit Turbo C++ für MS Dos von Borland. Da aber lange Zeit mein Atari noch der Hauptcomputer war, war auch meine Hauptsprache C und C++ wurde nur für wenige Projekte eingesetzt. Ende der 90er Jahre wurde der Atari stillgelegt und von da an wurde C++ immer mehr zu meiner Hauptsprache und C immer weniger eingesetzt. Habe ich früher noch zu allen Funktionen, die ich in C++ implementiert habe, auch ein C-Interface erstellt, so dass sie auch aus C aufgerufen werden konnten, sind mittlerweile fast alle Quellen, die ich heute noch pflege, nach C++ migriert und kaum noch C Quellen übrig.

Dieses Buch bietet eine Einführung und tiefer gehende Informationen zu den beiden Programmiersprachen C und C++. Obwohl C und C++ als zwei verschiedene Sprachen gelten, sind in meinem Augen die Gemeinsamkeiten viel zu groß als das es sinnvoll wäre, die Einführung in diese beiden Sprachen in zwei getrennten Büchern zu behandeln. Darüber hinaus halte ich es auch für C++-EntwicklerInnen sinnvoll, wenn sie auch Kenntnisse über die Programmiersprache C besitzen, um so einfacher ihre C-Programme nach C++ zu übertragen.

Wenn Sie ausschließlich an nur einer der beiden Programmiersprachen interessiert sind, können Sie getrost die speziellen Kapitel zur anderen Programmiersprache überspringen. Ich empfehle aber zumindest den C++-Interessierten, die Kapitel, die speziell für die Sprache C sind, zumindest zu überfliegen.

Wenn nichts anderes angegeben ist, gelten die Beschreibungen hier für beide Programmiersprachen C und C++. Manche Beschreibungen für C gelten zwar grundsätzlich auch für C++, wenn C++ aber bessere Sprachmittel zu Verfügung stellt, werde ich die Beschreibungen als C-spezifisch markieren.

Wenige Aspekte von C werden von einem C++-Compiler nicht akzeptiert. Ich empfehle daher grundsätzlich sich anzugewöhnen, C so zu verwenden, dass es auch von einem C++-Compiler akzeptiert wird. Dies erleichtert erstens eine spätere Portierung nach C++ und zweitens erhöht es die Qualität Ihrer Programme.

Dieses Buch sollte nicht als Referenzhandbuch verstanden werden. Jedes Kapitel setzt die vorherigen Kapitel voraus, erwartet aber keine Kenntnisse über die nachfolgenden Kapitel. Darüber hinaus würde eine ausführliche Behandlung aller Funktionen der Standardbibliothek den Rahmen dieses Buchs sprengen. Der aktuelle C++-Standard hat mehr als 1000 Seiten. Ich werde mich aber bemühen, Ihnen genügend Wissen zu vermitteln, dass Sie die meisten Entwicklungsprobleme selbst lösen können und nötigenfalls in der Lage sein werden, das fehlende Wissen in einem Referenzhandbuch oder im Internet selbst zu recherchieren.

Die Programmiersprache C wurde zum ersten mal 1989 vom ANSI-Komitee standardisiert. Die davor geltende Sprachbeschreibung K&R C benannt nach den beiden Erfindern Brian Kernighan und Dennis Ritchi ist nicht Gegenstand dieses Buchs. Der ANSI-standard wurde 1990 vom ISO-Komitee unverändert übernommen. Von 1999 bis heute erfolgten einige Erweiterung, die im Wesentlichen von C++ übernommen wurden. In diesem Buch werden die einzelnen Standards mit Ihren Kurzbezeichnungen C89, C99, C11, C17 und C23 verwendet.

C++ wurde zum ersten mal 1998 standardisiert. Später wurde der Standard bis heute mehrmals aktualisiert. Das derzeitige Standardisierungsverfahren soll noch 2026 abgeschlossen sein. Im Weiteren werden die Kurzbezeichnungen C++98, C++03, C++11, C++14, C++17, C++20, C++23 und C++26 verwendet.

Auf sogenannte experimentelle Erweiterungen und Zwischenrelease der Standards wird hier nicht näher eingegangen.

2. Grundlagen der Programmierung

Bevor wir uns mit der Programmierung eines Computers beschäftigen können, betrachten wir einige Grundlagen, die sprachunabhängig für fast alle Programmiersprachen gelten. Sollten Sie bereits eine Programmiersprache gelernt haben, können Sie möglicherweise dieses Kapitel auch überspringen und sofort bei den Grundbegriffen der C/C++-Programmierung einsteigen.

2.1. Grundfunktion eines Computersystems

Zuerst betrachten wir einmal wie die meisten Computer überhaupt funktionieren. Ein grundlegendes Verständnis über die Abläufe innerhalb eines Rechner ist immer sinnvoll. Ein genaues Wissen über die Abläufe innerhalb eines Computersystems ist nur dann notwendig, wenn Sie sich der Systemprogrammierung widmen wollen. Im folgenden sprechen wir nur von den üblichen Computern, wie sie in den meisten Haushalten, Büros oder Rechenzentren zu finden sind. Irgendwelche exotischen Architekturen wie z.B. Quantencomputer werden wir nicht betrachten.

Jeder Computer besteht aus mindestens je einen der folgenden Komponenten:

Eingabeeinheit, Ausgabeeinheit, Verarbeitungseinheit und Speicher.

Eine Eingabeeinheit hat die Aufgabe Daten von außerhalb des Computers in das System zu übertragen. Ohne eine solcher Einheit, kann der Computer keine Daten verarbeiten und wäre somit nutzlos. Beispiele wären Tastatur, Scanner, Netzwerkschnittstellen oder Sensoren für Temperatur oder andere Messgrößen. Aber auch Massenspeicher wie Festplatten können hierzu gezählt werden.

Analog zur Eingabeeinheit, hat die Ausgabeeinheit die Aufgabe, berechnete Daten wieder nach außen zu tragen. Ohne eine solche Einheit wäre ein Computer nutzlos, da er keine Daten weitergeben kann. Beispiele wären hier Bildschirm, Drucker, Netzwerkschnittstellen und auch hier wieder die Massenspeicher. Ein- und Ausgabeeinheiten werden für gewöhnlich im Ein-Ausgabesystem oder englisch I/O-System zusammengefasst.

Die Verarbeitungseinheit ist der Prozessor. Moderne Computersystem enthalten heutzutage mehrere Prozessoren oder zumindest Prozessoreinheiten (den sogenannten Cores) und können darüber auch noch weitere simulieren (Hyperthreading). Sofern wir nicht hoch performante Programme entwickeln wollen, gehen wir bei der Programmierung für gewöhnlich von nur einem Prozessorkern aus. Gegen Ende des Buches werden wir jedoch auf die Programmierung für mehrere Cores kurz eingehen. Eine tiefe Betrachtung dieses Themas würde aber den Rahmen dieses Buches übersteigen.

Ein Prozessor liest das fertige Programm Schritt für Schritt ein und führt die entsprechenden Befehle aus. Dabei wird er Daten in seinen internen Speicher den sogenannten Registern einlesen, Rechen- und/oder Prüfbefehle ausführen und die Ergebnisse wieder ausgeben. Dabei stehen auch bedingte und unbedingte Sprungbefehle zur Verfügung, die erlauben, das Programm nicht mit dem nächsten Befehl der Befehlsfolge sondern an einer beliebigen anderen Stelle fortzufahren.

Mikroprozessoren haben im Vergleich zu uns Menschen nur sehr eingeschränkte Fähigkeiten. Das wenige, das sie können, führen sie aber sehr schnell aus. Moderne Computer sind z.B. in der Lage etliche Milliarden Zahlen innerhalb von nur einer Sekunde zu addieren. Ein Mensch würde dafür wohl mehrere Monate wenn nicht gar Jahre benötigen. Allerdings können wir Menschen beliebig große Zahlen verarbeiten und haben auch mit rationalen Zahlen (Brüchen) kein Problem. Wir Menschen bilden Zahlen beispielsweise aus einem Vorrat von 10 Ziffern. Daher nennt man diese Zahlen auch Dezimalzahlen. Dezimal leitet sich vom lateinischen deci für 10 ab. Computer kennen aber nur die Ziffern 0 und 1, welche für die An- oder Abwesenheit einer bestimmten Spannung an einem bestimmten Punkt im Prozessor steht. Diese Zahlen werden auch Dualzahlen vom lateinischen duo für Zwei. In englischen Publikationen finden Sie häufig den Begriff binary (oder auf deutsch binär), das einen ähnlichen Ursprung hat.

So wie wir aus beliebig langen Kombinationen der Ziffern 0 bis 9 auch sehr große Zahlen bilden können, so erzeugt der Computer auch aus einer entsprechend langen Kombination von 0 und 1 ebenso große Zahlen. Das Problem hierbei ist, dass der Computer im Gegensatz zu uns nicht beliebig lange Zahlen auf diese Weise erzeugen kann. Während wir bei einer Dezimalzahl von einer Dezimalstelle sprechen (auch wenn dies häufig nur für die Nachkommastellen benutzt wird), so spricht man bei den Computerzahlen von Bits. Dabei werden auch die einzelnen Stellen je nach ihrer Wertigkeit von 0 bis n durchnummeriert. Die Einerstelle einer Dezimalzahl entspricht das Bit0, die Zehnerstelle das Bit1 usw. usf. Bei Dezimalzahlen verzehnfacht sich der Wert bei jeder weiteren Stelle, bei einer Dualzahl verdoppelt er sich.

Die ersten Prozessoren konnten gerade einmal mit 4 Bits gleichzeitig rechnen, so dass er gerade mal in der Lage war, mit ganzen Zahlen von 0 bis 15 zu rechnen. Moderne Computer können Ihre Zahlen aber aus bis zu 64 Bits bilden, so dass es hier einen Zahlenbereich von 0 bis 18 446 744 073 709 552 000 gibt, was für die meisten Anwendungsfälle ausreichen dürfte.

Mit rationalen Zahlen kann ein Prozessor überhaupt nicht rechnen. Dafür benutzt er

sogenannte Fließkommazahlen, die ebenfalls aus den Ziffern 0 und 1 gebildet werden. Fließkommazahlen verhalten sich ähnlich wie die rationalen Zahlen der Mathematik. Aber sie erreichen nicht die Genauigkeit, der rationalen Zahlen, so dass es gelegentlich Ungenauigkeiten wegen Rundungsfehlern geben kann, die nicht erwartet werden. Dies passiert vor allen dann, wenn rationale Zahlen von ihrer Dezimaldarstellung in das binäre Format des Prozessors und umgekehrt gewandelt werden müssen. Hierbei kann es zu Rundungsfehlern kommen, so dass eine binäre Fließkommazahl z.B. nicht mit einem festen Wert verglichen werden sollte. Auf diese Problematik werden wir aber später bei der Behandlung der mathematischen Funktionen von C/C++ genauer eingehen.

Der Speicher eines Computersystems dient der Zwischenspeicherung von eingelesenen Daten, Berechnungsergebnissen etc. Er ist im Grunde genommen eine Ansammlung von vielen Bits auf die der Prozessor jederzeit Zugriff hat. Da aber Prozessoren nur äußerst selten auf einzelne Bits zugreifen müssen, werden für gewöhnlich mehrere dieser Bits zu einer Speicherstelle zusammen gefasst. Bisher hat sich eingebürgert 8 Bits für eine Speicherstelle zusammen zu fassen. Jeder dieser Speicherstellen hat nun eine eindeutige Adresse, die mit einer natürlichen Zahl ab 0 durchnummeriert werden. Mit 8 Bits kann man natürliche Zahlen von 0 bis 255 speichern. Das genügt, um alle Buchstaben, alle Ziffern, verschiedene Sonderzeichen, Steuerzeichen und einige nationale Sonderzeichen wie die deutschen Umlaute kodieren zu können. Zur Kodierung folgt später noch ein eigener Abschnitt. Will der Computer größere Zahlen (man spricht dann auch von Wörtern oder Englisch word) speichern oder laden, so werden diese entsprechend ihrer Größe auf mehrere aufeinanderfolgende Speicherstellen verteilt. Hierbei gibt es aber einige Besonderheiten, die es zu beachten gilt:

Worte sollten nicht an ungeraden Adressen liegen. Ein Prozessor benötigt für den Zugriff auf Wörtern an ungerader Adresse mehrere Zyklen wohingegen er die Worte an geraden Adressen in einem Zyklus lesen oder schreiben kann. Manche Prozessoren weigern sich sogar, Worte an ungeraden Adressen zu lesen oder zu schreiben und unterbrechen stattdessen das laufende Programm. Bei vielen Compilern können Sie die Ausrichtung (Alignment) der Datenworte beeinflussen.

Die sogenannte Endianness besagt, in welcher Reihenfolge der Prozessor die einzelnen Bytes eines Wortes auf den Speicher verteilt. Big-endian Systeme speichern das höchstwertige Byte an erster Stelle und alle anderen Bytes entsprechend dahinter. Little-endian Systeme machen dies genau umgekehrt. D.h. das Byte mit der geringsten Wertigkeit wird zuerst gespeichert und alle anderen folgen dahinter. Da die Prozessoren der Firma Intel die bekanntesten sind, welche dieses Format verwenden, so wird dieses auch gerne mit Intelformat bezeichnet. Big-endian wird dagegen gerne auch mit Motorolaformat bezeichnet.

Gefürchtete Fehler bei Computerprogrammen sind der Überlauf (Overflow) und der Unterlauf (Underflow). Ein Überlauf passiert, wenn das Berechnungsergebnis einer Operation zu groß ist, um in einer Variable des entsprechenden Typs zu speichern. Beispiel für char:

```
char myTyp = 99 + 88 + 77;
```

Da char für gewöhnlich ein 8-Bitwert ist, können nur Werte von -128 bis +127, oder falls char ohne Voreichen ist, nur Werte von 0 bis 255. Ein Unterlauf kann bei Fließkommazahlen vorkommen, wenn das Berechnungsergebnis zu nah an der 0 ist, um es von dieser zu unterscheiden.

2.2. Kodierung

Ein nicht unerhebliches Problem der Mensch Maschine Schnittstelle ist die Tatsache, dass der Computer nur mit den beiden Symbolen 0 und 1 arbeiten kann, wohingegen, der Mensch sich einer Vielzahl von Symbolen bedient. Hierzu zählen beispielsweise die Buchstaben von A-Z aber auch a-z, die Ziffern 0-9 aber auch Sonderzeichen wie +, - etc.

Um dieses Problem zu lösen, wurde von einem amerikanischen Gremium eine Tabelle erarbeitet, die 128 Zeichen enthält welche auf 7 Bit langen Computerzahlen abgebildet werden. Diese Tabelle wird einfach ASCII-Tabelle (American Standard Code for Information Interchange) genannt und alle in der Tabelle enthaltenen Zeichen bilden den ASCII-Zeichensatz. Eine vollständige ASCII-Tabelle finden Sie sicherlich im Internet, ich spare mir daher hier die Auflistung.

Der Buchstabe A entspricht in der ASCII-Kodierung der Bitfolge 1000001 umgerechnet in das Dezimalformat entspricht diese Dualzahl der dezimalen Zahl 65. Der Buchstabe B ist eine Zeile in der Tabelle weiter und hat daher die Darstellung 1000010 bzw. 66. einige Steuerzeichen in der ASCII-Tabelle werden Ihnen immer wieder begegnen, so dass ich sie hier nicht vorenthalten will:

Ein Zeilenvorschub des Druckers oder am Bildschirm (Linefeed) entspricht 0001010 bzw dezimal 10. Ein Warenrücklauf ist 00001101 bzw dezimal 13. Ein Tabulator (Tab) ist 00001000 bzw dez. 8 usw.

Da es unüblich ist, die einzelnen Bitfolgen auszugeben, werden ich im weiteren Verlauf dieses Abschnittes darauf verzichten und erst dann, wenn die Bitoperationen behandelt werden, wieder darauf zurückgreifen. Stattdessen werde ich nur noch die dezimale Darstellung der Zeichen verwenden.

Die ASCII-Tabelle enthält in den ersten 32 Einträgen (0-31) nur Steuerzeichen für die Ausgabegeräte. Das sind die nicht lesbaren Zeichen. Dannach folgt das Leerzeichen (32), diverse Satz- und Sonderzeichen, alle lateinischen Buchstaben und die arabischen Ziffern. Es fehlen aber die in vielen westeuropäischen Sprachen üblichen Sonderzeichen wie die Umlaute Ä, Ö, Ü etc. Aus diesem Grund gibt es viele Erweiterungen der ASCII-Tabelle, die die Länge der Kodierung um 1 Bit erweitern und somit den möglichen Zeichenvorrat verdoppeln. Allen diesen Tabellen gemeinsam sind die ersten 128 Einträge, die denen der ASCII-Tabelle entsprechen.

Da dies aber immer noch nicht ausreicht, die Texte aller Sprachen dieser Welt darstellen zu können, hatten die meisten Betriebssysteme die Möglichkeit, zwischen den verschiedenen Erweiterungstabellen umzuschalten. In diesem Zusammenhang begegnet Ihnen dann häufig der Begriff der Codepage, der selbst heute noch eine gewisse Bedeutung hat. Eine Schwierigkeit war z.B. immer, wenn zwei Programme Daten austauschen wollen, die unterschiedliche Codepages benutzen. Heute noch

sieht man das Problem auf modernen Windowssystem, weil die Eingabeaufforderung immer noch mit der Zeichentabelle von MS-DOS arbeitet, während Windows selbst den ANSI-Zeichensatz benutzt.

Auf Grund der Schwierigkeit mit den verschiedenen Zeichensätzen und weil die Kodierung mit 8 Bit nicht ausreicht, chinesische Schriftzeichen und andere exotische Sprachen zu kodieren, wurde der Unicodestandard entworfen. Dieser Standard ist so gestaltet, dass es nicht mehr erforderlich ist, zwischen verschiedenen Zeichensätzen umzuschalten und er ist offen für Erweiterungen ohne das bestehende Programme geändert werden müssen. Darüber hinaus gibt es mit der UTF-8 Kodierung auch eine Erweiterung, die es erlaubt, auch Programme, die auf einen 8-Bit Zeichensatz aufsetzen ohne oder nur mit geringen Änderungen zu verwenden.

2.3. Zahlensysteme

Wie schon im vorherigen Kapitel angedeutet ist die Darstellung der Computerzahlen im Dualsystem für uns Menschen schwer interpretierbar, da wir das dezimale System gewohnt sind. Wenn wir aber auf einzelne Bits einer Computerzahl zugreifen wollen, ist aber die Dezimaldarstellung ungeeignet, da hiermit wiederum nicht so leicht auf die einzelnen Bits geschlossen werden kann. Daher werden für Computerzahlen auch oft (wenn sie eine technische Verwendung haben) in der hexadezimalen Darstellung oder verkürzt als Hexzahlen verwendet. Hexzahlen verwenden 16 Ziffern für die Darstellung nämlich die Dezimalziffern 0 bis 9 und die Buchstaben A bis F wobei Groß-/Kleinschreibung nicht unterschieden wird. Mit den 16 Ziffern des hexadezimalen Zahlensystems lassen sich genau 4 Bits kodieren. Dadurch kann ein Byte, das ja bekanntlich aus 8 Bits besteht als eine Folge von zwei Hexziffern dargestellt werden. Der Buchstabe A in der ASCII-Tabelle entspricht dabei der Hexzahl 41. Um eine Hexzahl von einer Dezimalzahl unterscheiden zu können wird i.d.R. ein Präfix 0x vorangestellt oder eine H hinten angehängt. Ich habe aber auch schon Darstellungen gesehen, wo ein H vorangestellt wurde. Zusammengefasst kann man sagen, dass die Bitfolge 01000001, 0x41, 41H und 65 alle für die gleiche Computerzahl stehen, welche den Buchstaben A bei der Ausgabe repräsentieren.

Die Umrechnung der Computerzahlen in die verschiedenen Darstellungsarten ist nicht sehr schwer. In einem der Übungskapitel werden wir ein Programm schreiben, das uns genau das erledigen wird, weshalb ich auf die weitere Erklärung an dieser Stelle verzichten werde.

2.4. Negative Zahlen

Ein weiteres Problem, das sich uns stellt, ist die Behandlung negativer Zahlen. Da der Prozessor nur die beiden Ziffern 0 und 1 kennt, fehlt ihm das Minuszeichen, um eine Zahl als negative Zahl zu interpretieren. Nun könnte man auf die Idee kommen, einfach ein Bit einer Zahl genau dafür zu reservieren, es also anders zu behandeln als alle anderen. Mit dieser Idee könnte man z.B. festlegen, dass die Bitfolge 01000001 wie +65 und 11000001 wie -65 zu interpretieren ist. In einem Byte könnte man damit statt den Zahlen von 0 bis 255 die Zahlen -127 bis +127 speichern. Das ist zwar durchaus möglich, würde aber die Recheneinheit wesentlich komplizierter und fehleranfälliger gestalten. Außerdem gäbe es zwei verschiedene Darstellungen

für die 0 nämlich +0 und -0. Tatsächlich benutzt man das Zweierkomplement, indem einfach alle Zahlen umgedreht werden und anschließend 1 addiert wird. Aus 01000001 (=65) wird im ersten Schritt 10111110 und anschließend 1 addiert macht 10111111. Mit dieser Methode können in einem Byte ganze Zahlen von -128 bis +127 gespeichert werden. Die Recheneinheit für das Addieren und Subtrahieren muss nicht zwischen positiven und negativen Zahlen unterscheiden auch der Test ob eine Zahl positiv oder negativ ist, lässt sich leicht implementieren. Nur bei wenigen Prozessorbefehlen muss die CPU zwischen vorzeichenbehafteten Zahlen und vorzeichenlosen Zahlen unterscheiden. Also ob 10111111 nun wie -65 behandelt werden soll oder wie 191. Diese Eigenart begegnet uns bei einigen Operation in C/C++, weshalb ich später noch einmal darauf eingehen werde.

2.5. Konstanten

Konstanten sind Werte in Ihrem Programm, die „fest verdrahtet“ sind und sich zur Laufzeit nicht ändern. Wenn Sie einen bestimmten Wert in Ihrem Programm mehrmals nutzen oder dann, wenn er sich irgendwann einmal doch ändern könnte, ist es sinnvoll, diesen an einer geeigneten Stelle einmal zu definieren, einen aussagekräftigen Namen zu geben und nur diesen Namen zu verwenden. Viele Programmiersprachen so auch C und C++ bieten geeignete Mittel, um solche konstanten Werte zu definieren ohne dabei Performanzeinbußen hinnehmen zu müssen.

2.6. Handle

Ein Handle ist eine Struktur der jeweiligen Bibliothek oder Betriebssystem, die dazu benutzt werden kann, um eine größere Struktur, die vorher erzeugt wurde, wieder zu finden. Handles können die verschiedensten Formen annehmen wie zum Beispiel ganze Zahlen, Speicheradressen etc. Das Handle ist in der Regel so lange gültig und eindeutig, bis die größere Struktur wieder gelöscht wurde. Solche Handle werden verwendet, weil diese oft deutlich kleiner sind wie die eigentliche Struktur und daher wesentlich einfacher innerhalb des Systems kopiert werden können. Betriebssystemhandles sind in der Regel innerhalb des ganzen Rechners eindeutig während Handles auf Bibliotheksobjekte nur innerhalb eines Programms eindeutig sind. Wie groß diese Handles sind und ob die binäre Darstellung eine bestimmte Bedeutung hat, muss uns nicht interessieren. Das System stellt und alles zur Verfügung, was wir wissen müssen. Bildlich gesehen ist ein Handle ein Handgriff für einen verschlossenen Koffer, an dem wir den Koffer herumtragen können, wenn wir aber etwas mit dem Inhalt machen müssen, geben wir den verschlossenen Koffer an die jeweilige Funktion und kümmern uns nicht um den Inhalt.

Beachten Sie aber folgende zwei Grundsätze:

Hütten Sie Ihre Handles, solange Sie noch nicht die dazugehörigen Ressourcen freigegeben haben. Andernfalls könnte es zu sogenannten Resourceleaks kommen, was dazu führen kann, dass Ihr Computersystem instabil oder extrem langsam läuft .

Vergessen Sie aber nach der Freigabe Ihr Handle und alle Kopien, da der Zugriff auf freigegebene Ressourcen unerwartete Ergebnisse nach sich ziehen kann.

2.7. Syntax/Semantik

Die beiden Begriffe Syntax und Semantik werden Ihnen häufig im Zusammenhang mit Beschreibungen von Programmiersprachen begegnen. Die Syntax einer Sprache oder eines einzelnen Befehls beschreibt, welche Symbole in einem Programm erlaubt sind und in welcher Reihenfolge der Übersetzer diese Symbole erwartet. Die Semantik sagt dann aus, was das fertige Programm genau ausführen wird. Ein Übersetzer interessiert sich i.d.R. ausschließlich für die Syntax, nicht aber für die Semantik. Wenn Sie eine Ausgabefunktion aufrufen, obwohl Sie eine Eingabefunktion hätten aufrufen müssen, wird dies Ihr Übersetzer möglicherweise klaglos übersetzen, das Programm macht aber nicht das, was sie erwarten. Ein anderes Beispiel wäre z.B. wenn Sie Unterprogramme mit falschen Parametern aufrufen. Wenn Sie eine Funktion aufrufen, die nicht Bestandteil des Sprachkerns ist (und das sind in C/C++ alle), und Ihr einen Parameter übergeben, der zwar den korrekten Typ besitzt, mit dessen Wert die Funktion aber nichts anzufangen weiß, wird auch hier, der Compiler das Programm klaglos übersetzen, jedoch werden Sie nicht das gewünschte Ergebnis erhalten.

Konsultieren Sie also stets vor dem Aufruf einer unbekannten Funktion die Dokumentation des Herstellers. Der Übersetzer kann mit seinen Fehlermeldungen bestenfalls syntaktische Hilfestellungen geben.

2.8. Programmierstile

Es gibt viele verschiedenen Programmiersprachen, die mehr oder weniger verschiedene Ansätze der Programmierung verfolgen. Ich werde nicht auf jeden einzelnen Stil eingehen, dennoch ist es hilfreich für das Erlernen einer Programmiersprache, die unterschiedlichen Ansätze zu kennen.

Der einfachste Ansatz sind die imperativen Programmiersprachen. Bei diesen Sprachen besteht ein Programm aus einer beliebigen Abfolge von Anweisungen für den Prozessor, die stur von vorne bis hinten abgearbeitet werden. Es gibt aber auch Sprungbefehle, um den Programmablauf auch an einer beliebigen anderen Stelle fortzufahren. Praktisch alle CPUs kennen nur diesen Ansatz aber auch die ersten BASIC versionen haben nur solche Sprachelemente eingebaut. Imperative Programmiersprachen sind leicht zu erlernen, da sie nur sehr wenige Sprachelemente kennen. Allerdings ist es recht schwierig fehlerfreie Programme zu erstellen, da der Programmablauf häufig schwierig zu verstehen ist.

Strukturierte Programmiersprachen beheben dieses Problem, in dem sie Sprachelemente einführen, die beliebige Sprungbefehle unnötig machen. Wenn Sie dann auch noch Ihren Quelltext so formatieren, dass die Struktur leicht erkennbar ist, sehen Sie schon auf den ersten Blick, wie der Programmablauf erfolgt. Ein Beispiel:

```
wenn eingabe größer als 0 dann
    mache irgendwas
andernfalls
    mache was anderes
```

Strukturierte Programmiersprachen bieten verschiedene Kontrollelemente wie Bedingungen, Wiederholungen und Unterprogramme. Wie werden später bei der

Beschreibung von C/C++ näher darauf eingehen.

Objektorientierte Programmiersprachen erweitern das Konzept der strukturieren Programmierung um Objekte. Bei Objekten werden Daten und Programmabläufe enger miteinander verknüpft wie bei anderen Sprachen. Dies hat den Vorteil, dass Programmierer, die eine objektorientierte Bibliothek verwenden, sich mehr um die Fähigkeiten kümmern können und weniger um die internen Datenstrukturen kümmern müssen. Nicht nur, dass sie das nicht müssen, sie dürfen es sogar überhaupt nicht, da der Zugriff auf die internen Strukturen weitgehend verboten ist. Darüber hinaus können Datenstrukturen mit dem Mittel der Vererbung erweitert werden, ohne die ursprüngliche Bibliothek ändern zu müssen. Grundsätzlich ist es auch in nicht objektorientierten Sprachen mit Objekten zu arbeiten, aber Sprachen, die dieses Konzept verinnerlicht haben, stellen Sprachmittel bereit, die garantieren, dass die gespeicherten Daten in den Objekten stets korrekt sind. In nicht objektorientierten Sprachen hängt dies von der Disziplin der Entwickler ab.

Generische Programmiersprachen stellen Sprachmittel bereit, die dem Entwickler erlauben sich ganz auf die Algorithmen zu konzentrieren und die Auswahl des konkreten Datentyps auf später zu verschieben. Dies ist praktisch, weil viele Algorithmen für unterschiedliche Datentypen (z.B. ganze Zahlen oder Fließkommazahlen) identisch implementiert werden können. Beispielsweise wird die Summe aller Zahlen einer Liste für ganze Zahlen genauso berechnet wie für Fließkommazahlen. Generische Programmiersprachen erlauben es, diesen Algorithmus genau einmal zu formulieren und erst beim Aufruf einer solchen Funktion wird vom Übersetzer die korrekte Implementierung erstellt.

Dynamische Programmiersprachen lösen vom Prinzip das gleiche Problem, wie es generische Programmiersprachen tun. Hier wird allerdings erst zur Laufzeit entschieden, wie der konkrete Algorithmus ausgeführt werden soll. Die Programmiersprachen JavaScript oder PHP verfolgen dieses Prinzip. Da diese aber nicht Thema dieses Buchs sind, werden wir diesen Punkt nicht weiter verfolgen.

Daneben gibt es noch eine Vielzahl weiterer Programmierstile, die jedoch in der C/C++ Entwicklung keine oder nur eine untergeordnete Rolle spielen, weshalb ich hier nicht weiter darauf eingehen möchte.

2.9. Compiler/Interpreter

Im vorherigen Abschnitt haben wir verschiedene Programmiersprachen nach Ihren Stilen unterteilt. Es gibt aber noch eine andere Möglichkeit nach der Programmiersprachen unterschieden werden können.

Bei einer Interpretersprache wird der Quelltext erst zur Laufzeit (also wenn das Programm ausgeführt wird) gelesen und analysiert, bevor die enthaltenen Befehle ausgeführt werden. Bei Compilersprachen wird der Quelltext vor der Ausführung einmalig in die Maschinensprache des Rechners übersetzt, ein fertiges, ausführbares Programm erstellt und dieses ausgeführt.

Dazwischen gibt es noch verschiedene Abwandlungen. So erzeugt ein Javacompiler keinen Maschinencode eines real existierenden Rechners sondern einen

Zwischencode, der dann zur Laufzeit von einem Interpreter ausgeführt wird.

Interpreter haben den Vorteil, dass das Programm sofort nach der Eingabe ausgeführt werden kann, da der Übersetzungsvorgang entfällt. Compiler haben dafür den Vorteil, dass die Zeit für die Ausführung eines Programms kürzer ist. Außerdem müssen Sie Ihren Quelltext nicht weitergeben.

Bei Java und ähnlichen Sprachen wie C# hat sich mittlerweile die Strategie durchgesetzt, dass der Interpreter den Zwischencode nicht einfach ausführt, sondern jede Funktion vor dem ersten Aufruf in Maschinencode übersetzt und bei jedem weiteren Aufruf nur noch diesen Maschinencode ausführt. Dadurch ist der Geschwindigkeitsvorteil von C/C++ gegenüber Java etwas geringer worden, dennoch immer noch vorhanden.

3. Der Übersetzungsvorgang

Der Weg von Ihrem C/C++ Programm gliedert sich in mehrere Schritte, die ich hier kurz erläutern möchte. Wichtig: Welche Befehle Sie genau in Ihrer Entwicklungsumgebung (Englisch Integrated Development Environment oder abgekürzt IDE) verwenden müssen, ist je nach Umgebung unterschiedlich. Da dieses Buch nicht für eine bestimmte Entwicklungsumgebung implementiert ist, sehen Sie bitte in der Anleitung Ihrer Entwicklungsumgebung (z.B. Visual C++, Xcode oder C++ Builder) nach.

Ihre Quelltexte werden mit einem Texteditor bearbeitet und in einfachen Textdateien mit den Dateinamenserweiterungen `.c` für C-Quelltexte oder `.cpp` für C++ Quelltexte gespeichert.

Wenn Ihre Programme sehr umfangreich geworden sind, können Sie Ihren Quelltext auch auf mehrere Dateien verteilen. Sie müssen dann alle Dateien zu einem Projekt in Ihrer IDE zusammenfassen. Dabei werden Sie von einem oder mehreren Quelltextdateien auf Elemente einer anderen Datei zugreifen müssen. Zu diesem Zweck können Sie in jeder Datei dem Compiler mitteilen, dass es solche Elemente gibt, oder besser Sie schreiben diese Information in einer getrennten, sogenannten Headerdatei und binden diese über spezielle Anweisungen in Ihre Quelltexte ein. Um den Überblick nicht zu verlieren, ist es sinnvoll, für jede Quelltextdatei genau eine solche Headerdatei zu erstellen, die die erforderlichen Informationen enthält. Diesen Headerdateien sollten Sie dann den gleichen Namen wie den Quelltextdateien geben nur mit der Erweiterung `.h` statt `.c` bzw. `.cpp`. Einige Implementierungen verwenden auch gerne für Headerdateien die Erweiterung `.hpp`, wenn sie zu einem C++ Quelltext gehören. Im aktuellen C++-Standard wird auf eine Erweiterung im Namen der Headerdateien sogar verzichtet. Ich persönlich ziehe es vor, grundsätzlich die Erweiterung `.h` zu verwenden. Sollte ich versehentlich einen C++ Header in einem C Quelltext eingebunden haben, wird mir dies der Compiler schon mit einer oder mehreren Fehlermeldungen quittieren.

Aufgabe des Compilers ist es, die Quelltextdatei zu lesen und hieraus die Maschinenbefehle für Ihren Prozessor zu erzeugen und diese in einer Objektdaten zu speichern. Die Objektdaten haben üblicherweise die Erweiterung `.o` oder `.obj`.

Diese Dateien können aber in der Regel nicht ausgeführt werden, da ihnen betriebssystemspezifische Information noch fehlen.

Ein besonderer Teil des Compilers ist der Präprozessor. Oft wird dieser auch als eigenständiges Programm dem Compiler vorangestellt. Mit Hilfe des Präprozessors können die Headerdateien in den Quelltext eingebunden, Teile der Datei von der Übersetzung ausgeschlossen (bedingte Kompilierung) und Makros und konstante Werte definiert und verwendet werden.

Wollen Sie eine oder mehrere Quelltexte in mehreren Projekten einsetzen, können Sie diese zu einer Bibliothek zusammenfassen. Ein spezielles Programm, der sogenannte Bibliotheksmanager (oder englisch Librarian) sammelt dann alle vom Compiler erstellten Objektdateien in einer Bibliotheksdatei zusammen. Sie müssen dann in anderen Projekten nur die Bibliotheksdatei und die Headerdateien einbinden. Die C/C++ Quelltexte sind nicht erforderlich. Alle Standardfunktionen der C/C++ Implementierung sind in solchen Bibliotheksdateien gespeichert und werden i.d.R. automatisch in Ihren eigenen Projekten eingebunden. Die Dateierweiterung hängt von der Art der Bibliothek und Ihrem Betriebssystem bzw Ihrer IDE ab. Da es hier keinen Standard gibt, werden wir in diesem Buch nicht näher darauf eingehen.

Weder Bibliotheks- noch die Objektdateien stellen ausführbare Programme dar, obwohl sie die Maschinenbefehle für den Prozessor bereits enthalten. Daher gibt es als letzten Schritt noch die Bindung (englisch linking). In diesem Schritt werden alle vom Compiler erstellten Objektdateien und alle erforderlichen Bibliotheksdateien zu einer Datei zusammengefasst, die Sie dann mit den Mitteln des Betriebssystems ausführen können. Das hierfür erforderliche Programm ist der Linker. Wie dieser Schritt genau funktioniert und welche Dateien Sie mit ausliefern müssen, um das Programm auch auf einem anderen Rechner zu starten, hängt wieder von der verwendeten IDE und dem Betriebssystem ab. In den einfachsten Fällen wird aber für gewöhnlich genau eine ausführbare Datei erzeugt, die Sie ohne weitere Zugaben weitergeben können.

Tipp: Compiler sind sicherlich nicht fehlerfrei, bedenken Sie aber, dass diese gerade am Anfang die Programmiersprachen besser kennen wie Sie. Viele Compiler bieten daher die Option, fehlerhafte Zeilen, die zwar syntaktisch korrekt sind aber möglicherweise trotzdem falsch sind, zu markieren und eine Warnung auszugeben. Schalten Sie diese Option ein, lesen Sie die Warnungen aufmerksam und prüfen, ob Sie tatsächlich einen Fehler haben, bevor Sie andere um Rat bitten. Viel zu häufig habe ich schon Anfängerfragen beantworten müssen, die den Programmieren schon der Compiler beantwortet hätte, wenn sie meinen Rat befolgt hätten.

Bevor Sie das ausführbare Programm weitergeben, müssen Sie auf jeden Fall prüfen, ob Ihr Programm fehlerfrei ist. Fehlerfreiheit ist war ein Wunsch, der nur sehr selten erreicht wird, aber Sie sollten stets versuchen, Ihre Programme mit möglichst wenigen Fehler auszuliefern.

Vorab können Sie einen sogenannten Unittest für Ihre Module implementieren, mit denen Ihre Module isoliert vom Rest des Programms testen können. Je nach Programmiersprache gibt es verschiedene Systeme wie JUnit oder Google Test mit denen Sie arbeiten können. Am Beginn Ihrer Karriere als Programmierer werden Sie

sicherlich nicht die Riesenprojekte stemmen, so dass Unittests unverzichtbar sind.

Sie haben auch die Möglichkeit, das Programm im Debugmodus zu übersetzen und das ausführbare Programm in einem Debugger laden. Im Debugmodus wird das Programm ohne Optimierungen übersetzt und in dem ausführbaren Programm Verweise auf die Quelldateien eingefügt (Debuginfo). Der Debugger lädt dann sowohl das Programm aus auch die Quelldateien und zeigt Ihnen Schritt für Schritt wie sich die Variablen bei welchen Statements ändern.

4. C/C ++ Begriffe

Bei der Entwicklung mit C/C++ trifft man immer wieder auf bestimmte Begriffe, die hier erklärt werden wollen. Die meisten dieser Begriffe finden Sie auch bei anderen Programmiersprachen, so dass es nicht schadet, diese zu kennen. Die Begriffe werde ich hier sowohl in Englischen als auch im Deutschen anführen, da bei Fehlermeldungen häufig die englischen Begriffe benutzt werden:

4.1. Anweisung - Statement

Eine Anweisung oder Statement ist ein Befehl für Computer etwas zu tun. Wie zum Beispiel, speichere einen bestimmten Wert in eine Zelle. Fahre mit dem Programm an einer anderen Stelle fort etc.

Anweisungen werden fast immer mit einem Strichpunkt ; beendet, mehrere Anweisungen können zu einer zusammen gefasst werden, in dem sie in geschweiften Klammern { ... } geschrieben werden. Dies ist erforderlich, weil manche Kontrollstrukturen genau eine Anweisung kontrollieren.

```
{  
    befehl1;  
    befehl2;  
}
```

Tipp: Leere Anweisungen, das sind einzelne Strichpunkte ohne einen Befehl davor, werden vom Compiler anstandslos akzeptiert, Die kann aber im Zusammenhang mit Abfragen oder Schleifen zu unerwünschten Ergebnissen führen. Achten Sie daher darauf, dass Sie leeren Anweisungen eine eigen Zeile spendieren, um späteren Lesern Ihres Quelltextes zu signalisieren, dass dies Absicht und korrekt ist.

4.2. Ausdruck - Expression

Ein Ausdruck ist ein Sprachelement, das einen beliebigen (berechneten) Wert darstellen kann. Dieser kann entweder eine Zahl, ein Zeichen, eine Zeichenfolge oder etwas Zusammengesetztes sein. Mehrere (Teil)ausdrücke können über Operatoren zu einem neuen Ausdruck zusammen gestellt werden. Ein Ausdruck, der mit einem Strichpunkt beendet wird, ist auch eine Anweisung.

```
123;  
'a';  
"Hallo kleine Maus";  
234.098;  
var1 = var2 + 768;
```

4.3. RValue - LValue

Von manchen Ausdrücken können die Adressen ermittelt werden. Nur solche können auch auf der linken Seite einer Zuweisung stehen, sie sind also veränderbar. Diese Ausdrücke heißen LValue (Locator Value oder auch Left Value). Ausdrücke, die nur auf der rechten Seite stehen dürfen, heißen RValue (Right Value).

4.4. Bezeichner - Identifier

Bezeichner sind die Namen aller Elemente, die selbst definiert wurden. Nähere Informationen finden Sie in einem eigenen Kapitel zum Sprachkern weiter unten.

4.5. Schlüsselwort - Keyword

Schlüsselworte sind Begriffe im Quelltext, die eine fest vorgegebene Bedeutung haben. Sie dürfen nicht für Bezeichner verwendet werden. Die meisten Entwicklungssysteme sind daher in der Lage, Schlüsselworte besonders zu kennzeichnen, so dass diese sofort erkannt werden.

4.6. Deklaration - Declaration

Eine Deklaration teilt dem Compiler mit, dass es eine Variable oder Funktion mit einem bestimmten Namen gibt und welchen Typ es hat bzw. auch welche Argumente erwartet werden. Es wird jedoch kein Speicherplatz für die Variable bzw. die Funktion bereitgestellt. Eine Variable bzw. eine Funktion darf beliebig oft in einem Programm deklariert werden. Sie müssen jedoch dafür sorgen, dass alle Deklarationen übereinstimmen. Daher ist es sinnvoll im gesamten Programm nur eine Deklaration zu schreiben und diese in allen Modulen, die sie brauchen, mit dem Präprozessor einzubinden.

4.7. Definition

Eine Definition teilt wie eine Deklaration dem Compiler mit, dass es eine Variable oder Funktion gibt. Nur wird auch zusätzlich der entsprechende Speicherplatz bereitgestellt. Im gesamten Programm darf es für eine Variable oder eine Funktion nur eine Definition geben. Andernfalls erhalten Sie eine Warnung oder einen Fehler vom Linker. Eine Definition ist grundsätzlich auch eine Deklaration.

4.8. Initialisierung

Wenn bei der Definition einer Variable gleich ein Wert zugewiesen wird, spricht man von einer Initialisierung. Deklarationen, bei denen der Variable ein Wert zugewiesen werden, sind automatisch auch die Definitionen der Variablen. Wird bei der Definition einer Variablen, diese nicht initialisiert, kann dies auch später geschehen. Die erste Verwendung muss aber immer eine Initialisierung sein. Viele Compiler können aber die Verwendung einer nicht initialisierten Variablen erkennen und geben eine Warnung aus.

4.9. White space

Sehr häufig finden Sie in Dokumentationen den Begriff white space. Eine vernünftige

deutsche Übersetzung dieses Begriffs kenne ich nicht. Mit white space sind alle Zeichen gemeint, die einen Abstand zwischen zwei druckbaren Zeichen (Buchstaben, Ziffern, Satzzeichen etc.) bewirken. Solche Zeichen sind das Leerzeichen (Space), Tabulator, Zeilen- (Line Feed) und Seitenvorschub (Form Feed) und der Wagenrücklauf (Carriage Return). Wann immer im Quelltext ein white space erscheinen darf oder muss, dürfen auch mehrere white space verwendet werden. Einzige Ausnahme sind Texte, die von Ihrem Programm bearbeitet werden und vom Compiler quasi nur durchgereicht werden. So haben folgende Statements unterschiedliche Ausgaben:

```
puts( "Hallo Welt" );  
puts( "Hallo      Welt" );  
puts( "      Hallo Welt" );
```

Die folgenden Statements sind aber für das Programm völlig identisch:

```
puts("Hallo Welt");  
puts(      "Hallo Welt"      )      ;  
puts(  
    "Hallo Welt"  
);
```

4.10. Ressource

Ressourcen sind vom Computer bereitgestellte Komponenten, die Sie anfordern und in Ihrem Programm nutzen können. Manche dieser Komponenten werden Ihrer Anwendung exklusiv zur Verfügung gestellt andere können gleichzeitig von mehreren Programmen benutzt werden. Die meisten Ressourcen werden wieder freigegeben, wenn Ihre Anwendung beendet wird. Sie sollten sich dennoch angewöhnen, Komponenten, die Sie nicht mehr brauchen, explizit dem System wieder zurückzugeben.

5. Der Sprachkern

Dieses Kapitel beschreibt den Sprachkern. Das ist der Teil der Sprache, der vom Compiler behandelt wird. Standardfunktionen wie z.B. die Eingabe von Daten werden vom Compiler nicht bearbeitet, sondern nur vorbereitet und später vom Linker dem fertigen Programm hinzugefügt. Der Erklärungen erfolgen daher relativ knapp, da ohne Standardfunktionen sinnvolle Beispielprogramme nicht möglich sind. Die meisten Beispiele hier sind daher nur Codefragmente und dienen ausschließlich der Demonstration der Syntax. Gerade die Erweiterungen von C++ sind aber für solche Erklärungen nicht geeignet und werden hier nur kurz erwähnt und auf weiter hinten verschoben, wenn wir mit der Standardbibliothek soweit sind, dass wir sinnvolle Beispielprogramme uns anschauen können.

Bei der Entwicklung Ihres C/C++ Programms müssen Sie i.d.R. folgende Elemente definieren: Variablen, Datentypen und Funktionen.

Mit Variablen beschreiben Sie welche Daten Ihr Programm verarbeiten kann. Sie können beliebig viele Variablen definieren. Jede Variable hat genau einen Datentyp, der beschreibt, welcher Art von Daten die Variable speichern kann. Da der Compiler

nur eine eingeschränkte Anzahl von Datentypen kennt, können Sie aus den bekannten Datentypen eigene sogenannte zusammengesetzte Typen (Aggregates) definieren. Achten Sie bitte darauf, dass Sie mit Ihren Datentypen und Variablen das zu lösende Problem und das gewünschte Ergebnis exakt modellieren. Daher spricht man auch von dem Datenmodell eines Programms. Je besser Ihre Typen und Variablen Ihren Anforderungen entsprechen, desto größer ist die Wahrscheinlichkeit, dass das fertige Programm auch tatsächlich das gewünschte Ergebnis liefert.

Bedenken Sie auch, dass der Gültigkeitsbereich einer Variablen oder eines Datentyps möglichst klein sein sollte. Denn nachträgliche Änderungen haben zwangsweise einen mehr oder weniger großen Einfluss auf Ihre Funktionsdefinitionen. Scheuen Sie aber trotzdem nicht, Ihr Datenmodell frühzeitig zu ändern, wenn das erforderlich sein sollte (z.B. durch neue Anforderungen) und nicht um fehlerhafte Datenmodelle herum zu programmieren. Sie mögen dabei zwar kurzfristig den Anpassungsaufwand einsparen, früher oder später wird dann aber das Programm so unübersichtlich, weil zu viele Sonderfälle beachtet werden müssen, so dass der Änderungsaufwand selbst für kleine Anpassungen unverhältnismäßig groß wird. Dann wird man irgendwann einmal an den Punkt gelangen, wo das ganze Programm komplett neu entwickelt werden muss und der in früheren Entwicklungsschritten eingesparte Aufwand um ein Vielfaches „zurückgezahlt“ werden muss. Man spricht hier auch von einer digitalen Schuld, mit der man sich belastet, wenn hier nicht sorgfältig gearbeitet wird.

5.1. Präprozessor

Der Präprozessor stellt den ersten Schritt einer Übersetzung dar. Mit seiner Hilfe können Sie andere Dateien in den Übersetzungsvorgang einbinden, Teile der Quelldatei von der Übersetzung ausschließen oder Makros definieren. Jede Quelltextzeile, die für den Präprozessor bestimmt ist, muss mit einem Nummernkreuz # anfangen. Wenn die Zeile umgebrochen werden muss, beenden Sie die Zeile mit einem Backslash \, die nachfolgende Zeile muss dann aber nicht mit dem Nummerkreuz anfangen.

Folgende Präprozessorkommandos gibt es:

a. include

Mit Hilfe der Präprozessoranweisung #include können Sie weitere Quelldateien in die Übersetzung einbinden. Für den Compiler sieht das dann so aus, als ob die eingebundene Datei, Bestandteil des Quelltextes wäre.

Üblicherweise benutzt man diese Anweisung, um Variablendeklarationen, Funktionsdeklarationen (Prototypen) und/oder Typdeklarationen anderer Module mit in den Quelltext aufzunehmen. Es ist zwar möglich auch andere Quelldateien einzubinden, das ist aber unüblich und kann zu unvorhergesehenen Seiteneffekten führen, die schwer zu beheben sind.

Folgt der Includeanweisung der Dateiname in doppelten Anführungszeichen, so wird die Datei zuerst im Verzeichnis des aktuellen Moduls gesucht und dann in den Pfaden für die Includedateien. Wird die Datei jedoch mit einem Kleiner- < und einen

Größerzeichen > eingerahmt, ist die Suchfolge umgekehrt.

```
#include "meinHeader.h"
#include <stdio.h>
```

Wenn Sie Funktionen der Standardbibliothek aufrufen wollen, müssen Sie eine oder mehrere Standardincludedateien in Ihrem Programm einbinden. C-Includedateien haben die Erweiterung h. C++-Includedateien verzichten auf eine Dateinamenserweiterung. C-Includedateien können Sie zwar auch in C++-Quellen einbinden, davon wird aber abgeraten, da diese nicht die Besonderheiten von C++ nutzen. Wenn Sie aber dennoch eine C-Funktion aufrufen müssen, includieren Sie bitte die C++-Version der C-Includedatei. Den korrekten Namen können Sie einfach dadurch bilden, indem Sie die Erweiterung h weglassen und dafür ein c voranstellen.

```
#include <stdio.h>           // Alt
#include <cstdio>             // Neu
```

Dies bewirkt, dass die C-Standardfunktionen in den besonderen Umfeld von C++ besser eingebunden werden.

b. define

Mit der Präprozessoranweisung `#define` können Sie Makros definieren. Makros verwenden Sie entweder für benannte Konstanten oder aber um kleine Funktionsschnipsel zu definieren. Makros können auch parametrisiert werden.

```
#define NUM_FIELDS          1024
#define NUM_ELEMENTS( array )  (sizeof( array ) / sizeof( array[0] ))
#define PRINT_ARRAY( array )  \
{ \
    int numElements = NUM_ELEMENTS( array ); \
    int i;\
    for( i=0; i<numElements; i++ ) \
        printf( "%03d %08d\n", i, array[i] ); \
}

int meinArray[NUM_FIELDS];
int numElements = NUM_ELEMENTS( meinArray );
PRINT_ARRAY( meinArray );
```

Für Funktionsschnipsel aber auch Konstanten verwenden Sie aber bitte lieber die Möglichkeiten des Compilers. Alle C++ Compiler und aktuelle C-Compiler haben hier bessere Möglichkeiten, die z.B. die Parameter der Funktionsschnipsel besser überprüfen können. Darüber hinaus kann es vorkommen, dass Sie unerwartete Seiteneffekte erhalten, wenn ein Makroparameter in der Ersetzung mehr als einmal verwendet wird und beim „Aufruf“ ein berechneter Ausdruck benutzt wird. In diesem Fall wird der Ausdruck mehr als einmal berechnet, was unerwünscht sein kann:

```
#define power( i ) (i)*(i)
int meineVariable = 10;
int quadrat = power( ++meineVariable );
```

Wenn bei einer Makrodefinition der Makroparameter mit einem doppelten Nummernkreuz vorangestellt wird, so wird beim Aufruf des Makros, der

entsprechende Aktualparameter als Stringliteral eingesetzt.

```
#include <stdio.h>
#define PRINT_VARIABLE( variable )          \
    printf( "%s=%d\n", ##variable, (variable) )

int main( void ) {
    int meinWert = 1234;
    PRINT_VARIABLE( meinWert );
    return 0;
}
```

c. ifdef - else - endif

Mit Hilfe dieses Konstrukts können Sie Teile des Quelltextes ausschließen bzw. einschließen, wenn ein Makro definiert ist. Der Else-zweig ist optional.

Die meisten Compiler definieren schon von sich aus, verschiedene Makros mit deren Hilfe Sie zum Übersetzungszeitpunkt bestimmen können, dass Ihre Anwendung bestimmte Befehle nur für besondere Umgebungen ausführen muss. Sehen Sie in der Dokumentation Ihres Compilers nach, welche definiert werden.

```
#define NDEBUG
#ifdef NDEBUG
    printf( "Hallo ich bin hier\n" );
#endif
#ifdef NDEBUG
    printf( "Hallo ich bin definiert 8-)\n" );
#else
    printf( "Hallo ich bin nicht definiert 8-(\n" );
#endif
#ifdef __WIN32__
    getchar();
#endif
```

d. ifndef - else - endif

Mit Hilfe dieses Konstrukts können Sie ebenfalls Teile des Quelltextes ausschließen bzw. einschließen, wenn ein Makro definiert ist. `ifndef` arbeitet aber genau umgekehrt wie `ifdef`.

```
#define NDEBUG
#ifndef NDEBUG
    printf( "Hallo ich bin nicht hier\n" );
#endif
#ifndef NDEBUG
    printf( "Hallo ich bin nicht definiert 8-(\n" );
#else
    printf( "Hallo ich bin definiert 8-)\n" );
#endif
```

Da Headerdateien auch von anderen Headerdateien eingebunden werden können, kann es bei größeren Projekten gerne passieren, dass manche Definitionen doppelt in Ihr Modul eingebunden werden. Um dies wirkungsvoll zu verhindern, werden Headerdateien gerne mit einer Kombination aus `ifndef` und `define` eingerahmt.

Hierfür wird häufig der Begriff „include guards“ verwendet.

```
#ifndef MEINE_HEADER_DATEI_H
#define MEINE_HEADER_DATEI_H
...
#endif
```

e. if - else - endif

Für komplexere Bedingungen können Sie auch die Präprozessoranweisung `#if` verwenden. Diese erlaubt auch die Verwendung der Operatoren `||` (or), `&&` (and), `!` (not) und den Vergleichsoperatoren und der Präprozessorfunktion `defined`.

```
#if !defined( __BORLANDC__ ) || __BORLANDC__ > 0x520
...
#endif
```

f. line

Mit diesem Befehl, können Sie dem Compiler mitgeben, welchen Dateinamen und welchen Offset für Zeilennummern er ab diesen Punkt in Fehlermeldungen angeben soll. Der Offset berechnet sich aus dem angegebenen Wert und die tatsächliche Zeilennummer. Das ist nützlich, wenn der C oder C++-Quellcode aus einer anderen Quelle generiert wurde und die Fehlermeldungen sich auf diesen anderen Code beziehen sollen. Der Dateiname ist optional.

```
#line 666 "IronMaiden.cpp"
```

g. Standardkonstanten

Der Präprozessor definiert einige Makros von sich aus und aktualisiert diese auch, wenn notwendig.

Konstante	Beschreibung
<code>__FILE__</code>	Dateinamen.
<code>__LINE__</code>	Zeilennummer.
<code>__DATE__</code>	Aktuelles Datum
<code>__TIME__</code>	Uhrzeit
<code>__STDC__</code>	Implementierungsabhängiger Wert. Typischerweise kann hier die Version des Standard ermittelt werden, nach dem der Compiler implementiert ist.
<code>__cplusplus</code>	Der C++-Standard.
<code>NDEBUG</code>	Dies ist eigentlich kein Standardmakro, wird aber von vielen Compilern automatisch gesetzt, wenn das Programm ohne Unterstützung des Debuggers übersetzt wird.

Die Expansion dieser Makros geschieht als letzten Schritt des Präprozessors. Das heißt, wenn eines der Konstanten in einem anderen Makro verwendet wird, so wird der Dateiname und die Zeilennummer verwendet, wo das Makro expandiert und nicht

definiert wurde.

5.2. Einfache Typen

Die einfachen Typen, den sogenannten intrinsics sind die Grundbestandteile Ihrer Variablen- und Typdefinitionen.

Folgende Typen sind bekannt: bool, char, wchar_t, int, float und double.

Variablen vom Typ bool können nur zwei verschiedene Werte annehmen und speichern den Wahrheitsgehalt einer Aussage. Mit den Schlüsselworten true für wahr und false für falsch geben Sie die entsprechenden Werte an. Bitte beachten Sie: Ältere C-compiler kennen diesen Typ noch nicht, da dieser erst bei C++ eingeführt wurde und erst später vom C-Standard übernommen wurde.

Mit dem Typ char können Sie einzelne Zeichen speichern. Der mögliche Wertebereich dieses Typs entspricht immer dem Wertebereich der einzeln adressierbaren Speicherzellen des Arbeitsspeichers Ihres Computers. Wie im Grundlagenteil dieses Buches schon geschrieben, ist die Breite einer solchen Zelle meistens 8 Bit, was ausreicht, um alle Zeichen des ANSI-Zeichensatzes zu speichern. Alternativ können Sie aber auch die entsprechende Dezimaldarstellung verwenden.

Die Zeichenkodierung ist Compiler- bzw. Betriebssystemabhängig. Die meisten Systeme verstehen zumindest den ASCII-Zeichensatz. Ein aktuelles Windows versteht in der Regel den ANSI-Zeichensatz- Linux und Mac OS X arbeitet mit der UTF-8-Kodierung. Achten Sie darauf, dass der Quelltexteditor, mit dem Sie arbeiten, die gleiche Zeichenkodierung verwendet wie Ihr Betriebssystem, andernfalls kann es passieren, wenn Sie z.B. deutsche Umlaute verwenden, dass Ihre Programme nicht übersetzt werden, obwohl im Editor alles richtig aussieht. Wenn Sie beabsichtigen, Ihre Programme auf mehreren Systemen zu übersetzen, sollten Sie daher Umlaute in Ihrem Quelltext vermeiden.

Der Standard verlangt auch nicht, ob der Typ vorzeichenbehaftet oder vorzeichenlos ist. Der Wertebereich kann daher von -128 bis + 127 oder von 0 bis 255 gehen. Die Prüfung, ob ein bestimmtes Zeichen im ASCII-Zeichensatz enthalten ist, erfolgt daher entweder über kleiner als 0 bei vorzeichenbehafteten char oder größer als 127 bei vorzeichenlosen char. Manche Compiler bieten aber auch eine Möglichkeit, dies einzustellen.

Der Typ wchar_t dient der Speicherung der Zeichen eines Unicodezeichensatzes. Bei C++ ist dies ein eingebauter Datentyp, ältere C-Compiler kennen ihn nur, wenn Sie die Headerdatei <stddef.h> eingebunden haben.

Der Typ int dient der Speicherung von ganzen Zahlen. In den meisten Fällen entspricht der Wertebereich dieses Typs dem Wertebereich der CPU-internen Speicher, den CPU-Registern, oder der Breite des Datenbusses. Bei den heutzutage üblichen 32- oder 64-Bit-Compilern ist die Breite 32 Bit. Der Wertebereich geht dann von -2147483648 bis +2147483647.

Float und double speichern Fließkommazahlen mit einfacher (float) oder doppelter

(double) Genauigkeit. Den Datentyp float sollten Sie nur nehmen, wenn Sie sehr große Datenmengen zu verarbeiten haben und Speicherplatz ein Problem sein könnte oder wenn eine hohe Rechenleistung benötigen und Genauigkeit eine untergeordnete Rolle spielt.

Zusätzlichen zu den aufgezählten Typen können Sie mit den sogenannten Modifizierern (type modifier) den Wertebereich der Variablen beeinflussen:

signed/unsigned: Hiermit legen Sie fest, ob die Variable mit oder ohne Vorzeichen verwendet werden soll. Bei signed gilt der Wertebereich von einem negativen Minimum bis zu einem positiven Maximum (z.B. -31768 bis + 31767) Bei unsigned ist das Minimum immer 0 und geht bis zu einem positiven Maximum (z.B. 0 bis 65535). Wenn Sie diesen Modifizierer nicht angeben, wird bei int immer signed angenommen, bei char und wchar_t hängt dies vom Compiler ab. Meistens ist das dann auch signed kann aber i.d.R. über einen Compilerschalter beeinflusst werden. Für die Typen bool, float und double steht dieser Modifizierer nicht zur Verfügung.

long/short vergrößert bzw. verkleinert den Wertebereich einer Integervariable. Die genauen Auswirkungen sind leider wieder abhängig vom Compiler. Long dürfen Sie bei neueren Compilern auch zweimal angeben. Damit wird die Wirkung sogar noch einmal verstärkt. Long darf auch im Gegensatz zu short auch bei Variablen vom Typ double angegeben werden. Auch hier kann die Genauigkeit der Variablen vergrößert werden. Es ist jedoch Abhängig vom Compiler und vom Prozessor, ob dies geschieht.

Leider macht der Standard keine Aussage über den Wertebereich eines Datentyps. Die einzige Bedingung, die erfüllt werden muss ist `short int <= int <= long int <= long long int`. Häufig stellt dies keine allzu große Limitierung dar, im späteren Übungsteil werde ich Ihnen jedoch zeigen, wie Sie mit diesen Problem umgehen können, falls das erforderlich sein sollte.

Da der Compiler bei vielen Angaben, wo ein Datentyp erwartet wird, int annimmt, falls die Angabe fehlt, können Sie hier die Variablendefinition verkürzen, indem Sie int weglassen, falls Sie einen der Modifizierer benutzen wollen. Das heißt statt `unsigned int` können sie auch nur einfach `unsigned` verwenden oder `short int` wird zu `short` verkürzt.

In C++11 und C99 wurden einige Datentypen eingeführt, die eine feste Breite haben. Das ist wichtig, um z.B. binäre Zahlenwerte in einer Datei ablegen und sicher wieder lesen zu können oder um einen garantierten Wertebereich zu haben. Diese Typen finden Sie in `<stdint.h>` bzw. `<cstdint>`. Die wichtigsten sind hier aufgelistet:

int8_t, int16_t, int32_t, int64_t	Vorzeichenbehafteter Integerwert mit 8, 12, 32 oder 64 Bit breite.
uint8_t, uint16_t, uint32_t, uint64_t	Vorzeichenloser Integerwert mit 8, 12, 32 oder 64 Bit breite.

Das Pendant zu `<stdint.h>` bzw. `<cstdint>` für Ganzzahlen ist in C++ ab C++23 die Datei `<stdfloat>`, die einige Fließkommatypen definiert mit fester Größe: `float16_t`,

float32_t, float64_t, float128_t und bfloat16_t.

5.3. Typqualifizierer

Mit dem Schlüsselwort `const` können Sie einen LValue definieren, der nicht verändert werden darf. Dies können Sie in C++ auch dazu nutzen konstante Werte in Ihrem Programm zu definieren ohne dabei auf die Präprozessoranweisung `#define` zurückgreifen zu müssen. In C++ ist es dem Compiler vorgeschrieben, dass er bei einem so definierten Lvalue den konstanten Wert direkt in den Programmcode einfügt, wenn der Wert auch bekannt ist.

```
const int NUM_FIELDS = 1000;
int array[NUM_FIELDS];
for( int i=0; i<NUM_FIELDS; i++ )
{
    ...
}
```

Diese Variante hat den Vorteil, dass Sie die Gültigkeit der Konstante einschränken können, was bei Präprozessorkonstanten nicht möglich ist. Dadurch können Sie die Gefahr von Mehrdeutigkeiten verringern. In C++ ist es daher auch möglich, eine konstante Variablendefinition auch in einer Headerdatei vorzunehmen ohne einen Linkerfehler wegen doppelt definierten Symbolen zu erhalten. Neuere C-Compiler sind dazu auch in der Lage.

Ein anderer Effekt dieses Schlüsselworts ist, dass der Compiler eine so definierte Variable in einen Speicherbereich ablegen kann, der schon von der Hardware aus nicht beschrieben werden kann. Jeder Versuch eine solche Variable zu ändern scheitert daher spätestens zur Laufzeit.

Mit Hilfe des Schlüsselwortes `const` können Sie dem Compiler mitteilen, dass die Speicherzelle, auf die ein Zeiger verweist nicht geändert werden darf.

```
int variable=10;
int variable2;
const int *zeiger = &variable;
*zeiger = 39;           // verboten;
variable2 = *zeiger;    // erlaubt
```

Ein Zeiger auf konstante Werte ist beispielsweise bei Funktionsparametern sehr sinnvoll, weil damit der Aufrufer weiß, dass der übergebene Wert von der aufgerufenen Funktion nicht geändert wird.

Ein Stringliteral alleine für sich genommen ist die Adresse eines Speichers, der nicht verändert werden darf und in dem der angegebene Text gespeichert ist. Man kann daher diese Adresse auch einem Zeiger zuweisen. Um sicher zu stellen, dass der Speicher nicht versehentlich verändert wird, muss der Zeiger mit `const` versehen werden.

```
const char *zeiger = "Mein Name ist Hase";
```

Wenn der Zeiger selbst nicht veränderbar sein soll, müssen Sie `const` zwischen dem `*` und dem Namen der Variablen setzen.

```
const char * const zeiger = "Mein Name ist Hase";
zeiger = "Hello world";      // verboten
```

Für Referenzen in C++ gilt das selbe wie für Zeiger. Eine const-Referenz kann nicht dazu benutzt werden, den referenzierten Wert zu verändern.

```
int variable=10;
int variable2;
const int &ref = variable;
ref = 39;          // verboten;
variable2 = ref;   // erlaubt
```

const-Referenzen verlängern auch die Lebensdauer von temporären Objekten. Diese sind solange gültig wie die const Referenz.

```
int variable=10;
int variable2;
const int &ref = variable + 20;
variable2 = ref;
```

Mit dem Schlüsselwort `volatile` erreichen Sie das ganze Gegenteil von `const`. Dieser Typqualifizierer teilt dem Compiler mit, dass sich der Inhalt der Variable jederzeit auch außerhalb der Kontrolle der aktuellen Funktion ändern kann. Daher ist es dem Compiler verboten, die Schreib- und Lesezugriffe auf diese Variable zu optimieren. Sie müssen immer sofort und in der selben Reihenfolge ausgeführt werden. `Volatile` wird beispielsweise benutzt, wenn Sie Anwendungen mit mehreren Threads oder Interrupthandlern implementieren.

```
volatile int variable=10;
int variable2;
volatile int &ref = variable;
ref = 39;
variable2 = ref;    // hier darf nicht einfach fest verdrahtet 39
                   // geschrieben werden
```

Der Typqualifizierer `mutable` von C++ spielt nur bei der Definition von Klassen eine Rolle. Er wird daher erst später behandelt.

5.4. Zeiger, Adressen und Referenzen

Manchmal ist es erforderlich, die Adresse einer Variable im Speicher zu ermitteln. Dies benötigen Sie beispielsweise dann, wenn eine Variable von einem Unterprogramm aus verändert werden soll. Aber auch wenn Variablen dynamisch auf dem sogenannten Heap (Haufen) erzeugt werden sollen, erhalten Sie nur eine Adresse von diesem Speicher.

Die Adresse einer Variablen ermitteln Sie mit dem Adressoperator `&` gefolgt von der Variablen:

```
int variable;
&variable;
```

Um die Adresse einer Variablen bzw. eine Speicheradresse in einer anderen Variablen speichern zu können benötigen Sie eine Zeigervariable bzw. kürzer einen Zeiger. Einen Zeiger definieren Sie mit einem Typ und dem Dereferenzoperator `*`.

Der Typ ist erforderlich, damit der Compiler weiß, welchen Typ der Inhalt des Speichers hat, der mit dem Zeiger adressiert werden soll:

```
int variable;
int *zeiger;
zeiger = &variable;
```

Um den Inhalt der Speicheradresse zu verändern oder zu lesen, verwenden Sie ebenfalls den Dereferenzoperator `*` gefolgt vom Namen des Zeigers:

```
int variable;
int *zeiger;
variable = 5;
zeiger = &variable;
*zeiger = 10;           // dies überschreibt variable
```

Zeiger, deren Typ Sie (noch) nicht kennen, dürfen mit dem Pseudotyp `void` definiert werden. Solche Zeiger können jedoch nicht dereferenziert werden, sie müssen daher vorher mit dem cast operator `()` auf den richtigen Typ gewandelt werden.

```
int variable;
void *zeiger;
variable = 5;
zeiger = &variable;
*zeiger = 10;           // Fehler
*(int*)zeiger = 10;     // korrekt
```

Beim Rechnen mit Zeigern wird grundsätzlich die Größe des Zeigertyps berücksichtigt. Wenn Sie beispielsweise einen Zeiger um eins erhöhen, bewirkt dies, dass der Zeiger nun auf den dahinter liegenden Wert verweist. Beachten Sie aber, dass das Rechnen mit Zeigern nur innerhalb eines Speicherblockes definiert ist.

```
int variable[15];           // Array siehe nächstes Kapitel
int *zeiger;
zeiger = &variable[0];
*zeiger = 10; // verändere den ersten Wert
zeiger++;    // zeige auf den nächsten Wert
*zeiger = 15; // verändere den zweiten Wert
```

Wenn Sie einen Zeiger auf eine Struktur haben, müssen Sie diesen nicht mit dem Dereferenzoperator `*` verwenden, um auf ein einzelnes Element der Struktur zuzugreifen. Stattdessen können Sie den Operator `->` verwenden.

```
struct myStruct
{
    int  element1,
        element2;
} variable;

struct myStruct *zeiger = &variable;
(*zeiger).element1 = 10;
zeiger->element2 = 20;
```

Grundsätzlich müssen Sie selbst dafür sorgen, dass Ihre Zeiger immer auf gültige Speicherzellen verweisen, die Sie lesen bzw beschreiben dürfen. Der Sprachkern

von C bzw. C++ sieht keinerlei Sprachmittel vor, dies wirksam zu garantieren. Wenn Sie ungültige Zeiger verwenden, ist das Verhalten nicht definiert. D.h. es kann alles mögliche passieren, auch ein schwerer Datenverlust oder Ihre Anwendung wird anfällig für Angriffe von außen.

Ein besonderer Wert für eine Adresse stellt der Wert 0 dar. Mit diesem Wert wird oft ein Fehlerfall oder die Tatsache, dass die Adresse (noch) nicht bekannt ist, angezeigt. Für diesen Wert ist die Konstante NULL in der Headerdatei <stddef.h> definiert. Ab C++11 können Sie statt NULL auch das neue Schlüsselwort nullptr verwenden. Das hat gegenüber dem Makro NULL den Vorteil, dass es auch gleich den korrekten Typ annehmen kann.

Als Alternative zu Zeigern können Sie bei C++ Referenzen benutzen. Referenzen werden ähnlich wie Zeiger definiert. Statt dem Dereferenzoperator wird jedoch der Adressoperator verwendet. Außerdem muss eine Referenz initialisiert werden und die Adresse, auf die die Referenz verweist, kann auch nachträglich nicht geändert werden. Auf der rechten Seite der Zuweisung muss ein LValue - also ein veränderbarer Wert - stehen. Den Adressoperator brauchen Sie bei der Definition nicht zu verwenden, das macht der Compiler automatisch. Bei der späteren Verwendung der Referenz müssen Sie diesen nicht explizit mit dem Dereferenzoperator dereferenzieren, auch das macht der Compiler automatisch. Wie auch bei den Zeigern müssen sie bei Referenzen selbst dafür sorgen, dass diese immer gültig bleiben. Insbesondere wenn eine Funktion eine Referenz zurückliefert, besteht die Gefahr, dass die Referenz ungültig wird. Lvalue-Referenzen auf lokale Variablen sind beim Funktionsergebnis nicht erlaubt. Rvalue Referenzen sind aber erlaubt. Trotzdem besteht die Gefahr, dass die Referenz nach der Funktion ungültig ist. Viele Compiler warnen daher davor, die Referenz auf einer lokalen Variable oder eines temporäres Objekt zurückzugeben.

```
int variable = 10;
int variable2;
int &falsch;           // Fehler: Definition fehlt
int &referenz = variable;
referenz = 20;         // verändert variable
variable2 = referenz;  // liest variable
```

Es ist auch nicht möglich die Adresse einer Referenzvariable zu ermitteln. Verwenden Sie den Adressoperator mit einer Referenzvariable, erhalten Sie die Adresse der Zielvariablen.

```
int variable = 10;
int variable2;
int &referenz = variable;
int *zeiger = &referenz;
variable2 = *zeiger;    // liest variable
```

Einen Zeiger auf eine Funktion können Sie auch sehr einfach deklarieren. Dazu schreiben Sie den Namen des Zeigers zusammen mit dem Zeigeroperator in Klammern gefolgt von der Parameterliste in Klammern. Ein Zeiger auf eine Funktion, die einen long zurückgibt einen ein Parameter von Typ int erwartet, kann so deklariert werden.

```
long (*myPointer)(int);
```

In einen solchen Zeiger wird die Adresse der Funktion gespeichert, indem Sie den Funktionsnamen ohne Argumentliste verwenden.

```
myPointer = myFunc;
```

Aufrufen können Sie die Funktion ganz normal über den Funktionszeiger wie jede andere Funktion auch.

```
result = myPointer( 2 );
```

Ab C++11 wurde das Konzept der RVALUE-Referenzen eingeführt. RVALUE-Referenzen referenzieren nicht Variablen im Speicher sondern temporäre Ergebnisse und verlängern deren Lebensdauer. Die Werte können nachträglich auch verändert werden. Normalerweise müssen die Ergebnisse eines Ausdrucks sofort einer Variable zugewiesen werden, wenn man sie nicht wieder verlieren will. Mit dieser neuen Art von Referenzen ist dies nicht notwendig. Eine RVALUE-Referenz wird deklariert indem zwei Zeichen & statt nur einer verwendet werden.

```
#include <iostream>
#include <string>

int main() {
    std::string s1 = "Test";
    // std::string&& r1 = s1;           // error: can't bind to lvalue
    std::string&& r2 = s1 + s1;        // okay: rvalue reference extends lifetime
    r2 += "Test";                      // okay: can modify through reference
    std::cout << r2 << '\n';
}
```

5.5. Arrays

Manchmal benötigen Sie mehrere Variablen ein und des selben Typs zur gemeinsamen Verarbeitung. Dies ist beispielsweise erforderlich, wenn Sie alle Primzahlen kleiner als 100 speichern und verarbeiten wollen. Ein Array können Sie einfach definieren: <typ> <name>[<Anzahl>];

```
int array[100];
```

Auf ein einzelnes Element greifen Sie über dessen Index mit dem Indexoperator [] zu. Der Index ist grundsätzlich eine Integerzahl von 0 bis zur Anzahl der Elemente des Arrays minus 1. Beachten Sie aber, dass weder der Compiler noch die Laufzeitumgebung die Korrektheit des Index überprüft. Wenn Sie versehentlich einen zu großen oder einen negativen Index verwenden, ist das Verhalten undefiniert und kann zum Datenverlust führen. Manche Compiler bieten jedoch eine Option, falsche Indizes zu überprüfen. Dadurch wird jedoch das Programm deutlich langsamer und es sollte daher nur zu Testzwecken eingeschaltet werden.

```
int array[100];
array[0] = 1;
array[1] = 2;
array[3] = 3;
array[4] = 5;
array[100] = 300;           // Index zu groß.
```

Wenn Sie ein Array schon bei der Definition mit Werten befüllen, dürfen Sie die Anzahl der Elemente des Arrays weglassen. Der Compiler ermittelt dies automatisch.

```
int array[] =
{
    1, 2, 3, 5, 7, 11
};
```

Mit Hilfe des Operators `sizeof` können Sie die Anzahl der Elemente eines Arrays ermitteln. Das ist praktisch, wenn das Array schon bei der Deklaration mit Werten befüllt wurde.

```
int array[] =
{
    1, 2, 3, 5, 7, 11
};

int anzahl = sizeof( array ) / sizeof( array[0] );
```

Wenn Sie den Namen eines Arrays ohne Indexoperator verwenden, erhalten Sie die Adresse des ersten Elements des Arrays. Sie benötigen dann nicht den Adressoperator. Alle weiteren Elemente liegen dann im Speicher dahinter. D.h. wenn der Zeiger mit dem Post-Increment-operator um 1 erhöht wird, zeigt er jeweils auf das nächste Element. Die Adresse können Sie auch ganz normal mit dem Dereferenzoperator dereferenzieren. Umgekehrt kann auch der Indexoperator `[ ]` auf Zeiger angewendet werden. Ein Index von 0 ändert den Wert, auf den der Zeiger verweist, 1 den dahinterliegenden Wert usw. usf.

```
int array[100];
int *zeiger = array;
// die ersten vier Statements schreiben alle den Wert 1 in
// das erste Element des Arrays.
Array[0] = 1;
*array = 1;
zeiger[0] = 1;
*zeiger++ = 1;
*zeiger++ = 2;
*zeiger++ = 3;
*zeiger++ = 5;
```

In älteren Standards war es erforderlich, dass die Anzahl der Elemente eines Arrays schon zur Übersetzung bekannt ist. Neuere Standards erlauben aber das Auslesen der Größe aus einer Variablen.

```
int myVariable = getNewSize();
int array[myVariable];
```

Prüfen Sie vor der Verwendung dieses Features, ob die angepeilten Compiler dies auch können. Im Zweifelsfall benutzen Sie einfach die herkömmliche Speicherverwaltung, wie sie weiter unten beschrieben wird.

5.6. Aufzählungstypen

Manchmal benötigen Sie einen Typ, der mehrere vordefinierte Werte aufnehmen

kann, um so beispielsweise alle Wochentage bearbeiten zu können. Hierfür gibt es das Schlüsselwort `enum`. Ein Beispiel:

```
enum wochenTage {  
    SONNTAG, MONTAG, DIENSTAG,  
    MITTWOCH, DONNERSTAG, FREITAG,  
    SAMSTAG  
};  
  
enum wochenTage derTag = MONTAG;
```

Aufzählungstypen werden intern wie Variablen vom Typ `int` gespeichert. Der erste Wert wird mit einer 0 gespeichert, der zweite mit einer 1 usw. usf. Wenn Sie einen anderen Wert vorgeben wollen, können Sie diesen jedoch leicht ändern:

```
enum wochenTage {  
    SONNTAG=1, MONTAG, DIENSTAG,  
    MITTWOCH, DONNERSTAG, FREITAG,  
    SAMSTAG  
};  
  
enum wochenTage derTag = MONTAG;
```

Die Zuweisung können Sie auch bei jedem anderen - also nicht nur beim ersten - machen. Alle Nachfolgenden Werte erhalten dann einen jeweils um 1 erhöhten Wert, Sie müssen aber darauf achten, dass kein Wert doppelt vergeben wird:

```
enum wochenTage {  
    SONNTAG, MONTAG, DIENSTAG,  
    MITTWOCH=20, DONNERSTAG, FREITAG,  
    SAMSTAG  
};  
  
enum wochenTage derTag = MONTAG;
```

Bei der Definition einer Variablen mit einem Aufzählungstyp können Sie in C++ das Schlüsselwort `enum` auch weglassen.

```
enum wochenTage {  
    SONNTAG, MONTAG, DIENSTAG,  
    MITTWOCH, DONNERSTAG, FREITAG,  
    SAMSTAG  
};  
  
wochenTage derTag = MONTAG;
```

5.7. Konstante - Literale

Nahezu alle Programme verwenden auch konstante Werte. In den vorangegangenen Kapiteln wurden schon einige Beispiele gezeigt. C/C++ kennt jedoch verschiedene Regeln, die beachtet werden müssen.

Ganze Zahlen ohne nähere Kennzeichnung haben immer den Typ `signed int` und die Basis 10 (es sind alle 10 Ziffern von 0 bis 9 erlaubt). Sie dürfen nicht mit einer 0 anfangen. Sie müssen selbst darauf achten, dass sie nicht zu groß wird für den Typ.

```
int variable = 123;
```

Ganze Zahlen, die mit einer 0 anfangen, haben die Basis 8 (es sind 8 Ziffern von 0 bis 7 erlaubt). Jede Ziffer entspricht dann 3 Bits der Variable.

```
int variable = 0123;
```

Ganze Zahlen, die mit 0x anfangen, haben die Basis 16 (es sind 16 Ziffern von 0 bis 9 und A-F erlaubt). Jede Ziffer entspricht dann 4 Bits der Variable. Bei den Buchstaben ist Groß-/Kleinschreibung irrelevant.

```
int variable = 0x123;  
int variable2 = 0XABC;  
int variable3 = 0xABC;  
int variable4 = 0xabc;
```

Negativen Zahlen muss das Minuszeichen vorangestellt werden, positiven Zahlen kann das Pluszeichen vorangestellt werden.

```
int variable = 123;  
int variable2 = +123;  
int variable3 = -123;
```

Wenn die Zahl für einen vorzeichenlosen Typ verwendet werden soll, muss die Konstante mit einem U enden.

```
unsigned variable = 123U;
```

Wenn die Zahl für den Typ long int verwendet werden soll, muss die Konstante mit einem L enden.

```
long variable = 123L;
```

Die Kennzeichnungen U und L können auch gemischt werden. Die Reihenfolge ist egal.

```
unsigned long variable = 123UL;
```

In den meisten Fällen findet die Konvertierung automatisch statt. D.h. das U und/oder das L kann auch weggelassen werden. In C++ kann es aber sehr häufig vorkommen, dass die Konvertierung nicht automatisch stattfinden kann.

Fließkommazahlen werden einfach so hingeschrieben, wie es in der Mathematik üblich ist. Als Dezimaltrenner wird der Punkt . verwendet.

```
double variable = 123.456;
```

Zehnerpotenzen der Fließkommazahlen werden mit einem e gefolgt vom Exponenten angegeben 1,2345 . * 10¹⁹ ist

```
double variable = 1.2345e19;
```

Einzelne Zeichen wie Buchstaben, Satzzeichen und ähnliches werden in einfachen Anführungszeichen eingeschlossen. Intern werden Sie jedoch automatisch in Ganzzahlen umgewandelt. Wie diese Konvertierung stattfindet hängt jedoch von der Plattform ab, mit der das Programm erstellt wird. Die meisten System kennen

zumindest den ASCII-Zeichensatz, andere können jedoch auch mit anderen Zeichensätzen umgehen. Für gewöhnlich findet der ANSI-Zeichensatz Verwendung. In aller Regel muss sich jedoch der Entwickler nicht darum kümmern, weshalb hier nicht näher darauf eingegangen wird.

Üblicherweise benutzt man für die Speicherung eines einzelnen Zeichens Variablen vom Typ `char`.

```
char variable = 'A';
```

Alle Standardfunktionen zum Verarbeiten von Zeichenketten verwenden Arrays von Typ `char` deren letztes Element 0 ist, da Arrays keine Information über deren Länge bereitstellen. Ein solches Array kann mit einem Stringliteral initialisiert werden. Ein Stringliteral ist eine Zeichenkette, die in doppelten Anführungszeichen eingeschlossen ist.

```
char zeichenKette[] = "Mein Name ist Hase";
```

Wenn Sie besondere Steuerzeichen in Ihrer Zeichenkette oder einzelner Zeichen verwenden wollen, können Sie benannte Zeichen einfügen, die mit einem Backslash `\` eingeleitet werden.

Folgende benannte Zeichen sind möglich:

<code>\b</code>	Backspace
<code>\t</code>	Horizontaler Tabulator
<code>\n</code>	Neue Zeile
<code>\v</code>	Vertikaler Tabulator
<code>\f</code>	Seitenvorschub
<code>\r</code>	Carriage Return (An den Anfang der Zeile)
<code>\\</code>	Der Backslash selber
<code>'</code>	Das einfache Anführungszeichen
<code>"</code>	Das doppelte Anführungszeichen
<code>\<octalZahl></code>	Das Zeichen mit der jeweiligen Kodierung als Oktalzahl (Beispiel <code>\010</code> für den Backspace)
<code>\x<hexZahl></code>	Das Zeichen mit der jeweiligen Kodierung als Hexzahl (Beispiel <code>\xA</code> für den Zeilenvorschub)

```
char newLine = '\n';
char *test = "C:\\WINDOWS\\win.ini";
char fehlermeldung[] = "Wollen Sie wirklich alles löschen?\nJa\tNein";
```

Mehrere Stringlitterale hintereinander, die nur durch Leerzeichen getrennt sind, werden wie eines behandelt.

```
puts(
    "Dateiformat von test.txt fehlerhaft.\n"
    "Datei wird ignoriert."
);
```

Wenn Sie einen konstanten Wert öfters in Ihrem Programm nutzen wollen, sollten Sie

mit dem Präprozessor eine benannte Konstante definieren. Dann müssen Sie, wenn der Wert später einmal sich ändern sollte, dies nur an einer Stelle berücksichtigen. Außerdem wird dadurch der Quelltext häufig leichter lesbar. Ein Makro definieren Sie einfach mit der Präprozessoranweisung `#define`.

```
#define NUM_FIELDS      1000
int array[NUM_FIELDS];
int i;
for( i=0; i<NUM_FIELDS; i++ ) {
    ...
}
```

5.8. Strukturen

Einfache Datentypen sind nicht immer geeignet die gewünschten Daten zu speichern. Daher ist es in C/C++ möglich, mehrere Daten zu einer Struktur zusammen zufassen. Hierfür wird mit den Schlüsselwort `struct` eine Strukturdefinition eingeleitet. Anschließend folgt optional ein Bezeichner für die Struktur und dann die eigentliche Definition. Die Definition ist eine in geschweiften Klammern eingerahmte Liste von Strukturelementen.

```
struct StrukturName {
    typ  elementName;
    ...
};
```

Als `typ` darf jeder beliebige Datentyp verwendet werden. Auch andere Strukturen:

```
struct NochNeStruktur {
    struct StrukturName  elementName;
    ...
};
```

Strukturdefinitionen dürfen auch geschachtelt werden. Dann kann der Name der Struktur weggelassen werden:

```
struct wiederNeStruktur {
    struct {
        typ  unterElement;
        ...
    } elementName;
    ...
};
```

Variablen dieses Typs werden ebenfalls mit dem Schlüsselwort `struct` deklariert. Hierzu benötigt man dann den Namen der Struktur: Auf ein einzelnes Element einer Struktur, kann dann mit dem Strukturoperator `.` zugegriffen werden.

```
struct StrukturName Variable;
...
Variable.elementName = 123;
```

Strukturvariablen dürfen auch schon bei der Definition mit Werten befüllt werden. Die Liste der Werte wird mit den geschweiften Klammern eingerahmt. Achten Sie darauf, dass die Typen der Werte auch zur Strukturdefinition passen.

```
struct StrukturName {
    int      element1,
           element2;
    double   element3;
} variable =
{
    1, 5, 5.0987
};
```

In C++ darf das Schlüsselwort struct auch weggelassen werden.

```
StrukturName Variable;
...
Variable.elementName = 123;
```

Wenn nur eine Variable einer Struktur definiert werden soll, muss die Struktur keinen Namen haben. Dann kann die Strukturdefinition und die Variablendefinition in einem Schritt erfolgen.

```
struct {
    typ elementName;
    ...
} variablenName;
```

Die Elemente einer Struktur werden im Speicher genau in der Reihenfolge abgelegt, in der Sie auch deklariert wurden. Manchmal ist es jedoch sinnvoll, dass alle Strukturelemente den gleichen Speicherplatz verwenden. Dann muss die Programmlogik entscheiden, welches Element gerade verwendet werden kann. Hierzu wird statt dem Schlüsselwort struct das Schlüsselwort union verwendet.

```
union unionName {
    typ element1;
    typ element2;
    ...
};
```

Bei Elementen, die ganze Zahlen speichern, können Sie auch deren Größe in Bits angeben um so beispielsweise einzelne Bits einer Ganzzahlvariable einfach ansprechen zu können. Sie dürfen dabei aber nicht mehr Bits angeben, als der entsprechende Typ eh schon hat. Auch können Sie von solchen Elementen nicht die Adresse ermitteln. Die Größe geben Sie durch Doppelpunkt vom Elementnamen getrennt an.

```
struct bitfield {
    int day:5;
    int month:4;
    int year:23;
};
```

In C++ können Strukturen statt mit dem Schlüsselwort struct mit dem Schlüsselwort class definiert werden. Für C++ gelten aber auch eine ganze Reihe weiterer Regeln, für die ein eigenes Kapitel vorgesehen ist.

Eine Struktur können Sie vorab deklarieren ohne sie zu definieren. Das ist praktisch, wenn Sie zum Beispiel einen Zeiger auf eine Struktur benötigen, die

Strukturdefinition aber noch nicht bekannt ist. Sie können alles mit dem Zeiger machen, wie z.B. an eine andere Funktion übergeben oder von einer anderen Funktion als Funktionsergebnis erhalten, wofür die Definition der Struktur noch nicht notwendig ist. Sie dürfen aber nichts machen, wofür die Strukturdefinition und -größe erforderlich sind.

5.9. Funktionen

Funktionen sind Teilprogramme Ihrer Anwendung, die Sie von jeder anderen Funktion aus beliebig häufig aufrufen können. Funktionen bestehen aus einem Funktionskopf (signature) und dem Funktionsrumpf (body). Im Funktionskopf wird festgelegt, welchen Typ der Rückgabewert hat, welchen Namen die Funktion besitzt und welche Parameter übergeben werden müssen. Im Funktionsrumpf befinden sich die Anweisungen, die durchgeführt werden müssen, wenn die Funktion aufgerufen wird. Der Funktionsrumpf wird in geschweiften Klammern eingerahmt. Die allgemeine Form einer einfachen Funktionsdefinition sieht so aus:

```
<Rückgabety> <Funktionsname>([Parameterliste]) {  
    <Anweisungen> ...  
}
```

Der Rückgabety darf in C auch weggelassen werden. Dann wird automatisch int angenommen. Dies ist aber nicht empfehlenswert, da es die Lesbarkeit behindert und in C++ verboten ist.

Wenn die Funktion keine Parameter erwartet, schreiben Sie statt der Parameterliste das Schlüsselwort void. In C++ können Sie void auch weglassen.¹ Mit dem Schlüsselwort return wird die Funktion beendet und der Wert hinter return als Rückgabewert (Funktionsergebnis) an den Aufrufer zurückgeliefert.

```
int funktion( void ) {  
    int var = 5;  
    printf( "%d", var );  
    return 0;  
}
```

return darf sich an jeder beliebigen Stelle innerhalb der Funktion befinden, die Funktion wird aber auf jeden Fall dann beendet, wenn die letzte Anweisung ausgeführt wurde. Beachten Sie aber, dass dann der Rückgabewert undefiniert ist. In C++ ist dieser Fall daher auch verboten und wird mit einer Fehlermeldung bei der Übersetzung quittiert.

¹ Viele C++ Programmierer runzeln die Stirn, wenn Sie das void in diesem Falle stehen lassen. Eine vernünftige Begründung habe ich dafür noch nicht gehört, außer dass man ein wenig Tipparbeit spart. Ich persönlich ziehe es vor, void stehen zu lassen, um leichter Funktionen nach C zu übertragen. Damit Sie aber in C++ Foren nicht schief angeschaut werden, versuche ich es hier in den C++-Beispielen wegzulassen.

```
int funktion( void ) {  
    int var = 5;  
    printf( "%d", var );  
    if( var == 5 )  
        return 3;  
    return 0;  
}
```

Wenn eine Funktion keinen Wert zurückliefert, verwenden Sie als Rückgabebetyp das Schlüsselwort `void`. In diesem Fall darf hinter dem Schlüsselwort `return` kein Rückgabewert stehen und der Funktionsaufruf darf auch nicht auf der rechten Seite einer Zuweisung stehen.

```
void funktion( void ) {  
    int var = 5;  
    printf( "%d", var );  
    return;  
}
```

Die einzelnen Parameter werden durch Kommata getrennt. Eine Parameterdefinition besteht aus dem Typ und dem Namen des Parameters. Sie können jeden beliebigen Typ übergeben. Wenn die Parameter innerhalb der Funktion verändert werden, werden die entsprechenden Variablen des Aufrufers nicht geändert.

```
void funktion( int param1, double param2 ) {  
    param2 = param2 * param1;  
    param1 = param1 * 2;  
    printf( "%d %f\n", param1, param2 );  
}
```

Bei einigen Funktionen ist die Liste der Parameter nicht vollständig spezifiziert. Diese Funktionen müssen aber mindestens einen spezifizierten Parameter erwarten. Statt den variablen Parametern werden einfach nur 3 Punkte angegeben.

```
void funktion( int ersterParameter, ... );
```

In einem solchen Fall kann der Compiler natürlich nicht überprüfen, ob die weiteren Parameter korrekt sind. Es obliegt Ihrer Verantwortung dieses zu kontrollieren. Wie sie eine solche Funktion selbst definieren können, behandeln wir in einem späteren Kapitel.

Wenn eine Funktion auch eine Variable des Aufrufers ändern soll und der Rückgabewert schon für etwas anderes benutzt wird, müssen Sie statt dem Inhalt der Variable die Adresse der Variable mit dem Adressoperator an die Funktion übergeben. Der entsprechende Parameter ist dann ein Zeiger.

```
int funktion( int *param1, double *param2 ) {
    *param2 = *param2 * *param1;
    *param1 = *param1 * 2;
    printf( "%d %f\n", *param1, *param2 );
    return 0;
}
...
int variable1;
double variable2
funktion( &variable1, &variable2 );
```

In C++ können Sie statt einem Zeiger auch eine Referenz übergeben.

```
int funktion( int &param1, double &param2 ) {
    param2 = param2 * param1;
    param1 = param1 * 2;
    printf( "%d %f\n", param1, param2 );
    return 0;
}
...
int variable1;
double variable2
funktion( variable1, variable2 );
```

Ich empfehle diese Schreibweise aber nicht, weil man ausschließlich am Funktionskopf erkennt, ob es sich um eine Referenz auf eine „fremde“ Variable oder um eine eigene lokale Variable handelt.

Arrays können nicht übergeben werden. Stattdessen müssen Sie die Adresse des Arrays übergeben. Den entsprechenden Parameter dürfen Sie aber trotzdem wie ein Array definieren - Sie können dabei aber die Anzahl der Elemente weglassen. Der Compiler wandelt ihn dann automatisch in einen Zeiger um.

```
void funktion( int param1[], int *param2 ) {
    ...
}
...
int array1[10], array2[10]
funktion( array1, array2 );
```

Beachten Sie aber bitte: Die Funktion erhält keine Information über die Größe des Array. Sie müssen selbst sicherstellen, dass nicht über die Grenzen des Arrays hinaus gelesen oder geschrieben wird.

Strukturen dürfen ebenfalls als Parameter übergeben werden. Wenn die Struktur sehr groß ist, muss aber unter Umständen eine große Datenmenge kopiert werden, was einiges an Zeit kosten kann. Daher werden Strukturen gerne als Zeiger übergeben und diese mit dem Schlüsselwort `const` versehen, um anzuzeigen, dass die Struktur nicht verändert wird.

```
void funktion( const struct Struktur *zeiger, const char *zeichenkette ) {
    printf( "%d", zeiger->member );
}
...
struct Struktur meineVariable;
char meinText[] = "Hello";

funktion( &meineVariable, meinText );
```

In C++ kann hier auch eine Referenz gewählt werden, was in diesem Fall auch angenehmer ist, weil die Struktur nicht explizit dereferenziert werden muss. Die fehlende Information, dass eine Referenz übergeben wird, spielt hier nur eine untergeordnete Rolle.

```
void funktion( const Struktur &referenz, const char *zeichenkette ) {
    printf( "%d", referenz.member );
}
...
Struktur meineVariable;
char meinText[] = "Hello";

funktion( meineVariable, meinText );
```

Grundsätzlich gilt: der Funktionsname muss eindeutig sein, d.h. es darf keine zwei Funktionen mit dem gleichen Namen geben. Es gibt aber einige Ausnahmen:

Eine Funktion, die mit dem Schlüsselwort `static` definiert wird, ist nur innerhalb des Moduls, in dem sie definiert wurde, sichtbar. D.h. in einer anderen Quelltextdatei des Projektes darf es eine Funktion mit dem gleichen Namen geben, sofern diese ebenfalls `static` deklariert wurde.

```
static void funktion( const struct Struktur *zeiger, const char
*zeichenkette )
{
    ...
}
```

Ein Funktionskopf ohne Rumpf ist ein Prototyp. Er deklariert eine Funktion ohne Definition und muss mit einem Strichpunkt abschließen. Sie müssen darauf achten, dass der Prototyp mit der Definition übereinstimmt. Ein Prototyp wird üblicherweise in einer Headerdatei geschrieben, die mit dem Präprozessor eingebunden wird.

```
void funktion( void );          // Prototyp
...
void funktion( void ) {        // Definition
    ...
}
```

In C ist es erlaubt, eine Funktion auch dann aufzurufen, wenn weder der Prototyp noch die Definition bekannt ist. Sie sollten dies aber vermeiden, da der Compiler nicht überprüfen kann, ob die Parameter korrekt übergeben werden und ob der Rückgabotyp übereinstimmt.

In C++ muss wenigstens ein Prototyp bekannt sein. Andernfalls erhalten Sie beim Aufruf einen Fehler.

Sie können eine Funktion aufrufen, in dem der Name der Funktion und, falls erforderlich, die Parameter verwendet wird.

```
void funktion( void ) {
    ...
}

int funktion2( int parameter ) {
    return parameter2 * 2;
}

void nochNeFunktion( void ) {
    int variable = 1;
    funktion();                // Aufruf ohne Parameter
    variable = funktion2( variable ); // Aufruf mit Parameter und
                                    // Rückgabetyt
    printf( "%d\n", variable );
    variable = funktion2( 1235 );
    printf( "%d\n", variable );
}
```

Der Aufruf einer Funktion ist ein RValue (darf also nur auf der rechten Seite einer Zuweisung stehen). Außer die Funktion liefert in C++ eine Referenz zurück.

Seit C99 und C++11 wird in jeder Funktion ein statisches Array `__func__` mit einer Zeichenkette definiert, die den unqualifizierten Namen der Funktion enthält. Das kann für Diagnosemeldungen während der Testphase sehr nützlich sein.

5.10. C++: Funktionsüberladung

In C++ dürfen Funktionen, die unterschiedliche Parameter erwarten, im gleichen Namensraum den gleichen Namen haben.

```
void funktion() {
    ...
}

void funktion( int parameter ) {
    ...
}
```

Das ist vor allem bei der generischen Programmierung mit Hilfe von templates (s.u.) bedeutsam. Ansonsten würde ich dieses Feature nur sparsam verwenden, weil es das Umschreiben (Refactoring) des Codes erschwert.

In C++ dürfen bei Funktionen den Parametern ein Vorgabewert mitgegeben werden, der übergeben wird, wenn beim Aufruf der Parameter weggelassen wird. Sie dürfen diese Vorgabewerte nur für die letzten Parameter der Liste angeben. Mittendrin ist nicht erlaubt. Sie müssen auch darauf achten, dass es keine weitere Funktion gibt, die die gleichen Parameter außer denen mit Vorgabewerten erwartet.

```
void funktion( int param1, int param2=3, int param3 ) { // falsch: nicht
                                                         // mittendrin
    ...
}
```

```

void funktion( int param1=1 ) {
    ...
}

/*
    falsch: es gibt schon eine Funktion mit diesem Namen, die einen int
    erwartet.
*/
void funktion( int param1, int param2=3, int param3=1 ) {
    ...
}

```

5.11. Variablen

Variablen sind Speicherbereiche, mit denen Sie Ihre Daten speichern und verarbeiten können. Zum Definieren einer Variable geben Sie immer einen Typ und den Namen der Variable an. Typ kann ein einfacher Typen, Strukturen, Zeiger und Referenzen, Aufzählungstyp oder Arrays sein. Der Name der Variable muss innerhalb seines Gültigkeitsbereichs eindeutig sein.

```

int          variable1;           // integer variable
struct stat  variable2;           // Struktur
unsigned char *variable3;         // Zeiger
const char   *variable4;
int          &variable5 = variable1; // Referenz
enum tage    variable6;           // Aufzählung
struct stat  *variable7[1024];    // Array von Zeigern auf eine
                                   // Struktur

```

Der Gültigkeitsbereich einer Variable bestimmt sich durch den Ort der Definition.

Innerhalb eines Anweisungsblockes in geschweiften Klammern (z.B. der Funktionsrumpf) bewirkt, dass die Variable nur innerhalb dieses Blockes gültig ist. In diesem Fall wird die Variable beim Eintritt in den Block angelegt und beim Verlassen des Blockes wieder zerstört. Außerhalb des Blockes ist die Variable undefiniert. Bei älteren C-Compilern muss eine solche Variable am Anfang des Blocks definiert werden. In C++ und aktuellen C-Compilern darf Sie auch innerhalb eines Blocks aber vor der ersten Verwendung definiert werden. Solche Variablen werden *auto-* oder *lokale Variablen* genannt. Das Schlüsselwort *auto* zeigt ebenfalls an, dass die Variable beim Eintritt in den Block erzeugt wird und beim Verlassen wieder gelöscht wird. Da dies aber das Standardverhalten ist, wird dies in der Regel nicht verwendet. Bei C++11 wurde daher die Bedeutung dieses Schlüsselwortes angepasst. Wenn bei der Definition einer Variable diese auch gleich initialisiert wird, kann statt dem Datentyp das Schlüsselwort *auto* verwendet werden und der Compiler bestimmt den korrekten Typ an Hand des Initialwertes der Variablen.

```

void function( void ) {
    int variable;
    if( ... )
    {
        int    nochNeVariable;
        ...
    }
}

```

Wenn eine lokale Variable mit dem Schlüsselwort `static` definiert wird, so wird sie schon beim Programmstart angelegt und beim Ende wieder zerstört. Der Inhalt der Variable bleibt dann auch über mehrere Aufrufe der Funktion erhalten. Es ist in diesem Fall meistens sinnvoll, die Variable bei der Definition auch gleich zu initialisieren.

```
void funtkion( void ) {
    static int aufrufzaehler = 0;
    aufrufzaehler += 1;
    printf( "Ich wurde %d mal aufgerufen\n", aufrufzaehler );
}
```

Lokale Variable, die mit dem Schlüsselwort `register` ausgezeichnet sind. Werden nach Möglichkeit in CPU-Registern abgelegt. Der Adressoperator kann auf solche Variablen nicht angewendet werden. Dies war aber schon immer eine Kannbestimmung und moderne Compiler können mir ihren Optimierern eh schon recht guten Code erzeugen, weshalb dieses Schlüsselwort nur noch selten benutzt wird und nur bis C++17 vorgeschrieben ist.

Variablen, die außerhalb einer Funktion deklariert werden, sind in allen nachfolgenden Funktionen gültig. Diese Variablen werden auch globale Variablen genannt.

```
int globaleVariable = 5;
void funktion1( void ) {
    globaleVariable = 30;
}
void funktion2( void ) {
    printf( "%d", globaleVariable );
}
```

Globale Variablen, die mit dem Schlüsselwort `static` definiert werden, sind jedoch nur innerhalb des Quelltextmoduls sichtbar. Wenn in einem anderen Quelltextmodul eine statische Variable mit dem gleichen Namen definieren, handelt es sich um eine andere Variable.

```
// Modul1.c
static int globaleVariable = 5;
void funktion1( void ) {
    globaleVariable = 30;
}
// Modul2.c
static int globaleVariable = 55;
void funktion2( void ) {
    printf( "%d", globaleVariable );
}
```

Mit dem Schlüsselwort `extern` deklarieren Sie eine Variable ohne sie zu definieren. Sie muss dann in einem anderen Modul definiert werden. Eine extern declaration mit gleichzeitiger Initialisierung ist auch eine Defintion.

```
// Modul1.c
extern int globaleVariable = 5;
int nochNeGlobaleVariable = 6;

void funktion1( void ) {
    globaleVariable = 30;
}

// Modul2.c
extern int globaleVariable;
extern int nochNeGlobaleVariable;

void funktion2( void ) {
    printf( "%d", globaleVariable );
}
```

Ab C++11 oder C23 kann mit `thread_local` eine Variable als statisch für einen Thread definiert werden. (Siehe 180)

5.12. Bezeichner

Bezeichner (identifiers) sind die Namen der Variablen, Typen, Funktionen und Konstanten, die Sie frei vergeben dürfen. Sie müssen jedoch verschiedene Regeln beachten:

- ☞ Sie müssen mit einem Buchstaben beginnen.
- ☞ Der Compiler akzeptiert zwar auch einen Unterstrich `_` am Anfang, jedoch sind solche Bezeichner für den Compilerhersteller reserviert, wenn sie global sind.
- ☞ Sie dürfen alle lateinischen Buchstaben (a-z und A-Z), die arabischen Ziffern (0-9) und den Unterstrich (`_`) benutzen.
- ☞ Groß- / Kleinschreibung wird grundsätzlich unterschieden `Anton` und `ANTON` sind zwei unterschiedliche Bezeichner.
- ☞ Innerhalb Ihres Gültigkeitsbereichs (scope) müssen Bezeichner eindeutig sein.
- ☞ Sie dürfen nicht mit einem Schlüsselwort übereinstimmen.

In der Regel können Bezeichner beliebig lang sein, jedoch kann es sein, dass manche Compiler bei der Unterscheidung nur eine geringe Anzahl von Zeichen (z.B. 8) zur Unterscheidung heranziehen. Sie sollten daher nicht zu lange Bezeichner verwenden. Die meisten Compiler dürften aber mit bis zu 256 Zeichen klar kommen.

Vergeben Sie auch sinnvolle Namen, die den Zweck der Variablen wiedergeben. Kurze Namen wie `a`, `b` oder `c` sind weniger aussagekräftig wie `numElements`, `anzahlKunden` etc.

Der Gültigkeitsbereich, in dem der Bezeichner erkannt wird, hängt davon ab, an welche Stelle in Ihrem Quelltext, der Bezeichner definiert wurde. Wird ein Bezeichner innerhalb eines Anweisungsblockes definiert (z.B. eine Variable) ist er nur dort gültig. Bezeichner, die außerhalb einer Funktion definiert werden, sind gültig ab der Definition bis zum Ende der gerade übersetzten C/C++ Quelltextdatei (compilation

unit).

Bezeichner, die innerhalb einer Strukturdefinition definiert wurden, können überall dort verwendet werden, wo auch die Strukturdefinition bekannt ist. Sie müssen dann aber auch im Zusammenhang mit dieser Struktur benutzt werden, weshalb ein Strukturelement durchaus den gleichen Namen wie eine andere Variable haben darf:

```
struct meineStruktur {
    int meinElement;
};
...
struct meineStruktur meineVariable;
...
int meinElement = meineVariable.meinElement;
```

Beim Definieren eines Bezeichners sollten Sie grundsätzlich einen möglichst kleinen Gültigkeitsbereich wählen. Hierfür gibt es mehrere Gründe:

Nachträgliche Änderungen an einer Variablen, Typ oder Funktion können Auswirkungen auf dem gesamten Gültigkeitsbereich haben. Je kleiner der Gültigkeitsbereich ist, desto geringer ist der Anpassungsaufwand.

Da Bezeichner innerhalb Ihres Gültigkeitsbereichs eindeutig sein müssen. Ist die Wahrscheinlichkeit größer, dass es zu Mehrdeutigkeiten kommt, die aufgelöst werden müssen, je größer der Gültigkeitsbereich ist.

Für Variablen gilt noch, dass Änderungen am Inhalt der Variablen schwieriger zu verfolgen und zu kontrollieren ist, je größer der Gültigkeitsbereich ist. Insbesondere wenn mehrere Threads einer Anwendung gleichzeitig eine Variablen benutzen, kann es unerwünschte Ergebnisse bringen, falls Sie keine entsprechenden Vorkehrungen treffen. Wie Sie mit mehreren Threads gleichzeitig auf eine globale Variable zugreifen können, betrachten wir in einem der Übungskapitel.

5.13. C++ Namensräume

Eine häufiges Problem bei der Namensfindung für globale Bezeichner ist, dass Sie nicht immer garantieren können, dass der Name auch wirklich eindeutig ist. Vor allen Dingen, wenn nachträglich eine Bibliothek eines anderen Herstellers eingebunden werden soll, kann es leicht zu Kollisionen kommen. Um dieses Problem zu beheben, wurden die Namensräume (namespace) in C++ eingeführt.

Einen Namensraum definieren Sie in C++ mit dem Schlüsselwort namespace gefolgt von einem Bezeichner. Anschließend folgt innerhalb von geschweiften Klammer die Definition Ihrer Typen, Variablen und Funktionen.

```
namespace meinNamensRaum {
    struct meineStruktur {
        int meinElement;
    };
}
```

```
namespace nochEinNamensRaum {
    struct meineStruktur {
        char *meinElement;
    };
}
```

In diesem Fall haben Sie zwei unterschiedliche Strukturen definiert, die zwar den selben Namen haben, aber in unterschiedlichen Namensräumen sich befinden und daher eindeutig sind.

Namensräume dürfen auch auf mehrere Dateien verteilt werden. Das ist wichtig, um 1. Deklarationen in Ihren Headerdateien vornehmen zu können, aber auch um mehrere Funktionen eines Namensraums auf unterschiedliche Quelltextdateien verteilen zu können:

```
// mein_raum.h
namespace meinNamensRaum {
    int funktion1();
    int funktion2();
};

// funktion1.cpp
#include "rmein_raum.h"

namespace meinNamensRaum {
    int funktion1() {
        ...
    }
};

// funktion2.cpp
#include "rmein_raum.h"

namespace meinNamensRaum {
    int funktion2() {
        ...
    }
};
```

Innerhalb Ihres Namensraums können Sie die Bezeichner des Namensraums ohne nähere Angaben verwenden. Außerhalb des Namensraums müssen Sie auch den Bezeichner des Namensraums gefolgt vom sogenannten scope-operator :: benutzen:

```
namespace meinNamensRaum {
    int funktion1() {
        ...
    }

    int funktion2() {
        funktion1();
    }
};

int funktion3() {
    meinNamensRaum::funktion2();
}
```

Wenn Sie häufig auf Bezeichner eines anderen Namensraums zugreifen müssen,

können Sie sich auch viel Tipparbeit sparen, indem Sie das Schlüsselwort `using` verwenden:

```
namespace meinNamensRaum {
    int funktion1() {
        ...
    }

    int funktion2() {
        funktion1();
    }
};

using namespace meinNamensRaum;

int funktion3() {
    funktion2();
}
```

Die Anweisung `using namespace <namensraum>;` bewirkt, dass der Compiler bei der Auflösung eines Bezeichners auch diesen Namensraum durchsucht. Seien Sie aber sehr vorsichtig mit dieser Anweisung, da Sie hiermit den Schutzmechanismus der Namensräume aushebeln. Als Faustregel gilt, verwenden Sie diese Anweisung niemals in einer Headerdatei.

Das Problem mit `using namespace` können Sie aber etwas mindern (wenn auch nicht beseitigen), indem Sie den Compiler mitteilen, welchen Bezeichner des Namensraums nutzen wollen. Dies geschieht ebenfalls mit dem Schlüsselwort `using`:

```
namespace meinNamensRaum {
    int funktion1() {
        ...
    }

    int funktion2() {
        funktion1();
    }
};

using meinNamensRaum::funktion2;

int funktion3() {
    funktion2();
}
```

Diese Anweisung ist weniger problematisch sollte aber auch in Headerdateien vermieden werden. Bei der Arbeit mit C++ Klassen, werden wir jedoch einen Anwendungsfall von `using` kennenlernen, der häufiger und auch in Headerdateien verwendet wird.

Alle Bezeichner die außerhalb eines Namensraums definiert wurden, werden dem „globalen Namensraum“ zugeordnet. Dieser hat keinen Namen, wenn Sie darauf zugreifen wollen jedoch im aktuellen Namensraum bereits den selben Bezeichner definiert haben, verwenden Sie den scope operator ohne Namensraum:

```

int funktion1() {
    ...
}

namespace meinNamensRaum {
    int funktion1() {
        ...
    }

    int funktion2() {
        funktion1();
        ::funktion1();
    }
};

```

In C++ befinden sich alle Standardbezeichner (Funktionen, Typen etc.), wenn sie nicht vom Preprozessor definiert wurden (dieser kennt keine Namensräume) im namespace std. Es ist Ihnen nicht erlaubt, eigene Bezeichner diesem namespace hinzuzufügen.

5.14. Operatoren

Operatoren verbinden ein bis drei Teilausdrücke - den Operanden - zu einem neuen Ausdruck. Das Ergebnis der Operation ist der Wert des neuen Ausdrucks. Die meisten Operatoren erwarten Operanden mit einem einfachen Typ. Es gibt aber auch Operatoren, die andere Arten von Operanden erwarten.

C und C++ kennen eine Vielzahl an Operatoren. Diese sind:

<i>Bezeichnung</i>	<i>Operator</i>	<i>Beschreibung</i>
Unäre Operatoren		
Unäre Operatoren haben nur einen Operanden		
Logisches NOT	!Ausdruck	Wenn der Operand ungleich true ist, ist der Wert der Operation false, wenn der Operand false ist, ist der Wert der Operation true. ¹
Binäres NOT	~Ausdruck	Jedes Bit des Operanden wird umgedreht, d.h. aus 0 wird 1 und aus 1 wird 0. Der Operand selber wird nicht verändert.
Post-Increment	LValue++	Der Wert des Operanden wird um 1 erhöht. Der alte Wert des Operanden ist der Wert der Operation.
Pre-Increment	++LValue	Der Wert des Operanden wird um 1 erhöht. Der neue Wert des Operanden ist der Wert der Operation.
Post-Decrement	LValue--	Der Wert des Operanden wird um 1 verringert. Der alte Wert des Operanden ist der Wert der Operation.

¹ In älteren C-Compilern gilt true ist ein Wert ungleich 0 und false = 0

Bezeichnung	Operator	Beschreibung
Pre-Decrement	--LValue	Der Wert des Operanden wird um 1 verringert. Der neue Wert des Operanden ist der Wert der Operation.
Adressoperator	&LValue	Der Wert der Operation ist die Speicheradresse des Operanden.
Dereferenzoperator	*Adresse	Der Wert der Operation ist der Inhalt der Speicheradresse im Operanden.
Typumwandlung	(Type)Ausdruck	Der Wert des Ausdrucks wird, wenn möglich, in den angegebenen Typ umgewandelt. Falls der neue Typ zu klein ist, werden überzählige Bits einfach kommentarlos abgeschnitten.
	const_cast<Typ>(Ausdruck)	Mit einem const_cast können Sie einem beliebigen Zeiger- oder Referenztyp den const modifier hinzufügen oder entfernen.
	dynamic_cast<Typ>(Ausdruck)	Mit einem dynamic_cast können Sie einen Zeiger- oder Referenztyp zu dem gleichen Typ einer abgeleiteten Klasse umwandeln. Allerdings ist das Ergebnis NULL bzw. nullptr nicht ein entsprechendes Objekt ist. Der cast funktioniert nur, wenn beim Objekt entsprechende Typinformationen gespeichert sind.
	reinterpret_cast<Typ>(Ausdruck)	Dieser cast bewirkt eine entsprechende Uminterpretation des Bitmusters in dem der Wert des Ausdrucks gespeichert ist. Es wird keinerlei Plausibilität weder zur Übersetzungs- noch zur Laufzeit geprüft.
	static_cast<type>(Ausdruck)	Dieser cast bewirkt eine entsprechende Uminterpretation des Bitmusters in dem der Wert des Ausdrucks gespeichert ist. Es wird keinerlei Plausibilität zur Laufzeit geprüft. Bei der Übersetzung wird aber geprüft, ob der Ausdruck tatsächlich ein entsprechendes Objekt sein könnte.
Größenbestimmung	sizeof(Typ/Ausdruck)	Bestimmt die Größe eines Typs oder Ausdrucks in Vielfachen eines char. Der Datentyp dieser Operators ist size_t und in <stdlib.h> deklariert. Für C++ ist der Typ std::size_t in <cstdlib> deklariert.
Typbestimmung	typeid(Typ/Ausdruck)	Liefert eine Referenz auf eine Compiler-spezifische Variable, die eindeutig für den Typ ist. Der Typ dieser Variable ist std::type_info und in <typeinfo> deklariert.

Bezeichnung	Operator	Beschreibung
Zweier Operatoren		
Zweier Operatoren haben zwei Operanden		
Addition	Ausdruck1 Ausdruck2	+ Der Wert der Operation ist die Summe aus beiden Operanden.
Subtraktion	Ausdruck1 Ausdruck2	- Der Wert der Operation ist die Differenz aus beiden Operanden.
Produkt	Ausdruck1 Ausdruck2	* Der Wert der Operation ist das Produkt aus beiden Operanden.
Division	Ausdruck1 Ausdruck2	/ Der Wert der Operation ist der Quotient aus beiden Operanden.
Modulo	Ausdruck1 Ausdruck2	% Der Wert der Operation ist der Divisionsrest aus dem Quotienten beider Operanden.
Logisches Und	Ausdruck1 Ausdruck2	&& Wenn beide Operanden ungleich 0 sind, ist der Wert der Operation 1 andernfalls 0. Beachte: Wenn schon der erste Ausdruck 0 ist, wird der zweite Operand nicht mehr ausgewertet.
Binäres Und	Ausdruck1 Ausdruck2	& Beide Operanden werden Bit für Bit UND verknüpft. UND heißt: Das Ergebnisbit ist dann 1, wenn beide Quellbits 1 sind.
Logisches Oder	Ausdruck1 Ausdruck2	Wenn mindestens ein Operand ungleich 0 ist, ist der Wert der Operation 1 andernfalls 0. Beachte: Wenn schon der erste Ausdruck 1 ist, wird der zweite Operand nicht mehr ausgewertet.
Binäres Oder	Ausdruck1 Ausdruck2	Beide Operanden werden Bit für Bit ODER verknüpft. ODER heißt: Das Ergebnisbit ist dann 1, wenn mindestens ein Quellbit 1 ist.
Exklusiv Oder	Ausdruck1 Ausdruck2	^ Beide Operanden werden Bit für Bit XOR verknüpft. XOR heißt: Das Ergebnisbit ist dann 1, wenn genau ein Quellbit 1 ist.
Links Shift	Ausdruck1 Ausdruck2	<< Alle Bits von Ausdruck 1 werden um die in Ausdruck 2 angegebene Anzahl von Stellen nach Links verschoben. Bits, die keinen Platz mehr haben, werden unterschlagen. Die frei werdenden Stellen werden mit 0 initialisiert.

Bezeichnung	Operator	Beschreibung
Rechts Shift	Ausdruck1 Ausdruck2	>> Alle Bits von Ausdruck 1 werden um die in Ausdruck 2 angegebene Anzahl von Stellen nach Rechts verschoben. Bits, die keinen Platz mehr haben, werden unterschlagen. Die frei werdenden Stellen werden mit dem Vorzeichen von Ausdruck 1 initialisiert. Bei Positiven Zahlen ist dies eine 0, bei negativen Zahlen ist die eine 1.
Zuweisung	LValue = Ausdruck	Der Wert des Ausdrucks Rechts des Operators wird in der Variablen auf der Linken Seite gespeichert. Dieser Wert ist auch gleichzeitig der Wert der Operation.
Kombinierte Zuweisung	LValue	+= Diese Operatoren sind eine Abkürzung für LValue = LValue <operator> (Ausdruck)
	Ausdruck	
	LValue -= Ausdruck	
	LValue *= Ausdruck	
	LValue /= Ausdruck	
	LValue %=	
	Ausdruck	
	LValue &=	
	Ausdruck	
	LValue = Ausdruck	
	LValue ^=	
	Ausdruck	
	LValue <<=	
	Ausdruck	
	LValue >>=	
	Ausdruck1	
Vergleiche	Ausdruck1 Ausdruck2	== Der Wert der Operation ist 1, wenn beide Operanden den gleichen Wert haben, andernfalls 0.
	Ausdruck1 Ausdruck2	<= Der Wert der Operation ist 1, wenn der Wert des ersten Operanden kleiner oder gleich dem Wert des zweiten Operanden ist, andernfalls 0.
	Ausdruck1 Ausdruck2	< Der Wert der Operation ist 1, wenn der Wert des ersten Operanden kleiner dem Wert des zweiten Operanden ist, andernfalls 0.
	Ausdruck1 Ausdruck2	> Der Wert der Operation ist 1, wenn der Wert des ersten Operanden größer dem Wert des zweiten Operanden ist, andernfalls 0.

Bezeichnung	Operator	Beschreibung
	Ausdruck1 Ausdruck2	>= Der Wert der Operation ist 1, wenn der Wert des ersten Operanden größer oder gleich dem Wert des zweiten Operanden ist, andernfalls 0.
	Ausdruck1 Ausdruck2	!= Der Wert der Operation ist 1, wenn beide Operanden nicht den gleichen Wert haben, andernfalls 0.
	Ausdruck1 Ausdruck2 (C++20)	<=> Dreivegevergleich: Der Wert der Operation ist -1, wenn der Wert des ersten Operanden kleiner dem Wert des zweiten Operanden ist, er ist 0 wenn beide Operanden gleich sind, andernfalls 1.
Elementzugriff	Ausdruck.Element	Der Wert der Operation ist der Wert des Elements "Element" aus der Struktur "Ausdruck". Ausdruck muss ein Strukturtyp sein. Und Element ein Element (Member) der Struktur.
	Ausdruck->Element	Der Wert der Operation ist der Wert des Elements "Element" aus der Struktur "Ausdruck". Ausdruck muss ein Zeiger auf einen Strukturtyp sein und Element ein Element (Member) der Struktur.
Arrayzugriff	array[index]	Der Wert der Operation ist der Wert des entsprechenden Elements im Array. array muss die Adresse des ersten Elements des Arrays ergeben.
Dreier Operator		
Ein Operator benötigt sogar drei Operatoren		
Bedingungsoperator	Ausdruck1 Ausdruck2 Ausdruck3	? Der Wert der Operation ist Ausdruck2 falls :Ausdruck1 ungleich 0 ist, andernfalls Ausdruck3.

Wenn mehrere Operatoren in einem Ausdruck verwendet werden, bestimmt sich die Zuordnung der Operanden zu den Operatoren nach dem Rang der Operatoren.

Beispiel: Im Ausdruck $5 + 10 * 3$ hat der Operator $*$ den Rang 3 während der Operator $+$ den Rang 4 hat. Daher ist die Zahl 10 ein Operand des Operators $*$ und das Ergebnis der Operation ist wiederum ein Operand des Operators $+$.

Wenn gleichrangige Operatoren verwendet werden, gibt die Richtung an, wie die Operanden den Operatoren zugeordnet werden. Beispiel: Bei $18 / 6 / 2$ ist die Richtung von links nach rechts, weshalb die Zahl 6 ein Operand des ersten Divisionsoperators ist.

Die Reihenfolge in der die Operatoren ausgewertet werden ist bis auf wenige Ausnahmen nicht festgelegt. Beispiel: Im Ausdruck $\text{funkt1}() + \text{funkt2}() * \text{funkt3}()$ darf

das Programm jede der drei Funktion in jeder beliebigen Reihenfolge aufrufen. Benötigen Sie aus irgendeinen Grund eine ganz bestimmte Reihenfolge, sollten Sie entweder Ihre Funktionsdefinition überdenken, so dass die Notwendigkeit nicht existiert, oder, falls dies nicht möglich ist, müssen Sie die Funktionsergebnisse in Variablen zwischenspeichern. Also z.B.

```
int a = funk1();
int b = funk2();
int c = funk3();
int result = a + b *c;
```

Mit Hilfe von Klammern kann die fest vorgegebene Rangfolge verändert werden.

Die einzigen Ausnahmen, wo die Reihenfolge der Operandenauswertung fest vorgeschrieben ist, sind die logischen Operatoren `||` und `&&` sowie der Kommaoperator. Bei diesen dreien muss der linke Operand zuerst ausgewertet werden. Für die logischen Operatoren `||` und `&&` gibt es dann auch noch die zusätzliche Forderung, dass, wenn das Ergebnis des ganzen Ausdrucks schon nach Auswertung des ersten Operanden feststeht, darf der zweite Operand nicht ausgewertet werden. Beispiel: `funk1() && funk2()`. Wenn `funk1` schon einen Wert `false` bzw. `0` zurückgeliefert hat, steht das Ergebnis der Operation schon fest und `funkt2` darf nicht mehr aufgerufen werden.

Rang	Operator	Richtung
	<code>[]</code> Arrayzugriff	
1	<code>.</code> -> Elementzugriff <code>++ --</code> Postfix <code>! ~</code> NOT <code>++ --</code> Prefix	links -> rechts
2	<code>&</code> Adressoperator <code>*</code> Dereferenz Operator <code>()</code> Typumwandlung	rechts -> links
3	<code>*</code> <code>/</code> <code>%</code> arithmetische Operatoren	links -> rechts
4	<code>+</code> <code>-</code> arithmetische Operatoren	
5	<code><<</code> <code>>></code> Shift operatoren	
6	<code><=></code> Dreiwegevergleich (C++20)	
7	<code><</code> <code><=</code> <code>>=</code> <code>></code> Vergleichsoperatoren	
8	<code>==</code> <code>!=</code> Vergleichsoperatoren	
9	<code>&</code> binäres UND	
10	<code>^</code> XOR	
11	<code> </code> binäres ODER	
12	<code>&&</code> logisches UND	

- 13 || logisches ODER
 14 ?: Bedingungsoperator
 15 = += -= *= /= %= |= &= ^= <<= >>= rechts -> links
 Zuweisungsoperatoren

5.15. C++: Operatorüberladung

In C++ ist es möglich, die Bedeutung der meisten Operatoren für eigene Struktur- und Aufzählungstypen zu definieren (operator overloading). Ein Operator wird grundsätzlich genauso definiert wie eine normale Funktion nur verwenden Sie statt dem Namen das Schlüsselwort **operator** gefolgt vom Operator. Die Funktion muss genauso viele Argumente erwarten wie der entsprechende Operator Operanden hat.

Beispiele:

```
bool operator ! ( const MyStruct &ref ) {
    return ref.myFlag;
}

Ratio operator * ( const Ratio &oper1,  const Ratio &oper2 ) {
    Ratio result;
    result.count = oper1.count * oper2.count;
    result.nominator = oper1.nominator * oper2.nominator ;

    return result;
}
```

Es ist nicht möglich, die Rangfolge der Operatoren zu ändern.

Bei den Inkrement- und Dekrement-Operatoren wird mit einem kleinen Trick gearbeitet, um zwischen Postinkrement (bzw. -dekrement) und Preinkrement (-dekrement) zu unterscheiden. Der Postinkrement erwartet noch zusätzlich einen Parameter vom Typ int, der aber nicht verwendet wird.

```
// operator.cpp: demo for operator overloading
struct MyStruct {
    int value;
    bool flag;
};

// pre increment
MyStruct operator ++ ( MyStruct &oper ) {
    ++oper.value;
    return oper;
}

// post increment
MyStruct operator ++ ( MyStruct &oper, int ) {
    MyStruct result = oper;
    oper.value++;
    return result;
}
```

```
int main() {
    MyStruct val = { 0, false };
    ++val;
    val++;
    return 0;
}
```

5.16. C++11 Benutzerdefinierte Literale

In C++ ab C++11 können Sie neue Suffixe für Ihre Literale einführen. Beachten Sie aber, bereits bestehende Suffixe wie U, L oder LL können Sie nicht ändern. Eigene Definitionen sind Funktionsdefinitionen in der folgenden Form:

```
<ret type> operator "" <Suffix> ( <inp type> val [, size_t len] )
```

<ret_type> ist ein beliebiger Typ, den die Funktion zurückliefern möchte. <Suffix> ist ein beliebiger Text, der als Suffix benutzt werden soll. Er muss aber mit einem Unterstrich beginnen, da andere Suffixe für zukünftige Erweiterungen reserviert sind. <inp type> und val ist der Typ und Wert, der vom Compiler ermittelt wurde. Len ist optional die Länge des erkannten Stringliterals.

```
MyType operator "" _mg ( long long val )
```

Definiert den Suffix _mg, der auf Integerliteralen angewendet werden kann und einen Wert vom Typ MyType erzeugt.

```
MyType variable = 1234_mg;
```

Ruft die definierte Funktion auf und initialisiert die Variable mit dem zurückgegebenen Wert.

5.17. Kontrollstrukturen

Kontrollstrukturen sind eine besondere Art von Anweisungen und dienen dazu den linearen Programmablauf von oben nach unten zu beeinflussen. Es gibt verschiedene Schlüsselworte, die in Abhängigkeit zu einer Bedingung den Programmlauf verändern. Wie schon im Abschnitt zu den Anweisungen weiter oben beschrieben, kontrollieren die Befehle genau eine Anweisung. Wollen Sie mehrere Anweisungen steuern, müssen Sie diese mit einem Anweisungsblock compound statement zusammenfassen.

```
{ anweisung1; Anweisung2; }
```

a. if - else

Mit dem Schlüsselwort if können Sie in Abhängigkeit von einem Ausdruck eine Anweisung ausführen lassen oder nicht. Folgt der ersten Anweisung unmittelbar das Schlüsselwort else, so wird die Anweisung hinter dem else ausgeführt, falls die Bedingung nicht erfüllt ist.

Die allgemeine Form der if Anweisung lautet:

```
if( ausdruck )
    Anweisung1;
[else
```

Anweisung2;]

Wenn `ausdruck` einen Wert ungleich 0 hat, wird `Anweisung1` ausgeführt, andernfalls `Anweisung2`.

```
if( i>0 )
    i -= 20;

if( i )
    i += 30;
else
    i = 5;

if( numCustomers < 30 ) {
    printf("Sie müssen noch %d mehr Kunden anwerben\n", 30-numCustomers);
    createAdvertising();
}
```

b. for

Mit dem Schlüsselwort `for`, können Sie eine Schleife programmieren, die so lange bzw. so oft ausgeführt wird, wie eine Bedingung erfüllt ist.

Die allgemeine Form der `for`-Schleife lautet:

```
for( Anweisung1; Ausdruck; Anweisung3 )
    Anweisung2;
```

Zuerst wird `Anweisung1` ausgeführt. Hiermit kann z.B. eine Laufvariable initialisiert werden. In C++ und aktuellen C-Compilern kann hier die Laufvariable auch gleich definiert werden. Sie ist dann nur innerhalb der `for`-Schleife gültig, Dann wird der Ausdruck ausgewertet, wenn er 0 ist, wird die Schleife sofort abgebrochen.

Anschließend werden `Anweisung2` und `Anweisung3` in dieser Reihenfolge ausgeführt.

Nun wird zur Auswertung des Ausdrucks gesprungen.

Hinweis: `Anweisung1` und `Anweisung3` können nicht mit der geschweiften Klammer aus mehreren Anweisungen zusammen gesetzt werden. Statt dessen müssen mehrere Anweisungen mit Kommata getrennt werden.

```
for( j=0, i=0; i<1000; j+=3, ++i )
    meinArray[i] = j*10+5;

for( int i=0; i<80; ++i ) {
    meinArray[i] = getCustomerID( i );
    printf( "Neuer Kunde %s\n", meinArray[i].name );
}
```

Für die `Anweisung3` wird häufig der sogenannte Inkrementoperator verwendet. Der Inkrementoperator kann sowohl als Präfix- (der Operator steht **vor** der Variable) als auch als Postfixoperator (der Operator steht **nach** der Variable) verwendet werden. In den hier gezeigten Beispielen ist es unerheblich, ob Sie den Präfix- oder den Postfixoperator wählen. In C++ gibt es aber Situationen, wo der Präfixoperator günstiger ist. Wir werden später noch einmal darauf eingehen.

c. while

Mit dem Schlüsselwort `while`, können Sie auch eine Schleife programmieren, die so lange bzw. so oft ausgeführt wird, wie eine Bedingung erfüllt ist.

Die allgemeine Form der `while`-Schleife lautet:

`while( Ausdruck )`

Anweisung;

Die Anweisung wird so lange bzw. oft ausgeführt, bis der Ausdruck 0 wird. Wenn der Ausdruck schon vor Eintritt in die Schleife 0 ist, wird sie sofort verlassen ohne die Anweisung auszuführen.

```
int i = 0;
int j=1000
while( i<1000 )
    meinArray[i++] = --j*10+5;

i = 0;
while( i<80 ) {
    meinArray[i] = getCustomerID( i );
    printf( "Neuer Kunde %s\n", meinArray[i].name );
    i++;
}
```

d. do - while

Mit dem Schlüsselworten `do` und `while`, können Sie auch eine Schleife programmieren, die so lange bzw. so oft ausgeführt wird, wie eine Bedingung erfüllt ist.

Die allgemeine Form der `do-while`-Schleife lautet:

`do`

Anweisung;

`while( Ausdruck )`

Die Anweisung wird so lange bzw. oft ausgeführt, bis der Ausdruck 0 wird. Im Gegensatz zur `while`-Schleife findet die Prüfung am Ende der Schleife statt, daher wird die Anweisung auch mindestens einmal ausgeführt.

```
int i = 0;
int j=1000
do
    meinArray[i++] = --j*10+5;
while( i<1000 );

i = 0;
do {
    meinArray[i] = getCustomerID( i );
    printf( "Neuer Kunde %s\n", meinArray[i].name );
    i++;
} while( i<80 )
```

e. switch - case - default

Mit den Schlüsselworten `switch`, `case` und `default`. Können Sie eine Sprungtabelle definieren.

Die allgemeine Form lautet:

```
switch( Ausdruck ) {  
    case Konstante1 : Anweisung1;  
    case Konstante2 : Anweisung2;  
    default : Anweisung3;  
}
```

Sie können beliebig viele Case-marken setzen aber nur eine Default-marke.

Die Reihenfolge der Marken bleibt Ihnen überlassen.

Der Wert des Ausdrucks bestimmt das Sprungziel. Es werden danach alle darunter liegenden Anweisungen ausgeführt. Auch über eventuell nachfolgende Sprungziele hinaus.

Wenn der Ausdruck mit keiner Konstante übereinstimmt und auch keine Default-marke existiert, wird an das Ende der Switch-anweisung gesprungen.

Die Konstanten müssen eindeutig sein.

```
enum tag { MO, DI, MI, DO, FR, SA, SO } meinTAG;  
meinTag = DI;  
switch( meinTag ) {  
    case MO:  
    case DI:  
    case MI:  
    case DO:  
    case FR:  
        arbeiten();  
    default:  
        schlafen();  
}
```

f. break

Mit dem Schlüsselwort break wird die aktuelle Schleife bzw. Sprungtabelle ohne weitere Prüfung beendet. Hinweis: Bei mehreren geschachtelten Schleifen und Sprungtabelle wird nur die innerste verlassen.

```
enum tag { MO, DI, MI, DO, FR, SA, SO } meinTAG;  
meinTag = DI;  
switch( Ausdruck ) {  
    case MO:  
    case DI:  
    case MI:  
    case DO:  
    case FR:  
        arbeiten();  
        break;  
    default:  
        schlafen();  
}
```

```
for( int i=0; i<80; i++ ) {
    meinArray[i] = getCustomerID( i );
    if( !meinArray[i] )
        break;
    printf( "Neuer Kunde %s\n"m meinArray[i].name );
}
```

g. continue

Mit dem Schlüsselwort `continue` wird der aktuelle Schleifendurchlauf abgebrochen und wieder an den Anfang zurückgekehrt.

Bei While- und Do-while-schleifen wird direkt zur Auswertung der Bedingung gesprungen.

Bei For-schleifen, wird direkt zur dritten Anweisung in der Klammer gesprungen.

Bei mehreren geschachtelten Schleifen, wird nur die innerste unterbrochen.

```
for( int i=0; i<80; i++ ) {
    meinArray[i] = getCustomerID( i );
    if( !meinArray[i] )
        continue;
    printf( "Neuer Kunde %s\n"m meinArray[i].name );
}
```

h. goto

Mit dem Schlüsselwort `goto` kann zu einer beliebigen Stelle innerhalb der Funktion gesprungen werden.

Da Programme, die dieses Konstrukt verwenden, dazu neigen unleserlich zu werden, verbieten viele Unternehmen den Einsatz dieses Befehls. Der einzige Anwendungsfall, der gelegentlich akzeptiert und verwendet wird, ist das Verlassen mehrerer geschachtelter Schleifen im Fehlerfall.

```
void funktion( void ) {
    int i, j;

    for( i=0; i<100; ++i ) {
        for( j=0; j<1000; ++j ) {
            bool success = andereFunktion( i, j );
            if( !success )
                goto errExit;
        }
    }

errExit:
    return;
}
```

i. try – catch -throw (Nur C++)

Im Falle eines schwerwiegenden Fehlers kann eine C++-Funktion eine Ausnahme (exception) „werfen“ (throw), die dann von einer der aufrufenden Funktionen aufgefangen (catch) werden kann. Dies ist sinnvoll, wenn eine Funktion

beispielsweise über Ihren Rückgabewert, keine Information über den Fehler zurückgeben kann. Der „Fänger“ der Ausnahme muss auch nicht direkt die werfende Funktion aufgerufen haben. Dies kann auch indirekt geschehen sein. Beispiele werden wir im Kapitel Fehlerbehandlung besprechen.

5.18. C++: Schablonen

Schablonen (templates) sind neben den Exceptions und Klassen die große Erweiterung von C++ gegenüber C. Die Idee, die dahinter steckt, kennen Sie vielleicht schon von einem Officeprogramm mit den Vorlagendateien. Es geht hierbei darum Funktionen oder Strukturen zu definieren ohne jedoch alle Konstanten oder Datentypen festzulegen. Diese werden erst dann angegeben, wenn die Funktion aufgerufen wird oder ein Objekt mit dieser Strukturen erzeugt wird.

Mit einem einfachen Beispiel will ich die Idee zeigen:

```
template <typename T> T maxValue( T data[], int numData ) {  
    T max = 0;  
  
    for( int i=0; i<numData; i++ )    {  
        if( max < data[i] )  
            max = data[i];  
    }  
  
    return max;  
}
```

Diese Funktion soll den größten Wert eines Arrays ermitteln und zurückgeben. Von welcher Art das Array ist, legen wir noch nicht fest, erst beim Aufruf der Funktion erzeugt der Compiler, wenn möglich, eine korrekte C++-Funktion, die den gewünschten Algorithmus enthält. Wenn wir diese Vorlage mehrmals mit unterschiedlichen Arraytypen aufrufen, erzeugt der Compiler jeweils eine eigene Version von maxValue. Wenn er das nicht kann, weil es z.B. die Operationen für den tatsächlichen Datentyp nicht gibt, wird eine Fehlermeldung ausgegeben und die Übersetzung des Programms verweigert.¹

Die Implementierung von Templates erfordert ein großes Maß an Sorgfalt, da Fehler nur sehr schwer zu finden sind. Beispielsweise versagt dieses Template, wenn ein Array mit ausschließlich negativen Werten übergeben wurde. Es dient daher nur zur Demonstration der Syntax.

In einem späteren Kapitel werden wir uns Templates genauer betrachten.

6. Das erste Programm

Bis jetzt haben wir nur graue Theorie behandelt und ich denke und hoffe, dass Sie schon lange darauf brennen, Ihr erstes C/C++ Programm übersetzen zu können. Traditionell beginnen die meisten Lehrbücher mit „Hello world“ oder „Hallo Welt“. So

¹ Alte Compiler haben dabei gerne die Meldung bei der Templatedefinition angezeigt, nicht aber beim Aufruf der Funktion. Das ist sehr ärgerlich gewesen, weil die Korrektur eines solchen Fehlers beim Aufruf erfolgen muss, nicht aber bei der Definition. Neuere Compiler zeigen glücklicherweise beide Stellen im Quelltext an.

auch hier, da dieses Programm die Mindestkomponenten eines C/C++ Programms zeigt.

```
/*
    hallo.c: Ausgabe von Hallo Welt
*/
#include <stdio.h>

int main( void ) {
    puts( "Hallo Welt" );
    return 0;
}
```

Obwohl dieses Programm auch korrekten C++ Code darstellt, würde man es in C++ aber anders formulieren:

```
//  hallo.cpp: Ausgabe von Hallo Welt
#include <iostream>

int main() {
    std::cout << "Hallo Welt\n";
    return 0;
}
```

Jedes Quelltextmodul sollte mit einem Kommentar beginnen, in dem Sie die Anwendung des Moduls kurz beschreiben.

Die Includezeile bindet die Deklarationen einer Standardbibliothek ein. Für die Ein-/Ausgabefunktionen ist dies in C die Datei `stdio.h` und in C++ `iostream`. In älteren C++ Programmen wird auch noch `iostream.h` stattdessen verwendet. Diese Datei stammt noch aus der Zeit bevor C++ zum ersten mal standardisiert wurde. Die Deklarationen sind dort ähnlich zu der Standardbibliothek unterschieden sich aber zum Beispiel darin, dass sie keinen Namensraum `std` verwenden bzw definieren. Diese macht gerade den Mischbetrieb (neue C++ Compiler und Prästandard Compiler) schwierig, weshalb ich z.B. meinen letzten Prästandard Compiler vor kurzen in Rente geschickt habe.

In der nächsten Anweisung beginnt die Definition der Funktion `main`. `Main` muss in jeden C/C++ Programm genau einmal definiert werden. Fehlt sie oder wird sie mehr als einmal definiert, erhalten Sie beim Linken der Anwendung einen Fehler. In unserem Beispiel erwartet die Funktion keinen Parameter, es ist aber möglich Kommandozeilenparameter Ihres Programms entgegen zu nehmen. In einem späteren Beispiel werden wir darauf eingehen.

Die erste Anweisung in `main` gibt nun den Text „Hallo Welt“ am Bildschirm aus. Hier unterscheiden sich die beiden Version dann doch sehr deutlich voneinander:

In C gibt es eine Standardfunktion `puts`, welche als einzigen Parameter eine Zeichenkette erwartet und diese Zeichenkette am Bildschirm ausgibt. Da `puts` automatisch hinterher den Cursor Ihres Bildschirms an den Anfang der nächsten Zeile setzt, müssen wir das entsprechende Steuerzeichen nicht mit in der Zeichenkette angeben.

In C++ verwenden wir nicht die Funktion `puts` sondern stattdessen die globale Variable `cout`, die im Namespace `std` deklariert wurde. `cout` ist vom Typ `ostream`, der den Operator `<<` überladen hat. Mit Hilfe dieses Operators können beliebige Werte (solange es dafür eine entsprechende Funktion gibt) ausgegeben werden. Wenn dem Operator eine Zeichenkette übergeben wird, setzt dieser allerdings am Ende der Ausgabe den Cursor Ihres Bildschirm nicht an den Anfang der nächsten Zeile. Aus diesem Grund müssen wir hier das dafür erforderliche Steuerzeichen angeben. Die erforderlichen Steuerzeichen sind das Carriage Return (`'\r'`) und das Line Feed (`'\n'`). `cout` fügt automatisch Carriage Return mit ein, wenn es ein Line Feed entdeckt, weshalb wir nur das Line Feed angeben müssen.

Die letzte Anweisung in der C-Version beendet `main` und gibt den Wert 0 zurück, um keinen Fehler zu signalisieren. In C++ darf (aber nur bei `main`) darauf verzichtet werden.

Spätestens jetzt ist es an der Zeit, dass Sie sich mit der Bedienung Ihrer Entwicklungsumgebung vertraut machen. Starten Sie ein neues Projekt und geben Sie den gewünschten Quelltext ein. Dann speichern Sie die Quelldatei und starten den Übersetzungsvorgang. Wenn Sie keine Tippfehler haben, sollte das Programm übersetzt worden sein und Sie können es starten.

Der Standard erlaubt eine eine von 3 möglichen Signaturen für die Funktion `main`:

```
int main( [void] );  
int main ( int argc, char *argv[] );  
int main( /* implementation defined */ );
```

Die erste Form haben wir bereits gesehen und ist für viele Anwendungen auch ausreichend. Die zweite Variante ist dann nützlich, wenn das Programm mit Tastaturbefehlen gestartet wird und dem Programm in der Eingabezeile auch weitere Argumente übergeben werden. In diesem Fall ist in `argc` die Anzahl der Komponenten in der Kommandozeile gespeichert und in `argv` erhalten Sie die Adresse eines Arrays mit den Zeigern auf die einzelnen Komponenten. Das Array ist dabei so lang wie `argc+1`, das erste Element enthält dabei immer den Programmnamen, die weiteren Elemente jeweils ein Programmargument und das letzte Element `argv[argc]` ist immer `NULL` bzw. `nullptr`. Die compilerspezifische Signatur finden Sie in dessen Benutzeranleitung. Unter MS-DOS, Windows, Unix, Linux oder Mac OS X ist es um Beispiel üblich, noch ein Array mit den Umgebungsvariablen zu speichern.

Wie Sie die Parameter benennen, bleibt ganz allein Ihnen überlassen, aus Gründen der Lesbarkeit würde ich aber empfehlen, bei `argc` und `argv` zu bleiben.

7. C++ Klassen Basiswissen

7.1. Klassen

Strukturdefinitionen sind in C++ wesentlich umfangreicher möglich als in C. Daher wird hier auch nicht der Begriff `Struktur`, sondern der Begriff `Klasse` (`class`) verwendet. Jede Instanz einer solchen Klasse wird auch `Objekt` genannt.

7.2. member function

Bei der Definition einer Klasse können zusätzlich zu normalen Elementen auch Klassenfunktionen definiert werden. Innerhalb einer solchen Funktionsdefinition können Sie auf die einzelnen Klassenelemente wie auf lokale Variablen und andere Funktionen zugreifen.

```
/*
    isLeap1.cpp: Zeige Schaltjahr
*/
#include <iostream>

struct Datum {
    int    tag,
          monat,
          jahr;

    bool isLeap() {
        if( (jahr % 4) != 0 )
            return false;
        if( (jahr % 400) == 0 )
            return true;
        if( (jahr % 100) == 0 )
            return false;

        return true;
    }
};

int main() {
    Datum datum = { 11, 6, 1964 };

    std::cout << datum.jahr << " ist " << (datum.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

    return 0;
}
```

7.3. this pointer

Implizit wird einer solchen Funktion ein versteckter Parameter **this** mit der Adresse des Objekts übergeben. Dies ist Beispielsweise praktisch, wenn eine lokale Variable einer solchen Funktion den selben Namen hat wie ein Klassenelement.

```
/*
    isLeap2.cpp: Zeige Schaltjahr
*/
#include <iostream>

struct Datum {
    int    tag,
          monat,
          jahr;
}
```

```
bool isLeap() {
    if( (jahr % 4) != 0 )
        return false;
    if( (jahr % 400) == 0 )
        return true;
    if( (jahr % 100) == 0 )
        return false;
    return true;
}

void setYear( int jahr ) {
    this->jahr = jahr;
}

};

int main() {
    Datum datum = { 11, 6, 1964 };

    std::cout << datum.jahr << " ist " << (datum.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

    datum.setYear( 1971 );

    std::cout << datum.jahr << " ist " << (datum.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

    return 0;
}
```

Grundsätzlich sollten Sie jedoch solche Konstellationen vermeiden. In einem späteren Kapitel werden Sie jedoch einen Fall lernen, wo Sie das Schlüsselwort `this` verwenden müssen.

7.4. Modularisierung

Bei sehr umfangreichen Klassen können die Definitionen über viele Tausend Zeilen lang werden. Um dies zu umgehen, können Sie die Definition einer Klassenfunktion auch außerhalb der Klassendefinition erledigen. Hierzu schreiben Sie innerhalb der Klassendefinition nur den Prototypen der Klassenfunktion und definieren die Klassenfunktion außerhalb. In diesem Fall müssen Sie bei der Definition der Funktion, den Klassennamen gefolgt vom Operator `::` dem Funktionsnamen voranstellen.

```
/*
    isLeap3.cpp: Zeige Schaltjahr
*/

#include <iostream>

struct Datum {
    int  tag,
        monat,
        jahr;

    bool isLeap();
    void setYear( int jahr ) {
        this->jahr = jahr;
    }
};
```

```
bool Datum::isLeap() {
    if( (jahr % 4) != 0 )
        return false;
    if( (jahr % 400) == 0 )
        return true;
    if( (jahr % 100) == 0 )
        return false;
    return true;
}

int main() {
    Datum datum = { 11, 6, 1964 };

    std::cout << datum.jahr << " ist " << (datum.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

    datum.setYear( 1971 );

    std::cout << datum.jahr << " ist " << (datum.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

    return 0;
}
```

7.5. Const Funktionen

Manche Elementfunktionen verändern keine der Klassenelemente. Es kann bei sehr umfangreichen Projekten für die einzelnen Projektmitglieder eine sehr nützliche Information sein, wenn Sie diese Tatsache auch dokumentieren. Darüber hinaus kann der Compiler diese Information nutzen eventuell besseren Code zu erzeugen, da er weiß, dass ein bestimmter Funktionsaufruf, dieses Objekt nicht verändern wird. Zu diesem Zweck stellen Sie dem Funktionskopf das Schlüsselwort `const` hinten an. Danach wird der Compiler jeden Versuch, das Objekt zu verändern mit einer Fehlermeldung quittieren.

```
/*
    isLeap4.cpp: Zeige Schaltjahr
*/

#include <iostream>

struct Datum {
    int    tag,
           monat,
           jahr;

    bool isLeap() const;
    void setYear( int jahr ) {
        this->jahr = jahr;
    }
};

bool Datum::isLeap() const {
    if( (jahr % 4) != 0 )
        return false;
    if( (jahr % 400) == 0 )
        return true;
    if( (jahr % 100) == 0 )
        return false;
```

```
        return true;
    }

    int main() {
        Datum datum = { 11, 6, 1964 };

        std::cout << datum.jahr << " ist " << (datum.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

        datum.setYear( 1971 );

        std::cout << datum.jahr << " ist " << (datum.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

        return 0;
    }
```

7.6. Statische Elemente

Gelegentlich kann es vorkommen, dass Sie eine Elementfunktion definieren müssen, die keine anderen Elemente der Klasse verwendet. In diesem Fall setzen Sie vor der Deklaration der Elementfunktion das Schlüsselwort `static`. Im Gegensatz zum Schlüsselwort `const` müssen Sie dies nur bei der Deklaration voranstellen. Bei der Definition ist dies nicht notwendig.

Innerhalb anderer Elementfunktionen rufen Sie die statischen Elementfunktion ganz normal auf. Von außerhalb müssen Sie noch den Klassennamen gefolgt vom Klassenoperator `::` voranstellen. Statische Funktion erhalten keinen Impliziten Zeiger `this` auf das aktuelle Objekt. Zugriff auf nicht statische Element sind daher nicht möglich.

```
/*
    isLeap5.cpp: Zeige Schaltjahr
*/

#include <iostream>

struct Datum {
    int    tag,
           monat,
           jahr;

    bool isLeap() const {
        return isLeap( jahr );
    }
    static bool isLeap( int jahr );

    void setYear( int jahr ) {
        this->jahr = jahr;
    }
};

bool Datum::isLeap( int jahr ) {
    if( (jahr % 4) != 0 )
        return false;
    if( (jahr % 400) == 0 )
        return true;
    if( (jahr % 100) == 0 )
        return false;
```

```

        return true;
    }

    int main() {
        Datum datum = { 11, 6, 1964 };
        std::cout << datum.jahr << " ist " << (datum.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

        datum.setYear( 1971 );
        std::cout << datum.jahr << " ist " << (datum.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

        std::cout << 2016 << " ist " << (Datum::isLeap( 2016 ) ? "ein" :
"kein") << " Schaltjahr." << std::endl;

        return 0;
    }

```

Wie bei Funktionen können Sie auch normale Klassenelemente als statisch deklarieren. Sie müssen in diesem Fall außerhalb der Klassendefinition einmal in einer Quelldatei definieren.

```

/*
    timel.cpp: Zeige Uhrzeit
*/
#include <iostream>

struct Zeit {
    static int timeZone;

    int stunde, minute, sekunde;

    int getLocalHour() const {
        int stunde = this->stunde - timeZone;
        if( stunde < 0 )
            stunde += 24;
        else if( stunde >= 24 )
            stunde -= 24;

        return stunde;
    }

    int getGMThour() const {
        return stunde;
    }
};

int Zeit::timeZone = -1;

int main() {
    Zeit uhrZeit = { 3, 35, 0 };

    std::cout << "Lokale Uhrzeit: " << uhrZeit.getLocalHour() << ':' <<
uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;
    std::cout << "GMT          : " << uhrZeit.getGMThour() << ':' <<
uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;

    return 0;
}

```

7.7. Konstruktoren

Um sicherzustellen, dass alle Elemente einer Klasse korrekt initialisiert werden, können Sie einen oder mehrere sogenannte Konstruktoren definieren. Ein Konstruktor ist eine Elementfunktion, die den selben Namen hat wie die Klasse, eine beliebige Anzahl Parameter erwartet und keinen Rückgabetyt besitzt. Wenn Sie mindestens einen Konstruktor definiert haben, muss einer davon bei der Definition eines Klassenobjekts aufgerufen werden. Dies geschieht, wenn Sie hinter dem Variablennamen in Klammern die Argumente für den Konstruktor angeben.

```
/*
    time2.cpp: Zeige Uhrzeit
*/
#include <iostream>

struct Zeit {
    static int timeZone;

    int stunde, minute, sekunde;

    Zeit( int stunde, int minute, int sekunde ) {
        this->stunde = stunde;
        this->minute = minute;
        this->sekunde = sekunde;
    }

    int getLocalHour() const {
        int stunde = this->stunde - timeZone;
        if( stunde < 0 )
            stunde += 24;
        else if( stunde >= 24 )
            stunde -= 24;

        return stunde;
    }

    int getGMThour() const {
        return stunde;
    }
};

int Zeit::timeZone = -1;

int main() {
    Zeit uhrZeit( 3, 35, 0 );

    std::cout << "Lokale Uhrzeit: " << uhrZeit.getLocalHour() << ':' <<
uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;
    std::cout << "GMT          : " << uhrZeit.getGMThour() << ':' <<
uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;

    return 0;
}
```

Ein temporäres Objekt erzeugen Sie, indem Sie den Klassennamen wie einen Funktionsaufruf verwenden. Dieses temporäre Objekt kann dann in einem beliebigen Ausdruck verwendet werden. Es wird frühestens dann gelöscht, wenn die gerade ausgeführte Anweisung beendet ist. Die „Lebensdauer“ solcher temporären Objekte wird verlängert, wenn Sie sie einer Referenz zuweisen.

```
const MyClass &obj = MyClass(); obj.perform();
```

Wenn ein Klassenelement ebenfalls einen Konstruktoraufruf erfordert, müssen Sie diesen nach dem Funktionskopf Ihres eigenen Konstruktors gefolgt von einem Doppelpunkt aber vor der Funktionsdefinition aufrufen. Da C++ auch für die einfachen Typen einen Standardkonstruktor definiert, können wir dies an Hand unseres einfachen Beispiels demonstrieren.

```
/*
    time3.cpp: Zeige Uhrzeit
*/
#include <iostream>

struct Zeit {
    static int timeZone;

    int stunde, minute, sekunde;

    Zeit( int stunde, int minute, int sekunde ) : stunde(stunde),
    minute(minute), sekunde(sekunde)
    {}

    int getLocalHour() const {
        int stunde = this->stunde - timeZone;
        if( stunde < 0 )
            stunde += 24;
        else if( stunde >= 24 )
            stunde -= 24;

        return stunde;
    }

    int getGMThour() const {
        return stunde;
    }
};

int Zeit::timeZone = -1;

int main() {
    Zeit uhrZeit( 3, 35, 0 );

    std::cout << "Lokale Uhrzeit: " << uhrZeit.getLocalHour() << ':' <<
    uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;
    std::cout << "GMT          : " << uhrZeit.getGMThour() << ':' <<
    uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;

    return 0;
}
```

Wie normale Elementfunktionen können natürlich auch Konstruktoren außerhalb der Klassendefinition definiert werden.

```
/*
    time4.cpp: Zeige Uhrzeit
*/
#include <iostream>

struct Zeit {
    static int timeZone;
```

```

    int stunde, minute, sekunde;
    Zeit( int stunde, int minute, int sekunde );
    int getLocalHour() const {
        int stunde = this->stunde - timeZone;
        if( stunde < 0 )
            stunde += 24;
        else if( stunde >= 24 )
            stunde -= 24;

        return stunde;
    }
    int getGMThour() const {
        return stunde;
    }
};

Zeit::Zeit( int stunde, int minute, int sekunde ) : stunde(stunde),
minute(minute), sekunde(sekunde)
{}

int Zeit::timeZone = -1;

int main() {
    Zeit uhrZeit( 3, 35, 0 );

    std::cout << "Lokale Uhrzeit: " << uhrZeit.getLocalHour() << ':' <<
uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;
    std::cout << "GMT          : " << uhrZeit.getGMThour() << ':' <<
uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;

    return 0;
}

```

Ein besonderer Konstruktor ist ein solcher, der als Argument einen Ausdruck vom gleichen Typ erwartet. Der Parameter muss dabei als Referenztyp definiert werden. Der copy constructor wird immer dann aufgerufen, wenn ein neues Objekt mit einem bereits vorhandenen Objekt initialisiert werden soll.

```

// copyc.cpp: demonstration how to define and use a copy constructor
#include <iostream>
struct MyStruct {
    int val;
    //MyStruct( myStruct src ); error
    MyStruct( int val ) : val(val) {}
    MyStruct( MyStruct &src ) : val(src.val) {}
};

int main() {
    MyStruct hello(1990);
    hello.val = 1964;
    MyStruct world1( hello );
    MyStruct world2 = hello;
    std::cout << world1.val << ' ' << world2.val;
    return 0;
}

```

Der copy constructor wird vom Compiler auch dann erzeugt, wenn Sie ihn nicht selbst definiert haben. In diesen Falle werden alle Elemente der Quelle wie bei einer

normalen C-Struktur eins zu eins kopiert. Dies ist dann ein Problem, wenn mindestens eines der Elemente auf Ressourcen verweist, die bei der Entfernung des Klassenobjekt frei gegeben werden sollen. Diese Ressource würde dann ein zweites Mal freigegeben werden, wenn die Kopie gelöscht wird. Um das zu vermeiden können Sie entweder im copy constructor eine Kopie der Ressource erzeugen, einen Zähler mitführen, den Aufruf des copy constructors verbieten oder ab C++11 den copy constructor löschen.

Zum Löschen des copy constructor verwenden Sie das Schlüsselwort delete mit dem Zuweisungsoperator.

```
MyStruct( MyStruct & ) = delete;
```

Wie Sie den Aufruf des copy constructor verbieten, lernen wir weiter unten.

Ab C++11 können Sie mit Hilfe von RVALUE-Referenzen einen sogenannten move constructor definieren. Der move constructor kann das Performanzproblem eliminieren, wenn Sie im copy constructor eine Ressource kopieren deren Quelle gleich danach gelöscht werden soll. Der move constructor wird den Zeiger auf die Ressource kopieren und in der Quelle löschen. Der move constructor wird dann aufgerufen, wenn ein neues Objekt mit einem temporären Objekt initialisiert werden soll.

```
MyStruct( MyStruct &&src ) {  
    ressourceHandle = src.ressourceHandle;  
    src.ressourceHandle = nullptr;  
}
```

Konstruktoren, die nur ein Argument erwarten, werden vom Compiler automatisch aufgerufen, wenn ein Typ in einen anderen Typ konvertiert werden muss. Mit dem Schlüsselwort explicit vor dem Konstruktor teilen Sie dem Compiler mit, dass dieser Konstruktor nicht automatisch verwendet werden soll.

```
explicit String( const char *text );  
function( "hello world" );  
function( String("hello world") );
```

7.8. Destruktoren

Neben den Konstruktoren können Sie auch sogenannte Destruktoren definieren. Diese dienen dazu, angeforderte Ressourcen wieder freizugeben, wenn das entsprechende Klassenobjekt gelöscht werden soll. Ein Destruktor ist eine Elementfunktion, die wie ein Konstruktor den selben Namen hat wie die Klasse jedoch wird eine Tilde vorangestellt. Darüber hinaus hat diese ebenfalls keinen Rückgabetyt und darf im Gegensatz zu den Konstruktoren keine Parameter erwarten. Pro Klasse kann nur ein Destruktor definiert werden.

Da wir noch nichts über Ressourcenbehandlung gelernt haben, dient das folgende Beispiel nur zur Demonstration von Destrukturen.

```
/*  
    time5.cpp: Zeige Uhrzeit  
*/
```

```

#include <iostream>

struct Zeit {
    static int timeZone;

    int stunde, minute, sekunde;

    Zeit( int stunde, int minute, int sekunde );
    ~Zeit();

    int getLocalHour() const {
        int stunde = this->stunde - timeZone;
        if( stunde < 0 )
            stunde += 24;
        else if( stunde >= 24 )
            stunde -= 24;

        return stunde;
    }

    int getGMThour() const {
        return stunde;
    }
};

Zeit::Zeit( int stunde, int minute, int sekunde ) : stunde(stunde),
minute(minute), sekunde(sekunde)
{}

Zeit::~~Zeit() {
    std::cout << "Auf wiedersehen" << std::endl;
}

int Zeit::timeZone = -1;

int main() {
    Zeit uhrZeit( 3, 35, 0 );

    std::cout << "Lokale Uhrzeit: " << uhrZeit.getLocalHour() << ':' <<
uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;
    std::cout << "GMT          : " << uhrZeit.getGMThour() << ':' <<
uhrZeit.minute << ':' << uhrZeit.sekunde << std::endl;

    std::cout << "Jetzt wird aufgeräumt" << std::endl;
    return 0;
}

```

7.9. Vererbung

Neue Klassendefinitionen können auch von bereits bestehenden Klassen abgeleitet werden. Hierzu notieren sie hinter dem neuen Klassennamen getrennt mit einem Doppelpunkt die kommasetrennte Liste der Klassen, von denen die neue Klasse abgeleitet werden soll. Die neue Klasse erbt dann alle Eigenschaften der Basisklassen. Zusätzlich zu den neu hinzugefügten Eigenschaften. Wurden in einer Ihrer Basisklassen ein Konstruktor definiert, müssen Sie diesen bei der Definition eines Konstruktors Ihrer neuen Klasse aufrufen. Die geschieht auf die selbe Art, wie Sie das schon bei Klassenelementen gesehen haben.

```

/*
    timestamp1.cpp: Zeige Uhrzeit und Schaltjahr
*/

```

```
#include <iostream>

struct Zeit {
    static int timeZone;

    int stunde, minute, sekunde;

    Zeit( int stunde, int minute, int sekunde );
    ~Zeit();
    int getLocalHour() const {
        int stunde = this->stunde - timeZone;
        if( stunde < 0 )
            stunde += 24;
        else if( stunde >= 24 )
            stunde -= 24;

        return stunde;
    }

    int getGMThour() const {
        return stunde;
    }
};

struct Datum {
    int tag,
        monat,
        jahr;

    Datum( int tag, int monat, int jahr ) : tag(tag), monat(monat),
    jahr(jahr)
    {}
    bool isLeap() const {
        return isLeap( jahr );
    }
    static bool isLeap( int jahr );
    void setYear( int jahr ) {
        this->jahr = jahr;
    }
};

bool Datum::isLeap( int jahr ) {
    if( (jahr % 4) != 0 )
        return false;
    if( (jahr % 400) == 0 )
        return true;
    if( (jahr % 100) == 0 )
        return false;

    return true;
}

struct Zeitstempel : Datum, Zeit {
    Zeitstempel(int tag, int monat, int jahr, int stunde, int minute, int
    sekunde) :
        Datum(tag, monat, jahr), Zeit(stunde, minute, sekunde)
    {}
};
```

```
Zeit::Zeit( int stunde, int minute, int sekunde ) : stunde(stunde),
minute(minute), sekunde(sekunde)
{}

Zeit::~~Zeit() {
    std::cout << "Auf wiedersehen" << std::endl;
}

int Zeit::timeZone = -1;

int main() {
    Zeitstempel zeit( 11, 3, 1964, 3, 35, 0 );

    std::cout << "Lokale Uhrzeit: " << zeit.getLocalHour() << ':' <<
zeit.minute << ':' << zeit.sekunde << std::endl;
    std::cout << "GMT          : " << zeit.getGMThour()   << ':' <<
zeit.minute << ':' << zeit.sekunde << std::endl;

    std::cout << zeit.jahr << " ist " << (zeit.isLeap() ? "ein" : "kein")
<< " Schaltjahr." << std::endl;

    std::cout << "Jetzt wird aufgeräumt" << std::endl;
    return 0;
}
```

7.10. Zugriffsmodifikator (Access Specifier)

Bisher haben wir Konstruktoren als Mittel kennengelernt, dafür zu sorgen, dass alle Elemente einer Klasse korrekt initialisiert werden. Nichts hindert uns aber bisher daran, nach der Erzeugung eines Objektes die Elemente eines Objektes von außerhalb der Klasse zu verändern. Ein gutes Klassendesign zeichnet sich aber dadurch aus, dass jeder ungültige Zustand eines Objektes vermieden wird. Um dies zu gewährleisten kann mit Hilfe der Schlüsselworte `private`, `protected`, `public` und `friend` festgelegt werden, welche Klassenelemente von außerhalb erreichbar sind und welche nicht.

Alle Elemente, die nach dem Schlüsselwort `private` gefolgt von einem Doppelpunkt deklariert oder definiert wurden, sind von außerhalb nicht erreichbar. Das Schlüsselwort `protected` bewirkt, dass die Elemente nur innerhalb der eigenen und alle abgeleiteten Klassen erreichbar ist. Das Schlüsselwort `public` macht das Element für alle Programmteile erreichbar.

Bei der Angabe der Basisklassen einer abgeleiteten Klassen können Sie ebenfalls mit Hilfe dieser Schlüsselworte festlegen, für welche Programmteile die Basisklassen erreichbar sind. Hier müssen Sie dies jedoch vor jeder Basisklasse einzeln angeben.

Wurde eine Klasse wie in unseren bisherigen Beispielen mit dem Schlüsselwort `struct` definiert, so sind alle Elemente `public`, falls Sie nichts angeben. Mit Hilfe des Schlüsselwortes `class` können Sie wie schon im Kapitel Strukturen bemerkt ebenfalls eine Klasse definieren. Dann sind aber alle Elemente `private`, falls Sie nichts angeben. In den allermeisten Fällen ist es sinnvoll, die Elementvariablen einer Klasse als `private` zu deklarieren, daher ist es üblich, das Schlüsselwort `class` zu bevorzugen.

```
/*
    timestamp2.cpp: Zeige Uhrzeit und Schaltjahr
*/
#include <iostream>

struct Zeit {
    protected:
        static int timeZone;

    private:
        int stunde, minute, sekunde;

    public:
        Zeit( int stunde, int minute, int sekunde );
        ~Zeit();

        int getLocalHour() const {
            int stunde = this->stunde - timeZone;
            if( stunde < 0 )
                stunde += 24;
            else if( stunde >= 24 )
                stunde -= 24;
            return stunde;
        }

        int getGMThour() const {
            return stunde;
        }

        int getMinute() const {
            return minute;
        }

        int getSekunde() const {
            return sekunde;
        }
};

struct Datum {
    private:
        int tag,
            monat,
            jahr;

    public:
        Datum( int tag, int monat, int jahr ) : tag(tag), monat(monat),
        jahr(jahr)
        {}
        bool isLeap() const {
            return isLeap( jahr );
        }
        static bool isLeap( int jahr );
};
```

```
void setYear( int jahr ) {
    this->jahr = jahr;
}
int getJahr() const {
    return jahr;
}
int getDay() const {
    return tag;
}
};

bool Datum::isLeap( int jahr ) {
    if( (jahr % 4) != 0 )
        return false;
    if( (jahr % 400) == 0 )
        return true;
    if( (jahr % 100) == 0 )
        return false;
    return true;
}

class Zeitstempel : public Datum, public Zeit {
public:
    Zeitstempel(int tag, int monat, int jahr, int stunde, int minute, int
sekunde) :
        Datum(tag, monat, jahr), Zeit(stunde, minute, sekunde)
    {}

    int getLocalDay() const {
        int tag = getDay();
        int stunde = getGMThour() - timeZone;
        if( stunde < 0 )
            --tag;
        else if( stunde >= 24 )
            ++tag;
        return tag;
    }
};

Zeit::Zeit( int stunde, int minute, int sekunde ) : stunde(stunde),
minute(minute), sekunde(sekunde)
{}

Zeit::~Zeit() {
    std::cout << "Auf wiedersehen" << std::endl;
}

int Zeit::timeZone = -1;
```

```
int main() {
    Zeitstempel zeit( 11, 3, 1964, 23, 35, 0 );

    std::cout << "Lokale Uhrzeit: " << zeit.getLocalDay() << ' ' <<
zeit.getLocalHour() << ':' << zeit.getMinute() << ':' << zeit.getSekunde()
<< std::endl;
    std::cout << "GMT          : " << zeit.getDay() << ' ' <<
zeit.getGMThour() << ':' << zeit.getMinute() << ':' << zeit.getSekunde()
<< std::endl;

    std::cout << zeit.getJahr() << " ist " << (zeit.isLeap() ? "ein" :
"kein") << " Schaltjahr." << std::endl;

    std::cout << "Jetzt wird aufgeräumt" << std::endl;
    return 0;
}
```

Mit Hilfe des Schlüsselwortes `friend` können Sie die aufgestellten Zugriffsregeln außer Kraft setzen und so ausgewählten Klassen und freien Funktionen Zugriff auf private Elemente gewähren. Machen Sie aber davon nur Gebrauch, wenn es Ihr Klassendesign unbedingt erfordert.

```
/*
    time6.cpp: Zeige Uhrzeit
*/
#include <iostream>

class Zeit {
    friend class Kalender;
    friend void show(const Zeit &meineZeit);

    int stunde, minute, sekunde;

public:
    Zeit( int stunde, int minute, int sekunde );

    int getHour() const {
        return stunde;
    }

    int getMinute() const {
        return minute;
    }

    int getSekunde() const {
        return sekunde;
    }
};

Zeit::Zeit( int stunde, int minute, int sekunde ) : stunde(stunde),
minute(minute), sekunde(sekunde)
{}

class Kalender {
    Zeit meineZeit;
```

```

    public:
        Kalender( int stunde, int minute, int sekunde ) : meineZeit( stunde,
minute, sekunde )
        {}
        Kalender( const Zeit &zeit ) : meineZeit( zeit ) {}
        void show() {
            std::cout << meineZeit.stunde << ':' << meineZeit.minute << ':' <<
meineZeit.sekunde << std::endl;
        }
};

void show(const Zeit &meineZeit) {
    std::cout << meineZeit.stunde << ':' << meineZeit.minute << ':' <<
meineZeit.sekunde << std::endl;
}

int main() {
    Zeit uhrZeit( 3, 35, 0 );
    Kalender kalender( uhrZeit );

    kalender.show();
    show( uhrZeit );

    return 0;
}

```

7.11. Zeiger und Referenzen

Zeigern und Referenzen auf Klassenobjekte können Sie auch die Adresse einer beliebigen abgeleiteten Klasse ohne explizite Typkonvertierung zuweisen. Dabei wird auch falls erforderlich automatisch die Adresse entsprechend korrigiert, so dass der Zeiger oder die Referenz auf den richtigen Bestandteil verweist.

```

/*
    pointer1.cpp: demonstration of pointer to base class
*/

#include <iostream>

class base1 {
    int value1;
};

class base2 {
    int value2;
};

class derived : public base1, public base2 {
    int value;
};

int showAddress1( const base1 &obj ){
    std::cout << &obj << std::endl;
}

int showAddress2( const base2 &obj ) {
    std::cout << &obj << std::endl;
}

```

```
int showAddress3( const derived &obj ) {
    std::cout << &obj << std::endl;
}

int main() {
    derived obj;

    showAddress1( obj );
    showAddress2( obj );
    showAddress3( obj );

    return 0;
}
```

Umgekehrt ist das nicht möglich und ergibt einen Compilerfehler.

```
int showAddress1( const base1 &obj ) {
    std::cout << &obj << std::endl;

    const derived *ptr = &obj;    // <-- Fehler
    std::cout << ptr << std::endl;
}
```

Wie Sie das dennoch erreichen können, wird in einem späteren Kapitel behandelt.

7.12. Virtuelle Funktionen

Wenn Sie einen Zeiger oder eine Referenz auf eine Basisklasse haben, können Sie nur die Elementfunktionen der Basisklasse aufrufen auch dann, wenn das Objekt, auf das der Zeiger bzw. die Referent verweist, von einer abgeleiteten Klasse ist. Wenn Sie jedoch eine Elementfunktion als virtual deklarieren, werden automatisch die selben Elementfunktionen in allen abgeleiteten Klassen virtual und beim Aufruf dieser Funktionen wird immer die richtige Version verwendet. Üblicherweise sollte man jedoch auch in den abgeleiteten Klassen die Funktionen als virtual deklarieren, damit jeder Entwickler auch dort sieht, dass es sich um eine virtuelle Funktion handelt. Es ist jedoch nicht unbedingt erforderlich.

Bei neueren C++ compiler ab C++11 können Sie mit override angeben, dass eine Funktion die gleiche Funktion der Basisklasse überladen soll.

```
struct B {
    virtual void f(int);
};

struct D : B {
    virtual void f(int) override;    // OK, D::f(int) overrides B::f(int)
    virtual void f(long) override;  // Error: f(long) does not override
                                    //      B::f(int)
};
```

Mit dem Schlüsselwort final können Sie ab C++11 angeben, dass eine Elementfunktion nicht überladen werden darf.

```
struct B {
    virtual void f() const final;
};
```

```
struct D : B {
    void f() const; // Error: D::f attempts to override final B::f
};
```

Wenn Sie innerhalb einer abgeleiteten Klasse die Version der Basisklasse aufrufen möchten, können Sie dies erreichen, in dem Sie den Namen der Basisklasse mit dem Klassenoperator davor stellen.

```
/*
    pointer2.cpp: demonstration of virtual functions
*/
#include <iostream>

class Punkt {
    double x, y;

    public:
    Punkt( double x, double y ) : x(x), y(y)
    {}

    virtual ~Punkt()
    {}

    virtual void show() const {
        std::cout << "Punkt (" << x << ', ' << y << ') ' << std::endl;
    }
};

class Kreis : public Punkt {
    double r;

    public:
    Kreis( double x, double y, double r ) : Punkt(x, y), r(r)
    {}

    virtual void show() const {
        Punkt::show();
        std::cout << "Radius " << r << std::endl;
    }
};

void showObject( const Punkt &obj ) {
    obj.show();
}

int main() {
    Punkt    punkt( 2, 3 );
    Kreis    kreis( 5, 6, 1 );

    showObject( punkt );
    showObject( kreis );

    return 0;
}
```

Gelegentlich kann es vorkommen, dass Sie eine Basisklasse definieren wollen, die zwar Elementfunktionen deklariert jedoch sollen sie nicht implementiert werden. Diese Basisklassen dienen dann dazu nur die Schnittstellen oder einen Teil davon zu definieren. Die Implementierung erfolgt dann in den abgeleiteten Klassen. Eine so definierte Basisklasse wird auch abstrakte Klasse genannt, da kein Objekt von

diesen Klassen erzeugt werden darf. Die Elementfunktionen, die zwar deklariert aber nicht implementiert werden, werden pure virtual genannt. Um eine Funktion so zu deklarieren, müssen Sie diese virtual deklarieren und hinter dem Funktionskopf die Zeichen = und 0 notieren.

```
/*
   pointer3.cpp: demonstration of abstract classes
*/
#include <iostream>

class Graphic {
public:
    virtual void show() const = 0;
    virtual ~Graphic()
    {}
};

class Punkt : public Graphic {
    double x, y;

public:
    Punkt( double x, double y ) : x(x), y(y)
    {}

    virtual void show() const {
        std::cout << "Punkt (" << x << ', ' << y << ') ' << std::endl;
    }
};

class Kreis : public Punkt
{
    double r;

public:
    Kreis( double x, double y, double r ) : Punkt(x, y), r(r)
    {}

    virtual void show() const {
        Punkt::show();
        std::cout << "Radius " << r << std::endl;
    }
};

void showObject( const Graphic &obj ) {
    obj.show();
}

int main() {
    Punkt    punkt( 2, 3 );
    Kreis    kreis( 5, 6, 1 );

    showObject( punkt );
    showObject( kreis );

    return 0;
}
```

Beachten Sie jedoch folgenden Umstand: Während der Laufzeit eines Konstruktors oder eines Destruktors einer Basisklasse wird beim direkten oder indirekten Aufruf einer virtuellen Funktion die Version der Basisklasse aufgerufen auch wenn ein

Objekt einer abgeleiteten Klasse erzeugt werden soll. Im Falle einer abstrakten Klasse führt dies zu einem Laufzeitfehler.

```
/*
    pointer4.cpp: demonstration of pure virtual function calls
*/
#include <iostream>

class Graphic {
public:
    virtual void show() const = 0;
    virtual void showName() const = 0;

    virtual ~Graphic() {
        print();
    }

    void print() {
        showName();
    }
};

class Punkt : public Graphic {
    double x, y;

public:
    Punkt( double x, double y ) : x(x), y(y) {}

    virtual void show() const {
        showName();
        std::cout << " (" << x << ', ' << y << ') ' << std::endl;
    }

    virtual void showName() const {
        std::cout << "Punkt";
    }
};

class Kreis : public Punkt {
    double r;

public:
    Kreis( double x, double y, double r ) : Punkt(x, y), r(r) {}

    virtual void show() const {
        Punkt::show();
        std::cout << "Radius " << r << std::endl;
    }

    virtual void showName() const {
        std::cout << "Kreis";
    }
};

void showObject( const Graphic &obj ) {
    obj.show();
}

int main() {
    Punkt    punkt( 2, 3 );
    Kreis    kreis( 5, 6, 1 );
```

```

        showObject( punkt );
        showObject( kreis );

        return 0;
}

```

7.13. Elementtypen (Member Types)

Klassen können nicht nur Variablen und Funktionen als Elemente haben sondern auch eigene Typdefinitionen.

```

/*
    isLeap6.cpp: Zeige Schaltjahr
*/
#include <iostream>

class Datum {
public:
    enum Monate {
        Januar, Februar, Maerz, April, Mail, Juni, July, August,
        September, Oktober, November, Deember
    };

private:
    int      tag,
           jahr;
    Monate   monat;

public:
    Datum( int tag, Monate monat, int jahr ) : tag(tag), monat(monat),
    jahr(jahr)
    {}

    bool isLeap() const {
        return isLeap( jahr );
    }
    static bool isLeap( int jahr );
    void setYear( int jahr ) {
        this->jahr = jahr;
    }
    int getYear() const {
        return jahr;
    }
};

bool Datum::isLeap( int jahr ) {
    if( (jahr % 4) != 0 )
        return false;
    if( (jahr % 400) == 0 )
        return true;
    if( (jahr % 100) == 0 )
        return false;
    return true;
}

int main() {
    Datum::Monate monat = Datum::Juni;
    Datum datum( 11, monat, 1964 );
}

```

```
std::cout << datum.getYear() << " ist " << (datum.isLeap() ? "ein" :  
"kein") << " Schaltjahr." << std::endl;  
datum.setYear( 1971 );  
std::cout << datum.getYear() << " ist " << (datum.isLeap() ? "ein" :  
"kein") << " Schaltjahr." << std::endl;  
std::cout << 2016 << " ist " << (Datum::isLeap( 2016 ) ? "ein" :  
"kein") << " Schaltjahr." << std::endl;  
return 0;  
}
```

Für's erste soll es an dieser Stelle genug sein. In einem späteren Kapitel betrachten wir weitere wichtigste Aspekte der Klassenentwicklung.

8. Ein- und Ausgabe von Daten

Alle Programme müssen in irgendeiner Form Daten einlesen und wieder ausgeben können. Andernfalls wären sie nutzlos. Auch wenn heute alle Personal Computer mit einer grafischen Benutzeroberfläche ausgestattet sind, die die Ein- Ausgabe über Bildschirmfenster (Dialog etc.) gestatten, werden auch heute noch eine Vielzahl an Programmen entwickelt, die sich auf die Eingabe via Tastatur und die Ausgabe auf dem Bildschirm beschränken. Windows verwendet hierfür z.B. die Eingabeaufforderung Mac OS X oder Linux verwenden ein Terminalfenster.

Da die Programmierung der Bildschirmfenster nicht normiert ist, ist sie nicht Thema dieses Buches. Wollen Sie dieses lernen verweise ich auf Themen wie MFC, VCL (jeweils nur Windows), Cocoa (Mac OS X), Xlib (versch. Unixsysteme) oder GTK, Qt (plattformübergreifend). Diese Liste erhebt keinen Anspruch auf Vollständigkeit.

Wie Sie bereits in unserem ersten Beispielprogramm sehen konnten, unterscheiden sich die Ein-/Ausgabefunktionen von C und C++ beträchtlich. Wir wollen sie daher getrennt betrachten und beginnen mit C.

8.1. Ein- und Ausgabe in C

Die erforderlichen Deklarationen für den Compiler befinden sich alle in der Headerdatei `<stdio.h>`. Daher muss diese Datei in jedem C-Programm, das eine oder mehrere dieser Funktionen aufrufen möchte, eingebunden werden (inkludieren).

a. puts/gets

Die erste Funktion „puts“ haben Sie bereits kennengelernt. Der Prototyp dieser Funktion lautet

```
int puts( const char *str );
```

Aus diesem Prototyp können wir sehen, dass puts einen Zeiger auf ein Zeichen erwartet, und garantiert, dass die Werte, auf die der Zeiger verweist, nicht geändert werden und einen Wert vom Typ int zurückliefert.

Aus den Prototypen können wir immer nur die Syntax entnehmen, wie eine Funktion aufgerufen wird. Das ist nämlich die einzige Information, für die sich der Compiler

interessiert, um überprüfen zu können, ob wir die Funktion syntaktisch korrekt aufrufen. Die Semantik, also wie die Funktion ihre Parameter interpretiert und wie der Aufrufer die zurückgegebenen Werte zu interpretieren hat, müssen wir dem Standard entnehmen.

Tatsächlich erwartet puts nicht einfach einen Zeiger auf ein Zeichen, sondern auf eine Zeichenkette, deren letztes Zeichen ein 0-Byte sein muss. Zur Erinnerung: Zeichenketten wurden im Abschnitt „Konstante – Literale“ weiter oben behandelt. Es wäre also falsch anzunehmen, puts könnte einfach nur ein einzelnes Zeichen ausgeben:

```
#include <stdio.h>

int main( void ) {
    char buchstabe = 'a';
    puts( &buchstabe );

    return 0;
}
```

Dieses Programm ist zwar syntaktisch korrekt und wird daher ohne zu murren übersetzt, dennoch werden Sie mit großer Wahrscheinlichkeit nicht das gewünschte Ergebnis erhalten. In solchen Fällen schreibt der Standard immer „the behaviour is undefined“ („das Verhalten ist undefiniert“). D.h. es kann alles mögliche passieren bis hin zum totalen Datenverlust. Seien Sie also stets vorsichtig und konsultieren Sie stets auch die Dokumentation, wenn Sie unsicher sind.

Der Rückgabewert von puts ist im Erfolgsfall eine nicht negative Zahl (also größer gleich 0) oder die in stdio.h vordefinierte Konstante EOF. Dies kann beispielsweise vorkommen, wenn der Benutzer die Verbindung zum Computer trennt, ehe Ihr Programm beendet wurde. Im allgemeinen müssen Sie darauf nicht reagieren, da in einem solchen Fall auch die Eingaberoutinen EOF (oder einen anderen Wert, der diesen Zustand signalisiert) zurückliefern würden, es genügt daher dabei den Rückgabewert auszuwerten. Sollte aber Ihr Programm lange keine Eingabe mehr machen wollen, sollten Sie durchaus den Wert beachten und Ihr Programm gegebenenfalls beenden.

Das Gegenstück zu puts heißt gets und liest eine Zeile von der Tastatur bis der Benutzer die Return- oder die Entertaste¹ gedrückt hat. Der Prototyp lautet

```
char *gets( char *str );
```

gets erwartet die Adresse eines Zeichenarrays und füllt diesen mit dem eingegebenen Text. Das Zeichenarray wird mit einem 0-Byte abgeschlossen, so dass Sie eine gültige Zeichenkette erhalten. Im Erfolgsfall erhalten Sie die Adresse des Arrays wieder zurück. Falls die Verbindung aus welchen Gründen auch immer beendet wurde, wird ein NULL-Zeiger zurückgeliefert. Da NULL in einer if-abfrage wie der boolsche Wert false interpretiert wird, können Sie diesen Zustand leicht überprüfen:

¹ Beide Tasten werden bei den meisten Systemen identisch behandelt, weshalb ich in Zukunft nur noch eine der beiden .Tasten nennen werde.

```
/* gets1.c */
#include <stdio.h>

int main( void ) {
    char text[80];

    puts( "Wie heißen Sie?" );
    if( gets( text ) ) {
        puts( "Hallo " );
        puts( text );
    }

    return 0;
}
```

Ein großes Problem von `gets` stellt die Tatsache dar, dass wir nicht die Größe des Arrays übergeben können. Wird zu viel Text eingegeben, findet ein Pufferüberlauf (buffer overflow) statt, der nicht erkannt wird, und der Speicher, der sich zufällig hinter Ihrem Array befindet, wird verändert. Daher gilt diese Funktion als veraltet und sollte nicht mehr verwendet werden. Eine mögliche Alternative werden wir im nächsten Abschnitt implementieren.

b. `putchar/getchar`

Diese beiden Funktionen lesen genau ein Zeichen von der Tastatur bzw schreiben genau ein Zeichen auf den Bildschirm. Ihre Prototypen sind:

```
int putchar( int c );
int getchar( void );
```

`putchar` wandelt den übergebenen Wert in einen `unsigned char` typ um (wobei überzählige Bits einfach verworfen werden) und gibt das Zeichen am Bildschirm aus. Wie `puts` liefert es den Wert EOF zurück, falls die Verbindung beendet wurde, oder das ausgegebene Zeichen als `unsigned char`.

`getchar` erwartet keinen Parameter und liefert das eingegebene Zeichen als `unsigned char` zurück. Im Falle eines Verbindungsabbruchs wird EOF zurückgeliefert.

Hier macht sich zum ersten mal bemerkbar, dass die Standardein-/ausgabefunktionen gepuffert arbeiten. Das heißt die Funktion liest von der Tastatur alle Zeichen bis der Benutzer die Returntaste gerückt hat, merkt sich diese Zeichen und gibt aber nur das erste Zeichen zurück. Beim nächsten Aufruf, werden die zwischengespeicherten Zeichen zurückgeliefert. Erst wenn der Zwischenspeicher leer ist, erhält der Benutzer die Gelegenheit, erneut Daten einzugeben.

Ein gängiger Fehler ist es auch, den Returnwert von `getchar` in einer Variablen vom Typ `char` zu speichern. In diesem Fall könnten Sie nämlich den Verbindungsabbruch nicht sicher erkennen und möglicherweise zu früh oder gar nicht reagieren.

```
/* getchar1.c */
#include <stdio.h>
```

```
int main( void ) {
    char c;

    while( 1 ) {
        c = getchar();
        if( c == EOF )
            break;
        putchar( c );
    }

    return 0;
}
```

Dieses Programm liest von der Tastatur ein Zeichen und gibt es sofort wieder aus. Da die Schleifenbedingung 1 ist, wird die Schleife im Schleifenkopf niemals beendet. Wir prüfen stattdessen innerhalb der Schleife den Rückgabewert von `getchar` und beenden die Schleife, wenn diese EOF geliefert hat.

Da die Variable jedoch von Typ `char` ist, ist die korrekte Erkennung nicht gewährleistet. Die korrekte Implementierung sieht daher so aus:

```
/* getchar2.c */
#include <stdio.h>

int main( void ) {
    int c;

    while( 1 ) {
        c = getchar();
        if( c == EOF )
            break;
        putchar( c );
    }
    return 0;
}
```

Eine kürzere Variante nutzt die Tatsache aus, dass Zuweisungen wiederum als Operand eines anderen Operators dienen können:

```
/* getchar3.c */
#include <stdio.h>

int main( void ) {
    int c;

    while( (c = getchar()) != EOF )
        putchar( c );

    return 0;
}
```

Hier wäre es dann auch kein Problem, `c` als `char` zu definieren, da ja hier nicht `c` auf EOF geprüft wird, sondern die Zuweisung, welche immer noch vom Typ `int` ist. Da aber `putchar` sowieso ein `int` haben will, belassen wir es dabei.

Mit Hilfe der Funktion `getchar` können wir nun eine Funktion implementieren, die das Problem von `gets` nicht kennt.

```
/* gets2.c */
#include <stdio.h>

static char *getline( char *data, int size ) {
    int c;
    int i = 0;

    if( data == NULL || size <= 0 )
        return NULL;

    size--;
    while( i < size ) {
        c = getchar();
        if( c == EOF || c == '\n' )
            break;
        data[i++] = (char)c;
    }

    /*
        wenn ende und noch nichts eingegeben wurde,
        signalisieren wir dies durch NULL
    */
    if( c == EOF && i==0 )
        return NULL;

    // Schreiben des 0 bytes
    data[i] = 0;

    return data;
}

int main( void ) {
    char text[80];

    puts( "Wie heißen Sie?" );
    if( getline( text, 80 ) ) {
        puts( "Hallo " );
        puts( text );
    }

    return 0;
}
```

In diesem Programm wird nun eine neue Funktion `getline` definiert, welche das Problem von `gets` vermeiden soll.

Wie jede vernünftige Implementierung einer Funktion, prüft sie am Anfang die Parameter auf Plausibilität und terminiert, falls diese ungültig sind. Bei Zeigern haben wir aber nur die Möglichkeit, auf den `NULL`-Zeiger zu prüfen. Bei der Größenangabe

prüfen wir, ob dieser zu klein ist.

Danach dekrementieren wir die Größe um 1, um sicherzustellen, dass wir das 0-Byte der Zeichenkette schreiben können. Die Schleifenbedingung prüfen dann, ob wir noch genügend Platz zur Verfügung haben.

Nun wird das nächste Zeichen mit `getchar` eingelesen. Wenn die Returntaste gedrückt wurde oder die Verbindung unterbrochen wurde (bei den meisten Implementierungen ist dies die Tastenkombination Strg-D), verlassen wir die Schleife. Andernfalls wird das eingelesene Zeichen in unser Array geschrieben und die Indexvariable um eins erhöht. Beachten Sie, dass für den Inkrement der Postinkrement verwendet wurde, weshalb die Indexvariable nach dem Speichern erhöht wird. Die Typumwandlung der Variablen `c` von `int` nach `char` ist nicht unbedingt erforderlich. Da aber die meisten Compiler ohne explizite Umwandlung eine Warnung anzeigen, wollen wir ihm hier mitteilen, dass dies beabsichtigt und korrekt ist.

Nach dem Ende der Schleife prüfen wir noch einmal auf Verbindungsabbruch. Wenn `getchar` EOF geliefert hat und wir noch kein Zeichen erhalten haben, signalisieren wir dies durch Rückgabe des NULL-Zeigers, so wie es `gets` auch machen würde. Andernfalls schreiben wir das 0-byte, um aus einem einfachen char-array eine gültige Zeichenkette zu machen und geben die Adresse des Arrays zurück.

Die Implementierung von `main` ist praktisch identisch zu derjenigen unseres Beispiels für `gets` nur wird statt `gets` eben unsere neue Funktion `getline` mit dem zusätzlichen Parameter für die Größe unseres Arrays.

Das Programm ist schon ganz brauchbar, aber es gibt noch ein paar mögliche Verbesserungen:

Die Größe des Arrays haben wir zweimal angegeben. Bei der Definition des Arrays und beim Aufruf von `getline`. So etwas ist fehleranfällig und sollte daher vermieden werden. Man könnte nun auf den Gedanken kommen, die Größe als Konstante zu definieren und diese Konstante dann zu benutzen:

```
#define ARRAY_SIZE 80

...

    char text[ARRAY_SIZE];

...

    if( getline( text, ARRAY_SIZE ) )

...

```

Dies ist möglich, da aber ein zusätzlicher Bezeichner eingeführt werden muss, birgt dies aber die Gefahr eines Namenskonflikts. Außerdem erschwert dies das Lesen des Quelltextes, da die Information nicht unbedingt erkennbar ist.¹ Wenn es nicht

¹ Moderne Entwicklungsumgebungen können dieses Problem zwar vermeiden, wir wollen uns aber nicht darauf verlassen, dass dies der Fall ist.

unbedingt erforderlich ist, sollte das daher vermieden werden. Mit Hilfe des `sizeof` operators kann aber die Größe vom Compiler automatisch bestimmt werden, und wir müssen keinen neuen Bezeichner einführen. Da die Arraydefinition beim Aufruf von `getline` bekannt ist, können wir `sizeof` ohne Probleme einsetzen:

```
if( getline( text, sizeof( text ) ) )
```

`sizeof` liefert aber einen Wert vom Typ `size_t` zurück, der in der Headerdatei `<stddef.h>` definiert wird. Da, wie wir später sehen werden, andere Funktionen in `<stdio.h>` ebenfalls diesen Typ verwendet, müssen wir `<stddef.h>` nicht inkludieren. Aber wir müssen die Definition von `getline` anpassen.

```
static char *getline( char *data, size_t size )
```

Der Datentyp `size_t` ist ein vorzeichenloser Typ. Eine Prüfung auf kleiner 0 ist daher nicht sinnvoll. Darüber hinaus können Prüfungen auf Werte gleich 0 mit dem Operator „logisch not“ (!) abgekürzt werden. Die if-abfragen sehen dann so aus:

```
if( !data || !size )
...
if( c == EOF && !i )
```

c. printf/scanf

Die nächste Gruppe der Ein-Ausgabefunktionen sind `printf` und `scanf`. Von diesen beiden Funktionen gibt es eine ganze Reihe von Varianten, von denen wir hier an dieser Stelle nicht alle besprechen wollen. Wir begnügen uns vorerst mit den beiden wichtigsten. Die Prototypen lauten:

```
int printf( const char *format, ... );
int scanf( const char *format, ... );
```

Beide Funktionen erwarten als ersten Parameter eine Zeichenkette (den Formatstring) und geben einen Wert mit dem Typ `int` zurück. An den jeweils drei Punkten sehen wir auch, dass wir zusätzliche Parameter übergeben können.

`printf` gibt den Text im Formatstring am Bildschirm aus. Dabei wird der Text nach Felddefinitionen durchsucht und, falls gefunden, der nächste Parameter gelesen, formatiert und statt der Felddefinition ausgegeben.

Eine Felddefinition beginnt immer mit einem Prozentzeichen und endet mit dem Typzeichen. Entsprechend der Reihenfolge der Felddefinitionen im Formatstring müssen Sie die zusätzlichen Parameter nach dem Formatstring mit übergeben. Folgende Typzeichen sind möglich:

d oder i	int	Ein vorzeichenbehafteter integer wird als Dezimalzahl ausgegeben.
u	int	Ein vorzeichenloser integer wird als Dezimalzahl ausgegeben.
o	int	Ein vorzeichenloser integer wird als Oktalzahl ausgegeben.
x oder X	int	Ein vorzeichenloser integer wird als Hexzahl ausgegeben. Bei kleinen x wird für die Hexziffern a-f Kleinbuchstaben benutzt, bei großem X sind es entsprechend große Ziffern.

f	double	Eine Fließkommazahl vom Typ double. Sie wird in der Form [-]ddd.ddd ¹ ausgegeben, also ohne Exponenten. Beispiel 512.5486
e	double	Eine Fließkommazahl vom Typ double. Sie wird in der Form [-]d.dddde[+/-]ddd ausgegeben, also mit Exponenten. Beispiel: 5.125486e003
g	double	Eine Fließkommazahl. Das Ausgabeformat wird jedoch an Hand des Wertes bestimmt.
E oder G	double	Wie e oder g, jedoch wird für den Exponenten der Großbuchstabe E verwendet.
c	char	Ein einzelnes Zeichen
s	char *	Eine Zeichenkette
p	void *	Ein Zeiger. Dieser wird in der Regel im Hexzahlenformat ausgegeben.

Die zweite Spalte gibt an, welchen Typ der entsprechende zusätzliche Parameter haben muss. Dabei ist es bei int unerheblich, ob Sie einen signed oder unsigned int übergeben. Printf formatiert den Parameter so, wie es das Typzeichen vorgibt. Dies kann unerwartete Ergebnisse bewirken. Die Zeichenkette bei s wird von printf nicht verändert, daher dürfen Sie auch bedenkenlos einen const Zeiger übergeben. Der Zeigertyp bei p ist unerheblich. Sie dürfen jeden beliebigen Zeiger übergeben. Hier nun ein einfaches Beispiel:

```
/* printf1.c */
#include <stdio.h>

int main( void ) {
    char        zeichen = 'A';
    int          zahl = 4711;
    unsigned     oktal = 4711;
    const char   *text = "Guten Abend";
    double       pi = 3.14;

    printf( "zeichen = %c\n", zeichen );
    printf( "zahl = %d\n", zahl );
    printf( "oktal = %o\n", oktal );
    printf( "text = %s\n", text );
    printf( "pi = %f\n", pi );

    return 0;
}
```

Zusätzlich zu den bisher gezeigten Typzeichen können Sie auch die Zeichen % und n angeben. Bei Prozent wird kein zusätzlicher Parameter erwartet, sondern stattdessen das Prozentzeichen selber ausgegeben.

Mit dem Typzeichen n müssen Sie die Adresse einer Variablen vom Typ int übergeben. In diesem Fall wird nichts am Bildschirm ausgegeben, sondern stattdessen schreibt printf die Anzahl der bisher ausgegebenen Zeichen in Ihre

¹ d steht hierbei für eine beliebige Ziffer.

Variable. Ich habe das bis jetzt noch nie gebraucht und mir fällt auch kein sinnvoller Zweck ein. `printf` liefert nämlich die Anzahl der insgesamt ausgegebenen Zeichen zurück und das kann schon eher von Bedeutung sein.

Da es von einigen Datentypen `int` und `double` verschiedene Größen (Wertebereiche bzw. Genauigkeiten) gibt, müssen Sie diese `printf` ebenfalls mitteilen können. Andernfalls wären Sie nicht in der Lage, Variablen vom Typ `long int` fehlerfrei auszugeben. Zu diesem Zweck gibt es noch den Größenmodifizierer, den Sie direkt vor dem Typzeichen angeben können.

Größenmodifizierer	Anwendbar auf	Erwartete Größe
<code>h</code>	<code>int</code>	<code>short int</code>
<code>l</code>	<code>int</code>	<code>long int</code>
<code>L</code>	<code>double</code>	<code>long double</code>
<code>ll</code>	<code>int</code>	<code>long long int</code>
<code>z</code>	<code>int</code>	<code>size_t</code>
<code>j</code>	<code>int</code>	<code>intmax_t</code>
<code>t</code>	<code>int</code>	<code>ptrdiff_t</code>

Die farblich hinterlegten Modifizierer wurden mit C99 eingeführt.

Beispiel:

```
long int variable = 25658754L;
printf( „%ld\n“, variable );
```

Zusätzlich können Sie einige weitere Modifikationen vornehmen. Die genaue Syntax einer Felddefinition lautet `%[option][breite][.präzision][größe]typ`. Typ und Größe haben wir schon besprochen. Außer dem Typ und dem Prozentzeichen sind alle Angaben optional, sie müssen aber in dieser Reihenfolge erscheinen. Für die Optionen können sie folgende Angaben machen:

- Der Wert wird linksbündig ausgegeben.
- + Bei positiven Zahlen wird auch ein Pluszeichen + ausgegeben.
- Leerzeichen Bei positiven Zahlen wird auch ein Leerzeichen statt dem Vorzeichen ausgegeben.
- 0 Zahlen werden entsprechend der Feldgröße mit 0 statt mit Leerzeichen aufgefüllt (002 statt 2)
- # Es wird ein alternatives Format verwendet:
 Oktalzahlen (Typ `o`) wird eine 0 vorangestellt.
 Hexzahlen (Typ `x` oder `X`) wird `0x` oder `0X` vorangestellt.
 Bei Fließkommazahlen mit den Typen `e`, `E` und `f` wird immer ein Dezimalpunkt mit ausgegeben, auch wenn keine Nachkommastellen folgen.
 Bei Fließkommazahlen mit den Typen `g` und `G` wird immer die Exponentialdarstellung benutzt aber nachfolgende Nullen nicht entfernt.

Für die Breite kann eine beliebige positive Zahl angegeben werden. Alternativ kann aber auch ein * angegeben werden, und die Breite wird aus dem nächsten Parameter, der vom Typ int sein muss, gelesen. Wenn der Wert breiter ist, wird nicht abgeschnitten sondern das Feld wird einfach breiter gemacht. Wenn er kürzer ist, so wird das Feld entsprechend der Angabe in Flags aufgefüllt. Wenn keine Breite angegeben wird, so wird das Feld so breit wie erforderlich um den Wert darzustellen.

Mit Hilfe der Präzision geben Sie die Anzahl der Nachkommastellen bzw. die Anzahl der signifikanten Stellen bei Fließkommazahlen an. Wie bei der Breite kann auch hier der Wert * angegeben werden und die Präzision über die Parameterliste übergeben werden.

```
/* printf2.c */
#include <stdio.h>

int main( void ) {
    int          i, breite = 20;

    double        pi = 3.14159265358979323846;
    double        zahlen[] =
    {
        5.256,
        3.14159265358979323846,
        2.71828182845904523536,
        10768.5478,
        1000
    };

    for( i=0; i<10; ++i ) {
        printf( "%20f %20g %20e\n", pi, pi, pi );

        pi *= 10;
    }

    pi = 3.14159265358979323846;
    for( i=0; i<10; ++i ) {
        printf( "%*f %*g %*e\n", breite, pi, breite, pi, breite, pi );

        pi /= 10;
    }

    pi = 3.14159265358979323846;
    for( i=0; i<10; ++i )
        printf( "%.*f\n", i, pi );

    for( i=0; i<sizeof( zahlen ) / sizeof zahlen[0]; ++ i )
        printf( "%20.3f %20g %#20g\n", zahlen[i], zahlen[i], zahlen[i] );

    return 0;
}
```

Das Gegenstück zu printf – scanf – erwartet ebenfalls einen Formatstring als ersten Parameter und entsprechend den Felddefinitionen zusätzliche Parameter. Im einfachsten Fall enthält der Formatstring ausschließlich Felddefinitionen, die ähnlich

aufgebaut werden wie bei printf. Statt einem Wert der formatiert wird, müssen Sie hier allerdings die Adresse einer Variablen angeben, in dem der eingegebene Wert gespeichert wird. Ich schreibe bewusst Adresse statt einem Zeiger, obwohl dies syntaktisch ebenso korrekt ist, da Anfänger gerne hier einen nicht initialisierten Zeiger übergeben, der zu unerwarteten Ergebnissen führen kann. Wenn Sie die Adresse vorher in einem Zeiger gespeichert haben, ist dies allerdings kein Problem. scanf sieht letztendlich intern auch nur einen Zeiger. Denken Sie aber daran, dass dieser eine korrekte Adresse enthalten muss. Ein kleines erstes Beispiel:

```
/* scanf1.c */
#include <stdio.h>

int main( void ) {
    int alter;

    puts( "Wie alt sind Sie?" );
    scanf( "%d", &alter );
    printf( "Sie sind also %d Jahre alt.\n", alter );

    return 0;
}
```

Beim Einlesen eines Feldes geht scanf aber wie folgt vor. Alle Leerzeichen werden ignoriert (außer beim Typzeichen c), dann werden alle Zeichen eingelesen, die für die Formatierung verwendet werden können, und gemäß der Felddefinition, in den gewünschten Wert umgewandelt und gespeichert. Mit Leerzeichen ist aber nicht nur einfach die Leertaste gemeint, sondern auch Tabulator, Zeilenvorschub, Wagenrücklauf und ähnliche gemeint.

Trifft scanf auf ein Zeichen, das nicht umgewandelt werden kann, (z.B. weil ein Buchstabe, statt einer Zahl eingegeben wurde, beendet scanf die Umwandlung und gibt die Anzahl der bis dahin gelesenen Felder zurück. Dabei wird das ungültige Zeichen im Eingabepuffer stehen gelassen.

Für die Abfrage einer Zahl ist daher das oben gezeigte Beispielprogramm ungeeignet. Erst recht, wenn wir mehrere Werte vom Benutzer erfragen, gibt es Probleme, wenn wir auf Falscheingaben nicht reagieren. Eine bessere Variante prüft daher auch den Rückgabewert von scanf und warnt den Benutzer davor, fehlerhafte Eingaben zu tätigen. Bei der Gelegenheit prüfen wir auch, ob die Verbindung abgebrochen wurde, indem wir unser Programm beenden.

```
/* scanf2.c */
#include <stdio.h>

static void leerePuffer( void ) {
    int c;
    while( (c=getchar()) != EOF && c != '\n' )
        ;
}
```

```
static int eingabe( const char *prompt ) {
    int      eingabeWert;
    char      abschluss;
    int      scanfWert;

    puts( prompt );

    while( 1 ) {
        scanfWert = scanf( "%d%c", &eingabeWert, &abschluss );
        if( scanfWert == EOF ) {
            eingabeWert = -1;
            break;
        }
        else if( !scanfWert || eingabeWert < 0 || abschluss != '\n' ) {
            puts( "Sie haben einen falschen Wert eingegeben!" );
            leerePuffer();
        }
        else
            break;
    }
    return eingabeWert;
}

int main( void ) {
    int  alter;

    alter = eingabe( "Wie alt sind Sie?" );

    if( alter >= 0 )
        printf( "Sie sind also %d Jahre alt.\n", alter );
    else
        puts( "Spielverderber" );
    return 0;
}
```

Die Funktion `leerePuffer` löscht alle vorhandenen Zeichen im Eingabepuffer bis zur Returnntaste. Sie sollte nur aufgerufen werden, wenn sich tatsächlich noch Zeichen im Eingabepuffer befinden. Andernfalls werden wir vom Benutzer eine Eingabe erwarten, die wir gleich wieder verwerfen. Ein Ende der Eingabe mit EOF berücksichtigen wir zwar auch, geben diese Information aber nicht weiter, weil `scanf` dieses und auch noch einmal mitteilen würde.

Die Funktion `eingabe` erwartet eine Zeichenkette als Parameter, welche sie als erstes als Prompt ausgibt. Anschließend wird in einer Schleife der Benutzer solange nach einer Zahl gefragt, bis er den gewünschten Wert eingibt. Wir prüfen auch das Zeichen hinter dem Wert, damit wir sicherstellen können, dass auch dort kein ungültiges Zeichen steht. Bei korrekter Eingabe muss hier die abschließende Returnntaste kommen.

Wenn `scanf` das Ende der Verbindung erkannt hat. Speichern wir eine -1 in unsere Variable, die wir zurückliefern, um den Aufrufer (also die Funktion `main`) dieses zu signalisieren.

Wenn `scanf` keinen Wert interpretieren konnte, der Benutzer eine negative Zahl eingegeben hat oder nach der Zahl noch ein weiteres Zeichen eingegeben wurde. Zeigen wir eine Fehlermeldung an und bleiben in der Schleife.

Ist alles korrekt, befindet sich in unserer Variablen, das geforderte Alter und wir können die Schleife beenden und den Wert zurückliefern.

Die Implementierung von `main` führt nichts neues ein und sollte daher keine weiteren Erklärungen brauchen.

Das genaue Format einer Felddefinition bei `scanf` sieht so aus: `%[*][breite][größe]typ`. Größe und Typ entsprechen der jeweiligen Beschreibungen bei `printf`. Beim Typ `n` wird die Anzahl der bis dahin gelesenen Zeichen gespeichert. Beim Typ `s` wird bis zum ersten Leerzeichen gelesen. Die Breite gibt an, wie viele Zeichen maximal für dieses Feld gelesen werden dürfen. Wenn nach dem Prozentzeichen ein Stern (*) folgt, wird das Feld war aus dem Eingabepuffer gelesen und interpretiert, der konvertierte Wert wird aber nicht gespeichert.

Für alle anderen Zeichen im Formatstring gilt:

Ist das Zeichen ein Leerzeichen werden alle Leerzeichen im Eingabepuffer überlesen. Für alle anderen Zeichen muss auch genau dieses Zeichen im Eingabepuffer gelesen werden. Andernfalls bricht `scanf` ab.

Für die Eingabe von der Tastatur ist es unüblich, den Stern (*), eine Feldbreite oder ein Zeichen außerhalb einer Felddefinition zu verwenden, da wir somit die korrekte Eingabe unnötig erschweren. Beim Lesen aus einer Datei kann dies aber durchaus sinnvoll sein. `scanf` erlaubt zwar selbst nicht das Lesen aus einer Datei, die üblichen Betriebssysteme erlauben aber die Umleitung der Tastatureingaben und Bildschirmausgaben von bzw. zu einer Datei, so dass dies auch hier sinnvoll sein kann.

In der Headerdatei `stdio.h` gibt es noch verschiedene weitere Deklarationen von Abwandlungen von `scanf` und `printf`, die wir aber auf ein späteres Kapitel verschieben wollen.

d. `fopen/fclose`

Nun haben wir die wichtigsten Funktionen zur Tastatureingabe und Bildschirmausgabe kennengelernt. Häufig müssen wir aber Daten aus Dateien lesen und/oder schreiben. Dateien gehören zu den sogenannten Ressourcen, die wir vom System anfordern, und, wenn wir fertig sind, wieder freigeben müssen. Für die Anforderung einer Datei gibt es die Funktion `fopen` und die Freigabe geschieht mit der Funktion `fclose`. Die Prototypen lauten

```
FILE *fopen( const char *filename, const char *mode );
int fclose( FILE *handle );
```

`fopen` erwartet als ersten Parameter eine Zeichenkette mit den Dateinamen und als zweiten Parameter einen Modus. Der Aufbau eines Dateinamens ist Betriebssystem-abhängig. Meistens wird der Dateiname aber vom Benutzer erfragt, so dass es in seiner Verantwortung liegt, korrekte Dateinamen anzugeben. In unseren Beispielen

werden wir Dateinamen verwenden, die praktisch alle System verstehen.

Der Modusparameter ist ebenfalls eine Zeichenkette und beschreibt, welche Operationen wir planen auszuführen und welche Bedingungen die Datei erfüllen muss:

"r"	Datei wird zum Lesen geöffnet, sie muss schon existieren.
"w"	Datei wird zum Schreiben geöffnet. Wenn sie nicht existiert, wird sie erzeugt andernfalls wird der komplette Inhalt gelöscht.
"a"	Datei wird zum Lesen und Schreiben geöffnet. Neue Daten werden aber ans Ende der Datei angehängt.
"r+"	Datei wird zum Lesen und Schreiben geöffnet, sie muss schon existieren.
"w+"	Datei wird zum Lesen und Schreiben geöffnet. Wenn sie nicht existiert, wird sie erzeugt andernfalls wird der komplette Inhalt gelöscht.
"a+"	Datei wird zum Lesen und Schreiben geöffnet. Neue Daten werden aber ans Ende der Datei angehängt.

In allen Fällen werden die Dateien im Textmodus geöffnet. D.h. Wir erzeugen eine Datei, die mit einem normalen Texteditor bearbeitet werden kann. Die bewirkt beispielsweise bei manchen Systemen, dass beim Schreiben eines Newlinezeichens '\n' automatisch ein Zeilenvorschub '\r' eingefügt wird. Wenn dies nicht gewünscht ist, hängen Sie dem Modus ein b an, um die Datei im Binärmodus zu öffnen.

fopen liefert im Erfolgsfall die Adresse einer Variablen mit dem Typ FILE („file handle“ oder „file pointer“) zurück, die Sie in einer entsprechenden Zeigervariablen speichern müssen. Im Fehlerfall wird die Konstante NULL zurückgeliefert. Prüfen Sie in jedem Fall den Rückgabewert bevor Sie den Zeiger verwenden.

fclose schließt die Datei. Nach dem Schließen, dürfen Sie Ihren Zeiger nicht mehr weiter verwenden, außer zum Öffnen einer anderen Datei. Ihr Quelltext sollte immer in etwa so ausschauen:

```
FILE *fp = fopen( "meineDatei.txt", "w" );
if( fp ) {
    ...
    fclose( fp );
}
else
    puts( "Kann Datei meineDatei.txt nicht öffnen." );
```

Wenn ich in C eine Datei öffne, schreibe ich immer zuerst so ein Codefragment, um sicherzustellen, dass die wesentlichen Funktionsaufrufe nicht fehlen. Wenn ich mehrere Dateien öffnen muss, wird die zweite Datei immer im Erfolgsblock der vorhergehenden Datei geöffnet und der Code entsprechend eingerückt. Dadurch sieht man schon von weitem, dass der Code alles wichtige enthält:

```
FILE *fp = fopen( "meineDatei.txt", "r" );
if( fp ) {
    FILE *fp2 = fopen( "andereDatei.txt", "w" );
    if( fp2 ) {
        ...
        fclose( fp2 );
    }
    else
        puts( "Kann Datei andereDatei.txt nicht öffnen." );
    fclose( fp );
}
else
    puts( "Kann Datei meineDatei.txt nicht öffnen." );
```

Tipp: Seien Sie bei Fehlermeldungen nicht zu geizig mit Informationen für den Benutzer. Zu häufig habe ich schon Fehlermeldungen lesen müssen wie „cannot open file.“ und mich dann gefragt, welche Datei das Programm wohl gerade öffnen wollte. Für weitere Informationen zu diesem Problem folgt später ein eigenes Kapitel.

e. fprintf/fscanf

Haben Sie eine Datei im Textmodus geöffnet, können Sie die Funktionen fprintf oder fscanf zum Schreiben oder Lesen verwenden. Die Prototypen sind:

```
int fprintf( FILE *fp, const char *format, ... );
int fscanf( FILE *fp, const char *format, ... );
```

Diese beiden Funktionen arbeiten exakt identisch zu ihren Pendants für die Tastatureingabe und Bildschirmausgabe, nur dass Sie hier einen „file pointer“ angeben müssen. Wenn Sie die Datei nur zum Lesen geöffnet haben jedoch versuchen zu schreiben, wird fprintf mit EOF antworten. Umgekehrt macht dies fscanf, wenn die Datei nur zum Schreiben geöffnet wurde.

Beide Funktionen können auch aufgerufen werden, wenn die Datei im Binärmodus geöffnet wird. Empfehlenswert ist das allerdings nicht, da in den meisten Fällen das Ergebnis unbrauchbar sein wird.

```
/* fscanf.c */
#include <stdio.h>

static void leerePuffer( void ) {
    int c;
    while( (c=getchar()) != EOF && c != '\n' )
        ;
}
```

```
static int eingabe( const char *prompt ) {
    int      eingabeWert;
    char abschluss;
    int      scanfWert;

    puts( prompt );

    while( 1 ) {
        scanfWert = scanf( "%d%c", &eingabeWert, &abschluss );
        if( scanfWert == EOF ) {
            eingabeWert = -1;
            break;
        }
        else if( !scanfWert || eingabeWert < 0 || abschluss != '\n' ) {
            puts( "Sie haben einen falschen Wert eingegeben!" );
            leerePuffer();
        }
        else
            break;
    }
    return eingabeWert;
}

#define DATEINAME      "letzteAngabe.txt"
#define DATEIFORMAT    "Alter: %d\n"

int main( void ) {
    int  alter,
        letztesAlter;

    FILE *fp = fopen( DATEINAME, "r" );
    if( fp ) {
        if( !fscanf( fp, DATEIFORMAT, &letztesAlter ) ) {
            puts(
                "Dateiformat von " DATEINAME " fehlerhaft.\n"
                "Datei wird ignoriert."
            );
            letztesAlter = 0;
        }
        fclose( fp );
    }
    else
        letztesAlter = 0;
}
```

```

alter = eingabe( "Wie alt sind Sie?" );
if( alter >= 0 ) {
    printf( "Sie sind also %d Jahre alt.\n", alter );
    if( alter < letztesAlter )
        puts( "Sie Schlingel! Jungbrunnen erfunden?" );
    else if( alter > letztesAlter ) {
        FILE *fp = fopen( DATEINAME, "w" );
        if( fp ) {
            if( fprintf( fp, DATEIFORMAT, alter ) == EOF ) {
                puts(
                    "Datei " DATEINAME " konnte nicht geschrieben "
                    "werden.\n"
                    "Festplatte voll?"
                );
            }
            fclose( fp );
        }
        else {
            puts(
                "Datei " DATEINAME " konnte nicht erstellt "
                "werden."
            );
        }
    }
}
else
    puts( "Spielverderber" );
return 0;
}

```

f. fputs/fgets

Analog zu den Funktionen `gets` und `puts` gibt es für die Dateiein- und -ausgabe die Funktionen `fputs` und `fgets`.

Die Prototypen lauten:

```

int fputs(const char *s, FILE *stream);
char *fgets(char *s, int size, FILE *stream);

```

Während `fprintf` und `fscanf` sich exakt identisch zu ihren Bildschirm- bzw. Tastaturpendants verhalten, gibt es bei `fputs` und `fgets` kleine Unterschiede.

Der Unterschied bei `fgets` fällt sofort auf, da diese Funktion die Länge des Zeichenarrays als Parameter möchte. Bei ihr entfällt daher das Problem des Pufferüberlaufs von `gets`.

Bei `fputs` sieht man den Unterschied nicht am Prototypen. Während `puts` nach der Ausgabe der Zeichenkette noch ein Line Feed und Carriage Return ausgibt, wird dies `fputs` nicht machen.

g. fputc/putc/fgetc

Wie Sie sich vielleicht denken können, gibt es auch für das zeichenweise Lesen und Schreiben von Daten Pendanten zu den Bildschirm- bzw. Tastaturversionen. Leider

wurden sie nicht einfach umbenannt indem ein f vorangestellt wurde und dann gibt es auch jeweils zwei verschiedene Versionen. Die Prototypen lauten:

```
int fputc( int c, FILE *stream );
int putc( int c, FILE *stream );
int fgetc( FILE *stream );
int getc( FILE *stream );
```

fputc und putc entsprechen der bekannten Funktion putchar und fgetc bzw. getc entsprechen getchar. Während fputc und fgetc als Funktionen implementiert sein müssen, dürfen putc und getc als Präprozessormakro definiert sein. Dies kann, wie wir im Abschnitt Präprozessor gesehen haben, unerwünschte Folgen haben.

h. fwrite/fread

Zum Schreiben und Lesen von binären Daten gibt es die Funktionen fwrite und fread. Beim Schreiben von Binärdaten werden diese in einem Format gespeichert, die ohne Konvertierung auskommt und nach dem Lesen direkt weiterverarbeitet werden können. Dies hat den Vorteil, dass aufwendige Konvertierungen entfallen können. Der bedeutende Nachteil aber ist, dass diese Daten nur von Ihrem Programm problemlos weiterverarbeitet werden können. Übersetzen Sie Ihr Programm auf einen Rechner mit anderer Architektur (z.B. mit Power PC CPU statt Intel x86 CPU oder anderer Datenbreite (64 Bit statt 32 Bit) oder anderen Compilerschaltern (Alignment), können Sie für gewöhnlich die Daten nicht mehr verarbeiten und müssen diese konvertieren.

Die Prototypen dieser beiden Funktionen sehen so aus:

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

Der erste Parameter ist die Adresse des Speichers (Variable), dessen Inhalt Sie in der Datei speichern bzw. in das Sie lesen wollen. Der zweite Parameter gibt die Größe eines Datensatzes an. Verwenden Sie hier unbedingt den sizeof operator, um zu garantieren, dass die richtige Größe verwendet wird. Der nächste Parameter gibt die Anzahl der Datensätze an, die geschrieben bzw. gelesen werden sollen. Bei einem Array geben Sie hier die Anzahl der Elemente an, bei einer Variablen einfach die Konstante 1. Der letzte Parameter ist das Dateihandle, das Sie von fopen erhalten haben.

Wichtig: Die Datei muss im Binärmodus geöffnet worden sein. Andernfalls können Sie fehlerhafte Dateien erzeugen. Unixartige Betriebssysteme werden i.d.R. keine Unterschiede machen, so dass es passieren kann, dass Sie diesen Fehler nicht bemerken. Portieren Sie Ihr Programm dann nach Windows, kann es sein, dass Sie diesen Fehler schnell feststellen werden.

fwrite und fread liefern die Anzahl der Datensätze zurück, die erfolgreich gelesen bzw. geschrieben werden konnten. Der Rückgabewert sollte nie ignoriert werden.

Das folgende Beispiel ist eine Modifikation des fprintf/fscanf Beispiels des vorherigen Abschnitts. Da nur die Funktion main geändert wurde, wird hier nur diese abgebildet:

```

/* fread.c */
...
#define DATEINAME "letzteAngabe.dat"

int main( void ) {
    int alter,
        letztesAlter;

    FILE *fp = fopen( DATEINAME, "rb" );
    if( fp ) {
        if( fread( &letztesAlter, sizeof( letztesAlter ), 1, fp ) != 1 ) {
            puts(
                "Dateiformat von " DATEINAME " fehlerhaft.\n"
                "Datei wird ignoriert."
            );
            letztesAlter = 0;
        }
        fclose( fp );
    }
    else
        letztesAlter = 0;

    alter = eingabe( "Wie alt sind Sie?" );
    if( alter >= 0 ) {
        printf( "Sie sind also %d Jahre alt.\n", alter );
        if( alter < letztesAlter )
            puts( "Sie Schlingel! Jungbrunnen erfunden?" );
        else if( alter > letztesAlter ) {
            FILE *fp = fopen( DATEINAME, "wb" );
            if( fp ) {
                if( fwrite( &alter, sizeof( alter ), 1, fp ) != 1 ) {
                    puts(
                        "Datei " DATEINAME " konnte nicht geschrieben "
                        "werden.\n"
                        "Festplatte voll?"
                    );
                }
                fclose( fp );
            }
            else {
                puts(
                    "Datei " DATEINAME " konnte nicht erstellt "
                    "werden."
                );
            }
        }
    }
    else
        puts( "Spielverderber" );
    return 0;
}

```

i. Weitere Funktionen

Es folgen einige weitere Funktionen, die hier kurz beschrieben werden:

```
int ferror( FILE *fp );
```

Liefert den Wert 0 zurück, wenn beim letzte Schreib- oder Lesevorgang kein Fehler passiert ist, andernfalls ein Wert != 0.

```
int feof( FILE *fp );
```

Liefert den Wert 0 zurück, wenn beim letzte Schreib- oder Lesevorgang kein Dateiende erkannt wurde, andernfalls ein Wert != 0.

```
void clearerr( FILE *fp );
```

Löscht sowohl den Fehler- als auch den EOF-zustand. Wird aber die Ursache des Fehlers nicht korrigiert, wird der nächste Schreib- oder Leseversuch, diesen Fehler sofort wieder melden.

```
int ungetc( int c, FILE *fp );
```

Mit ungetc kann ein einmal gelesenes Zeichen wieder zurück in den Eingangspuffer geschrieben werden. Beim nächsten Lesevorgang wird dann zuerst diese Zeichen wieder gelesen. Diese Funktion ist für mehrstufige Parser praktisch, da eine Teilfunktion des Parsers, wenn sie ein Zeichen erkennt, das sie nicht interessiert, dieses wieder zurückschreiben kann und somit für die nächste Stufe des Parsers wieder verfügbar ist.

```
int c;
while( 1 ) {
    c = fgetc( fp );
    if( c == EOF )
        break;
    if( c >= '0' || c <= '9' ) {
        ungetc( c, fp );
        break;
    }
    ...
}
```

Dieses Beispiel verarbeitet eine Datei solange bis es eine Ziffer findet und überlässt diese der nächsten Stufe.

```
long ftell( FILE *fp );
int fgetpos( FILE *fp, fpos_t *pos );
int fseek( FILE *fp, long offset, int whence );
int fsetpos( FILE *fp, const fpos_t *pos );
void rewind( FILE *fp );
```

Diese fünf Funktionen behandeln die Position des nächsten Schreib- oder Lesevorgangs in der Datei. Mit ftell oder fgetpos erhalten Sie die aktuelle Position. Ftell wird für Dateien benutzt, die im Binärmodus geöffnet wurden, fgetpos ist für Dateien im Textmodus. fgetpos erwartet hierfür die Adresse einer Variablen mit dem Typ fpos_t (ebenfalls in stdio.h definiert). Der Wert dieser Variablen darf nur für nachfolgende Aufrufe von fsetpos verwendet werden.

fseek kann für eine Positionierung an einer beliebigen Stelle innerhalb der Datei in Bytes erfolgen. Im zweiten Parameter „offset“ geben Sie an, um wie viele Bytes der Positionszeiger verschoben werden soll. Negative Werte verschieben die Position in

Richtung Dateianfang, positive Werte in Richtung Ende. Im zweiten Parameter „whence“ geben Sie eine der vordefinierten Konstanten, um anzugeben, ab welcher Stelle. SEEK_SET ab den Anfang der Datei (offset muss dann positiv sein), SEEK_CUR ab der aktuellen Position (offset darf positiv und negativ sein), SEEK_END ab dem Ende der Datei (offset muss negativ sein).

Die Funktion rewind setzt den Positionszeiger an den Anfang der Datei. In allen Fällen, die den Positionszeiger ändern, werden Fehler- und EOF-Zustände wieder zurückgesetzt.

Häufig werden fseek und ftell dazu verwendet, die Größe einer Datei zu ermitteln:

```
long getFileSize( const char *fileName ) {
    long size;
    FILE *fp = fopen( fileName, "rb" );
    if( fp ) {
        fseek( fp, 0, SEEK_END );
        size = ftell( fp );
    }
    else
        size = -1;
    return size;
}
```

Dies wird häufig funktionieren. Die Funktion versagt aber, wenn die Datei größer ist wie der maximale Wertebereich von long int erlaubt. ANSI-C und auch der POSIX-Standard, der von vielen Compilern unterstützt wird, bieten hier leider keine vernünftige Lösung. Versuchen Sie also nach Möglichkeit ohne die Dateigröße auszukommen.

```
FILE *stdin, *stdout, *stderr;
```

Diese drei vordefinierten, globalen Variablen enthalten Dateihandles, die automatisch beim Start Ihres Programms geöffnet werden. stdin dient zum Lesen von Tastatur, stdout und stderr zum Schreiben am Bildschirm. stdout wird für die normale Ausgabe verwendet und stderr bei Fehlermeldungen. Da viele Betriebssysteme die getrennte Umleitung in verschiedene Dateien erlauben, können so die Benutzer diese getrennt speichern lassen.

Mit diesen Variablen können die Standardfunktionen für die Bildschirmausgabe und Tastatureingabe leicht selbst implementiert werden:

```
/* stdio.c */
#include <stdio.h>

int puts( const char *str ) {
    int result = fputs( str, stdout );

    if( result != EOF )
        result = fputc( '\n', stdout );
    return result;
}
```

```
int putchar( int c ) {  
    return fputc( stdout, c );  
}
```

```
int getchar( void ) {  
    return fgetc( stdin );  
}
```

gets habe ich ausgelassen, weil das eh nicht verwendet werden soll. printf und scanf lass ich weg, weil für deren Implementierung uns noch nicht alle erforderlichen Techniken bekannt sind.

```
FILE *freopen(const char *filename, const char *mode, FILE *stream);
```

Diese Funktion ist dazu da, ein bereits geöffnetes Dateihandle in eine andere Datei umzuleiten. Normalerweise wird die nur für die drei Standardhandles stdin, stdout und stderr benötigt. Der Rückgabewert muss nicht gespeichert werden, da im Erfolgsfall das selbe Handle wieder zurückgegeben wird. Es genügt also, wenn auf NULL geprüft wird:

```
/* freopen.c */  
#include <stdio.h>  
  
int main( void ) {  
    if( freopen( "ausgabe.log", "w", stdout ) )  
        puts( "Hallo Welt" );  
    else  
        fputs( "Fehler bei der Umleitung", stderr );  
    return 0;  
}
```

Dieses Beispiel schreibt des Text „Hello World“ nicht auf dem Bildschirm, obwohl die Bildschirmfunktion verwendet wird.

```
int setvbuf( FILE *fp, char *buf, int type, size_t size );  
void setbuf( FILE *fp, char *buf );  
int fflush( FILE *fp );
```

Im Abschnitt für die Bildschirmausgabe und Tastatureingabe habe ich schon erwähnt, dass alle Funktion mit einem Puffer arbeiten und erst wenn dieser voll ist (beim Schreiben) oder leer ist (beim Lesen) findet ein tatsächlicher Schreib- bzw. Lesevorgang statt.

Mit diesen Funktionen kann gesteuert werden, wie die Pufferung durchgeführt werden muss:

Im Parameter fp wird das Dateihandle angegeben. In buf übergeben Sie die Adresse eines Speicherbereichs, der zu Pufferung benutzt werden soll. Übergeben Sie hier einen NULL-Zeiger, besorgt sich die Bibliothek selbst einen solchen Puffer, falls erforderlich. Im Parameter type geben Sie eine der drei möglichen Konstanten an, um die Art der Pufferung festzulegen: `_IOBF` Datei wird komplett gepuffert, `_IOLBF` Datei wird Zeilenweise gepuffert (Standard bei stdin, stdout und stderr), `_IONBF` keine Pufferung, jeder Lese- und Schreibvorgang greift sofort auf die Festplatte zu.

Dieser Modus ist selten sinnvoll und sollte nur in besonderen Fällen benutzt werden, da es die Leistung Ihres Programms extrem beeinträchtigen würde.

Die Funktion `flush` darf ausschließlich für Ausgabedateien aufgerufen werden. Es werden alle noch nicht gespeicherten Schreibvorgänge durchgeführt. Da `stdout` und `stderr` Zeilengepuffert arbeiten, müssen Sie diese Funktion aufrufen, wenn Sie unvollständige Zeilen ausgeben wollen und diese auf dem Bildschirm erscheinen sollen.

```
/* fflush.c */
#include <stdio.h>

static char *getline( char *data, int size ) {
    int c;
    int i = 0;

    if( data == NULL || size <= 0 )
        return NULL;

    size--; // platz für das 0 byte
    while( i < size ) { // wenn zu viel Zeichen Abbruch
        c = getchar();
        if( c == EOF || c == '\n' )
            break;
        data[i++] = (char)c;
    }

    /*
        wenn ende und noch nichts eingegeben wurde,
        signalisieren wir dies durch NULL
    */
    if( c == EOF && i==0 )
        return NULL;

    // Schreiben des 0 bytes
    data[i] = 0;
    return data;
}

int main( void ) {
    char text[80];

    printf( "%s", "Wie heißen Sie? " );
    fflush( stdout );
    if( getline( text, sizeof( text ) ) )
        printf( "%s %s\n", "Hallo", text );

    return 0;
}
```

Grundsätzlich können Sie alle C-Ein-/Ausgabefunktionen auch in C++ einsetzen. Es ist aber nicht garantiert, dass diese Funktionen noch von zukünftigen Standards abgedeckt sind. Darüber hinaus bietet C++ einige bessere Methoden, die auf die Art der C++-Entwicklung besser abgestimmt ist. Müssen Sie aber trotzdem diese Funktionen in Ihren C++-Quellen benutzen, verwenden Sie bitte statt der Headerdatei `<stdio.h>` die Headerdatei `<cstdio>`. In dieser Datei werden die gleichen Funktionen deklariert allerdings im namespace `std`. Hallo Welt würde dann so ausschauen:

```

/*  hallo.c: Ausgabe von Hallo Welt */
#include <cstdio>

int main() {
    std::puts( "Hallo Welt" );
}

```

8.2. Ein- und Ausgabe in C++ (iostream)

In C++ erfolgt die Ein-/Ausgabe über die Klassen und Funktionen die in `<iostream>` deklariert werden.

a. Handles

Im Gegensatz zu C müssen Sie in C++ das Handle für die Ein-/Ausgabe immer angeben. Daher werden Sie die Handles für die Standardgeräte häufiger brauchen wie in C. Deshalb werden wir hier die Standardhandles gegenüberstellen:

Legacyobjekt	C++	Verwendung
stdin	std::cin	Standardeingabe von der Tastatur
stdout	std::cout	Standardausgabe auf den Bildschirm
stderr	std::cerr	Standardausgabe auf den Bildschirm speziell für Fehlermeldungen.

Die Legacyobjekte können Sie ganz normal wie im vorigen Kapitel verwenden.

Gibt es in C nur den Typ FILE sowohl für Ein- als auch Ausgabe gibt es in C++ mehrere Klassen. Wir wollen hier erst mal nur die wichtigsten Typen betrachten. Zur Verbesserung der Übersichtlichkeit, lass ich hier den Namespace std weg.

Typ	Verwendung
istream	Allgemeiner Typ zum Lesen
ostream	Allgemeiner Typ zum Schreiben
fstream	Typ zum Lesen und Schreiben aus bzw. in einer Datei. Dieser Stream ist im Header <code><fstream></code> definiert.
ifstream	Typ zum Lesen aus einer Datei. Dieser Stream ist im Header <code><fstream></code> definiert.
ofstream	Typ zum Schreiben in eine Datei. Dieser Stream ist im Header <code><fstream></code> definiert.
stringstream	Typ zum Lesen und Schreiben aus bzw. in einer Zeichenkette. Dieser Stream ist im Header <code><sstream></code> definiert.
istringstream	Typ zum Lesen aus einer Zeichenkette. Dieser Stream ist im Header <code><sstream></code> definiert.
ostringstream	Typ zum Schreiben in eine Zeichenkette. Dieser Stream ist im Header <code><sstream></code> definiert.

Die hier aufgelisteten Klassen sind Templateklassen, die aus den gleichnamigen Klassentemplates mit dem Prefix `basic_` für den Typ `char` definiert wurden. Für `wchar_t` gibt es eigene Instanzen des Klassentemplates, die wir uns aber im Rahmen dieses Buchs nicht näher anschauen wollen. Die wichtigsten Templates sind:

<code>basic_ostream</code>	<code>basic_istream</code>	<code>basic_iostream</code>
<code>basic_ofstream</code>	<code>basic_ifstream</code>	<code>basic_fstream</code>

Wenn Sie für einen eigenen Datentyp zum Schreiben in eine Datei eine Funktion definieren wollen, verwenden Sie den Datentyp `ostream`. Dieser kann nämlich auch mit `ofstream` versorgt werden aber nicht umgekehrt. Das ist, wie wir später sehen werden, von Bedeutung, weil es noch mehr Typen gibt, wie die hier aufgelisteten.

Vor der Benutzung eines Handles müssen Sie es öffnen. Das kann direkt mit dem Konstruktor des Handles erfolgen oder mit der Memberfunktion `open`. Zum Schließen eines Handles benutzen Sie entweder die Memberfunktion `close` oder lassen es beim beenden des Blocks, der Funktion oder des Programms automatisch mit dem Destruktor erledigen.

Wenn mein Öffnen einer Datei zum Lesen mit der Klasse `ifstream` die Datei nicht existiert, wird die Datei nicht geöffnet. Mit der Memberfunktion `is_open` kann geprüft werden, ob das Öffnen erfolgreich war. Beim Öffnen zum Schreiben mit `ofstream` wird die Datei erzeugt.

```
#include <iostream>
#include <fstream>

int main() {
    std::ifstream inp( "test.txt" );
    if( !inp.is_open() ) {
        std::cout << "Erzeuge Datei" << std::endl;
        std::ofstream out( "test.txt" );

        out << "1234" << std::endl;
        out.close();

        inp.open( "test.txt" );
    }
    int zahl;
    inp >> zahl;
    std::cout << "Die gelesene Zahl ist " << zahl << std::endl;
    return 0;
}
```

Beim Öffnen können Sie dem Handle noch einige Modusbits entweder im Konstruktor oder in der Methode `open` mitgeben.

<code>std::ios_base::app</code>	Neue Daten werden immer an das Ende der Datei angehängt.
---------------------------------	--

<code>std::ios_base::binary</code>	Datei wird im Binärmodus geöffnet. Das bedeutet unter Windows, dass bei der Ausgabe des Linefeed-zeichens <code>\n</code> kein Carriage Return angefügt wird.
<code>std::ios_base::in</code>	Datei wird zum Lesen geöffnet. Braucht bei <code>ifstream</code> nicht angegeben werden.
<code>std::ios_base::out</code>	Datei wird zum Schreiben geöffnet. Braucht bei <code>ofstream</code> nicht angegeben werden.
<code>std::ios_base::trunc</code>	Leert die Datei nach dem Öffnen.
<code>std::ios_base::ate</code>	Positioniert den Schreib/Lesezeiger nach dem Öffnen direkt an das Ende der Datei.
<code>std::ios_base::noreplace (C++23)</code>	Öffnet die Datei exklusiv. D.h. sie darf nicht von einem anderen Programm geöffnet sein.

b. Ausgabe mit `<<`, Eingabe mit `>>`

Für die C++-Ausgabehandles wurde der Linksshiftoperator `>>` überladen, so dass der im rechten Operanden angegebene Wert auf dem Kanal des im linken Operanden angegebenen Handles ausgegeben wird. Danach wird der Ausgabekanal zurück gegeben, so dass mit einem weiteren Shiftoperator ein weiterer Wert im selben Befehl ausgegeben werden kann.

```
std::cout << "Hello " << variable;
```

Für die meisten im Standard definierten Datentypen können Sie den Shiftoperator zusammen mit einem Ausgabekanal verwenden. Wenn die Verwendung nicht möglich ist, erhalten Sie beim Übersetzen eine Fehlermeldung. Das ist auch der Grund, weshalb ich die Ein-/Ausgabe mit C++ der alten Variante von C vorziehe, da die C++-version typischer ist. Viele moderne C-Compiler können zwar die Verwendung von `printf` etc. zur Übersetzung überprüfen, dies ist jedoch nicht im Standard gewährleistet und ich würde mich darauf auch nicht verlassen.

Ein weiterer großer Vorteil der `iostream` library ist, dass Sie für eigene Datentypen eigene Shiftoperatoren implementieren können, damit so ihre eigenen Typen genau wie die Standardintrinsic ausgegeben werden können.

```
inline std::ostream &operator << (std::ostream &theStream, const STRING
&theSource) {
    if( theSource.text ) {
        theStream << theSource.text->string;
    }

    return theStream;
}
```

Analog zum Linksshiftoperator kann für die Eingabe der Rechtsshiftoperator verwendet werden. Grundsätzlich gilt hier das gleiche, wie oben beschrieben. Insbesondere können Sie für eigene Typen auch eigene Funktionen zum Lesen

definieren.

c. IO-Manipulatoren

Für die formatierte Ausgabe ist es erforderlich, die Art der Umwandlung vor der Ausgabe zu manipulieren. Grundsätzlich werden zwar Zahlen vor der Ausgabe in eine für den Menschen lesbare Form gebracht, allerdings muss man manchmal, wie z.B. auch bei printf möglich, die Ausgabebreite festlegen. Einen Manipulator können Sie einfach ganz normal wie jede andere Variable ein bzw. ausgeben, indem Sie den Manipulator per Shiftoperator an das Handle übergeben:

```
std::cout << "Hello World" << std::endl;
```

Ein Manipulator kann sich auf den ganz Stream beziehen oder nur auf das nächste Element. Hier folgt nun eine Liste der wichtigsten Manipulatoren:

Header	Name	Verwendung
<ios>	boolalpha nonboolalpha	Legt fest, ob Variablen vom Typ bool als Text oder als Zahlenwert ausgegeben werden soll.
	fixed scientific hexfloat (c++11) defaultfloat (c++11)	Legt fest, wie Fließkommazahlen dargestellt werden.
<ostream>	ends	Gibt das Ende einer Zeichenkette (Bytewert 0) aus.
	flush	Flushed den Schreibpuffer auf das Ausgabegerät. Spätestens dann wird der Text sichtbar.
	endl	Schreibt ein Zeilenende und flushed den Schreibpuffer.
<iomanip>	setbase(n)	Legt die Basis für die Ein-/Ausgabe von Integerzahlen fest. Wenn der Manipulator nicht benutzt wird, so wird die Basis 10 angenommen.
	setfil(c)	Legt fest, welchen Zeichen zum Auffüllen von Feldern benutzt werden soll, wenn der Wert kürzer ist, wie die gewünschte Feldbreite. Wenn der Manipulator nicht benutzt wird, so wird die das Leerzeichen verwendet.
	setprecision(n)	Gibt die Präzision an
	setw(n)	Gibt die Breite des nächsten Feldes an.

Beispiel:

```
double pi = 3.141592653589793239  
std::cout << std::setprecision(4) << pi << std::endl;
```

Dies sind nur die wichtigsten Manipulatoren. Um eine aktuelle Auflistung aller Manipulatoren zu erhalten, konsultieren Sie bitte eine der Verfügbaren Onlinereferenzen.

d. Funktionen

Neben den Funktionen zum Öffnen und Schließen von Streams und den Operatoren zum formatierten Lesen und Schreiben gibt es noch weitere Funktionen. Hier wollen wir uns die wichtigsten anschauen.

Funktion	Benutzung
get	<code>int get();</code> Liest ein einzelnes Zeichen aus dem Stream.
peek	<code>int peek();</code> Wie get nur wird das Zeichen nicht aus dem Stream entfernt
unget	<code>basic_istream &unget();</code> Legt das zuletzt gelesene Zeichen zurück in den Stream. Da der Stream auch wieder zurückgegeben wird, kann dieser für andere Funktionen gleich verwendet werden.
putback	<code>basic_istream& putback(char_type ch);</code> Legt das Zeichen in den Eingabepuffer, so dass es beim nächsten Lesen ausgeliefert wird.
getline	<code>basic_istream& getline(char_type* s, std::streamsize count);</code> <code>basic_istream& getline(char_type* s, std::streamsize count, char_type delim);</code> Liest mehrere Zeichen aus den Eingabestream bis count Zeichen gelesen wurden, das Zeilenende erkannt wurde oder bis das Begrenzungszeichen delim erkannt wurde.
ignore	<code>basic_istream& ignore(std::streamsize count = 1, int_type delim = Traits::eof());</code> „Überliest“ die angegebene Anzahl Zeichen oder bis das Begrenzungszeichen delim erkannt wurde.
read	<code>basic_istream& read(char_type* s, std::streamsize count);</code> Liest mehrere Zeichen aus den Eingabestream bis count Zeichen gelesen wurden
tellg	<code>pos_type tellg();</code> Gibt die aktuelle Position des Lesezeigers innerhalb der Datei zurück.

Funktion	Benutzung
seekg	<pre>basic_istream& seekg(pos_type pos); basic_istream& seekg(off_type off, std::ios_base::seekdir dir);</pre> Positioniert den Lesezeiger innerhalb der Datei neu. Die zweite Methode positioniert den Zeiger relativ zum Anfang (dir == std::ios_base::beg), zum Ende (dir == std::ios_base::end) oder zur aktuellen Position (dir == std::ios_base::cur)
put	<pre>basic_ostream& put(char_type ch);</pre> Schreibt ein einzelnes Zeichen in den Outputstream.
write	<pre>basic_ostream& write(const char_type* s, std::streamsize count);</pre> Schreibt mehrere Zeichen in den Outputstream.
tellp	<pre>pos_type tellp();</pre> Gibt die aktuelle Position des Schreibzeigers innerhalb der Datei zurück.
seekp	<pre>basic_ostream& seekp(pos_type pos); basic_ostream& seekp(off_type off, std::ios_base::seekdir dir);</pre> Positioniert den Schreibzeiger innerhalb der Datei neu. Die zweite Methode positioniert den Zeiger relativ zum Anfang (dir == std::ios_base::beg), zum Ende (dir == std::ios_base::end) oder zur aktuellen Position (dir == std::ios_base::cur)
flush	<pre>basic_ostream& flush();</pre> Leert den Ausgabepuffer und schreibt alle Daten in den Outputstream. Das ist z.B. nützlich, wenn der Outoutstream zum Bildschirm geht oder andere Geräte steuert.
good	<pre>bool good() const;</pre> Liefert true zurück, wenn die letzte I/O-Operation erfolgreich war.
eof	<pre>bool eof() const;</pre> Liefert true zurück, wenn das Dateiende erreicht wurde.
fail	<pre>bool fail() const;</pre> Liefert true zurück, wenn am Ein- oder Ausgabestrom ein Formatierungsfehler festgestellt wurde.
bad	<pre>bool bad() const;</pre> Liefert true zurück, wenn am Ein- oder Ausgabestrom ein Fehler festgestellt wurde.
operator !	<pre>bool operator!() const;</pre> Wie fail()

Funktion	Benutzung
operator bool	operator bool() const; Wie !fail()

8.3. Ergänzungen

Im folgenden zeige ich eine Tabelle, die C-Variablen und -Typen deren jeweiligen C++-Pendants gegenüberstellt.

FILE	std::fstream std::ofstream std::ifstream
stdout	std::cout
stdin	std::cin
stderr	std::cerr

Die Memberfunktionen heißen sehr ähnlich zu den C-Pendants, einzig die formatierte Ein-/Ausgabe sieht wegen den Shiftoperatoren, die die C++-Klassen verwenden, etwas anders aus.

Wenn Sie Zahlen in ihrer Binärform speichern wollen, müssen Sie darauf achten, dass das Lesen nur auf einer kompatiblen Architektur funktionieren wird, falls Sie keine entsprechende Massnahmen ergreifen. Nach Möglichkeit sollten Sie nur Datentypen verwenden, deren Größe definiert und bekannt ist. Wenn Sie einen nicht zu alten Compiler verwenden, können Sie die Datentypen verwenden, die in `<stdint.h>` bzw. `<cstdint>` definiert wurden. Darüberhinaus sollten Sie einen „Magic“ an den Dateianfang schreiben, um sicherzustellen, dass die Datei mit einer kompatiblen Architektur geschrieben wurde. Auch ist das Schreiben der Größe der Daten vor den eigentlichen Daten ganz hilfreich, um unerwartetes Verhalten zu vermeiden.

```
#include <iostream>
#include <fstream>
#include <cstdint>

struct DATA {
    int64_t value;
};

int main( ) {
    int16_t magic = 0x1234;
    int16_t size = sizeof( DATA );
    DATA data;

    data.value = 12345678;
```

```

    {
        std::ofstream myFile("temp.gak", std::ios_base::binary);
        myFile.write( reinterpret_cast<const char*>(&magic),
sizeof( magic ) );
        myFile.write( reinterpret_cast<const char*>(&size),
sizeof( size ) );
        myFile.write( reinterpret_cast<const char*>(&data),
sizeof( data ) );
    }

    {
        int16_t new_magic;
        int16_t new_size;
        DATA new_data;
        std::ifstream myFile("temp.gak", std::ios_base::binary);
        myFile.read( reinterpret_cast<char*>(&new_magic),
sizeof( new_magic ) );

        if( new_magic == 0x3412 ) {
            std::cerr << "Falsche Endianess" << std::endl;
        }
        else if( new_magic != magic ) {
            std::cerr << "Falsches Magic" << std::endl;
        }
        else {
            myFile.read( reinterpret_cast<char*>(&new_size), sizeof( new_size ) );
            if( size != new_size ) {
                std::cerr << "Falsche Größe" << std::endl;
            }
            else {
                myFile.read( reinterpret_cast<char*>(&new_data),
sizeof( new_data ) );
                std::cout << "Gefunden: " << new_data.value <<
std::endl;
            }
        }
    }
    return 0;
}

```

9. C++ Templates

Wie schon in einem vorherigen Abschnitt beschrieben, dienen Templates als Vorlage für vom Compiler zu generierende Funktionen und Klassen. Daher spricht man auch von generischer Programmierung.

9.1. Definition

Das Schlüsselwort `template` leitet eine Vorlagendefinition ein. Anschließend folgen in spitzen Klammern die Templateparameter. Danach folgt die eigentliche Templatedefinition. Die Definition kann eine ganz normale Klasse oder Funktion sein. Neuere Standards erlauben auch Variable. Innerhalb der Definition können Sie die Templateparameter ganz normal verwenden, wie wenn Sie bereits bekannte Typen oder Konstanten wären.

Typparameter werden mit den Schlüsselworten `typename` oder `class` eingeleitet, Integerkonstanten durch den Typ (`char`, `short`, `int`, `long`, `signed`, `unsigned`)

```
template <typename T, int N> ...
```

9.2. Funktionstemplates

Beim Aufruf einer Templatefunktion müssen Sie die Templateargumente ebenfalls in spitzen Klammern angeben. Beim Generieren der Templatefunktion ersetzt der Compiler die formalen Templateparameter durch die übergebenen Argumente. Wenn es für den Compiler auch möglich ist, die Templateargumente aus den Funktionsargumenten herzuleiten, müssen Sie die Templateargumente nicht angeben. In älteren Standards war es nicht möglich, namenlose, lokale Typen zu übergeben. Gnu C++ 4 hat hier ein Fehler generiert:

```
static struct {
    Position::MoveFunc move;
    const char          *name;
} moves[] =
{
    ...
};
myFunc(moves);
```

Andere Compiler, die den Standard nicht so genau genommen haben, übersetzten das wie gewünscht. Seit C++11 ist dies auch erlaubtes Verhalten.

Wenn die Templateargumente Datentypen (bzw. Klassen) sein sollen, setzen Sie die Schlüsselworte `typename` oder `class` voran. Ich persönlich ziehe `typename` vor, da nicht nur Klassen sondern auch andere Typen verwendet werden können.

```
/*
    templatel.cpp: show max value
*/
#include <iostream>

template <typename T> T maxValue( T data[], int numData ) {
    T max = 0;
    for( int i=0; i<numData; ++i ) {
        if( max < data[i] )
            max = data[i];
    }
    return max;
}

int main() {
    int intArray[] = { 0, 5, 2, 6, 9, 10, 1, 3 };
    double doubleArray[] = { 2.12, 3.14, 99.09, 5.34 };

    std::cout << "max von intArray = " << maxValue<int>( intArray,
sizeof(intArray) / sizeof(intArray[0]) ) << std::endl;
    std::cout << "max von doubleArray = " << maxValue( doubleArray,
sizeof(doubleArray) / sizeof(doubleArray[0]) ) << std::endl;

    return 0;
}
```

9.3. Skalare Templateparameter

Als Templateparameter dürfen nicht nur Typen verwendet werden sondern auch Integerkonstanten. Beim Aufruf müssen Sie dann in spitzen Klammern den konstanten Wert angeben. Der konstante Wert kann auch durch den Compiler zur Übersetzungszeit berechnet werden.

```
/*
    template2.cpp: show max value
*/

#include <iostream>

template <int numData, typename T> T maxValue( T data[] ) {
    T max = 0;

    for( int i=0; i<numData; i++ ) {
        if( max < data[i] )
            max = data[i];
    }
    return max;
}

int main() {
    int intArray[] = { 0, 5, 2, 6, 9, 10, 1, 3 };
    double doubleArray[] = { 2.12, 3.14, 99.09, 5.34 };

    std::cout << "max von intArray = " << maxValue<sizeof(intArray) /
sizeof(intArray[0])>( intArray ) << std::endl;
    std::cout << "max von doubleArray = " <<
maxValue<sizeof(doubleArray) / sizeof(doubleArray[0])>( doubleArray ) <<
std::endl;

    return 0;
}
```

Konstante Parameter können auch aus Arraygrößen automatisch ermittelt werden.

```
template <typename T, size_t N>
size_t cppArraySize(T (&)[N]) {
    return N;
}

int myArray[] = { 2, 3, 6, 10 };
std::cout << cppArraySize( myArray );
```

9.4. Klassentemplates

Nicht nur Funktionen sondern auch Klassen können mit Vorlagen definiert werden. Hierzu wird nach der Templatedeklaration und deren Parameter einfach die Klasse ganz normal definiert, wobei die Templateparameter verwendet werden können. Wenn Sie ein Klassentemplate instanziierten (also ein Objekt einer solchen Klasse erzeugen), müssen Sie in spitzen Klammern hinter dem Namen des Klassentemplates die Argumente angeben.

```
/*
    template3.cpp: show distance
*/

#include <iostream>
#include <cmath>

template<typename ScalarT> class Point {
public:
    typedef ScalarT    Scalar;

private:
    Scalar            x, y;

public:
    Point( Scalar x, Scalar y ) : x(x), y(y)
    {}

    Scalar distance( const Point &other ) const {
        Scalar kat1 = x - other.x;
        Scalar kat2 = y - other.y;
        return std::sqrt( kat1*kat1 + kat2*kat2 );
    }
};

int main() {
    Point<float>    point1(0,0);
    Point<float>    point2(1,1);

    std::cout << "Abstand ist " << point1.distance( point2 ) << std::endl;
}
```

Obwohl es in diesem konkreten Beispiel nicht notwendig ist, ist es sehr häufig sinnvoll, wenn Sie in Klassentemplates für die Typparameter des Templates eigene Elementtypen definieren.

Für Templateparameter können auch Defaultargumente angegeben werden, die verwendet werden, wenn sie bei der Instanziierung des Templates nicht angegeben wurden. Hierzu notieren Sie hinter der Parameterdeklaration das Gleichheitszeichen und das Defaultargument.

```
/*
    template4.cpp: show distance
*/

#include <iostream>
#include <cmath>

template<typename ScalarT=double> class Point {
public:
    typedef ScalarT    Scalar;

private:
    Scalar            x, y;

public:
    Point( Scalar x, Scalar y ) : x(x), y(y)
    {}
}
```

```

        Scalar distance( const Point &other ) const {
            Scalar kat1 = x - other.x;
            Scalar kat2 = y - other.y;

            return std::sqrt( kat1*kat1 + kat2*kat2 );
        }
};

int main() {
    Point<>    point1(0,0);
    Point<>    point2(1,1);

    std::cout << "Abstand ist " << point1.distance( point2 ) << std::endl;
}

```

9.5. Klassenfunktionstemplates

Auch Elementfunktionen einer Klassen können als Template definiert werden. Selbst dann, wenn die Klasse selbst schon als Template definiert wurde.

```

/*
    template5.cpp: show distance
*/

#include <iostream>
#include <cmath>

template<typename ScalarT=double> class Point {
public:
    typedef ScalarT    Scalar;

private:
    Scalar            x, y;

public:
    Point( Scalar x, Scalar y ) : x(x), y(y)
    {}

    Scalar getX() const {
        return x;
    }

    Scalar getY() const {
        return y;
    }

    template<typename PointT>
    Scalar distance( const PointT &other ) const {
        Scalar kat1 = x - other.getX();
        Scalar kat2 = y - other.getY();
        return std::sqrt( kat1*kat1 + kat2*kat2 );
    }
};

int main() {
    Point<float>    point1(0,0);
    Point<>         point2(1,1);

    std::cout << "Abstand ist " << point1.distance( point2 ) << std::endl;
}

```

Elementfunktionen, die als Template definiert werden, dürfen aber nicht virtual sein.

9.6. Spezialisierung

Das bisherige Beispiel des Klassentemplates Point liefert für einen Templateargument ein sehr ungenaues Ergebnis für den Abstand zweier Punkte. Eine mögliche Lösung des Problems besteht in der sogenannten Templatespezialisierung. Hiermit geben wir dem Compiler die Anweisung, wie die Templateklasse für ein bestimmtes Templateargument zu erstellen ist.

Eine Templatespezialisierung können Sie wie folgt definieren:

- ☞ Nach dem Schlüsselwort `template` wird die Parameterliste leer gelassen.
- ☞ Nach dem Namen des Template schreiben Sie in spitzen Klammern das oder die Argumente, für die diese spezielle Variante verwendet werden soll.

```
/*
    template6.cpp: show distance
*/

#include <iostream>
#include <cmath>

template<typename ScalarT=double> class Point {
public:
    typedef ScalarT    Scalar;

private:
    Scalar              x, y;

public:
    Point( Scalar x, Scalar y ) : x(x), y(y)
    {}

    Scalar getX() const {
        return x;
    }

    Scalar getY() const {
        return y;
    }

    template<typename PointT>
    Scalar distance( const PointT &other ) const {
        Scalar kat1 = x - other.getX();
        Scalar kat2 = y - other.getY();
        return std::sqrt( kat1*kat1 + kat2*kat2 );
    }
};

template<> class Point<int> {
public:
    typedef int    Scalar;

private:
    Scalar          x, y;
```

```

public:
Point( Scalar x, Scalar y ) : x(x), y(y)
{}

Scalar getX() const {
    return x;
}

Scalar getY() const {
    return y;
}

template<typename PointT>
double distance( const PointT &other ) const {
    double kat1 = x - other.getX();
    double kat2 = y - other.getY();
    return std::sqrt( kat1*kat1 + kat2*kat2 );
}
};

int main() {
    Point<int>    point1(0,0);
    Point<int>    point2(1,1);
    std::cout << "Abstand ist " << point1.distance( point2 ) << std::endl;
}

```

Klassentemplates können auch teilweise spezialisiert werden. Das ist zum Beispiel sinnvoll, wenn Sie für Zeiger ein anderes Template benötigen:

```

template <typename ElementT>
struct ElementPrinter
{
    inline static std::ostream &printElement( std::ostream &stream, const
ElementT &elem )
    {
        stream << elem;
        return stream;
    }
};

template <typename ElementT>
struct ElementPrinter<ElementT*>
{
    inline static std::ostream &printElement( std::ostream &stream, const
ElementT *elem )
    {
        if( elem )
            stream << *elem;
        else
            stream << "<NULL>";

        return stream;
    }
};

```

9.7. Ableitungen

Wenn Sie in einem abgeleiteten Klassentemplate auf ein Element der Basisklasse zugreifen wollen, müssen Sie auf dieses über den `this` Zeiger zugreifen. Dies war in einer früheren C++ Spezifikation nicht notwendig. Daher bieten manche Compiler einen Schalter an, diese Regel zu umgehen.

```
/*
    template7.cpp: show distance
*/

#include <iostream>
#include <cmath>

template<typename ScalarT=double> class Point {
public:
    typedef ScalarT    Scalar;

private:
    Scalar            x, y;

public:
    Point( Scalar x, Scalar y ) : x(x), y(y)
    {}

    Scalar getX() const {
        return x;
    }

    Scalar getY() const {
        return y;
    }
};

template<typename ScalarT> class Point3D : public Point<ScalarT> {
public:
    typedef ScalarT    Scalar;

private:
    Scalar            z;

public:
    Point3D( Scalar x, Scalar y, Scalar z ) : Point<Scalar>(x,y), z(z)
    {}

    Scalar getZ() const {
        return z;
    }

    template<typename PointT>
    Scalar distance( const PointT &other ) const {
        Scalar kat1 = this->getX() - other.getX();
        Scalar kat2 = this->getY() - other.getY();

        Scalar kat3 = std::sqrt( kat1*kat1 + kat2*kat2 );
        Scalar kat4 = z - other.getZ();
        return std::sqrt( kat3*kat3 + kat4*kat4 );
    }
};
```

```
int main() {
    Point3D<double>    point1(0,0,0);
    Point3D<double>    point2(1,1,1);
    std::cout << "Abstand ist " << point1.distance( point2 ) << std::endl;
}
```

9.8. Template-Template-Parameter

Nicht nur Typen und Konstanten können Templates als Argumente übergeben werden sondern auch andere Klassentemplates. Template template parameter (ja das heißt so) werden mit dem Schlüsselwort `template` eingeleitet. Anschließend folgen in spitzen Klammern die Art der Templateparameter des template template parameters. Dannach folgen das Schlüsselwort `class` und der Name des template template parameters optional gefolgt vom Standardwert des Parameters. Ab C++-11 kann statt dem Schlüsselwort `class` auch das Schlüsselwort `typename` verwendet werden.

```
/*
    template8.cpp: simple map
*/

#include <iostream>
#include <math.h>

template<typename ValueT, int numData=10> class Array {
public:
    typedef ValueT value_type;

private:
    value_type    data[numData];

public:
    void set( int index, const value_type &val ) {
        if( index >= 0 && index < numData )
            data[index] = val;
    }

    value_type get( int index ) const {
        if( index >= 0 && index < numData )
            return data[index];
        else
            return value_type();
    }
};

template<typename KeyT, typename ValueT, template<typename, int> class
ContainerT=Array, int maxNumData=10>
class Map {
public:
    typedef KeyT                key_type;
    typedef ValueT              value_type;
    typedef ContainerT<key_type, maxNumData> key_container_type;
    typedef ContainerT<value_type, maxNumData> value_container_type;

private:
    key_container_type    keys;
    value_container_type  values;
    int                  numData;
};
```

```

public:
Map() : numData(0)
{}

void add( const key_type &key, const value_type &value ) {
    if( numData < maxNumData-1 ) {
        keys.set( numData, key );
        values.set( numData, value );
        ++numData;
    }
}

value_type get( const key_type &key ) {
    for( int i=0; i<numData; ++i ) {
        if( keys.get( i ) == key )
            return values.get( i );
    }
    return value_type();
}
};

int main() {
    Map<char, double> theMap;

    theMap.add( 'p', 3.142 );
    theMap.add( 'e', 2.718 );

    std::cout << "e=" << theMap.get( 'e' ) << std::endl;
    std::cout << "pi=" << theMap.get( 'p' ) << std::endl;
}

```

9.9. Templates und Elementtypen (member types)

Wenn Sie innerhalb einer Templatedefinition auf einen Elementtyp einer übergebenen Klasse zugreifen wollen, müssen Sie dies mit dem speziellen Schlüsselwort `typename` kennzeichnen. Dies ist erforderlich, damit der Compiler schon bei der ersten Analyse Ihrer Templatedefinition noch vor der ersten Instanziierung die Richtigkeit gewährleisten kann.

```

/*
    template9.cpp: simple map
*/

#include <iostream>
#include <math.h>

template<typename ValueT, int numData=10> class Array {
public:
    typedef ValueT value_type;
private:
    value_type    data[numData];
public:
    void set( int index, const value_type &val ) {
        if( index >= 0 && index < numData )
            data[index] = val;
    }
}

```

```

        value_type get( int index ) const {
            if( index >= 0 && index < numData )
                return data[index];
            else
                return value_type();
        }
};

template<typename KeyContainerT, typename ValueContainerT, int
maxNumData=10>
class Map {
public:
    typedef KeyContainerT                key_container_type;
    typedef ValueContainerT              value_container_type;
    typedef typename key_container_type::value_type    key_type;
    typedef typename value_container_type::value_type value_type;

private:
    key_container_type    keys;
    value_container_type  values;
    int                  numData;

public:
    Map() : numData(0)
    {}

    void add( const key_type &key, const value_type &value ){
        if( numData < maxNumData-1 ){
            keys.set( numData, key );
            values.set( numData, value );
            ++numData;
        }
    }

    value_type get( const key_type &key ) {
        for( int i=0; i<numData; ++i ) {
            if( keys.get( i ) == key )
                return values.get( i );
        }
        return value_type();
    }
};

int main() {
    Map<Array<char>, Array<double> > theMap;

    theMap.add( 'p', 3.142 );
    theMap.add( 'e', 2.718 );

    std::cout << "e=" << theMap.get( 'e' ) << std::endl;
    std::cout << "pi=" << theMap.get( 'p' ) << std::endl;
}

```

9.10. Templatequellen

Templatedefinitionen müssen bei der Instanziierung dem Compiler bekannt sein. Daher sollten sie sich immer in Headerdateien befinden. Wenn Sie die gleiche Templateklasse (=Instanziierung eines Klassentemplates) oder die gleiche Templatefunktion (=Aufruf eines Funktionstemplates) in unterschiedlichen Modulen erzeugen, muss der Linker die jeweiligen (Element)funktionen beim Binden Ihrer

Anwendung zusammenführen. Er darf also weder eine Fehlermeldung wegen doppelt definierte Symbole ausgeben noch Ihre Anwendung unnötig aufblähen.

Spezialisierungen von Templates enthalten aber alles, was der Compiler benötigt, um sie zu instanziierten, sie sollten daher genauso behandelt werden, wie normale Funktionen, also in einer normalen Quelldatei statt einem Header abgelegt werden, andernfalls erhalten Sie einen Fehler wegen doppelten Symbolen.

9.11. Neue Standards

Ab C++ 11 wurden die parameter packs eingeführt. parameter packs werden bei den Templateparameter definiert und haben dies Formate

typ... <packname> für konstante Parameter und typename|class... <packname> für Typparameter.

Beim Instanziierten eines solchen Templates werden für den parameter pack 0, 1 oder mehrere Argumente angegeben.

```
template <typename FMT_T, typename... VARY>
void MyFunction {VARY... param} { }
MyFunction <std::string, int, float>("Hello", 1, 3.14);
```

Innerhalb der Vorlagendefinition werden die packs über den Templateparameter oder den Funktionsparameter verwendet gefolgt von drei Punkten. Der Templateparameter wird durch die Templateargumente ersetzt, der Funktionsparameter wird durch die Funktionsargumente ersetzt.

Ab C++ 20 können Sie mit den Schlüsselworten concept und requires verschiedene Eigenschaften formulieren, die ein Templateargument erfüllen muss. Der Sinn dieser Möglichkeit ist, dass der Compiler beim Instanziierten eines solchen Templates bessere Fehlermeldungen anzeigen kann aber auch dass weitere Prüfungen erfolgen können, die sonst möglicherweise zur Laufzeit erst zum Tragen kommen. Im Header <concepts> finden Sie verschiedene Hilfsklassen und -templates, die Ihnen bei der Formulierung dieser Bedingungen helfen sollen.

Fürs erste soll dies zum Thema Templates genügen. Weitere Informationen folgen in einem späteren Kapitel.

10. Zeichenketten

Zeichenketten dürften wohl eine der am häufigsten verwendeten Datenstrukturen bei der Programmierung sein. Die einfachste Form – die Stringlitterale – haben wir schon kennengelernt. In diesem Kapitel lernen wir, wie wir sie verarbeiten können. Obwohl C++ eigene Klassen für die Verarbeitung von Zeichenketten kennt, werden Sie möglicherweise trotzdem hin und wieder die C-Funktionen benutzen müssen.

10.1. C-Zeichenketten

Zum Speichern einer Zeichenkette verwenden Sie in C entweder ein Array von char oder einen Zeiger auf char. Wenn die Zeichenkette nicht verändert werden darf, wird

ein const-Zeiger verwendet. Wie auch immer, Sie müssen stets gewährleisten, dass Sie genügend Speicher angefordert haben, um die ganze Zeichenkette zu speichern. Wenn Sie hier ungenügende Prüfungen implementieren, kann dies schnell zu schwerwiegenden Fehlern führen.

Die Deklaration der C Standardbibliothek für die Zeichenketten befindet sich in den Headern `<string.h>` für C und `<cstring>` für C++. Wie immer gilt für C++, dass sich die Funktionen im Namensraum `std` befinden.

```
char *strcpy( char *dest, const char *src );
```

Kopiert die Zeichenkette, auf die `src` verweist, in den Speicher auf den `dest` verweist. Zurückgegeben wird der Wert von `dest`. Achten Sie darauf, dass der Speicherbereich, auf den `dest` verweist, genügend groß ist.

```
char *strncpy( char *dest, const char *src, size_t maxLen );
```

Wie `strcpy` jedoch werden maximal `maxLen` Zeichen kopiert.

```
char *strcat( char *dest, const char *src );
```

Hängt die Zeichenkette, auf die `src` verweist, an die Zeichenkette, auf die `dest` verweist, an. Achten Sie darauf, dass `dest` tatsächlich auf eine gültige Zeichenkette, also mit einem 0-Byte am Ende, verweist. Auch muss der Speicherbereich auf den `dest` verweist genügend groß sein. Es wird der Wert von `dest` wieder zurückgegeben.

```
char *strncat( char *dest, const char *src, size_t maxLen );
```

Wie `strcat` allerdings werden maximal `maxLen` Zeichen kopiert.

```
char *strdup( const char *src );
```

Erzeugt eine Kopie der Zeichenkette, auf die `src` verweist, und gibt die Adresse der Kopie zurück. Sie müssen nach Gebrauch den Speicher mit der Funktion `free` wieder freigeben.

```
int strcmp( const char *s1, const char *s2 );
```

Vergleicht die beiden Zeichenketten und liefert -1 zurück, wenn `s1` vor `s2` liegt, 0 wenn beide Zeichenketten identisch sind, 1, wenn `s2` vor `s1` liegt.

Zum Vergleich geht die Funktion jedes Zeichen der beiden Zeichenketten durch und beendet den Vergleich, wenn ein unterschiedlicher Wert oder das 0-char, das die Zeichenkette beendet gefunden wird. Wenn das Zeichen in `s1` kleiner als in `s2` ist, wird -1 zurückgegeben. Wenn das Zeichen identisch ist (also beide 0 sind), so ist das Ergebnis 0. Wenn der Code in `s1` größer als in `s2` ist, wird eine 1 zurückgegeben.

Beachten Sie, dass diese Funktion nicht dazu geeignet ist, deutsche Texte korrekt zu sortieren, da die Umlaute nicht Bestandteil der ASCII-Tabelle sind und der ANSI-Zeichensatz nicht zur Sortierung geeignet ist.

```
int strncmp( const char *s1, const char *s2, size_t n );
```

wie `strcmp`, jedoch endet der Vergleich spätestens nach `n` Zeichen.

```
int strcoll( const char *s1, const char *s2 );
```

Wie `strcmp` allerdings berücksichtigt es die aktuellen Ländereinstellungen. D.h. deutsche Umlaute werden anders verglichen, so wie im Deutschen üblich. Bei `strcmp` werden Umlaute entsprechend ihrer binären Codierung sortiert.

```
size_t strlen( const char *s );
```

Liefert die Länge der übergebenen Zeichenkette zurück.

```
char *strchr( const char *s, int c );          /* nur C */  
const char *strchr( const char *s, int c );    // nur C++  
char *strchr( char *s, int c );                // nur C++
```

Liefert die Adresse des ersten Zeichens in `s` mit dem Code `c`. Falls das Zeichen nicht gefunden wird, wird `NULL` zurückgegeben. Bedenken Sie in C: Wenn Sie als Zeichenkette eine Stringliteral übergeben, erhalten Sie zwar einen non-const-Zeiger, Sie dürfen aber das Zeichen trotzdem nicht ändern, da Sie sonst einen Laufzeitfehler erhalten können.

```
char *strrchr( const char *s, int c );          /* nur C */  
const char *strrchr( const char *s, int c );    // nur C++  
char *strrchr( char *s, int c );                // nur C++
```

Liefert die Adresse des letzten Zeichens in `s` mit dem Code `c`. Falls das Zeichen nicht gefunden wird, wird `NULL` zurückgegeben. Für C gilt hier der gleiche Hinweis, wie bei der vorherigen Funktion `strchr`.

```
size_t strcspn( const char *s1, const char *s2 );
```

Sucht das erste Zeichen in `s1`, das nicht an einer beliebigen Position in `s2` ist und liefert dessen Index zurück. Falls kein Zeichen in `s1` gefunden wird, das nicht in `s2` enthalten ist, gibt die Funktion die Länge von `s1` zurück.

```
size_t strspn( const char *s1, const char *s2 );
```

Sucht das erste Zeichen in `s1`, das an einer beliebigen Position in `s2` ist und liefert dessen Index zurück. Falls kein Zeichen in `s1` gefunden wird, das auch in `s2` enthalten ist, gibt die Funktion die Länge von `s1` zurück.

```
char *strpbrk( const char *s1, const char *s2 );          /* nur C */  
const char *strpbrk( const char *s1, const char *s2 );    // nur C++  
char *strpbrk( char *s1, const char *s2 );                // nur C++
```

Wie `strspn` allerdings wird nicht der Index sondern die Adresse zurückgegeben. Wenn keines gefunden wurde, wird `NULL` zurückgegeben.

```
char *strstr( const char *s1, const char *s2 );          /* nur C */  
const char *strstr( const char *s1, const char *s2 );    // nur C++  
char *strstr( char *s1, const char *s2 );                // nur C++
```

Durchsucht `s1` nach dem ersten Vorkommen von `s2`. Wird es gefunden, so liefert es die Adresse der gefundenen Zeichenkette in `s1`, andernfalls wird `NULL` zurückgegeben.

```
char *strtok( char *s1, const char *s2 );
```

Dient zum Tokenisieren einer Zeichenkette. Es wird die Zeichenkette s1 nach dem ersten Vorkommen eines Zeichens aus s2 durchsucht, durch eine 0 ersetzt und s1 wieder zurückgegeben. Um die restlichen Token zu erhalten, muss man nur die Funktion in einer Schleife erneut aufrufen, wobei statt s1 eine NULL übergeben wird, bis NULL zurückgegeben wird.

```
char* token = strtok(input, " ");
while (token) {
    puts(token);
    token = strtok(NULL, " ");
}

size_t strxfrm( char *target, const char *source, size_t n );
```

Dient zum Umwandeln einer länderspezifischen Zeichenkette in eine Darstellung, die verglichen werden kann.

```
char *strlwr( char *s );
```

Wandelt die übergebene Zeichenkette in Kleinbuchstaben um und berücksichtigt dabei die aktuellen Ländereinstellungen. Der Rückgabewert ist der Wert von s.

```
char *strupr( char *s );
```

Wandelt die übergebene Zeichenkette in Großbuchstaben um und berücksichtigt dabei die aktuellen Ländereinstellungen. Der Rückgabewert ist der Wert von s. Obwohl strupr und strlwr keine Standardfunktion sind, sind sie in vielen Compilern enthalten weshalb ich sie hier aufliste.

10.2. C++ Zeichenketten

Zum Bearbeiten von Zeichenketten in C++ gibt es die Klassen `std::string` und `std::wstring`. In neueren C++-Standards gibt es noch mehr Klassen, die aber erst später behandelt werden sollen. Hier geht es nur um die beiden genannten, da diese am häufigsten verwendet werden. `std::string` ist eine Templateinstanz von `std::basic_string` mit dem Templateargument `char` für die einzelnen Zeichen und `std::wstring` verwendet das Templateargument `wchar_t`.

`std::string` können Sie für die meisten westlichen Zeichensätze benutzen, die auf den lateinischen Buchstaben basieren. Wenn Sie flexibler sein wollen, nehmen sie stattdessen `std::wstring` allerdings mit dem Nachteil, dass Sie dann hier mehr Speicher brauchen.

Der große Vorteil von `std::string` und verwandten Klassen ist, dass Sie sich nicht um die Speicherverwaltung kümmern müssen, da diese Klassen Speicher anfordern bzw. freigeben, wenn es erforderlich ist.

Den Konstruktor von `String` können Sie unter anderen mit diesen Argumenten aufrufen:

```
std::string()
std::wstring()
```

Erzeugt eine leere Zeichenkette.

```
std::string(std::size_t count, char c)
std::wstring(std::size_t count, w_char c)
```

Erzeugt eine Zeichenkette bestehend aus der angegebenen Anzahl count des angegebenen Zeichens c.

```
std::string(const char *str [, std::size_t count])
std::wstring(const w_char *str [, std::size_t count] )
```

Erzeugt eine Zeichenkette bestehend aus den ersten count Zeichen der Zeichenkette str. Achten Sie darauf, dass count kleiner oder gleich der tatsächlichen Anzahl Zeichen ist, da ansonsten das Verhalten nicht definiert ist. Wenn count weggelassen wird, werden alle Zeichen in die neue Zeichenkette kopiert.

```
std::string( const std::string& other )
std::wstring( const std::wstring& other )
std::string( std::string&& other )
std::wstring( std::wstring&& other )
```

Dies sind die Kopier- und Movekonstruktoren. (Move- ab C++11)

Daneben gibt es noch einige andere Argumente, die verwendet werden können, diese sollen aber hier in der Einführung keine Rolle spielen und sind für fortgeschrittene Methoden von Bedeutung.

Zusätzlich zu den Konstruktoren gibt es noch verschiedene andere Funktionen, mit denen die Zeichenketten bearbeitet werden können. Im folgenden beschreibe ich der Übersichtlichkeit wegen nur die Parameter für std::string. Für std::wstring müssen Sie die Variante mit wchar_t statt char verwenden. Auch hier werden ich nur die wichtigsten für diese Einführung aufführen. Für fortgeschrittene Themen verwenden Sie bitte ein Referenzhandbuch.

```
operator =
assign()
```

Mit dem Zuweisungsoperator oder Memberfunktion assign können Sie der Zeichenkette einen neuen Wert übergeben. Als rechter Operand oder Argument können Sie eine andere C++ Zeichenkette, eine C-Zeichenkette oder ein einzelnes Zeichen angeben. Andere möglichen Operanden oder Argumente sollen uns hier jetzt nicht interessieren.

```
clear()
```

Leert die komplette Zeichenkette. Dabei ist nicht garantiert, dass der belegte Speicherplatz auch freigegeben wird.

```
insert( size_t index, size_t count, char c )
```

Fügt die angegebene Anzahl count das Zeichen c ab der angegebenen Position index ein.

```
insert( std::size_t index, char *str [, std::size_t count] )
```

Fügt die ersten count Zeichen der C-Zeichenkette str ab der Position index ein. Wenn count weggelassen wird, so wird die ganze Zeichenkette eingefügt.

```
insert( std::size_t index, const std::string &str [, std::size_t s_index,
std::size_t count] )
```

Fügt die Teilzeichenkette ab der Position `s_index` in der Quellzeichenkette `str` mit der angegebenen Anzahl `count` Zeichen ab der Position `index` in die Zielzeichenkette ein. Beispiel:

```
std::string hello="Hello world";
std::string name="Martin Richard Gäckler";
hello.insert( 5, name, 7, 8 );
std::cout << hello « std::endl;
```

Übung: Erstellen Sie aus diesem Codeschnippel ein fertiges Programm und prüfen Sie die Ausgabe. Versuchen Sie die Ausgabe vorherzusagen.

```
std::string& erase( size_t index = 0, size_t count = npos )
```

Entfernt ab der Position `index` die angegebene Anzahl Zeichen aus der Zeichenkette. Wenn `count` fehlt, wird bis zum Ende alles gelöscht. Wenn `index` fehlt, wird alles gelöscht.

```
operator +=
append()
```

Hängt den zweiten Operanden bzw. das Argument an die Zeichenkette an. Sie können unter anderen eine C-Zeichenkette, ein einzelnes Zeichen oder eine andere C++Zeichenkette angeben.

```
replace( std::size_t pos, std::size_t count, ... )
```

Ersetzt ab der angegebenen Position die angegebene Anzahl Zeichen durch einen Ersatz. Als Ersatz können Sie angeben eine andere C++Zeichenkette, eine andere C-Zeichenkette oder ein einzelnes Zeichen. Dabei haben Sie auch die Möglichkeit, den Ersatz weiter zu präzisieren, hier soll uns aber diese Information genügen.

Zum Inspizieren einer Zeichenkette gibt es noch weitere Funktionen, von denen auch hier wieder die wichtigsten aufgelistet werden.

```
bool empty() const
```

Liefert `true` zurück, wenn die Zeichenkette leer ist.

```
bool size() const
```

Liefert die Anzahl der Zeichen in der Zeichenkette zurück

```
std::size_t find( char c, std::size_t pos ) const
std::size_t find( const std::string &str, std::size_t pos=0 ) const
std::size_t find( const char *s, std::size_t pos, size_t count ) const
```

Sucht ab der angegebenen Position `pos` nach dem ersten Vorkommen des Zeichens `c`, der C++-Zeichenkette `str` oder der C-Zeichenkette `s`. Im letzteren Fall können Sie angeben, nach wie vielen Zeichen in `s` gesucht werden soll.

Der Rückgabewert ist der 0-basierte Index der gefundenen Zeichenkette bzw. des gefundenen Zeichens. Wenn die Zeichenkette oder das Zeichen nicht gefunden wird,

so wird die Konstante `std::string::npos` zurückgeliefert.

```
std::string text = "Hello World";
std::size_t fPos = text.find( "Martin" );
if( std::string::npos == fPos) {
    std::cerr << "Text nicht gefunden" << std::endl;
}

std::size_t rfind( char c, std::size_t pos ) const
std::size_t rfind( const std::string &str, std::size_t pos=0 ) const
std::size_t rfind( const char *s, std::size_t pos, size_t count ) const
```

`rfind` arbeitet wie `find` nur sucht es nicht das erste Vorkommen sondern das letzte Vorkommen.

```
int compare( const std::string& str ) const;
int compare( const char *s ) const;
```

Vergleicht wie die C-Funktion `strcmp` zwei Zeichenketten. D.h. es wird -1 zurückgegeben, wenn die eigene Zeichenkette vor der übergebenen Zeichenkette liegt. Es wird 0 zurückgegeben, wenn sie identisch sind und es wird 1 zurückgegeben, wenn die übergebene Zeichenkette vor der eigenen Zeichenkette liegt.

```
std::string substr( std::size_t pos = 0, std::size_t count = npos ) const
```

Liefert einen Teilstring mit `count` Zeichen ab der Position `pos` der eigenen Zeichenkette zurück. Wenn `count` fehlt, wird ab `pos` der Rest der eigenen Zeichenkette zurückgegeben, wenn `pos` fehlt, wird eine Kopie der eigenen Zeichenkette zurückgegeben.

```
const char *c_str() const
```

Diese Funktion `c_str` liefert einen C-Zeichenkette, die nicht verändert werden darf. Sie können diese Funktion benutzen, um alte C-Funktionen aufzurufen, die einen `const char` Zeiger erwarten.

```
char& operator []( std::size_t pos )
const char& operator []( std::size_t pos ) const
```

Liefert eine Referenz auf das Zeichen an der angegebenen Position. Wenn Sie die Zeichenkette verändern dürfen, können Sie auch das Zeichen verändern, weshalb in diesem Fall, der Ausdruck auf der linken Seite einer Zuweisung stehen darf. Ein Codeschnippel soll dies verdeutlichen.

```
std::string hello = "wosld";
if( 'r' != hello[3] )
    hello[3] = 'r';
```

Sie müssen selbst dafür sorgen, dass der Index auch im gültigen Bereich liegt. Wenn der Index zu groß ist, ist bei älteren Compilern das Verhalten undefiniert, bei neueren Compilern darf auch ein Fehler generiert werden (siehe unten).

```
char& at( std::size_t pos )
const char& at( std::size_t pos ) const
```

At arbeitet wie der operator [] nur wird hier in jedem Fall ein Fehler ausgelöst, wenn der Index zu groß ist.

```
std::string operator +
```

Erzeugt eine neue Zeichenkette mit dem ersten und dem zweiten angehängten Operator.

```
operator ==
```

Vergleicht zwei Zeichenketten.

```
operator >>
```

```
operator <<
```

Dient zur Dateneingabe bzw. -ausgabe von Zeichenketten.

11. Typumwandlungen

Ein häufiges Problem, das Ihnen bei der Programmierung begegnen wird, ist die Aufgabe einen Wert von einem Typ zu einem anderen umzuwandeln. Sowohl C als auch C++ bieten Ihnen verschiedene Optionen, dieses Problem zu lösen. Wobei sich jedoch C in vielen Punkten von C++ unterscheidet.

11.1. Implizite Typumwandlung

Bei der impliziten Typumwandlung wird diese nicht direkt angegeben sondern automatisch von Ihrem Programm zur Laufzeit oder zur Übersetzungszeit vorgenommen. Dabei gelten folgende Regeln:

- ☞ Numerische Typen (float, double, int und char) werden jeweils von einem zum anderen umgewandelt.
- ☞ Wenn der Zieltyp dabei kleiner ist wie der Ausgangstyp (Beispielsweise long int → short int) werden die überzähligen höchstwertigen Bits abgeschnitten. In der Regel erhalten Sie dann vom Compiler eine Warnung.
- ☞ Wenn der Zieltyp größer ist, wird der Ausgangswert unter Beibehaltung seines Vorzeichens entsprechend aufgefüllt (positive Werte mit binären 0 negative Werte mit 1).
- ☞ Bei Vergleichs- und arithmetischen Operatoren wird der kleinere Operand stets erst in der größeren umgewandelt, falls sie eine unterschiedliche Größe haben. Wenn einer der Operanden eine Fließkommazahl ist der andere jedoch nicht, so wird der Ganzzahlwert in eine Fließkommazahl konvertiert.
- ☞ Bei der Konvertierung eines Fließkommatyps (float oder double) in einen Ganzzahltyp werden die Nachkommastellen abgeschnitten. Auch hier erhalten Sie in der Regel eine Warnung vom Compiler.
- ☞ Aufzählungstypen können nach numerischen Typen konvertiert werden. Umgekehrt gilt dies jedoch nur in C. C++ erlaubt dies nicht, hier müssen Sie den Wert explizit umwandeln.

- ☞ Wenn der Zieltyp `bool` ist, werden alle Werte ungleich 0 bzw. `NULL` zu `true`. 0 und `NULL` werden zu `false`.
- ☞ Wenn der Ausgangstyp `bool` ist, so wird `true` zu 1 und `false` zu 0.
- ☞ In C können Zeiger beliebig konvertiert werden. Da dies in den meisten Fällen Unsinn ist, erhalten Sie in der Regel von Ihrem Compiler eine Warnung, die Sie beachten sollten. In C++ gelten deutlich strengere Regeln.
- ☞ Ein Zeiger oder eine Referenz auf eine Klasse kann in einen beliebigen Basisklassenzeiger bzw. -referenz (Upcast) konvertiert werden.
- ☞ Ein Zeiger oder eine Referenz kann auch auf einen `const`-Zeiger bzw. `const`-Referenz des selben Typs konvertiert werden.
- ☞ Ein beliebiger Zeiger kann auf einen Zeiger vom Typ `void` konvertiert werden.
- ☞ In C++ werden Konstruktoren, die mit nur einem Parameter aufgerufen werden können, ebenfalls für die implizite Typumwandlung verwendet, wenn diese nicht mit dem Schlüsselwort `explicit` davon ausgenommen wurden. Das Schlüsselwort `explicit` muss vor dem Konstruktor kommen.
- ☞ Wenn in einer C++ Klasse die Typumwandlung definiert wurde, so wird diese Funktion auch für die implizite Typumwandlung verwendet.

11.2. Explizite Typumwandlung in C

In C können Sie mit dem Cast operator jeden skalaren Typ (`bool`, `char`, `int`, `float`, `double` und beliebige Zeiger) in jeden anderen skalaren Typ ohne weitere Prüfung umwandeln. Hierzu schreiben Sie vor dem umzuwandelnden Wert den gewünschten Zieltyp in Klammern. Bitte beachten Sie: Der Compiler wird Ihnen keinerlei Warnung anzeigen, seien Sie also Vorsichtig besonders, wenn Sie Zeiger bzw. Adressen umwandeln.

11.3. Explizite Typumwandlung in C++

C++ kennt vier verschiedene Cast operatoren, die alle den selben Aufbau aufweisen:

`cast_operator<Zieltyp>(Ausgangswert)`

Zieltyp kann dabei jeder beliebige Typ auch selbst definierte Typen sein. Der Ausgangswert kann jeden beliebigen Wert und Typ aufweisen. Wenn einer der beiden Typen ein nicht skalarer Typ ist, so müssen Sie jedoch eine geeignete Konvertierungsfunktion bereitstellen.

`const_cast`: Dieser Operator ist nur dazu geeignet, um einem Zeiger oder einer Referenz den Typqualifizierer `const` hinzuzufügen oder zu entfernen. Beachten Sie jedoch, dass das Hinzufügen des Typqualifizierers auch mit einer impliziten Konvertierung möglich ist, so dass dieser meistens nur zum Entfernen des Typqualifizierers verwendet wird.

```
const char *ccp = "hello world";  
char *cp = const_cast<char*>(ccp);
```

Darüber hinaus müssen Sie auch noch darauf achten, dass der Speicherbereich dadurch nicht automatisch beschreibbar wird. Wenn im oben gezeigten Beispiel Stringlitterale in einem nicht beschreibbaren Speicher (z.B. dem ROM) abgelegt werden, so können Sie diesen auch nach Anwendung des Operators `const_cast` nicht beschreiben. Das Beispiel ist daher nicht zur Nachahmung geeignet und soll nur die syntaktische Verwendung des Operators demonstrieren. Eine sinnvolle Anwendung könnte sein, wenn Sie die sogenannte „lazy initialization“ Ihrer Klassen vor dem Anwender verbergen wollen:

```
int MyClass::getValue() const {
    if( !myValue ) {
        const_cast<MyClass*>(this)->myValue = 1516;
    }
    return myValue;
}
```

Generell wird aber empfohlen, diesen Operator nur sparsam einzusetzen.

static_cast: Dieser Operator wird dazu benutzt um numerische Typen zu konvertieren oder um einen Zeiger eine Referenz in einen „kompatiblen“ Zeiger zu konvertieren.

Die Umwandlung von numerischen Werten funktioniert zwar auch über die implizite Konvertierung, jedoch zeigt in diesen Fällen häufig der Compiler eine Warnung an, da er nicht weiß, ob dies Absicht oder ein Versehen ist. Wenn der Zieltyp mit nur einem Schlüsselwort beschrieben werden kann, bevorzuge ich jedoch in der Regel einen Konstruktoraufruf, da dies weniger Zeichen erfordert und deshalb schneller zu lesen ist.

```
long    l = 123456;
short   s1 = short(l);
short   s2 = static_cast<short>(l);
```

Die Konvertierung von und/oder nach Klassenobjekten ist auch möglich, wenn die Klasse(n) eine geeignete Konvertierungsfunktion bereitstellen.

Neben den schon bei den impliziten Typkonvertierungen erlaubten Zeiger- bzw. Referenzumwandlungen können Sie mit dem Operator `static_cast` jeden Zeiger vom Typ `void` in jeden beliebigen anderen Zeiger oder jeden Klassenzeiger/-referenz in jeden Zeiger/Referenz auf abgeleiteten Klassen konvertieren - also Downcasts. Es ist jedoch einzig und allein Ihre Verantwortung, dafür zu sorgen, dass diese Zeiger- bzw. Referenzkonvertierungen auch tatsächlich korrekt sind. Es findet weder zur Übersetzungs- noch zur Laufzeit eine Überprüfung statt. In der Regel erhalten Sie bei falschen Zeiger-/Referenzkonvertierungen ein instabiles Programm. Wenn Sie Pech haben, erhalten Sie ein Programm, das Funktionen startet, die Sie ganz sicher nicht wollen. Seien Sie also sehr vorsichtig.

```
/* convert1.c */
#include <stdio.h>
```

```
int main( void ) {
    #ifndef __cplusplus
    /*
       enums <-> int
    */
    {
        enum Test {
            HELLO, WORLD
        } enumVal = WORLD;
        int iVal = enumVal;
        enum Test eVal = iVal;

        printf( "%d\n", iVal );
        printf( "%d\n", (int)eVal );
    }
    #endif
    #ifdef __cplusplus
    /*
       bool <-> int
    */
    {
        bool bVal = true;
        int iVal = bVal;
        int i2 = 5;
        bool b2 = i2;
        int i3 = b2;

        printf( "%d\n", iVal );
        printf( "%d\n", (int)b2 );
        printf( "%d\n", (int)i3 );
    }
    #endif
    /*
       unsigned short <-> signed int
    */
    {
        unsigned short usVal = 0xFFFF;
        int iVal = usVal;

        printf( "%d\n", iVal );
    }
    /*
       signed short <-> unsigned int
    */
    {
        short sVal = -1;
        unsigned uVal = sVal;

        printf( "%u %x\n", uVal, uVal );
    }
}
```

```
/*
float with int compare
*/
{
    double    flt = 1.5;
    int       i = 1;

    if( flt > i )
        printf( "%f > %d\n", flt, i );
    if( i < flt )
        printf( "%d < %f\n", i, flt );
}

/*
long long with int compare
*/
{
    long long i64 = 0x123456789LL;
    int       i32 = 0x23456789;

    if( i64 == i32 )
        printf( "%010lld == %010d\n", i64, i32 );
    else
        printf( "%010lld != %010d\n", i64, i32 );
}

#ifdef __cplusplus
/*
pointer conversions
*/
{
    struct tst1 {
        int tst1;
    };

    struct tst2 {
        int tst2;
    };

    void *vPtr = 0x123456;
    int *iPtr = vPtr;
    long *lPtr = iPtr;
    void *v2Ptr = lPtr;
    struct tst1 *s1Ptr = lPtr;
    struct tst2 *s2Ptr = s1Ptr;
    int i = iPtr;
    char *cPtr = i;

    printf( "%x\n", s2Ptr );
}
#endif
```

```

#ifdef __cplusplus
{
    class Integer {
        int value;

    public:
        Integer( int i, int factor=1 ) : value(i*factor)
        {}

        operator int () {
            return value;
        }
    };
    Integer iVal = 5;
    int i = iVal + 5;

    printf( "%d\n", i );
}
#endif

{
    struct Integer {
        int value;
    } iVal;

    int i=5;
    int j;
    iVal = (struct Integer)i;
    j = (int)iVal;

    printf( "%d\n", j );
}
return 0;
}

```

dynamic_cast: Mit Hilfe des Operators `dynamic_cast` ist es möglich einen sicheren Downcast von Zeigern und Referenzen durchzuführen. Wenn es sich bei dem Objekt, auf das der Zeiger verweist, tatsächlich um die gewünschte Klasse oder um eine davon abgeleitete Klasse handelt, wird der Downcast ausgeführt. Handelt es sich jedoch um eine andere Klasse, so wird bei der Zeigerkonvertierung die Adresse `nullptr` zurückgegeben und bei einer Referenzkonvertierung eine Exception `std::bad_cast` ausgelöst.

```

// convert2.cpp
#include <iostream>
#include <typeinfo>

struct base {
    int b;

    virtual int getValue() const {
        return b;
    }
};

struct derived1 : public base {
    int d1;
}

```

```

        virtual int getValue() const {
            return d1;
        }
};

struct derived2 : public base {
    int d2;
    virtual int getValue() const {
        return d2;
    }
};

int main() {
    derived1 d1;
    derived2 d2;

    base *b1 = &d1;
    base &b2 = d2;

    derived1 *dp1 = dynamic_cast<derived1*>(b1);
    derived2 *dp2 = dynamic_cast<derived2*>(b1);

    if( dp1 )
        std::cout << "Zeiger b1 erfolgreich nach derived1* konvertiert" <<
std::endl;
    if( !dp2 )
        std::cout << "Zeiger b1 erfolglos nach derived2* konvertiert" <<
std::endl;

    try {
        derived2 &d2 = dynamic_cast<derived2&>(b2);
        std::cout << "Referenz b2 erfolgreich nach derived2& konvertiert"
<< std::endl;
    }
    catch( std::bad_cast &e )
    {}

    try {
        derived1 &d1 = dynamic_cast<derived1&>(b2);
    }
    catch( std::bad_cast &e ) {
        std::cout << "Referenz b2 erfolglos nach derived1& konvertiert" <<
std::endl;
    }

    return 0;
}

```

reinterpret_cast: dieser Operator kann dazu verwendet werden, Referenzen bzw. Zeiger beliebig zu konvertieren. Es können auch Zeiger zu Ganzzahltypen und wieder zurück konvertiert werden. Es wird auch garantiert, dass der Wert eines Zeigers nicht verändert wird, wenn der Ganzzahltyp groß genug ist, auch eine Adresse zu speichern.

```

// convert3.cpp
#include <iostream>

class Integer {
    int value;

```

```

    public:
    Integer( int i ) : value(i) {}

    int getValue() const {
        return value;
    }
};

class SuperInt : public Integer {
    int factor;

    public:
    SuperInt( int i, int f ) : Integer(i), factor(f) {}

    long getValue() const {
        return long(Integer::getValue()) * factor;
    }
};

class Long {
    long value;

    public:
    Long( long i ) : value(i) {}
    long getValue() const {
        return value;
    }
};

int main() {
    {
        Integer      i(123);
        SuperInt  si(123,456);
        Integer    *ip = &si;
        //SuperInt  *sip = ip;                                verboten
        SuperInt  *sip = static_cast<SuperInt *>(ip);
        // Long    *l = static_cast<Long *>(&i);                verboten
        std::cout << sip->getValue() << std::endl;
    }

    {
        int      i = 123;
        int      *ip = &i;
        //long    *lp = ip;                                    verboten
        //long    *lp = static_cast<long*>(ip);                  verboten

        void *vp = ip;
        // int    *ip2 = vp;                                    verboten
        int      *ip2 = static_cast<int*>(vp);

        std::cout << *ip2 << std::endl;
    }

    {
        Integer      i(123);
        SuperInt  si(123,456);
        //Long      l = reinterpret_cast<Long>(i);              verboten

        short      s0 = 123;
        short      s1 = 456;
        short      s2 = 789;
    }
}

```

```

//long      l1 = reinterpret_cast<long>(s1); verboten
long      *lp = reinterpret_cast<long*>(&s1);
long      l2 = reinterpret_cast<long>(&s1);
SuperInt  *sp = reinterpret_cast<SuperInt>*>(l2);

// short s = reinterpret_cast<short>(&s1); verboten
short     s = 123;
long      *lp2 = reinterpret_cast<long*>(s);

std::cout << s0 << ' ' << *lp << ' ' << s2 << std::endl;
}

{
    Integer    i(123);
    Integer    &ir = i;
    Long       &lr = reinterpret_cast<Long&>( ir );
    std::cout << lr.getValue() << std::endl;        // undefiniertes
verhalten
}
return 0;
}

```

11.4. Klassenzeiger- und -referenzumwandlung in C++

In C++ gibt es bei der Konvertierung von Zeigern und Referenzen zwei besondere Fälle:

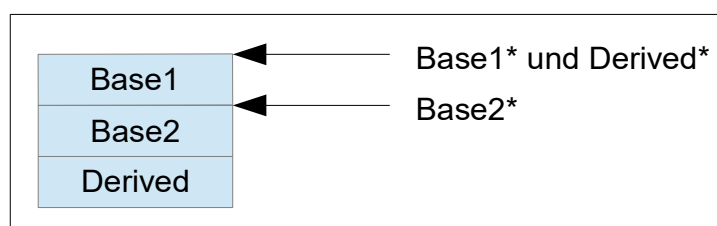
- ☞ **Upcast:** die Konvertierung eines Zeigers oder Referenz auf eine abgeleitete Klasse nach einem Basisklassenzeiger bzw. -referenz. Diese Umwandlung ist möglich, wenn die Basisklasse sichtbar ist.
- ☞ **Downcast:** Ist der umgekehrte Weg also von einer Basisklasse zu einer abgeleiteten Klasse. Auch diese Umwandlung ist möglich, wenn die Basisklasse sichtbar ist.

In beiden Fällen ist zu beachten, dass die Adresse, auf die ein Zeiger bzw. eine Referenz verweist, entsprechend angepasst wird, sollte eine Klasse mehr als eine Basisklasse besitzen.

```

class Base1 {
    ...
};
class Base2 {
    ...
};
class Derived : public Base1, public Base2 {
    ...
};

```



```
// convert4.cpp
#include <iostream>

struct base1 {
    int b1;
};

struct base2 {
    int b2;
};

struct derived : public base1, base2 {
    int d;
};

int main() {
    derived d;
    derived *dp1 = &d;
    base1 *b1 = dp1;
    base2 *b2 = dp1;
    derived *dp2 = static_cast<derived*>(b2);

    std::cout << dp1 << ' ' << b1 << ' ' << b2 << ' ' << dp2 << std::endl;

    return 0;
}
```

11.5. Umwandlungsmethoden

a. C: Konvertierung einer Zeichenkette in einen numerischen Wert

Für die Konvertierung einer Zeichenkette in einen numerischen Wert stellt die Standardbibliothek verschiedene Funktionen zur Verfügung.

```
double strtod( const char *s, char **endptr );
double atof( const char *s );
```

Beide Funktionen interpretieren eine Zeichenkette und geben den gelesenen Wert als double zurück. Dabei werden zuerst alle führenden Leerzeichen (white spaces) übersprungen und dann alle Zeichen aus denen eine Fließkommazahl gebildet werden können interpretiert. strtod erwartet als zweiten Parameter die Adresse eines char-Zeigers und speichert dort die Adresse des ersten nicht interpretierbaren Zeichen. Diese Funktion ist daher vorzuziehen vor allen, wenn Sie Benutzereingaben interpretieren wollen. Wenn keine Fließkommazahl erkannt wurde, wird 0 zurückgegeben.

```
int atoi( const char *s );
long atol( const char *s );
```

Diese beiden Funktionen lesen ebenfalls eine Zeichenkette, liefern jedoch die Interpretation als int bzw long int zurück. Auch hier werden zuerst führende Leerzeichen übersprungen und die Interpretation endet beim ersten ungültigen

Zeichen. Wenn kein gültiger Ganzzahlwert erkannt wurde, wird 0 zurückgegeben. Diese Funktionen eignen sich nur bedingt zur Interpretation von Benutzereingaben

```
long strtol(const char *s, char **endptr, int radix);
unsigned long strtoul(const char *s, char **endptr, int radix);
```

Diese beiden Funktionen interpretieren die Zeichenketten als long int bzw. unsigned long int. Wie bei strtod können Sie hier die Adresse eines char pointers übergeben, in dem die beiden Funktionen die Adresse des ersten nicht interpretierbaren Zeichen speichern. Darüber hinaus können Sie mit dem Parameter radix das Zahlensystem angeben, das interpretiert werden soll.

```
int sscanf( const char *buffer, const char *format, ... );
```

sscanf ist die mächtigste Funktion zum Konvertieren von Zeichenketten in numerische Werte. Sie funktioniert genau wie die bereits beschriebenen Funktionen scanf und fscanf, nur dass hier die Werte nicht von einem Eingabestrom (input stream) gelesen werden sondern von einer Zeichenkette, die Sie im ersten Argument übergeben müssen.

```
/* convert5.c */
#include <stdio.h>
#include <stdlib.h>

int main( void ) {
    {
        const char *integerString = "123456ABCD";
        int i = atoi( integerString );
        printf( "Es wurde %d eingegeben\n", i );
    }
    {
        const char *integerString = "1234ABCD";
        char *endPtr;
        long i = strtol( integerString, &endPtr, 10 );
        printf( "Es wurde %ld eingegeben\n", i );
        printf( "Falsches Zeichen %c\n", *endPtr );

        i = strtol( integerString, &endPtr, 16 );
        printf( "Es wurde %lx eingegeben\n", i );
        if( *endPtr )
            printf( "Falsches Zeichen %c\n", *endPtr );
    }
    {
        const char *floatString = "-1234.5678e-10";
        double d = atof( floatString );

        printf( "Es wurde %lg eingegeben\n", d );
    }
    {
        const char *floatString = "-1234.5678e-10ABCD";
        char *endPtr;
        double d = strtod( floatString, &endPtr );

        printf( "Es wurde %lg eingegeben\n", d );
        printf( "Falsches Zeichen %c\n", *endPtr );
    }
}
```

```
    return 0;
}
```

b. C++: Konvertierung eines Strings in einen numerischen Wert

Bei C++11 wurden mehrere freie Funktionen für verschiedene numerische Umwandlungen eingeführt:

```
int std::stoi( const std::string &str, std::size_t *pos = nullptr, int base
= 10)
long std::stol( const std::string &str, std::size_t *pos = nullptr, int
base = 10)
long long std::stoll( const std::string &str, std::size_t *pos = nullptr,
int base = 10)
```

Liefert die Zeichenkette als int, long oder long long zurück. Wenn Sie die Adresse einer Variablen vom Typ std::size_t übergeben, wird dort der Index des ersten Zeichens gespeichert, der für die Konvertierung nicht verwendet wurde. Bei Angabe einer Basis, können Sie die Basis der Zahl festlegen also zum Beispiel Dualzahl, Dezimalzahl oder Hexadezimalzahl.

```
unsigned long std::stoul( const std::string &str, std::size_t *pos =
nullptr, int base = 10)
unsigned long long std::stoull( const std::string &str, std::size_t *pos =
nullptr, int base = 10)
```

Diese Funktionen arbeiten wie die vorherigen nur liefern sie einen vorzeichenlosen Wert zurück.

```
float std::stof( const std::string &str, std::size_t *pos = nullptr )
double std::stod( const std::string &str, std::size_t *pos = nullptr )
long double std::stold( const std::string &str, std::size_t *pos = nullptr
)
```

Diese drei Funktionen arbeiten wie die vorherigen nur liefern sie eine Fließkommazahl zurück und Sie können auch die Basis nicht angeben.

c. Konvertierung eines numerischen Werts in eine Zeichenkette

```
int sprintf( char *buffer, const char *fmt, ... )
int std::sprintf( char *buffer, const char *fmt, ... )
```

Die Funktion sprintf in <stdio.h> bzw. <cstdio> kann benutzt werden, um einen beliebigen numerischen Wert in eine Zeichenkette umzuwandeln. Im Speicherbereich buffer wird das Ergebnis abgelegt. Sie müssen allerdings selbst dafür sorgen, dass ein genügend großer Speicher vorhanden ist. Im Parameter fmt geben Sie den Formatstring an, wie er auch bei printf (siehe Seite 91) verwendet wird.

```
char * itoa( int value, char *buffer, int radix );
char * ltoa( long value, char *buffer, int radix );
char * ultoa( unsigned long value, char *buffer, int radix );
```

Diese drei Routinen sind eigentlich nicht Bestandteil des C- bzw C++-Standards. Sie sind aber Teil des Posix-Standards und daher häufig auch bei anderen Compilern für Windows oder andere Systeme implementiert. Der Parameter value erwartet den

umzuwandelnden Wert. Buffer erwartet den Speicherbereich, wo die Zeichenkette abgelegt wird. Sie müssen selbst dafür sorgen, dass dieser groß genug ist. Der dritte Parameter erwartet das Zahlensystem, das verwendet werden soll.

In C++ gibt es noch diese Möglichkeit:

```
std::ostream  
std::wostream
```

Diese beiden Klassen können dazu benutzt werden, um beliebige Datentypen, sofern der entsprechende Shiftoperator definiert wurde, in Zeichenketten umzuwandeln. Erzeugen Sie einfach einen Stream, Verwenden sie eine oder mehrere der Ausgabefunktionen, flushen Sie den Schreibpuffer und holen sich anschließend die Zeichenkette. Das folgende Codefragment soll dies verdeutlichen;

```
int value = 123;  
std::ostream stream;  
stream << value;  
stream.flush();  
std::string result = stream.str();
```

Ab C++11 stehen ihnen noch die wie folgenden Routinen zur Verfügung:

```
std::string std::to_string( ... )  
std::wstring std::to_wstring( ... )
```

Hiermit können Sie die Umkehrung aufrufen von einer Zahl zu einer Zeichenkette.

d. C++ Sonstiges

Ab C++17 wurden in `<charconv>` ebenfalls weitere Methoden und Klassen zur Konvertierung von und nach Zeichenketten eingeführt.

12. Fehlerbehandlung

1980 hat uns mein erster Informatiklehrer beigebracht, dass unsere Programme auch dann stabil laufen müssen, wenn der Benutzer seinen Hintern auf die Tastatur setzt. Diese Forderung war damals wichtig, ist heute wichtig und wird es auch immer sein. Gerade heute, wo viele elektronische Geräte über das Internet miteinander vernetzt sind und bösartige Programme immer wieder versuchen, Fehler in der Programmierung für ihre eigenen Ziele auszunutzen, ist es wichtig, dass Ihre Programme auf alle erdenklichen Fehlersituationen korrekt reagieren.

Die Fehlerbehandlung erstreckt sich in drei Schritten

Fehlererkennung. Reaktion, Meldung.

Wichtig ist, dass alle Benutzereingaben peinlich genau auf ihre Korrektheit überprüft werden müssen, bevor sie weiter verarbeitet werden. Aber auch Werte, die Sie aus Dateien oder Datenbanken gelesen haben, sollten nicht ungeprüft weiter verarbeitet werden, da Sie nie sicherstellen können, dass diese Werte nicht auch außerhalb Ihrer Anwendung manipuliert oder auf Grund von Hardwarefehler zerstört wurden. In

manchen Unternehmen ist es vorgeschrieben, dass alle Programme, die auf Kundengeräten ausgeführt werden, vor der Auslieferung einen Scrumbletest bestehen müssen, bei dem gezielt gespeicherte Daten manipuliert oder zerstört werden.

Der grundsätzliche Aufbau Ihrer Funktionen sollte in etwa so zusammengesetzt sein:

Überprüfung der Argumente

```
Wenn Fehler entdeckt
    Fehlerbehandlung
    Ende mit Fehlercode
```

Fordere Ressource an

```
Wenn erfolgreich
    Verarbeitung der Argumente und Berechnung des Ergebnis
    Wenn Verarbeitung fehlerhaft
        Fehlerbehandlung
        Setze Ergebnis auf Fehlercode
    Freigabe der Ressource
sonst
    Fehlerbehandlung
    Setze Ergebnis auf Fehlercode
```

Ende mit Ergebnis

Natürlich müssen nicht in jeder Funktion alle Komponenten dieses Aufbaus enthalten sein. Wenn zum Beispiel Ihre Funktion keine Argumente benötigt, gibt es auch nichts zu überprüfen.

Wie nun die einzelnen Komponenten zu implementieren sind, hängt sehr stark von Ihren Anforderungen ab. In der Regel genügt es, die Argumente daraufhin zu überprüfen, ob der gültige Wertebereich eingehalten wurde.

Wenn die Argumente von Ihrer Anwendung berechnet wurden und ein Verlassen des gültigen Wertebereichs auf einen Programmfehler hinweist, können Sie zur Überprüfung der Argumente das Funktionsmakro `assert` verwenden. `Assert` schreibt bei der Entwicklung Ihrer Anwendung nach Erkennung eines Fehlers eine Fehlermeldung in der Regel mit dem Namen Ihrer Quelltextdatei und der Zeilennummer und beendet das Programm. Wenn Ihr Programm in einem Debugger läuft, wird meistens das Programm unterbrochen und Sie erhalten Gelegenheit, den aktuellen Zustand zu untersuchen, um so die Ursache des Problems zu ermitteln. Ist der Entwicklungstest abgeschlossen und sie fertigen eine Releaseversion Ihrer Anwendung für die Integrationstests und spätere Auslieferung, so macht `assert` nichts und benötigt daher weder Speicher noch CPU-Zeit oder sonstige Ressourcen. Intern setzt der Compiler beim Übersetzen der Releaseversion die Preprozessor-konstante `NDEBUG`, welches anzeigt, dass `assert` nichts machen muss.

Die Definition von `assert` befindet sich in `<assert.h>`. In C++ können Sie auch `<cassert>` verwenden, was den Namenskonventionen der C-Header in C++ entspricht. Der Aufruf von `assert` sieht so aus:

```
assert( condition );
```

Ein Wert für condition von 0 gilt als Fehler.

```
/*
    assert.c: demonstration of assert
*/

#include <stdio.h>
#include <assert.h>

static void showDigit( int ziffer ) {
    assert( ziffer >= 0 && ziffer <= 9 );
    printf( "%d", ziffer );
}

int main( void ) {
    showDigit( 1 );
    showDigit( 9 );
    showDigit( 6 );
    showDigit( 4 );
    showDigit( 1964 );

    return 0;
}
```

Bedenken Sie auch in C++, dass das Makro vom Preprozessor ausgewertet wird und dieser die Eigenarten von C++ nicht kennt. Enthält Ihr Aufruf von `assert` ein Template mit mehreren Templateargumenten, kann dies zu einem Syntaxfehler führen. Um das Problem zu beheben, müssen Sie Ihre Bedingung daher noch einmal in Klammern setzen:

```
assert( max<int, long>( 0,1 ) );    // Fehler
assert( ( max<int, long>( 0,1 ) ) ); // OK
```

Wie die Fehlerbehandlung zu implementieren ist, hängt natürlich sehr stark von den Anforderungen an Ihrer Funktion, Ihres Programms und Ihrer Umgebung ab. Der einfachste Fall wäre sicherlich, den Fehler zu korrigieren und dann ganz normal weiter zu machen. Wenn dies nicht möglich ist, müssen Sie entscheiden, ob und wie der Benutzer informiert werden muss. Wenn Sie den Benutzer nicht sofort informieren wollen, sollten Sie an dieser Stelle dennoch auf jeden Fall darauf achten, dass alle Informationen, die der Benutzer eventuell benötigt, um das Problem zu beseitigen erhalten bleiben. Schon häufig habe ich Programme erlebt, die mit dem Fehler, dass sie eine Datei nicht öffnen können, beendet werden, den Namen der Datei aber verschweigen.

Ob und wie Sie die aufrufende Funktion über einen Fehler informieren, müssen Sie natürlich selbst entscheiden. Bei Funktionen, die positive Integers zurückgeben sollen, eignen sich zum Beispiel negative Werte für bestimmte Fehler. Funktionen, die einen Zeiger zurückgeben, ist der Wert `NULL` bzw. `nullptr` gut geeignet.

Die Standardbibliothek von C gibt im Fehlerfall -1 oder `NULL` zurück und schreibt in der globalen Variable `errno` einen Code für die genauere Begründung. Wenn ein Fehler in der Standardbibliothek auftritt, sollten Sie sofort `errno` in einen eigenen Speicherbereich kopieren, da sonst jeder weitere Aufruf einer Standardfunktion den Wert von `errno` überschreiben kann.

Um `errno` verwenden zu können gibt es verschiedene Standardfunktionen und Macros, die die Auswertung erleichtern. `Errno` selbst ist ein Präprozessormakro und in der Includedatei `<errno.h>` definiert. In C++ können Sie stattdessen auch die Includedatei `<cerrno>` verwenden.

Die Funktion `perror` ist in den Includedateien `<stdio.h>` (C) und `<cstdio>` (C++) deklariert. Ihr Prototyp lautet:

```
void perror( const char *text );
```

Sie gibt den übergebenen Text gefolgt von einem Erklärungstext am Standardfehlerkanal aus.

Die Beispiele in C und C++ weichen voneinander ab. Daher bringe ich hier beide.

```
/*
    perror.c: perror example
*/
#include <stdio.h>
int main( void ) {
    FILE *f = fopen( "gibts nicht", "r" );
    if( NULL != f ) {
        fclose( f );
    }
    else {
        perror( "gibts nicht" );
    }
    return 0;
}

/*
    perror.cpp: perror example
*/
#include <cstdio>
#include <fstream>
int main() {
    std::ifstream f( "gibts nicht" );
    if( !f.is_open() ) {
        std::perror( "gibts nicht" );
    }
    return 0;
}
```

Der große Nachteil von `perror` ist, dass es die Fehlermeldung an der Konsole bzw genauer gesagt am Standardfehlerkanal ausgibt. Dies ist nicht immer sinnvoll, daher gibt es im Standard auch die Funktion `strerror`, welche einen Zeiger auf einen Fehlertext zurückgibt, der zum übergebenen Fehlercode passt.

Die Funktion `strerror` ist in den Includedateien `<string.h>` (C) und `<cstring>` (C++) deklariert. Ihr Prototyp lautet:

```
char *strerror( int errCode );
```

Trotz des zurückgegebenen Typs darf der Text nicht verändert werden (korrekt wäre also `const char *` statt `char *`) und der Text kann auch bei jedem weiteren Aufruf überschrieben werden. Sie sollten ihn also sofort weiterverarbeiten.

```
/*
    strerror.c: strerror example
*/

#include <errno.h>
#include <stdio.h>
#include <string.h>

static void myPerror( const char *text ) {
    fprintf( stderr, "Fehler bei %s: %s\n", text, strerror(errno) );
}

int main( void ) {
    FILE *f = fopen( "gibts nicht", "r" );
    if( NULL != f ) {
        fclose( f );
    }
    else {
        myPerror( "gibts nicht" );
    }

    return 0;
}

/*
    strerror.cpp: strerror example
*/

#include <cstring>
#include <cerrno>
#include <iostream>
#include <fstream>

static void myPerror( const char *text ) {
    std::cerr << "Fehler bei " << text << ": " << std::strerror(errno) <<
std::endl;
}

int main() {
    std::ifstream f( "gibts nicht" );
    if( !f.is_open() ) {
        myPerror( "gibts nicht" );
    }
    return 0;
}
```

Andere Funktionen ab C/C++11

Ab C++11 werden in `<system_error>` verschiedene Klassen und Funktionen definiert, um mit verschiedenen Fehlerkategorien besser umgehen zu können.

Ab C++23 werden im Header `<stacktrace>` verschiedene Funktionen und Typen definiert bzw. deklariert, die es erlauben, die Aufrufhierarchie zu bestimmen und in einer Fehlermeldung auszugeben. Dies ist für die Fehlersuche vor allen außerhalb eines Debuggers sehr hilfreich.

Ab C++26 mit Hilfe von `<contracts>` verschiedenen Bedingungen konfiguriert werden, wie ein Programmfehler erkannt wird, und wie darauf reagiert werden soll. Darüber hinaus kann mit `<debugging>` verschiedenen Aktionen gestartet werden, wenn das Programm in einem Debugger läuft.

In C++ werden sehr häufig sogenannte Exceptions „geworfen“ (throw), wenn ein Fehler auftritt. Daher müssen Sie, wenn Ihre Funktion Ressourcen freizugeben hat, diese auffangen. Ihr Verarbeitungsblock muss dann so aussehen:

```
try {
    Verarbeitung;
    Freigabe;
}
catch( ... ) {
    Freigabe;
    Fehlerbehandlung;
}
```

Wie Sie hier sehen können, müssen Sie die Freigabe Ihrer Ressourcen zweimal codieren. Da es außerdem in C++ möglich ist, mehrere catch-Blöcke dem try-Block folgen zu lassen, muss die Freigabe entsprechend häufig codiert werden. Daher ist es in der Regel am besten, die Handles der Ressourcen in Klassen zu speichern, die beim Beenden der Funktion die Ressourcen automatisch wieder freigeben. Die Standardbibliothek von C++ bietet eine ganze Reihe vordefinierter Klassen, die Sie für diesen Zweck benutzen können.

Das Schlüsselwort `try` leitet einen try-catch-Block ein. Es folgen in geschweiften Klammern die Anweisungen, welche eine Ausnahme auslösen könnten. Danach folgen ein oder mehrere catch-Blöcke.

Ein catch-Block wird durch das Schlüsselwort `catch` eingeleitet. Danach folgt in Klammern eine Variablendefinition, die nur innerhalb des catch-Blocks gültig ist. Anschließend folgen in geschweiften Klammern die Anweisungen, die ausgeführt werden, wenn die entsprechende Ausnahme ausgelöst wurde.

Mit dem Schlüsselwort `throw` können Sie eine beliebige Variable oder ein temporäres Object „werfen“, diese wird dann vom ersten catch-Block in der aktuellen Aufrufhierarchie aufgefangen, der in seiner Variablendefinition den selben Typ benutzt. Bei einem Klassenobjekt wird auch eine Basisklasse akzeptiert. Beim Werfen einer Exception können Sie das Objekt frisch erzeugen, es muss also nicht unbedingt vorher in einer Variablen gespeichert werden. Innerhalb eines catch-Blocks kann die Ausnahme auch weggelassen werden. In diesem Fall wird die eben gefangene Ausnahme wieder weitergegeben.

```
Try {
    /// TODO
} catch( ... ) {
    /// TODO
    throw;
}
```

Eine besondere Form des catch-Blocks enthält keine Variablendefinition sondern drei

Punkte. Dies bewirkt, dass Sie wirklich jeden Fehler auffangen können, der nicht schon von einem anderen catch-Block behandelt werden kann. Allerdings haben Sie dann innerhalb des catch-Blocks keine Informationen über die Art des Fehlers. Da Sie aber mit throw ohne Angabe einer Ausnahme diese trotzdem weitergeben können, ist dies völlig ausreichend, wenn Sie in Ihrem catch-Block nur die Ressourcen wieder freigeben wollen.

Grundsätzlich können Sie zwar jeden beliebigen Datentyp für Ihre Ausnahmen benutzen, es ist aber üblich für Ihre Ausnahmen eine von der Standardklasse exception abgeleitete Klasse zu verwenden. Sie können entweder eine selbst definieren oder eine der im Standard definierten Ausnahmen verwenden: Die Standardklassen sind im Header <exception> oder <stdexcept> definiert.

1. logic_error
 1. domain_error
 2. invalid_argument
 3. length_error
 4. out_of_range
 5. future_error (ab C++11)
 6. bad_optional_access (ab C++17)
 2. runtime_error
 1. range_error
 2. overflow_error
 3. underflow_error
 4. regex_error (ab C++11)
 5. system_error (ab C++11)
 1. ios_base::failure (ab C++11)
 2. filesystem::filesystem_error (ab C++17)
 3. bad_typeid
 4. bad_cast
 1. bad_any_cast (ab C++17)
 5. bad_weak_ptr (ab C++11)
 6. bad_function_call (ab C++11)
 7. bad_alloc (definiert in <new>)
 1. bad_array_new_length (ab C++11, definiert in <new>)
-

8. `bad_exception`9. `ios_base::failure` (bis C++11)10. `bad_variant_access` (ab C++17)

All diese Klassen erwarten in Ihrem Konstruktor eine Zeichenkette, in der Sie genauere Informationen über den Fehler angeben sollten. Die Zeichenkette wird vom Konstruktor kopiert, sollten Sie diese Zeichenkette dynamisch alloziert haben, müssten Sie diesen Bereich selbst wieder freigeben, was natürlich nach dem Werfen der Ausnahme nicht möglich ist. Da hierfür aber üblicherweise entweder eine Zeichenkettenliteral oder die Standardklasse `string` verwendet wird, soll uns das hier nicht mehr weiter interessieren.

Die Basisklasse `exception` definiert eine virtuelle Memberfunktion `what`, die einen Zeiger auf diesen Text wieder zurückgibt.

```
virtual const char *what();
```

Der zurückgegebene Zeiger ist natürlich nur solange gültig, wie das Exceptionobjekt existiert, also nur innerhalb des `catch`-Blocks, das die Ausnahme gefangen hat.

```
/*
    exception1.cpp: example for try, catch and throw
*/
#include <iostream>
#include <exception>

static void werfer() {
    throw std::range_error( "Eine Ausahme" );
}

int main() {
    try {
        werfer();
        std::cout << "Keine Ausnahme" << std::endl;
    }

    catch( std::range_error &e ) {
        std::cerr << "range_error: " << e.what() << std::endl;
    }

    catch( std::runtime_error &e ) {
        std::cerr << "runtime_error: " << e.what() << std::endl;
    }

    catch( std::exception &e ) {
        std::cerr << "exception: " << e.what() << std::endl;
        throw;
    }

    catch( ... ) {
        std::cerr << "Unbekannter Fehler" << std::endl;
        throw;
    }

    return 0;
}
```

Der Vorteil von Exceptions ist die Möglichkeit genügend Informationen zu speichern, damit entweder der Benutzer eine aussagekräftige Fehlermeldung erhalten kann oder eine Funktion Ihres Programms den Fehler möglicherweise selbst beheben kann. Darüber hinaus können Funktionen, die weder selbst Fehler generieren noch explizit irgendwelche Ressourcen freigeben müssen, die Fehlerbehandlung völlig ignorieren, was wiederum für die Lesbarkeit Ihres Quelltexts sehr dienlich ist.

Wichtig ist, versuchen Sie Ihr Programm zu implementieren, dass jeder Fehler auch erkannt wird. Wenn möglich sollte Ihr Programm den Fehler selbst korrigieren, wenn das nicht möglich ist, sollte eine Fehlermeldung generiert werden, die dem Benutzer möglich alle Informationen gibt, die er/sie braucht, um den Fehler zu korrigieren. Nichts ist schlimmer, wenn das System schreibt:

„Cannot find the file specified“

aber verschweigt, welche Datei eigentlich gesucht wurde.

In C++11 können Sie eine Funktion so definieren bzw. deklarieren, so dass angezeigt wird, ob eine Exception geworfen wird oder nicht. Hierfür können die Schlüsselworte `noexcept` oder `throw` verwendet werden. `noexcept`, `noexcept(<ausdruck>)` oder `throw()` können verwendet werden. Ausdruck muss dabei zu einem boolschen Wert `true` oder `false` ausgewertet werden. Wenn dieser `true` ist, kann die Funktion keine Exception auslösen. `noexcept` und `throw()` ist gleichbedeutend mit `noexcept(true)`. Bis C++17 konnte bei `throw()` auch eine Liste von Exception angegeben werden, die eine Funktion auslösen kann.

13. Speicherverwaltung

13.1. Konzepte

Bei allen Programmen geht es darum, Daten zu lesen, speichern, diese wieder zu finden, laden, verändern und wieder zu speichern. Hier betrachten wir grundlegende Konzepte, wobei tatsächlich das Speichermedium selber übertragbar ist. Das heißt die gleichen Konzepte finden bei Magnetbändern genauso Anwendung wie im Arbeitsspeicher des Computers.

Im Array werden alle Daten in einem einzigen Block gespeichert. Wenn neue Daten hinzugefügt werden sollen und dieser Block nicht groß genug ist, muss ein neuer Block angefordert, die alten Daten vom alten Block in den neuen kopiert und dann die neuen Daten eingefügt werden. Wenn Sie ein C-Array benutzen wollen, müssen Sie schon zur Übersetzungszeit wissen, wie groß es sein soll. Ein nachträgliches Vergrößern oder verkleinern ist nicht möglich. Der Zugriff auf ein einzelnes Element ist dafür aber recht schnell, da Sie nur den Anfang und den Index kennen müssen. Aus dem Anfang und den Index errechnet sich einfach und schnell die Adresse des gesuchten Objekts. Selbst bei einem Magnetband kann dann der Datensatz erstaunlich schnell gefunden werden., da einfach nur an die korrekte Stelle gespult werden muss. Das Löschen einzelner Objekte kann dafür recht lange dauern, da alle nachfolgenden Objekte verschoben werden müssen.

Einfach verkettete Listen speichern zu jedem Objekt die Adresse des nächsten

Objekts. Zur Verwaltung einer Liste speichern Sie die Adresse des ersten (head) und des letzten (tail) Objekt der Liste. Wenn neue Objekte angehängt werden sollen, muss nur ein Block für ein Objekt und den Zeiger angefordert werden und das letzte Objekt muss auf den neuen Block verweisen. Der Zeiger im letzten Block ist dann immer ein Nullzeiger. Das Hinzufügen von neuen Objekten geht relativ schnell, wenn man sich neben dem ersten Objekt immer auch das letzte merkt. Zum Zugriff auf ein bestimmtes Element mit dem Index *i* müssen Sie beginnend vom Anfang der Liste den Zeiger zum nächsten Objekt *i*-mal dereferenzieren bis das gewünschte Objekt gefunden wird. Der folgende Codeschnippel soll das demonstrieren.

```
extern MyStruct *head;
size_t index = 123;
MyStruct *ptr = head;
for( size_t i = 0; i<index; ++i ) {
    if( !ptr ) {
        // TODO: Fehlermeldung
        break;
    }
    ptr = ptr->next;
}
```

Beim Löschen eines Objektes muss der Zeiger des Vorgängers auf den Nachfolger geändert werden. Wenn es gar keinen Vorgänger gibt, muss der Kopfzeiger stattdessen geändert werden.

```
extern MyStruct *head;
// TODO:: find object and its previous
MyStruct *prev = ...;
MyStruct *ptr = ...;
if( prev ) {
    assert( prev->next == ptr );
    prev->next = ptr->next;
}
else
    head = ptr->next;
```

Doppelt verkettete Listen arbeiten wie die einfachen Listen nur werden bei jedem Objekt nicht nur ein Zeiger auf den Nachfolger sondern auch auf den Vorgänger gespeichert. Dadurch kann das Löschen robuster implementiert werden, da Sie nur den Zeiger auf das zu löschende Objekt benötigen und eventuell vorhandene Fehler können leichter erkannt und behoben werden. Der Nachteil dafür ist, dass die Implementierung etwas aufwendiger ist und vor allem, dass Sie pro Objekt noch einen zusätzlichen Zeiger brauchen.

Arrays sind immer dann nützlich, wenn kleine Datenstrukturen verwaltet werden müssen. Mehrere Integers in einer verketteten List ist eine ziemliche Speicherverschwendung, da Sie zu jedem Integer noch ein oder gar zwei Zeiger benötigen, die auch nicht gerade klein sind. Auch ist der wahlfreie Zugriff auf einzelne Elemente unschlagbar schnell, die dafür benötigte Zeit ist nämlich im Arbeitsspeicher immer gleich. Wenn Sie Arrays auch mit einer ausreichenden Größe allozieren, kann der Nachteil beim Anfügen neuer Elemente minimiert werden. Ein weiteres Problem kann

sein, dass Zeiger auf einzelne Elemente im Array plötzlich ungültig werden oder auf falsche Elemente zeigen, wenn neue Elemente hinzugefügt oder alte Elemente entfernt werden. Ein weiteres Problem beim Arbeiten mit Array kann sein, wenn Sie häufig neue Daten nicht anhängen sondern mitten drinnen einfügen müssen. Dann müssen auch möglicherweise viele Elemente verschoben werden.

Wenn die Nachteile den Vorteilen beim wahlfreien Zugriff überwiegen, können Sie aber darüber nachdenken, einfach oder doppelt verkettete Listen zu verwenden. Verkettete Listen benötigen pro Objekt ein oder gar zwei zusätzliche Zeiger, die wieder Speicher brauchen. Die Verwaltung von Arrays braucht unabhängig von der Anzahl der Objekte immer gleich viel Speicher, nämlich den Zeiger auf das erste Objekt und die Anzahl der Objekte. Verkettete Listen brauchen mindestens den Zeiger auf den Anfang (head) besser noch zusätzlich den Zeiger auf das Ende (tail). Eine gute Idee, vor allem bei doppelt verketteten Listen, ist auch die Speicherung der Anzahl der Objekte. Achten Sie dann aber darauf, dass diese wirklich immer korrekt gepflegt ist. Falsche Daten sind nämlich oft schlimmer als gar keine Daten, die berechnet werden können.

Alternativ kann auch über eine Kombination aus Arrays und verketteten Listen nachgedacht werden. Bei den C++ Containern finden Sie eine Struktur, die beide Konzepte miteinander verbindet.

Neben den gerade angesprochenen Strukturen gibt es noch weitere Konzepte, die darauf aufbauen.

Ein Stack oder Stapel ist eine sogenannter LIFO Struktur. LIFO steht für last in first out. Das können Sie sich wie einen Stapel Arbeitsblätter vorstellen. Beim Erteilen neuer Aufträge wird ein neuer Zettel oben auf dem Stapel abgelegt. Beim Bearbeiten der Aufträge werden diese Blätter von oben wieder entnommen. Der erste Auftrag wird als letztes bearbeitet. Um einen Stack zu realisieren brauchen Sie vom Prinzip nur einen Zeiger. Wenn ein neues Objekt hinzugefügt werden soll, wird der Zeiger inkrementiert und dann das neue Objekt an dieser Adresse gespeichert. Beim Entnehmen des letzten Objekts wird der Zeiger zuerst dereferenziert, die Daten gelesen und dann der Zeiger dekrementiert. Grundsätzlich kann das aber auch genau anders herum realisiert werden. Sie müssen nur aufpassen, dass Lesen und Schreiben zueinander passen. Wenn Sie beim Schreiben zuerst inkrementieren und dann schreiben, wird beim Lesen zuerst gelesen und dann dekrementiert. Wenn Sie aber mögen, können Sie aber beim Schreiben zuerst schreiben und dann dekrementieren. Dann muss es beim Lesen aber wieder umgekehrt sein, also zuerst inkrementieren und dann lesen. Die anderen möglichen Kombination erspare ich mir jetzt aber hier aufzulisten.

Dem gegenüber steht die Queue oder Warteschlange. Dies ist eine FIFO-Struktur und FIFO steht, sie können es sich sicherlich denken, für first in first out. Zum Implementieren einer Queue benötigen Sie je einen Zeiger auf das erste (head) und das letzte (tail) Element. Zum Hinzufügen eines Objekts hängen Sie dieses an die Liste an und korrigieren Sie tail. Zum Lesen verwenden Sie head und korrigieren Sie diesen dann mit dem Nachfolger.

Stacks werden üblicherweise mit Arrays implementiert wohingegen Queues am

besten mit Listen implementiert werden. Bei der Implementierung des Stapels oder der Warteschlangen müssen Sie aber auf die Fälle berücksichtigen, dass Elemente entnommen werden sollen, obwohl keines vorhanden ist, oder dass Elemente hinzugefügt werden sollen, obwohl kein Speicher mehr verfügbar ist.

13.2. C Speicherverwaltung

Bei der Definition eines Arrays muss die Anzahl der Elemente als Konstante vorliegen. Wenn Sie zum Übersetzungszeitpunkt noch nicht wissen, wie viele Elemente gespeichert werden sollen, müssen Sie das Array dynamisch auf dem Heap (Haufen) anlegen. Hierzu wird in C die Standardfunktion malloc verwendet.

malloc hat den folgenden Prototypen:

```
void *malloc( size_t size );
```

malloc erzeugt ein Array im Speicher und liefert die Adresse des ersten Elements zurück. Sie müssen dann diese Adresse einem Zeiger zuweisen, damit Sie das Array verwenden können. Wenn das Array nicht mehr benötigt wird, müssen Sie es mit der Standardfunktion free wieder freigeben.

Der Prototyp von free lautet:

```
void free( void *address );
```

Beachten Sie aber, dass der Zeiger mit der Adresse danach nicht mehr gültig ist. Sie dürfen ihn daher nicht mehr benutzen ohne ihm vorher einen neuen Wert zugewiesen zu haben. Andernfalls riskieren Sie einen Datenverlust.

```
int anzahl = 100;
int *zeiger = malloc( anzahl * sizeof( int ) );
zeiger[0] = 1;
zeiger[1] = 2;
zeiger[2] = 3;
zeiger[3] = 5;
...
free( zeiger );
```

Beachten Sie: der Speicher, der mit malloc reserviert wird, wird **nicht** initialisiert. Der Inhalt der Speicherzellen ist also mehr oder weniger zufällig. Lesen sollten Sie also erst, wenn Sie vorher gültige Werte hineingeschrieben haben.

Benötigen Sie ein definiertes Verhalten beim Lesen, müssen Sie die Funktion calloc verwenden. Der Prototyp lautet:

```
void *calloc( size_t numElements, size_t size );
```

Der Speicherbereich, der mit calloc angefordert wurde, wird mit Nullen initialisiert. Ein Beispiel:

```
int *zeiger = calloc( 255, 1 );
int i;
const char *text = "Hello world";
char *c;
for( c=text; c; ++c )
    zeiger[*c]++;
...
free( zeiger );
```

13.3. C++ Speicherverwaltung

In C++ können solche dynamischen Arrays auch mit dem Schlüsselwort `new` erzeugt und mit dem Schlüsselwort `delete` wieder gelöscht werden. Auch hier gilt: Der Zeiger darf nach dem Löschen nicht mehr verwendet werden, bevor er nicht einen neuen Wert erhalten hat.

```
try {
    int anzahl = 100;
    int *zeiger = new int [anzahl];
    zeiger[0] = 1;
    zeiger[1] = 2;
    zeiger[2] = 3;
    zeiger[3] = 5;
    ...
    delete zeiger[];
}
catch( ... ) {
    delete zeiger[];
}
```

Üblicherweise werden Sie jedoch nur in Ausnahmefällen, Objekte mit `new` anfordern müssen. In den meisten Fällen genügen die in C++ definierten Standardcontainer, die im folgenden Kapitel behandelt werden.

Um einzelne Objekte einer Klasse zu erzeugen, können Sie statt der Kombination aus `new` und `delete` auch die sogenannten Smartpointer verwenden. Diese sind in `<memory>` als Template definiert.

```
std::auto_ptr<TYPE>
```

Mit diesem Template erzeugen Sie einen Zeiger auf ein verwaltetes Objekt; Im Destruktor wird der Zeiger und auch das verwaltete Objekt gelöscht. Mit den Operator `*` und `->` können Sie den Zeiger wie gewohnt Dereferenzieren. Ein kleines Codeschnipsel:

```
#include <memory>
struct MyStruct {
    int value;
};

std::auto_ptr<MyStruct> ptr( new MyStruct );
ptr->value = 123;
doSomething( *ptr );
```

In neuen Code soll diese Klasse eigentlich nicht mehr verwendet werden und neuen

Compiler darf auch die Unterstützung entfernt werden.

Als Ersatz für `auto_ptr` kann ab C++11

```
std::unique_ptr<TYPE [, DELETER]>
```

verwendet werden. Der Templateparameter `TYPE` gibt wieder die Art des Zeigers, der erzeugt werden soll. Der optionale Parameter `DELETER` gibt die Funktion an, die zum Löschen verwendet werden soll. Sie soll uns hier nicht weiter interessieren.

Wie `auto_ptr` kann auch beim `unique_ptr` mit den Operatoren `*` und `->` dieser wie ein normaler Zeiger benutzt werden. Mit dem freien Funktionstemplate `make_unique` kann ein neues Objekt `unique_ptr` erzeugt werden.

```
#include <memory>
struct MyStruct {
    int m_value;
    MyStruct(int value) : m_value(value) {}
};
std::unique_ptr<MyStruct> ptr = std::make_unique<MyStruct>( 123 );
doSomething( *ptr );
```

Die Zuweisung eines `unique_ptr`-objects ist nicht erlaubt. Dafür können Sie das Klassentemplate

```
std::shared_ptr<TYPE>
```

verwenden. `Shared_pointer` kann genauso verwendet werden wie `unique_pointer` jedoch haben Sie die Möglichkeit mehr als einen Zeiger auf das verwaltete Objekt zu verwenden. Erst wenn der letzte Zeiger gelöscht wurde, wird auch das verwaltete Objekt entfernt. Wie auch bei `unique_ptr` gibt es ein Funktionstemplate zum Erzeugen neuer Zeiger.

```
#include <iostream>
#include <memory>
struct MyStruct {
    int m_value;
    MyStruct(int value) : m_value(value) {}
    ~MyStruct() { std::cout << "Here is my end 8-(\n" }
};
{
    std::shared_ptr<MyStruct> longStorage;
    {
        std::shared_ptr<MyStruct> ptr = std::make_shared<MyStruct>( 123 );
        doSomething( *ptr );
        longStorage = ptr;
    }
    std::cout << "Here is the end ptr\n";
}
std::cout << "Here is the end longStorage\n";
```

Eine Falle auf die Sie achten müssen, sind Objekte, die sich gegenseitig mit einem `shared_pointer` referenzieren. Wenn Sie da keinen zusätzlichen Mechanismus implementieren, kann es ihnen passieren, dass diese Objekte nie wieder freigegeben werden und Sie einen memory leak haben. Ich hatte diesen Fall in meinem DMS als ich dort smart pointer eingeführt habe. Ein Ordner hatte eine Liste aller Kinder in

einem Array von `shared_pointer` gespeichert und jedes Kind hat sein Elternobjekt als `shared_pointer` abgelegt. Das Fenster, das den Ordnerinhalt angezeigt hat, hat zwar seinen smart pointer auf den Ordner gelöscht. Da die Kinder aber immer noch einen `shared pointer` auf diesen Ordner hatten, wurde der Ordner nicht gelöscht und deshalb wurden auch die Kinder nicht gelöscht. Am Ende war mein Speicher mit lauter ungenutzten Objekten gefüllt.

Natürlich kann noch einiges mehr über smart pointer geschrieben werden, für das erste in dieser Einführung soll es hier aber genügen.

In der Headerdatei `<new>` gibt es neben nützlichen Funktionen, die nur selten gebraucht werden, und daher hier nicht behandelt werden sollen, die Definition zweier Exceptionklassen, die sie verwenden können, wenn Sie eine besondere Behandlung von Speicherfehler wünschen.

Ab C++11 finden Sie in `<scoped_allocator>` und ab C++17 in `<memory_resource>` weitere lowlevel Hilfsfunktionen und -klassen zur Speicherverwaltung, die nur in besonderen Fällen notwendig sind.

14. Nützliches

In C++ sind einige nützliche Basisklassen oder -templates definiert, die immer wieder in anderen Klassen oder Templates verwendet werden, Ich werde hier die wichtigsten vorstellen.

14.1. Variable Anzahl Argumente

Falls es erforderlich sein sollte, können Sie eine Funktion so definieren, dass sie eine variable Anzahl an Argumente akzeptiert. `Printf` und `scanf` sind Beispiele der Standardbibliothek. Grundsätzlich sollten Sie das nur machen, wenn es unbedingt erforderlich ist, da der Compiler hier beim Aufruf weniger Prüfmöglichkeiten hat.

Alle Funktionen und Makros sind in `<stdarg.h>` bzw. `<cstdarg>`. Diese müssen also vor der Benutzung inkludiert werden. Bei der Definition schreiben Sie in der Parameterliste der Funktion mindestens einen festen Parameter gefolgt von einem Komma und 3 Punkten statt weiterer Parameter.

```
void funktion( int param1, int oper, ... )
```

Im Funktionskopf deklarieren Sie eine lokale Variable vom Typ `va_list`.

```
va_list  args;
```

Bevor Sie ein Argument lesen können, muss die eben deklarierte lokale Variable initialisiert werden. Dazu benutzen Sie das Makro `va_start`

```
va_start( args, oper );
```

Dann können Sie Argumente mit dem Makro `va_arg` lesen. `va_arg` benötigt als erstes Argument den Namen unserer `va_list` Variable und als zweites Argument den Typ des gewünschten Funktionsarguments. Grundsätzlich müssen Sie die Argumente in der gleichen Reihenfolge lesen, in der sie geschrieben wurden.

```
int operand = va_arg( args, int );
```

Am Ende müssen Sie mit dem Makro `va_end` den Vorgang abschließen. Danach dürfen Sie die `va_list` Variable nicht mehr benutzen.

```
va_end(args);
```

Achten Sie auf jeden Fall darauf, dass die Argumente in der gleichen Reihenfolge gelesen werden wie sie geschrieben wurden. Auch sehr wichtig ist, dass sie mit dem korrekten Typ gelesen werden. Andernfalls ist das Verhalten undefiniert. Ein einfaches Beispiel:

```
// vaargsdemo.c: va_args demonstration
#include <stdio.h>
#include <stdarg.h>

int superfunc( int oper, ... ) {
    va_list  args;
    int  result = oper;
    va_start( args, oper );
    while( 1 ) {
        int operand = va_arg( args, int );
        if( !operand )
            break;

        if( oper )
            result *= operand;
        else
            result += operand;
    }
    va_end(args);
    return result;
}

int main( void ) {
    int result1 = superfunc( 1, 1, 2, 3, 4, 5, 0 );
    int result2 = superfunc( 0, 1, 2, 3, 4, 5, 0 );
    printf( "%d %d\n", result1, result2 );
    return 0;
}
```

14.2. C++ Ratio

Seit C++11 wurde das Hilfstemplate `std::ratio` im Header `<ratio>` definiert.

```
template<std::intmax_t Num, std::intmax_t Denom = 1> class ratio;
```

In der Headerdatei sind auch einige Hilfstypen für kilo, milli etc. definiert, so dass einfach physikalische Größen umgerechnet werden können.

```
/* ratio.cpp - demonstration of ratio template */
#include <iostream>
#include <ratio>
```

```

template <typename FROM, typename TO>
double convert( double value ) {
    value *= double(FROM::num) / double(TO::num);
    value *= double(TO::den) / double(FROM::den);
    return value;
}

int main() {
    double source = 1;

    double target = convert<std::kilo, std::milli>( source );
    std::cout << source << ' ' << target << std::endl;

    target = convert<std::kilo, std::mega>( source );
    std::cout << source << ' ' << target << std::endl;

    target = convert<std::kilo, std::ratio<1> >( source );
    std::cout << source << ' ' << target << std::endl;

    return 0;
}

```

14.3. C++ Hash

Ein Hash ist eine Bytefolge, die man benutzen kann, um die Gültigkeit einer größeren Datenstruktur überprüfen zu können. Motivation ist, dass ein Datenblock, der von einer anderen Stelle übertragen wurde, überprüft werden muss, dass er fehlerfrei übertragen wurde. Dazu berechnet der Sender eines Datenblocks einen Hash mit einem vorgegebenen Algorithmus und überträgt diesen Hash zusammen mit dem Datenblock. Der Empfänger kann nun den Hash mit dem selben Algorithmus ebenfalls berechnen und wenn beide Hashs identisch sind, kann der Empfänger davon ausgehen, dass der Datenblock korrekt übertragen wurde. Je nach Qualität der Hashfunktion können mehrere Fehler erkannt werden oder sich sogar gegenseitig aufheben. Daneben ist es auch wichtig zu wissen, ob eine Hashfunktion kryptografischen Ansprüchen genügt. Denn einfache Hashfunktionen erlauben es, einen Block so zu verändern, dass der Hash aber trotzdem sich nicht ändert. Dadurch sind auch illegale Veränderungen des Datenblocks möglich, die vom Empfänger nicht erkannt werden.

Hash ist ein Template, erwartet den Objekttyp als Templateargument und ist nur eine forward declaration für die Struktur `std::hash`. Für verschiedene Standardklassen gibt es eine eigene Spezialisierung. Für Ihre eigenen Klassen müssen Sie eigene Spezialisierungen definieren. Die Spezialisierungen müssen den Funktionsoperator überladen, der ein Rvalue vom Typ des Templateargument erwartet und ein `size_t` zurückgibt.

Der Standard sagt nichts über die Qualität der Hashfunktion aus. Daher empfehle ich, diesen Hash nur zum Schutz vor Hardwarefehlern und Irrtümern zu verwenden. Bei stärkeren Anforderungen empfehle ich daher andere Bibliotheken wie zum Beispiel `openssl` zu verwenden.

```

#include <cstdint>
#include <functional>
#include <iostream>
#include <string>

```

```

struct Employee {
    std::string name;
    std::uint64_t ID;
};

namespace std {
    template <>
    class hash<Employee> {
    public:
        std::size_t operator()(const Employee& employee) const {
            // computes the hash of an employee using a variant
            // of the Fowler-Noll-Vo hash function
            constexpr std::uint64_t prime{0x1000000001B3};
            std::size_t result{0xcbf29ce484222325};
            for (std::size_t i=0, ie = employee.name.size(); i!=ie; ++i)
                result = (result * prime) ^ employee.name[i];
            return result ^ (employee.ID << 1);
        }
    };
}

int main() {
    Employee employee;
    employee.name = "Zaphod Beeblebrox";
    employee.ID = 42;
    std::hash<Employee> hash_fn;
    std::cout << hash_fn(employee) << '\n';
}

```

14.4. C++: Funktionsobjekte, Lambdaausdrücke

Funktionsobjekte sind Instanzen von Klassen, die mindestens einen Funktionsoperator überladen haben.

```
class MyStruct { int operator () (int); };
```

Ein Objekt dieser Klasse, kann wie eine Funktion benutzt werden.

```
MyStruct object; int result = object( 123 );
```

Der Vorteil dieser Funktionsobjekte ist vor allem bei der Templateprogrammierung spürbar, weil Funktionszeiger und Funktionsobjekt leicht ausgetauscht werden können.

```

template <typename SUPERTYPE>
void process( SUPERTYPE &processor ) {
    processor(123);
}

```

Der Funktion process kann nun sowohl ein Funktionszeiger als auch ein Funktionsobjekt übergeben werden.

Je nach Erfordernis wird der Compiler für unterschiedliche Aufrufe auch die passenden Funktionen generieren.

Im Header <functional> werden nützliche Klassen, Funktionen und Templates definiert bzw. deklariert, die in dieser Einführung aber nicht näher betrachtet werden

sollen..

Ab C++ 11 ist es möglich namenlose Funktionsobjekte zu definieren. In der einfachsten Form wird ein Lambdaausdruck mit zwei Teilen definiert:

```
[captures] { function body }
```

Function-body ist wie ein ganz normaler Functionsbody, in dem Sie auch lokale Variablen definieren können. Captures beschreiben wie auf Variablen von außerhalb innerhalb des function body zugegriffen werden kann. Captures haben die Form:

```
capture-default  
capture-list  
capture-default, capture-list
```

capture-default ist ein = oder &. = (Zuweisung) besagt, dass alle Variablen nur gelesen werden können. & (Referenz) bewirkt, dass die Variablen auch geändert werden können. Capture-list ist eine Komma separierte Liste von Variablen, die gelesen werden können. Ist vor einen Eintrag der Referenzoperator, kann die Variable auch verändert werden.

```
[=] { std::cout << variable; }
```

Optional kann dem Lambdaausdruck noch wie bei Funktionen eine Parameterliste zwischen den captures und dem fuction body hinzugefügt werden.

```
[captures] (parameter-list) { function body }
```

Die Namen der Parameter müssen sich aber von den Namen der captures unterscheiden.

```
[=] (int factor) { std::cout << variable * factor; }
```

Der return type des Aufrufs kann entweder mit dem trailing type angegeben werden oder er wird automatisch ermittelt, wenn der function body nur aus einem return statement besteht. Der trailing type ist ein Pfeil -> gefolgt vom Typ.

```
[=] (int factor) -> int { return variable * factor; }
```

Auch für die Lambdaexpressions gilt das gleiche wie bei den Funktionsobjekten, dass Sie leicht eingesetzt werden können, wenn auch ein Funktionszeiger möglich ist.

```
// lambda.cpp: demo for lanbdaexpressions  
#include <iostream>  
  
template <typename TYPE>  
int perform( const TYPE &proc, int param ) {  
    return proc( param );  
}
```

```
int main() {
    int result;
    int capture = 2;
    int ret = perform(
        [=, &result](int param) -> int {
            result = param * capture;
            return result;
        }, 4
    );
    std::cout << ret << ' ' << result;
    return 0;
}
```

Das soll es für diese Einführung gewesen sein. Ich denke die wichtigsten Aspekte der Lambdaexpressions sollten hier betrachtet worden sein.

15. C++ Container

15.1. Container Templates

Zum Speichern mehrere Objekte auf einmal haben Sie in C nur das Array, das Sie entweder lokal in einer Funktion oder global für alle Funktionen definieren oder per malloc bzw calloc anfordern und mit free wieder frei geben müssen. In C++ gibt es dafür das Konzept der Container, das Sie benutzen können, um mehrere Objekte auf einmal zu speichern. Diese Container übernehmen dann, die Anforderung des Speichers und geben diesen auch wieder frei, wenn er nicht mehr benötigt wird. Es gibt eine ganze Reihe von Containerklassen mit unterschiedlichen Verhalten und Speicherlayout, die aber alle ähnlich funktionieren. Weshalb es auch viele Gemeinsamkeiten gibt.

Container sind grundsätzlich als Template definiert mit dem Datentyp den Sie speichern wollen als ersten Templateparameter. D.H. Sie können grundsätzlich nur Objekte vom gleichen Typ in den Container ablegen. Wenn Sie unterschiedliche Objekte ablegen wollen, können Sie eigentlich nur über einen Trick über Zeiger bzw. Smartpointer vorgehen.

Sequenz Container dienen dazu die Elemente der Reihe nach zu bearbeiten

std::vector	Ein dynamisch angelegtes C-Array von Objekten. Die Größe kann zur Laufzeit variieren. Der Container ist im Header <vector> definiert.
std::array (C++11)	Ein C-Array fester Größe. Ist vom Prinzip das gleiche wie das klassische C-Array selbst, da es aber auch viele Funktionen wie std::vector auch kennt, ist es manchmal ganz nützlich. Der Container ist im Header <array> definiert.

<code>std::deque</code>	Ein deque arbeitet wie ein vector, allerdings werden nicht alle Objekte in einem Speicherblock gespeichert, sondern es werden mehrere kleinere Blöcke verwendet. Das hat den Vorteil, dass wenn ein neues Objekt hinzugefügt werden soll und der reservierte Speicher nicht groß genug ist, muss nicht der ganze Speicher umkopiert werden, sondern es wird einfach nur ein neuer Block angefordert. Der indizierte Zugriff auf einzelne Elemente dauert dafür länger, da zusätzliche Verwaltung notwendig ist. Der Container ist im Header <code><deque></code> definiert.
<code>std::list</code>	Doppelt verkettete Liste, der Container ist im Header <code><list></code> definiert.
<code>std::forward_list</code> (C++11)	Einfach verkettete Liste, der Container ist im Header <code><forward_list></code> definiert.

Assoziativ Container speichern die Objekte als Key-Value-Pair, so dass einzelne Elemente leicht gefunden werden können.

<code>std::set</code>	Set speichert nur den Schlüssel. Diese sind eindeutig, die Elemente sortiert. Der Container ist im Header <code><set></code> definiert.
<code>std::map</code>	Die Schlüssel sind eindeutig, alle Elemente sind nach den Schlüsseln sortiert. Der Container ist im Header <code><map></code> definiert.
<code>std::multiset</code>	Wie set nur sind die Schlüssel nicht eindeutig. Der Container ist im Header <code><set></code> definiert.
<code>std::multimap</code>	Wie map nur sind die Schlüssel nicht eindeutig. Der Container ist im Header <code><map></code> definiert.

Seit C++11 gibt es auch die unsortierte Varianten der eben gezeigten Container, die aber ein anderes Laufzeitverhalten haben. Je nachdem welche Daten Sie haben und wie häufig Daten entfernt, hinzugefügt oder gelesen werden, kann sich eine Verbesserung ergeben. Die neuen Templates heißen dann einfach `unordered_set`, `unordered_map`, `unordered_multiset` und `unordered_multimap`.

Container Adapter verwenden die oben genannten Container, um verschiedene Konzepte zu implementieren.

<code>std::stack</code>	Nützlich um einen Stapel (Seite 155) zu implementieren. Der Container ist im Header <code><stack></code> definiert.
<code>std::queue</code>	Hiermit wird eine Warteschlange (Seite 155) implementiert. Der Container ist im Header <code><queue></code> definiert.
<code>std::priority_queue</code>	Hiermit wird eine besondere Warteschlange definiert, in der manche Objekte bevorzugt behandelt werden. Der Container ist im Header <code><queue></code> definiert.

Viele Container haben die gleichen Memberfunktionen wie wir sie schon bei den Klassen `std::string` und `std::wstring` gesehen haben. In der folgenden Tabelle wollen wir die wichtigsten member, die viele Container unterstützen, auflisten.

Name	Beschreibung
<code>assign</code>	<code>assign(std::size_t count, const T&value)</code> Ersetzt den kompletten Inhalt des Containers durch die angegebene Anzahl an Kopien des übergebenen Objekts.
<code>begin</code> <code>cbegin</code>	Ermöglicht den Zugriff auf das erste Element.
<code>end</code> <code>cend</code>	Zeigt „hinter“ dem letzten Element.
<code>rbegin</code> <code>crbegin</code>	Zeigt auf das letzte Element
<code>rend</code> <code>crend</code>	Zeigt „vor“ dem ersten Element
<code>at</code>	<code>at(std::size_t pos)</code> Liefert eine Referenz auf das Element an einer bestimmten Position. Wirft eine Exception, wenn die Position ungültig ist.
<code>operator []</code>	<code>operator [] (std::size_t pos)</code> Liefert eine Referenz auf das Element an einer bestimmten Position. Das Verhalten ist undefiniert, wenn die Position ungültig ist.
<code>data</code>	Liefert einen Zeiger auf den Datenblock, der alle Daten enthält.
<code>front</code>	Liefert eine Referenz auf das erste Element.
<code>back</code>	Liefert eine Referenz auf das letzte Element.
<code>empty</code>	Liefert <code>true</code> zurück, wenn der Container leer ist.
<code>size</code>	Liefert die Anzahl der Elemente in dem Container zurück.
<code>resize</code>	<code>void resize(size_type count [, const value_type& value]);</code> Vergrößert bzw. verkleinert den Container auf die angegebene Größe. Wenn die neue Größe kleiner wie die aktuelle Größe ist, werden überzählige Elemente einfach gelöscht. Wenn sie größer ist, werden neue Elemente entweder mit den Defaultkonstruktor erzeugt oder die neuen Elemente erhalten eine Kopie des übergebenen Objekts.
<code>capacity</code>	<code>std::size_t capacity() const</code> Maximale Anzahl der Elemente, die der aktuelle Speicherblock aufnehmen kann ohne dass dieser vergrößert werden muss.

Name	Beschreibung
reserve	<code>void reserve(std::size_t new_cap)</code> Vergrößert den Speicherblock, so dass er die angegebene Anzahl an Elemente aufnehmen kann, ohne wieder vergrößert werden zu müssen. Das ist sinnvoll, wenn schon vorher bekannt ist, dass sehr viele Elemente angehängt werden müssen. Dann kann schon vorher der Speicher angefordert werden und er muss nicht mehrmals umkopiert werden.
clear	Löscht den Inhalt des Containers.
push_front	<code>void push_front(const value_type& value)</code> Fügt ein neues Element an den Anfang des Containers ein.
push_back	<code>void push_back(const value_type& value)</code> Hängt ein neues Element an das Ende des Containers an.

15.2. Iterationen

a. Einleitung

Sehr häufig werden Sie vor dem Problem stehen, alle Elemente eines Containers nacheinander bearbeiten zu müssen. In den ersten C- und C++-Standards standen dafür nur einfache Operationen zur Verfügung. In Arrays konnten Sie einfach alle möglichen Indices mit einer For-Schleife abarbeiten.

```
for( size_t i=0; i<size; ++i )
```

wenn Sie ein festes Array haben, können Sie die Ermittlung der Obergrenze auch automatisieren,

```
for( size_t i=0; i<sizeof(liste)/sizeof(liste[0]); ++i )
```

Das funktioniert aber nur, wenn die Größe des Arrays an der Stelle auch bekannt ist und nicht nur ein Zeiger vorliegt.

Ab C++11 wurden die range-based for loops – aus anderen Sprachen auch als for each loop bekannt. In der einfachsten Form folgen sie dieser Syntax:

```
for( item-declaration : range ) statement;
```

item-declaration deklariert eine Variable, die ein einzelnen Element der Liste, die bearbeitet werden muß, aufnehmen kann. Sie kann auch eine Referenz oder const-referenz sein, Range deklariert die Liste, die bearbeitet werden muss. Das kann entweder eine initialisierter Liste in geschweiften Klammer oder ein Container sein.

```
// forEach.cpp - demonstration of a range based for loop
#include <iostream>
#include <vector>
int main () {
    std::vector<int> v = { 1, 2, 3 };
    for( int &i : v ) {
        std::cout << i << std::endl;
        i *= 2;
    }
    for( const int &i : v ) {
        std::cout << i << std::endl;
    }
    return 0;
}
```

Für diese Einführung soll uns das hier bei den range-base for loops genügen.

Für verkettete Listen, können Sie einen Zeiger verwenden und mit diesem durch Ihre Elemente iterieren, im Codeschnipsel könnte das so ausschauen:

```
for( MyType *item = head; item; item = item->next )
```

Wenn Sie die Reihenfolge der Elemente umdrehen wollen. müssen Sie beim Zugriff über einen Index darauf achten, dass Sie keine Endlosschleife programmieren, wenn Sie das Ende falsch formulieren, oder einen Speicherzugriffsfehler:

```
for( size_t i = size; i >= 0; --i )
```

Die Initialisierung ist fehlerhaft, da der größte erlaubte Index $\text{size} - 1$ ist. Dazu ist das Abbruchkriterium falsch, da ein vorzeichenloser Typ nie negativ werden kann. Um das zu beheben, können Sie eine do-while-schleife benutzen.

```
size_t i=size;
do {
    --i;
    // TODO: fetch and process item
} while( i );
```

Einfach verkettete Listen können nicht rückwärts bearbeitet werden. Das funktioniert nur mit einer doppelt verketteten List. Dann aber analog zur Bearbeitung einer Liste in „normaler“ Reihenfolge.

```
for( MyType *item = tail; item; item = item->prev )
```

Der Nachteil der bisher beschriebenen Verfahren ist, dass sie nur für einen bestimmten Containertyp funktionieren. Auch ist es aufwendiger den Code anzupassen, wenn Sie zum Beispiel die Richtung ändern wollen. Statt Indizes, Referenzen oder Zeiger zu verwenden, wurden in C++ die Iteratoren eingeführt, die auch unabhängig vom verwendeten Container benutzt werden können.

b. Iteratorkategorien

In C++ unterscheiden wir mehrere Kategorien von Iteratoren. Die im folgenden beschrieben werden.

Input Iteratoren

Input Iteratoren sind dazu da, Daten aus dem Container zu lesen. Sie unterstützen folgende Operation.

```
<InputIterator>( const <InputIterator> & );
```

Copy constructor. Sie können jeden Iterator beliebig oft kopieren.

```
const <Element> &operator *() const;
```

Dereferenz operator Sie können den Iterator wie einen Zeiger dereferenzieren und erhalten das Element, auf das er zeigt. Das Element ist aber nicht änderbar.

```
const <InputIterator> &operator ++ ();  
<InputIterator> operator ++ ( int );
```

Preinkrement, Postinkrement. Der Iterator zeigt auf das nächste Element, das bearbeitet werden soll.

```
bool operator == (const <InputIterator> &) const;  
bool operator != (const <InputIterator> &) const;
```

Vergleichsoperatoren, Sie können zwei Iteratoren auf Gleichheit oder Ungleichheit überprüfen.

Output Iteratoren

Output Iteratoren sind dazu da, Daten im Container zu schreiben. Sie unterstützen folgende Operation.

```
<OutputIterator>( const <OutputIterator> & );
```

Copy constructor. Sie können jeden Iterator beliebig oft kopieren.

```
<Element> &<operator *() const;
```

Dereferenz operator Sie können den Iterator wie einen Zeiger dereferenzieren und erhalten das Element, auf das er zeigt. Im Gegensatz zum Inputiterator können Sie das Element aber auch ändern.

```
const <OutputIterator> &operator ++ ();  
<OutputIterator> operator ++ ( int );
```

Preinkrement, Postinkrement. Der Iterator zeigt auf das nächste Element, das bearbeitet werden soll.

Die folgenden Iteratorkategorien sind nur der Vollständigkeit aufgelistet und sollen uns erst mal weiter nicht interessieren.

Forward Iteratoren:

```
<ForwardIterator>( const <ForwardIterator> & );  
const <Element> &operator *() const;  
<Element> &operator *();  
const <ForwardIterator> &operator ++ ();
```

```
<ForwardIterator> operator ++ ( int );
bool operator == (const <ForwardIterator> &) const;
bool operator != (const <ForwardIterator> &) const;
```

Neu:

```
const <ForwardIterator> &operator = (const <ForwardIterator> &);
const <Element> *operator ->() const;
<Element> *operator ->();
```

Bidirectional Iteratoren:

```
<BidirectionalIterator>( const <BidirectionalIterator> & );
const <Element> &operator *() const;
<Element> &operator *();
const <BidirectionalIterator> &operator ++ ();
<BidirectionalIterator> operator ++ ( int );
bool operator == (const <BidirectionalIterator> &) const;
bool operator != (const <BidirectionalIterator> &) const;
const <BidirectionalIterator> &operator=(const <BidirectionalIterator> &);
const <Element> *operator ->() const;
<Element> *operator ->();
```

Neu:

```
const <BidirectionalIterator> &operator -- ();
<BidirectionalIterator> operator -- ( int );
```

Random Access Iteratoren:

```
<RandomAccessIterator>( const <RandomAccessIterator> & );
const <Element> &operator *() const;
<Element> &operator *();
const <RandomAccessIterator> &operator ++ ();
<RandomAccessIterator> operator ++ ( int );
bool operator == (const <RandomAccessIterator> &) const;
bool operator != (const <RandomAccessIterator> &) const;
const <RandomAccessIterator> &operator = (const <RandomAccessIterator> &);
const <Element> *operator ->() const;
<Element> *operator ->();
const <RandomAccessIterator> &operator -- ();
<RandomAccessIterator> operator -- ( int );
```

Neu:

```
const <Element> &operator [] ( size_type ) const;
```

```
<Element> &operator [] ( size_type );  
const <RandomAccessIterator> &operator += ( difference_type );  
<RandomAccessIterator> operator + ( difference_type ) const;  
const <RandomAccessIterator> &operator -= ( difference_type );  
<RandomAccessIterator> operator - ( difference_type ) const;  
difference_type operator - (const <RandomAccessIterator> &) const  
bool operator < (const <RandomAccessIterator> &) const;  
bool operator <= (const <RandomAccessIterator> &) const;  
bool operator >= (const <RandomAccessIterator> &) const;  
bool operator > (const <RandomAccessIterator> &) const;
```

c. Methoden

Um mit einem Iterator arbeiten zu können, müssen Sie erst einmal einen passenden Iterator für den zu bearbeitenden Container erhalten. Hierzu haben die meisten Container einen oder mehrere der folgenden Memberfunktionen.

```
begin - end  
cbegin - cend  
rbegin - rend  
crbegin - crend
```

- Die Funktionen ...begin liefern einen Iterator auf das erste zu bearbeitende Element im Container.
- Die Funktionen ...end liefern einen Iterator, die das Ende der Bearbeitungsschleifen markieren
- Die Funktionen ohne dem Prefix r werden verwendet, um die Liste von vorne bis hinten zu bearbeiten.
- Die Funktionen mit dem Prefix r werden verwendet, um die Liste in umgekehrter Reihenfolge von hinten bis vorne zu bearbeiten.
- Die Funktionen ohne dem Prefix c werden verwendet, um mit dem Iterator die Elemente verändern zu können.
- Die Funktionen mit dem Prefix c geben einen Iterator zurück, mit dem die Elemente nicht verändert werden können.
- Wichtig: Den Prefix, der bei ..begin verwendet wurde, muß auch bei ..end verwendet werden, sonst wird Ihr Programm nicht compilieren.

Nicht alle Container bieten alle hier aufgelisteten Memberfunktionen.

Den Iterator, den Sie mit ..begin erhalten haben, können Sie so oft inkrementieren, bis dieser identisch ist mit dem Wert von ..end. Dieser darf dann nicht mehr dereferenziert werden.

Beim Inkrementieren empfehle ich grundsätzlich den Preinkrementoperator zu verwenden. Da der Preinkrement keine temporäre Kopie des Iterators erzeugen

muss. Dies kann zwar häufig vom Optimierer des Compilers berücksichtigt werden, das ist aber nicht immer möglich, deshalb habe ich mir angewöhnt, den Preinkrement zu verwenden, selbst wenn das sicher keinen Einfluss auf den Code hat.

Alle Operationen auf den Container, die die Anzahl der Elemente im Container verändern, können alle Iteratoren ungültig machen. Vermeiden Sie solche Operationen innerhalb Ihrer Schleife oder prüfen Sie in der Referenzdokumentation, welche Iteratoren davon betroffen sein könnten.

Den Typ des Iterators erhalten Sie vom Container selbst, dort werden die Membertypen definiert: `iterator`, `const_iterator`, `reverse_iterator` und `const_reverse_iterator`.

Ein Codeschnipsel, der das demonstrieren soll:

```
for(VectorType::iterator it=vec.begin(), endIT=vec.end(); it!=endIT; ++it)
```

d. Insert Iteratoren

Neben den eben behandelten Iteratoren zum Bearbeiten bereits vorhandener Elemente gibt es noch drei Iteratoren zum Einfügen neuer Elemente in bestehende Container. Diese Iteratoren sind aber als eigene Templates definiert mit dem jeweiligen Container als Templateargument. Die Iteratoren, um die es hier geht, befinden sich im Header `<iterator>` und sind im namespace `std` definiert. Wenn Sie einen solchen Iterator dereferenzieren wird ein neues Element im zugrundeliegenden Container erzeugt und eine Referenz auf das neue Element zurückgegeben. Die Dereferenzierung kann also auf der linken Seite einer Zuweisung stehen.

front_insert_iterator

Der `front_insert_iterator` erzeugt neue Elemente am Anfang des Containers. Sie erhalten einen solchen Iterator mit dem freien Funktionstemplate `front_inserter` und dem container objekt als Argument.

```
std::front_insert_iterator<ContainerType> front =  
    std::front_inserter<ContainerType>( container );
```

insert_iterator

Der `insert_iterator` erzeugt neue Elemente an einer beliebigen Position im Container. Sie erhalten einen solchen Iterator mit dem freien Funktionstemplate `inserter` und dem container objekt und einen iterator als Argument.

```
std::insert_iterator<ContainerType> ins =  
    std::inserter<ContainerType>( container, ++container.begin() );
```

back_insert_iterator

Der `back_insert_iterator` erzeugt neue Elemente am Ende des Containers. Sie erhalten einen solchen Iterator mit dem freien Funktionstemplate `back_inserter` und dem container objekt als Argument.

```
std::back_insert_iterator<ContainerType> append =  
    std::back_inserter<ContainerType>( container );
```

Beachten Sie, dass nicht alle Containertypen alle diese Iteratoren unterstützen. Im Zweifelsfall prüfen Sie bitte das Referenzhandbuch zu C++. Hier folgt ein Beispiel, das die Nutzung der Operatoren demonstrieren soll.

```
// iterator.cpp: demo of how to use an iterator
#include <deque>
#include <iterator>
#include <iostream>

int main () {
    typedef std::deque<int>          MyContainer;
    typedef MyContainer::const_iterator      IntConstIT;
    typedef std::front_insert_iterator<MyContainer>      IntFrontIT;
    typedef std::insert_iterator<MyContainer>      IntInserter;
    typedef std::back_insert_iterator<MyContainer>      IntApendIT;

    MyContainer v;

    std::cout << "----Step 1-----" << std::endl;
    IntApendIT back = std::back_inserter<MyContainer>(v);

    *back = 1;
    *back = 2;
    *back = 3;

    for( IntConstIT it = v.cbegin(); it != v.cend(); ++it )
        std::cout << *it << std::endl;

    std::cout << "----Step 2-----" << std::endl;
    IntInserter ins = std::inserter<MyContainer>(v, ++v.begin() );
    *ins = 4;
    *ins = 5;
    *ins = 6;

    for( IntConstIT it = v.cbegin(); it != v.cend(); ++it )
        std::cout << *it << std::endl;

    std::cout << "----Step 3-----" << std::endl;
    IntFrontIT front = std::front_inserter<MyContainer>(v);
    *front = 7;
    *front = 8;
    *front = 9;

    for( IntConstIT it = v.cbegin(); it != v.cend(); ++it )
        std::cout << *it << std::endl;

    std::cout << "----END-----" << std::endl;
    return 0;
}
```

16. Datum, Zeitstempel und Stoppuhren

Immer wieder werden Sie mit Kalenderdaten, Zeitstempel und Laufzeiten zu tun haben. Dabei werden Sie von C mit den wichtigsten Operation versorgt wohingegen C++ hier schon deutlich mehr Operation zur Verfügung stellt.

Einen wichtigen Fehler, der leider immer wieder vorkommt, ist, dass in der falschen Zeitzone gearbeitet wird, oder dass gar keine Zeitzoneneinformationen gespeichert wurde. Das gibt vor allen dann ein Problem, wenn dann zusätzlich Sommer- und

Winterzeit falsch verwendet werden. Ich empfehle daher grundsätzlich die Zeitzone UTC zu speichern, mit Ihr zu rechnen und auch die UTC-Zeiten als Triggerzeiten zu verwenden. Nicht selten kommt es vor, dass beim Wechsel von Sommer- auf Winterzeit oder umgekehrt eine Aktion zu oft gestartet oder vergessen wurde, weil das System nicht klar kommt, dass eine Uhrzeit entweder ausgelassen oder nach einer Stunde wiederholt wurde.

16.1. Funktionen und Typen in C <time.h>

<time.h> bzw. <ctime>

Name	Beschreibung
time_t	Ganzzahltyp der die Systemzeit speichern kann. Obwohl das ein systemspezifischer Typ ist, wird dort meist time_t stets die Anzahl der Sekunden seit 1.1.1970 00:00 gespeichert.
tm	<pre>struct tm;</pre> Kalenderdatum und -zeit mit tm_sec Sekunden tm_min Minuten tm_hour Stunden tm_mday Tag im Monat tm_mon Monat tm_year Jahr tm_yday Tag im Jahr tm_isdst Sommerzeit: >0 wenn Sommerzeit ist, 0, wenn Normalzeit ist, <0 wenn es unbekannt ist.
difftime	<pre>double difftime(time_t time_end, time_t time_beg);</pre> Berechnet den Unterschied zwischen zwei Zeitstempel in Sekunden.
time	<pre>time_t time(time_t* arg);</pre> Liefert die aktuelle Uhrzeit zurück. Das genaue Format der Uhrzeit ist abhängig vom Compiler. Die aktuelle Uhrzeit kann aber auch in die Variable gespeichert werden, auf die arg zeigt, wenn nicht NULL übergeben wurde.
clock	<pre>clock_t clock(void);</pre> Liefert die CPU-Zeit seit Programmstart zurück. Wenn Sie den Wert durch CLOCKS_PER_SEC dividieren, erhalten Sie die Zeit in Sekunden. Beachten Sie, manche Systeme unterscheiden nicht zwischen CPU- und Benutzerzeit. Der Vergleich zwischen unterschiedlichen Systemen kann hiermit daher nicht erfolgen.

Name	Beschreibung
asctime, asctime_s	<pre>char *asctime(const struct tm* time_ptr);</pre> <pre>errno_t asctime_s(char* buf, rsize_t bufsz, const struct tm* time_ptr);</pre> Konvertiert die Zeitangabe auf die das Argument zeigt, in eine Textdarstellung. Die erste Variante liefert einen Zeiger auf einen statischen Speicher. Jeder weitere Aufruf überschreibt diesen Speicher. Daher soll ab C11 die zweite Variante aufgerufen werden, die einen Zeiger auf den Speicher und die Größe erwartet.
ctime, ctime_s	<pre>char *ctime(const time_t* timer_ptr);</pre> <pre>errno_t ctime_s(char* buf, rsize_t bufsz, const time_t* timer_ptr);</pre> Arbeitet wie asctime bzw asctime_s nur wird ein anderes Zeittyp erwartet.
strftime	<pre>size_t strftime(char* str, size_t count, const char *format, const struct tm *tp);</pre> Wandelt einen Zeitstempel in struct tm in eine benutzerspezifisches Format. Die Funktion erwartet folgende Parameter: str: die Zieladresse, an der der Text geschrieben werden soll. count: die Größe des Speichers. format: das Format des Zeitstempels. tp: der Zeitstempel. Der Formatstring ist ähnlich aufgebaut wie bei printf. Alle Zeichen werden in die Zieladresse kopiert außer %, welches ein Feld einleitet. Unter anderen sind folgende Felder möglich: %: das Prozentzeichen wird ausgegeben. Y: die vierstellige Jahreszahl y: die zweistellige Jahreszahl b: abgekürzter Monatsname B: voll ausgeschriebener Monatsname m: zweistellige Monatszahl U: die zweistellige Wochenzahl mit dem Sonntag als Basis. W: die zweistellige Wochenzahl mit dem Montag als Basis. j: die Tageszahl im Jahr. d: die Tageszahl im Monat. a: der abgekürzte Wochentag A: der ausgeschriebene Wochentag. w: die Tageszahl in der Woche, die Zählung fängt aber bei 0 an. H: die zweistellige Stundenzahl von 0 bis 23 I: die zweistellige Stundenzahl von 0 bis 12 M: zweistellige Minuten S: zweistellige Sekunden c: ein Standardformat für den ganzen Zeitstempel. p: die Bezeichnung des Tagesabschnitts (AM/PM) Z: der abgekürzte Name der Zeitzone-

Name	Beschreibung
gmtime	<code>struct tm* gmtime(const time_t* timer);</code> Erzeugt aus einem Wert vom Typ <code>time_t</code> eine Struktur <code>tm</code> , die Greenwichmeantime (GMT) enthält.
localtime	<code>struct tm* localtime(const time_t* timer);</code> Erzeugt aus einem Wert vom Typ <code>time_t</code> eine Struktur <code>tm</code> , die den lokale Zeitstempel enthält.

```
// time.c - examples for <time.h>
#include <time.h>
#include <stdio.h>

static void showTime( const struct tm *theTime ) {
    char localBuffer[128];
    asctime_s( localBuffer, sizeof( localBuffer ), theTime );
    puts( localBuffer );
}

int main( void ) {
    time_t theTime = time( 0 );
    struct tm local = *localtime( &theTime );
    struct tm iis = *gmtime( &theTime );
    puts( "Zeit in Linz:" );
    showTime(&local);
    puts( "Zeit auf der IIS:" );
    showTime(&iis);
    return 0;
};
```

16.2. Funktionen und Klassen in C++

Ab C++11 wurde die Headerdatei `<chrono>` eingeführt. In dieser Datei werden im Namespace `chrono` unter anderen diese Definitionen und Deklarationen eingeführt.

Name	Beschreibung
<code>std::chrono::duration</code>	<code>template<class Rep, class Period = std::ratio<1> > class duration;</code> Klassentemplate zur Speicherung eines Zeitraums. <code>Rep</code> muss ein numerischer Typ, <code>Period</code> bestimmt die Ganularität.

Name	Beschreibung
std::chrono::nanoseconds std::chrono::microseconds std::chrono::milliseconds std::chrono::seconds std::chrono::minutes std::chrono::hours std::chrono::days (ab C++20) std::chrono::weeks (ab C++20) std::chrono::months (ab C++20) std::chrono::years (ab C++20)	Diese Typen speichern eine Zeitdauer mit der angegebenen Auflösung.
std::chrono::time_point	<pre>template<class Clock, class Duration = typename Clock::duration> class time_point;</pre> Klassentemplate zur Speicherung eines Zeitpunkts. Der Templateparameter Clock erwartet einen Uhrentyp, Duration die Auflösung als Argument.
std::chrono::system_clock	Mit Variablen dieses Typs, können Sie die aktuelle Systemzeit ermitteln.
std::chrono::steady_clock	Mit Variablen dieses Typs erhalten Sie eine Uhr, die garantiert nie zurückspringt auch dann nicht, wenn die Systemzeit verändert wird.
std::chrono::high_resolution_clock	Hiermit bekommen Sie eine hoch-auflösende Uhr.
std::chrono::utc_clock (ab C++20)	Hiermit bekommen Sie die definierte Zeit der Zone Coordinated Universal Time UTC
std::chrono::tai_clock (ab C++20)	Hiermit bekommen Sie die definierte Zeit der Zone International Atomic Time TAI
std::chrono::gps_clock (ab C++20)	Hiermit bekommen Sie die definierte Zeit des Global Positioning System GPS.
std::literals::chrono_literals (ab C++14)	Ein namespace, der verschiedene Suffixe für Nanosekunden bis Stunde definiert. Ab C++20 befinden sich dort auch die Suffixe für Tag und Jahr.

```
// chrono.cpp - demo for <chrono>
#include <chrono>
#include <iostream>
#include <iomanip>
#include <numeric>
#include <vector>
```

```
static volatile int sink; // prevents optimization

static void do_some_work(std::size_t size) {
    std::vector<int> v(size, 42);
    sink = std::accumulate(v.begin(), v.end(), 0); // ensures side effect
}

int main() {
    std::cout << std::fixed << std::setprecision(9) << std::left;
    for(unsigned long long size = 1ull; size<1000000000ull; size*=100) {
        const std::chrono::utc_clock::time_point start =
            std::chrono::utc_clock::now();
        do_some_work(size);
        const std::chrono::utc_clock::time_point end =
            std::chrono::utc_clock::now();
        const std::chrono::duration<double> diff = end - start;
        std::cout << "start = " << start << "; end = " << end << ";\n";
        std::cout << "diff = " << diff << "; size = " << size << '\n';
    }
}
```

17. Threads

17.1. Wissenswertes

Moderne Betriebssysteme können nicht nur mehrere Programme „quasi“ gleichzeitig ausführen, sie erlauben auch, dass innerhalb eines Programms mehrere Programmablaufpläne „quasi“ gleichzeitig ausgeführt werden. Mittlerweile können die meisten (oder alle?) Prozessoren tatsächlich mehrere Programmabläufe gleichzeitig ausführen aber das soll hier jetzt weniger interessieren. Das ist auch sehr betriebssystem- und CPU-abhängig und daher nicht Thema dieses Buchs. Wir wollen zuerst die Konzepte Sprachunabhängig betrachten. Dazu erklären wir ein paar wichtige Grundbegriffe.

Thread	Ein Thread ist ein Programmablaufplan, der asynchron von anderen Threads abgearbeitet wird. Beim Starten eines Programms wird automatisch ein Thread gestartet, der nach dem Startcode die Funktion main aufruft. Beim Erzeugen oder Starten eines weiteren Threads müssen Sie eine andere Funktion angeben, die Threadfunktion, die aufgerufen werden kann. Der Aufrufer der Funktion läuft dann unabhängig von der aufgerufenen Funktion weiter. Insbesondere erhält er auch kein Funktionsergebnis. Die Threadfunktion kann auch andere Funktionen ganz normal aufrufen, der Thread beendet aber spätestens dann, wenn die Threadfunktion fertig ist. Eine Schwierigkeit besteht dann, wenn mehr als ein Thread gleichzeitig, die selbe Variable lesen und schreiben wollen. Dann benötigen Sie einen Mechanismus mit dem die Zugriffe synchronisiert werden können. Wenn die Threads gemeinsame Daten nur lesen wollen und nur eigene Variablen schreiben wollen, benötigen Sie wahrscheinlich keine Synchronisation, im schlimmsten Fall, muss der Hauptthread nur Warten, bis die andern Threads beendet sind. Das sollte er aber sowieso, da oft das Programm beendet wird, wenn der Hauptthread vorzeitig beendet wird.
Mutex	Mit einem Mutex, wechselseitiger Ausschluss, engl mutual exclusion, können Sie dafür sorgen, dass eine Ressource immer nur von einem Thread gleichzeitig beansprucht werden kann. Ein einfacher Mutex wird z.B. verwendet, wenn Sie sicherstellen wollen, dass eine Funktion immer nur von einem Thread gleichzeitig ausgeführt werden kann. <pre>void function(void) { enterMutex(); // TODO do something exitMutex(); }</pre>
Lock	Ein Lock wird verwendet, wenn mehrere Funktionen in unterschiedlichen Threads auf die selbe Ressource zugreifen wollen. <pre>Lock myLock; Type myData; void function(void) { Locker myLocker = lockData(myLock); // TODO do something unlockData(myLocker); }</pre>
Semaphore	Ein Semaphore ist eine leistungsfähigere Variante eines Mutex. Sie werden besonders dann benutzt, wenn mehrere Prozesse synchronisiert werden müssen.
Condition Variables	Mit einer Condition Variable kann ein Thread einen anderen signalisieren, dass ein bestimmtes Ereignis eingetreten ist.

Thread Local Storage	Ein Thread Local Storage ist ein Speicher, der zwar statisch oder gar global definiert ist, aber für jeden Thread eine eigene Kopie besitzt. In der Analogie zu den bisher gelernten ist das wie eine statische, lokale Variable, die zwar nur in einer Funktion benutzt werden kann, ihren Wert aber nicht am Ende der Funktion verliert.
Deadlock	Ein Deadlock tritt auf, wenn zwei Prozesse oder Threads je eine Ressource für sich reserviert haben und nun auf die Freigabe der anderen Ressource warten. Das können Sie verhindern, indem Sie in Ihrem Programm dafür sorgen, dass alle Ressourcen immer nur in der gleichen Reihenfolge angefordert werden. Alternativ wäre es auch möglich, beim Reservieren einer Ressource einen Timeout anzugeben, bis wann die Ressource spätestens zur Verfügung stehen muß. Dann haben Sie zumindest eine Chance, dem Benutzer eine aussagekräftige Fehlermeldung anzuzeigen. Ich finde nichts schrecklicher wie, wenn die Anwendung nur einen sich drehenden Kreis anzeigt und ansonsten den Benutzer völlig im Unklaren lässt, was hier noch passiert.
Race Condition	Eine Race Condition tritt auf, wenn die Zugriffe auf eine oder mehrere Ressourcen schlecht oder gar nicht synchronisiert wurde. Dann kann das Programm- bzw. Funktionsergebniss hochgradig davon abhängig sein, wie der Taskscheduler des Betriebssystems die CPU-Zeit auf die einzelnen Prozesse bzw. Threads verteilt hat.
Atomic	Eine atomare Variable garantiert die Unteilbarkeit der Zugriffe um Race Conditions zu vermeiden. Einfachste Beispiele für einen atomaren Zugriff sind die Inkrement- oder Dekrementoperatoren. Diese Operationen auf eine nicht atomare Variable haben ein unvorhersehbares Ergebnis, wenn mehrere Threads gleichzeitig die gleiche Operation auf die selbe Variable ausführen. Meistens werden Sie aber sowieso mit einem Lock oder ähnlichen arbeiten müssen weshalb wie hier in diesen Buch nicht näher darauf eingehen werden. Der Vorteil der atomischen Variablen ist aber, dass Sie keinen Mutex und andere Lockmechanismen brauchen, weshalb Sie damit keine Deadlocks befürchten müssen.

17.2. Threads mit C

In der Headerdatei <threads.h> befinden sich verschiedene Definitionen und Deklarationen für Threads, Mutexes, Condition Variables und Thread Local Storages. Der Rückgabewert der Funktionen ist, wenn nicht anderes angegeben wurde, `thrd_success` in Erfolgsfall oder ein Fehlercode.

Name	Beschreibung
<code>thrd_t</code>	Dieser Typ wird benutzt, um einen Thread zu identifizieren.
<code>thrd_success</code>	Kein Fehler

Name	Beschreibung
thrd_nomem	Speicherfehler
thrd_timeout	Timeout
thrd_busy	Ressource temporär nicht verfügbar
thrd_error	Nicht näher spezifizierter Fehler.
thrd_start_t	typedef int (*thrd_start_t)(void *) Funktionszeigertyp
thrd_create	int thrd_create(thrd_t *thr, thrd_start_t func, void *arg); Erzeugt und startet einen Thread. In thr wird der neue Threadidentifizier gespeichert, func gibt die Adresse der Threadfunktion an und arg das Argument für die Threadfunktion.
thrd_equal	int thrd_equal(thrd_t lhs, thrd_t rhs); Vergleicht zwei Threadhandles. Der Rückgabewert ist 0, wenn beide den selben Thread betreffen.
thrd_current	thrd_t thrd_current(void); Liefert das Threadhandle des gerade laufenden Threads zurück.
thrd_sleep	int thrd_sleep(const struct timespec* duration, struct timespec* remaining); Unterbricht die Ausführung des aktuellen Threads. Duration enthält die Zeitspanne, in der gewartet werden soll, in remaining wird die Restzeit gespeichert, falls die Wartezeit vorzeitig unterbrochen wurde. Remaining darf auf den selben Speicher zeigen wie duration. Remaining darf auch NULL sein, wenn die Restzeit nicht gespeichert werden soll. Es wird 0 zurückgegeben, wenn die Zeit abgelaufen ist, -1, wenn die Wartezeit unterbrochen wurde, ein anderer negativer Wert, wenn ein Fehler passiert ist.
thrd_yield	void thrd_yield(void); Mit dieser Funktion können Sie dem Betriebssystem mitteilen, dass es den Thread unterbrechen kann, um andere Threads ausführen zu können. Die genaue Wartezeit ist nicht definiert.
thrd_exit	void thrd_exit(int res); Beendet den aktuellen Thread. Es wird nicht wieder zurückgekehrt. Mit res können Sie einen Wert zurückgeben, der anderweitig mit dem Threadhandle abgefragt werden kann.

Name	Beschreibung
thrd_detach	<pre>int thrd_detach(thrd_t thr);</pre> Entfernt den Thread aus der Beobachtung ohne ihn zu beenden. Threadspezifische Ressourcen werden erst freigegeben, wenn er endete.
thrd_join	<pre>int thrd_join(thrd_t thr, int *res);</pre> Wartet darauf, bis der angegebene Thread beendet wurde. In res wird das Threadergebnis gespeichert. Das Ergebnis eines Threads ist entweder der Rückgabewert der Threadfunktion oder das Argument von thread_exit. Der Rückgabewert von thread_join ist thrd_success, wenn alles ok ist, thrd_error im Falle eines Fehlers. Danach wird der Thread aus der Beobachtung entfernt. Das Verhalten ist undefiniert, wenn der Thread vor dem Aufruf schon aus der Beobachtung entfernt wurde.
mtx_t	In Typ für ein Handle auf einen Mutex.
mtx_init	<pre>int mtx_init(mtx_t* mutex, int type);</pre> Erzeugt einen Mutex. In Mutex wird das Handle gespeichert, type erwartet die Art des Mutex. Dabei sind folgende Werte erlaubt: mtx_plain, eine einfache Variante mtx_timed, ein Mutex, der Timeout unterstützt, mtx_plain mtx_recursive, ein rekursiver Mutex, der von einem Thread mehrmals gesperrt werden kann, der dann aber auch entsprechend oft freigegeben werden muß, bevor er von andern Threads verwendet werden kann, mtx_timed mtx_recursive, ein rekursiver Mutex, der auch Timeout unterstützt.
mtx_lock	<pre>int mtx_lock(mtx_t* mutex);</pre> Blockiert den aktuellen Thread, bis der Mutex exklusiv für den Thread zur Verfügung steht. Danach kann der Mutex von anderen Threads nicht reserviert werden und werden blockiert bis der Mutex wieder freigegeben wurde. Das Verhalten ist undefiniert, wenn der Mutex nicht als rekursiver Mutex erzeugt wurde und der Mutex bereits für den aktuellen Thread reserviert ist. Ansonsten muss der Mutex so oft freigegeben werden wie er gesperrt wurde. Sie müssen sich das wie ein Schloss vorstellen. Wenn Sie beim Zusperrern den Schlüssel zweimal umdrehen, müssen Sie das beim Öffnen auch tun.

Name	Beschreibung
mtx_timedlock	<pre>int mtx_timedlock(mtx_t *mutex, const struct timespec * time_point);</pre> Blockiert den aktuellen Thread, bis der Mutex exklusiv für den Thread zur Verfügung steht oder bis der angegebene Zeitpunkt erreicht ist. Das Verhalten ist undefiniert, wenn der Mutex keinen Timeout unterstützt. Bei den Zeitpunkt handelt es sich um eine absolute UTC-Angabe.
mtx_trylock	<pre>int mtx_trylock(mtx_t *mutex);</pre> mtx_trylock arbeitet wie mtx_lock allerdings wird nicht gewartet bis der Mutex reserviert wurde stattdessen wird ein Fehler gemeldet.
mtx_unlock	<pre>int mtx_unlock(mtx_t *mutex);</pre> Unlock gibt einen zuvor gesperrten Mutex wieder frei. Das Verhalten ist undefiniert, wenn der Mutex gar nicht von diesem Thread gesperrt wurde. Bei einem rekursiven Mutex, müssen Sie die Funktion so oft aufrufen wie der Mutex gesperrt wurde ehe die Freigabe tatsächlich erfolgt.
mtx_destroy	<pre>void mtx_destroy(mtx_t *mutex);</pre> Diese Funktion Zerstört den Mutex. Wenn andere Threads auf den Mutex warten, ist das Verhalten undefiniert.
call_once	<pre>void call_once(once_flag* flag, void (*func)(void));</pre> Diese Funktion ruft eine andere Funktion genau einmal auf, auch wenn sie von mehreren Threads gleichzeitig aufgerufen wurde. Flag ist ein Zeiger auf eine Variable vom Typ once_flag, die von allen Threads gelesen und beschrieben werden kann. Diese Variable muss mit dem Wert ONCE_FLAG_INIT vorbelegt werden. Func ist ein Funktionszeiger.
cnd_t	Handle für eine condition variable
cnd_init	<pre>int cnd_init(cnd_t* cond);</pre> Erzeugt eine neue condition variable. Cond zeigt auf die Variable, wo das Handle der constition variable gespeichert wird.

Name	Beschreibung
<code>cnd_signal</code>	<pre>int cnd_signal(cnd_t *cond);</pre> Signalisiert einen Thread, der auf die condition variable wartet, dass es weiterlaufen kann. Wenn gar kein Thread wartet, passiert nichts und es wird Erfolg gemeldet.
<code>cnd_broadcast</code>	<pre>int cnd_broadcast(cnd_t *cond);</pre> Signalisiert allen Threads, die auf eine condition variable warten, dass es weiterlaufen kann. Wenn gar kein Thread wartet, passiert nichts und es wird Erfolg gemeldet.
<code>cnd_wait</code>	<pre>int cnd_wait(cnd_t* cond, mtx_t* mutex);</pre> Gibt den Mutex frei, auf den mutex verweist, und wartet auf die condition variable bis ein anderer Thread <code>cnd_broadcast</code> aufruft oder bis der aktuelle Thread bei <code>cnd_signal</code> ausgewählt wurde. Das Verhalten ist undefiniert, wenn der Mutex nicht vom aktuellen Thread gesperrt wurde.
<code>cnd_timedwait</code>	<pre>int cnd_timedwait(cnd_t* cond, mtx_t* mutex, const struct timespec* time_point);</pre> Arbeitet wie <code>cnd_wait</code> nur wird hier noch die späteste Zeitpunkt angegeben, bis dahin <i>gewartet</i> werden soll.
<code>cnd_destroy</code>	<pre>void cnd_destroy(cnd_t* cond);</pre> Löscht die condition variable. Das Handle darf anschließend nicht mehr benutzt werden.
<code>thread_local</code>	Mit <code>thread_local</code> markierte Variablen sind statisch für den Thread. Seit C23 ist dies ein Schlüsselwort, weshalb vor der Benutzung <code><thread.h></code> nicht inkludiert werden muss.
<code>tss_t</code>	Ein Handle für Threaddaten.
<code>TSS_DTOR_ITERATIONS</code>	Ein konstanter Wert, der angibt, wie oft Threaddaten maximal erstört werden.
<code>tss_dtor_t</code>	<pre>typedef void(*tss_dtor_t)(void*)</pre> Ein Funktionszeiger auf die Löschfunktion der Threaddaten.

Name	Beschreibung
tss_create	<pre>int tss_create(tss_t* tss_key, tss_dtor_t destructor);</pre> Erzeugt einen neuen Speicherbereich, für den gerade laufende Thread. In tss_key erhalten Sie das handle des Speichers, mit destructor geben Sie die Adresse der Funktion an, die beim Löschen aufgerufen werden soll.
tss_set	<pre>int tss_set(tss_t tss_id, void *val);</pre> Mit dieser Funktion setzen Sie einen threadspezifischen Wert. Mit tss_id übergeben Sie das Handle, das vorher mit tss_create erzeugt wurde, mit val, wird das threadspezifische Datum übergeben.
tss_get	<pre>void *tss_get(tss_t tss_key);</pre> Mit dieser Funktion erhalten Sie den threadspezifischen Wert wieder zurück.
tss_delete	<pre>void tss_delete(tss_t tss_id);</pre> Löscht den threadspeicher für alle Threads. Sie können es zwar vom Prinzip aufrufen, wenn die Threads noch laufen, aber es ist unklar, welche Destruktoren noch aufgerufen werden sollen.

```
/* thread.c - demonstration of <thread.h> */
#include <stdio.h>
#include <stdlib.h>
#include <threads.h>
#include <time.h>

static mtx_t s_mutex;
static void syncConsoleOut( const char *text, FILE *handle ) {
    mtx_lock( &s_mutex );
    fputs(text, handle);
    mtx_unlock( &s_mutex );
}

static void setData(tss_t dataHandle) {
    clock_t *clockTime = malloc( sizeof( clock_t ) );
    if( !clockTime ) {
        syncConsoleOut( "Fehler bei malloc\n", stderr);
    }
    else {
        *clockTime = clock();
        if( tss_set(dataHandle, clockTime) != thrd_success ) {
            syncConsoleOut( "Fehler beim Setzen der Threaddaten\n",
stderr);
        }
    }
}

}
```

```
static clock_t readData(tss_t dataHandle) {
    clock_t result = 0;
    clock_t *threadData = tss_get(dataHandle);
    if( threadData ) {
        result = *threadData;
    }
    else {
        syncConsoleOut( "Fehler beim Lesen der Threaddaten\n", stderr);
    }
    return result;}

static int theThreadFunc(void *arg) {
    tss_t *threadData = arg;
    setData(*threadData);
    return (int)readData(*threadData);
}

static void theThreadDestroy(void *arg) {
    free( arg );
    printf( "Freed %p\n", arg );
}

int main(void) {
    tss_t threadData;
    thr_t thr1, thr2;
    int result1, result2;

    if( mtx_init( &s_mutex, mtx_plain ) != thr_success ) {
        syncConsoleOut( "Fehler beim Erzeugen des Mutex\n", stderr);
        exit( -1 );
    }

    if( tss_create( &threadData, theThreadDestroy ) != thr_success ) {
        syncConsoleOut( "Fehler beim Erzeugen der Threaddaten\n",
stderr);
        exit( -1 );
    }
    setData(threadData);

    thr_sleep(&(struct timespec){.tv_sec=1}, NULL);
    if( thr_create( &thr1, theThreadFunc, &threadData ) != thr_success )
{
        syncConsoleOut( "Fehler beim Erzeugen des Thread 1\n", stderr);
        exit( -1 );
    }

    thr_sleep(&(struct timespec){.tv_sec=1}, NULL);
    if( thr_create( &thr2, theThreadFunc, &threadData ) != thr_success )
{
        syncConsoleOut( "Fehler beim Erzeugen des Thread 2\n", stderr);
        exit( -1 );
    }
}
```

```

    if( thrd_join( thr1, &result1 ) != thrd_success ) {
        syncConsoleOut( "Fehler beim Warten auf Thread 1\n", stderr);
        exit( -1 );
    }
    if( thrd_join( thr2, &result2 ) != thrd_success ) {
        syncConsoleOut( "Fehler beim Warten auf Thread 2\n", stderr);
        exit( -1 );
    }

    clock_t timer = readData(threadData);
    printf("Main: %ld Thread1 %d Thread2 %d\n", (long)timer, result1,
result2);

    tss_delete(threadData);
    mtx_destroy(&s_mutex);
    return 0;
}

```

Für die Anwendung von ConditionVariables gibt es ein Extrabeispiel:

```

/* condition.c - demonstration of ConditionVariable */
#include <stdio.h>
#include <stdlib.h>
#include <threads.h>
#include <time.h>

static const int THREAD_DELAY = 1;
static const int MAIN_PATIENCE = 2;

static mtx_t s_consoleMutex;
static mtx_t s_conditionMutex;
static cnd_t s_condition;

static void syncConsoleOut( const char *text, FILE *handle ) {
    mtx_lock( &s_consoleMutex);
    fputs(text, handle);
    mtx_unlock( &s_consoleMutex);
}

static int theThreadFunc(void *arg) {
    thrd_sleep(&(struct timespec) { .tv_sec = THREAD_DELAY }, NULL);
    if (cnd_signal(&s_condition) != thrd_success) {
        syncConsoleOut("Fehler beim Signalisieren der Condition Variable\n",
stderr);
        exit(-1);
    }
    return 0;
}

int main(void) {
    thrd_t theThread;

    if (mtx_init(&s_consoleMutex, mtx_plain) != thrd_success) {
        syncConsoleOut("Fehler beim Erzeugen des s_consoleMutex\n",
stderr);
        exit(-1);
    }
}

```

```
    if (mtx_init(&s_conditionMutex, mtx_plain) != thrd_success) {
        syncConsoleOut("Fehler beim Erzeugen des s_conditionMutex\n",
stderr);
        exit(-1);
    }
    if (cnd_init(&s_condition) != thrd_success) {
        syncConsoleOut("Fehler beim Erzeugen der Condition Variable\n",
stderr);
        exit(-1);
    }
    syncConsoleOut("Start...\n", stdout);
    if (mtx_lock(&s_conditionMutex) != thrd_success) {
        syncConsoleOut("Fehler beim Sperren des s_conditionMutex\n",
stderr);
        exit(-1);
    }
    if (thrd_create(&theThread, theThreadFunc, (void*)NULL) !=
thrd_success) {
        syncConsoleOut("Fehler beim Erzeugen des Thread\n", stderr);
        exit(-1);
    }
    time_t now = time(NULL);
    if (cnd_timedwait(&s_condition, &s_conditionMutex, &(struct timespec)
{.tv_sec = now + MAIN_PATIENCE }) != thrd_success) {
        syncConsoleOut("Fehler beim Warten auf s_condition\n", stderr);
        exit(-1);
    }
    int dummy;
    if( thrd_join(theThread, &dummy ) != thrd_success ) {
        syncConsoleOut( "Fehler beim Warten auf Thread\n", stderr);
        exit( -1 );
    }
    mtx_unlock(&s_conditionMutex);
    cnd_destroy(&s_condition);
    mtx_destroy(&s_conditionMutex);
    mtx_destroy(&s_consoleMutex);
    syncConsoleOut("Alles OK!\n", stdout);
    return 0;
}
```

In der Headerdatei <stdatomic.h> finden Sie alle Deklarationen und Definitionen, die Sie für atomare Variablen und Zugriffe brauchen.

17.3. Threads mit C++

Für Threads mit C++ gibt es ab C++11 mehrere Klassen, die auch in eigenen Headern definiert sind. In <thread> ist u.a. die Klasse thread deklariert. Die Klasse hat folgende Funktionen.

Name	Beschreibung
id	Klasse, die einen Thread eindeutig identifizieren kann.

Name	Beschreibung
thread	<pre>thread() noexcept; thread(thread&& other) noexcept; template< class F, class... Args > explicit thread(F&& f, Args&&... args); thread(const thread&) = delete;</pre> Ohne Argument, wird ein Thread-objekt ohne verknüpften Programmthread erzeugt. Der move constructor erzeugt ein neues Threadobjekt aus einem bestehenden. Das alte Objekt ist dann nicht mehr mit dem Programmthread verknüpft. Mit einem Funktionsobjekt und Argumenten für dieses können Sie ein verknüpftes Threadobjekt erzeugen und den Programmthread sofort starten. Der copy constructor ist nicht aufrufbar.
operator=	<pre>thread& operator=(thread&& other) noexcept; jthread& operator=(jthread&& other) noexcept;</pre> Wie der jeweilige move constructor.
joinable	<pre>bool joinable() const noexcept;</pre> Gibt true zurück, wenn der Thread schon gestartet wurde und noch kein anderer Thread auf das Ende des Threads gewartet hat.
get_id	<pre>std::thread::id get_id() const noexcept; std::jthread::id get_id() const noexcept;</pre> Liefert eine eindeutige id des Threads.
join	<pre>void join();</pre> Wartet auf das Ende des Programmthreads. Das Verhalten ist undefiniert, wenn mehrere Threads gleichzeitig auf den Thread warten. Danach wird das Threadobjekt entkoppelt.
detach	<pre>void detach();</pre> Entkoppelt das Threadobjekt mit dem Programmthread. Eventuell angeforderte Ressourcen werden erst am Ende des Programmthread freigegeben.
swap	<pre>void swap(std::thread& other) noexcept;</pre> Vertauscht zwei Threadobjekte, so dass sie mit dem jeweils anderen Programmthread verknüpft sind.
~thread	Der Destruktor des Threadobjekts. Wenn das Threadobjekt noch mit einem Programmthread verknüpft ist, wird versucht, dieses abubrechen.

In C++20 wurde im Header `<thread>` die neue Klasse `jthread` eingeführt. `jthread` ist wie `thread`, im Destruktor wartet das Objekt aber auf das Ende des Programmthreads.

Bevor wir uns mit den Neuerungen bei `jthread` befassen, benötigen wir zwei neue Hilfsklassen, die in `<stop_token>` deklariert werden. Die Klasse `stop_source` wird

benutzt, um Programmthreads, stoppen zu können. Von einem Objekt der Klasse `stop_source` erhalten Sie ein Objekt der Klasse `stop_token`. Dieses `stop_token` objekt können Sie den Programmthreads übergeben, so dass diese überprüfen können, ob die Threads beenden sollen.

Name	Beschreibung
std::stop_source	
<code>get_token</code>	<code>stop_token get_token() const noexcept;</code> Mit dieser Funktion erhalten Sie ein Token, das Sie den Threads übergeben müssen, die Sie steuern wollen.
<code>stop_possible</code>	<code>bool stop_possible() const noexcept;</code> Prüft, ob ein stop request möglich ist.
<code>stop_requested</code>	<code>bool stop_requested() const noexcept;</code> Prüft ob ein stop request bereits angefordert wurde.
<code>request_stop</code>	<code>bool request_stop() noexcept;</code> Verschickt ein stop request. Alle Threads, denen das <code>stop_token</code> übergeben wurde, können darauf reagieren.
std::stop_token	
<code>stop_requested</code>	<code>bool stop_requested() const noexcept;</code> Prüft, ob ein stop request verschickt wurde.
<code>stop_possible</code>	<code>bool stop_possible() const noexcept;</code> Prüft, ob ein stop request überhaupt möglich ist.
std::jthread	
<code>jthread</code>	Wenn das Funktionsobjekt als ersten Parameter ein <code>stop_token</code> erwartet, brauchen Sie dieses dem Konstruktor nicht übergeben, das übernimmt die Klasse selber. Die anderen Argumente müssen Sie aber übergeben.
<code>get_stop_source</code>	<code>stop_source get_stop_source() noexcept;</code> Liefert ein <code>stop_source</code> zurück, mit der der Thread beendet werden kann.
<code>get_stop_token</code>	<code>stop_token get_stop_token() const noexcept;</code> Gibt das dazugehörige <code>stop_token</code> zurück.
<code>request_stop</code>	<code>bool request_stop() noexcept;</code> Verschickt ein <code>stop_request</code> .
<code>~jthread</code>	Der Destruktor verschickt ein <code>stop_request</code> und wartet anschließend auf das Ende des dazugehörigen Programthreads.

Im Gegensatz zu den Threads in C, wo Sie bei `join` auch das Ergebnis der Threadfunktion erhalten, wurden in C++ im Header `<future>` grundlegende Funktionen und

Klassen eingeführt, die wir hier kurz vorstellen wollen.

Name	Beschreibung
promise	<pre>template< class T > class promise;</pre> In einem Objekt des Klassentemplates kann ein Thread einen Wert vom Typ T abspeichern, den ein anderer Thread später abholen kann.
future	<pre>template< class T > class future;</pre> In einem Objekt des Klassentemplates kann ein Thread einen Wert vom Typ T abholen, den ein anderer Thread vorher gespeichert hat.
shared_future	<pre>template< class T > class shared_future;</pre> Wie future nur dass mehr als ein Thread gleichzeitig auf das Ergebnis warten kann.
packaged_task	<pre>template< class R, class ...ArgTypes > class packaged_task<R(ArgTypes...)>;</pre> Mit diesem Template können Sie Funktionsobjekte definieren, die asynchron ausgeführt werden. Das Ergebnis können Sie mit einem future abholen.
async	<pre>template<class F, class... Args> std::future<> async(F&& f, Args&&... args);</pre> Führt eine Funktion asynchron aus. Das Funktionsergebnis ist ein future mit dem auf das Ende des asynchronen Funktion gewartet und das Ergebnis abgeholt werden kann

In der Headerdatei <mutex> finden Sie Deklarationen und Templates für Klassen, um beliebige Strukturen für den exklusiven Zugriff durch einen Thread zu markieren. Ich werde die wichtigsten vorstellen.

Name	Beschreibung
std::mutex	
lock	<pre>void lock();</pre> Reserviert den Mutex für den aktuellen Thread solange bis der Mutex wieder freigegeben wurde. Wenn der Mutex von einem anderen Thread belegt ist, wird gewartet, bis der andere Thread den Mutex freigegeben hat. Wenn der Mutex bereits reserviert wurde, ist das Verhalten undefiniert.
try_lock	<pre>bool try_lock();</pre> Wie lock allerdings wird nicht gewartet, wenn der Mutex bereits belegt ist, sondern false zurückgegeben. Wenn der Mutex reserviert wurde, so wird true zurückgegeben.

Name	Beschreibung
unlock	<pre>void unlock();</pre> Gibt einen Mutex wieder frei, so dass er von anderen Threads reserviert werden kann. Wenn der Mutex vom aktuellen Thread gar nicht reserviert wurde, ist das Verhalten undefiniert.
~mutex	Der Destruktor. Das Verhalten ist undefiniert, wenn der Mutex noch für einen Thread reserviert ist oder ein Thread, der ihn reserviert hat, endete ohne den Mutex freizugeben.
std::timed_mutex Diese Klasse ist wie mutex nur enthält sie zwei weitere Funktionen, die es erlauben, die Wartezeit auf die Reservierung zu begrenzen.	
try_lock_for	<pre>template< class Rep, class Period > bool try_lock_for(const std::chrono::duration<Rep, Period>& timeout_duration);</pre> Wie lock allerdings wartet Sie die angegebene Zeitspanne, ob der Mutex nicht doch reserviert werden kann.
try_lock_until	<pre>template< class Clock, class Duration > bool try_lock_until(const std::chrono::time_point<Clock, Duration>& timeout_time);</pre> Wie try_lock_for allerdings wird gewartet, bis der angegebene Zeitpunkt erreicht wurde.
std::recursive_mutex Diese Klasse ist wie mutex allerdings wird gezählt wie oft ein Thread den Mutex reserviert hat. Der Thread muss den Mutex genauso oft wieder freigeben wie er ihn reserviert hat.	
std::recursive_timed_mutex Diese Klasse ist nun eine Kombination aus timed_mutex und recursive_mutex.	
std::lock_guard <pre>template< class Mutex > class lock_guard;</pre>	
lock_guard	<pre>explicit lock_guard(Mutex& m);</pre> Reserviert den angegebenen Mutex und wartet gegebenenfalls bis dieser von einem anderen Thread freigegeben wurde.
~lock_guard	Der Destruktor gibt den Mutex wieder frei. Das Verhalten ist undefiniert, wenn der Mutex nicht mehr existiert oder bereits freigegeben wurde.
Andere Klassen und Typen	
unique_lock	Ein unique_lock ist ein Template für allgemeine Aufgaben, wenn mehrere Threads oder Objekte den selben Mutex bearbeiten wollen.
scoped_lock	Ein scoped lock ist ähnlich wie ein lock_guard nur dass mehrere Mutexes auf einmal reserviert werden können um dead locks zu vermeiden.

Name	Beschreibung
Freie Funktionen	
lock	<pre>template< class Lockable1, class Lockable2, class... LockableN > void lock(Lockable1& lock1, Lockable2& lock2, LockableN&... lockn);</pre> lock versucht mehrere reservierbare Objekte zu reservieren. Die Reihenfolge der Reservierungen ist unspezifiziert, es wird aber versucht, Deadlocks zu vermeiden.
try_lock	<pre>template< class Lockable1, class Lockable2, class... LockableN > int try_lock(Lockable1& lock1, Lockable2& lock2, LockableN&... lockn);</pre> Try_lock versucht mehrere reservierbare Objekte zu reservieren. Wenn auch nur ein Mutex nicht reserviert wurde, werden all bereits reservierten Mutexe wieder freigegeben.
call_once	<pre>template< class Callable, class... Args > void call_once(std::once_flag& flag, Callable&& f, Args&&... args);</pre> Dieses Template ruft ein Funktionsobjekt genau einmal auf. Auch wenn mehrere Threads dieses genau gleichzeitig machen. Für jedes Funktionsobjekt müssen Sie genau eine Variable vom Typ <code>std::once_flag</code> definieren und dieses <code>call_once</code> übergeben..

Im Header `<condition_variable>` finden Sie alle Definitionen und Deklarationen für ConditionVariables.

Name	Beschreibung
std::condition_variable	
notify_one	<pre>void notify_one() noexcept;</pre> Benachrichtigt einen Thread, der auf die ConditionVariable wartet.
notify_all	<pre>void notify_all() noexcept;</pre> Benachrichtigt alle Threads, die auf die ConditionVariable warten.
wait	<pre>void wait(std::unique_lock<std::mutex>& lock);</pre> <pre>template< class Predicate > void wait(std::unique_lock<std::mutex>& lock, Predicate pred);</pre> Entsperrt den Mutex und wartet auf die Benachrichtigung der ConditionVariable. Wenn noch ein Funktionsobjekt übergeben wurde, wird so lange bzw. so oft auf die ConditionVariable gewartet, wie das Funktionsobjekt false zurückliefert. <pre>while(pred()) wait(lock);</pre>

Name	Beschreibung
wait_for	<pre>template< class Rep, class Period > std::cv_status wait_for(std::unique_lock<std::mutex>& lock, const std::chrono::duration<Rep, Period>& rel_time);</pre> <pre>template< class Rep, class Period, class Predicate > bool wait_for(std::unique_lock<std::mutex>& lock, const std::chrono::duration<Rep, Period>& rel_time, Predicate pred);</pre> Die Funktion arbeitet wie wait nur dass zusätzlich eine Zeitspanne angegeben werden kann, wie lange maximal gewartet werden soll. Der Rückgabewert vom Typ <code>std::cv_status</code> zeigt an, ob der Thread benachrichtigt wurde (<code>no_timeout</code>) oder nicht (<code>timeout</code>).
wait_until	<pre>template< class Clock, class Duration > std::cv_status wait_until(std::unique_lock<std::mutex>& lock, const std::chrono::time_point<Clock, Duration>& abs_time);</pre> <pre>template< class Clock, class Duration, class Predicate > bool wait_until(std::unique_lock<std::mutex>& lock, const std::chrono::time_point<Clock, Duration>& abs_time, Predicate pred);</pre> Dieser Member arbeitet wie <code>wait_for</code> nur wird hier statt der maximalen Wartedauer der Zeitpunkt, an dem spätestens die Wartezeit aufhören soll.
std::condition_variable_any Diese Klasse arbeitet wie <code>condition_variable</code> sie ist nur flexibler, was den Mutex angeht, da sie jede Klasse akzeptiert, die das korrekte Interface anbietet,	
std::notify_all_at_thread_exit Diese freie Funktion erlaubt es, alle Threads zu benachrichtigen, wenn der aktuelle Thread beendet ist.	

Andere Headerdateien

<code><atomic></code>	Hier finden Sie alle Deklarationen und Definitionen, die Sie für atomare Variablen und Zugriffe brauchen.
<code><barrier></code> (C++20)	Klassentemplate für eine Barriere.
<code><hazard_pointer></code> (C++26)	Bietet einen Mechanismus ähnlich wie <code>shared_mutex</code> , um mehrere Konsumenten und ein Produzent zu synchronisieren.
<code><latch></code> (C++20)	Latch bietet einen count down zähler, der von einem Thread herunter gezählt werden kann, währen ein anderer Thread auf das Erreichen von 0 wartet.
<code><rcu></code> (C++26)	Bietet einen weiteren Synchronisationsmechanismus.
<code><semaphore></code> (C++20)	Bietet zwei weiteren Synchronisationsmechanismen <code>counting_semaphore</code> und <code>binary_semaphore</code>

<shared_mutex> (C++14)	Dieser Header bietet Mechanismen zur kooperativen Zusammenarbeit. Mehrere Threads können lesend auf ein Objekt zugreifen, es kann aber immer nur 1 Thread schreiben. Solange ein Thread noch lesend auf ein Objekt zugreifen will, kann niemand schreiben und umgekehrt.
------------------------	--

```

/* thread.cpp - demonstration of concurrent library */
#include <iostream>
#include <chrono>
#include <ctime>
#include <mutex>
#include <future>

using namespace std::chrono_literals;
thread_local clock_t s_clockTime;

static void syncConsoleOut(const char* text, std::ostream& handle) {
    static std::mutex s_mutex;
    std::lock_guard lock(s_mutex);
    handle << text;
}

static void setData() {
    s_clockTime = clock();
}

static clock_t readData() {
    clock_t result = s_clockTime;
    return result;
}

static void theThreadFunc(void* arg) {
    std::promise<std::clock_t>& thePromise
=*reinterpret_cast<std::promise<std::clock_t*>>(arg);

    setData();
    thePromise.set_value(readData());
}

int main(void) {
    setData();

    std::this_thread::sleep_for(1s);

    std::promise<std::clock_t> thread1promise;
    std::future<std::clock_t> thread1future =
thread1promise.get_future();
    std::thread myThread1(theThreadFunc, &thread1promise);

    std::this_thread::sleep_for(1s);

    std::promise<std::clock_t> thread2promise;
    std::future<std::clock_t> thread2future =
thread2promise.get_future();
    std::thread myThread2(theThreadFunc, &thread2promise);

    clock_t timer = readData();

    std::cout << "Main: " << timer << " Thread1 " << thread1future.get()
<< " Thread2 " << thread2future.get() << std::endl;

```

```
myThread1.join();  
myThread2.join();  
return 0;  
}
```

18. C Standardbibliothek

Der C Standard kennt eine Vielzahl vordefinierter Funktionen, Konstanten, Variablen und Typen, die Sie in Ihrem Programm verwenden können. Sie werden mit dem Linker hinzugebunden und bilden zusammen mit Ihrem eigenen Code das ausführbare Programm. Wie diese Funktionen hinzugebunden werden müssen, entnehmen Sie bitte der Dokumentation zu Ihrem Entwicklungssystem.

Um die Bibliothek nutzen zu können, müssen Sie in jedem Fall die Deklarationen bzw. Definitionen mit dem Präprozessor in Ihren Quelltext einbinden. Auf dieser Seite werden die wichtigsten Funktionen gruppiert nach der entsprechenden Headerdatei vorgestellt:

18.1. Standardfunktionen <stdlib.h>

Die Funktionen der Standardbibliothek haben verschiedene Aufgaben. Unter anderen zur Speicherverwaltung, Prozeßkontrolle und Typumwandlung. Die wichtigsten sind:

a. Speicherverwaltung

Funktion	Beschreibung
calloc	<code>void *calloc(size_t size)</code> Fordert neuen Speicher vom Betriebssystem an und initialisiert diesen mit 0. Wenn nicht genügend Speicher zur Verfügung steht, wird NULL zurückgeliefert.
malloc	<code>void *malloc(size_t size)</code> Fordert neuen Speicher vom Betriebssystem an. Wenn nicht genügend Speicher zur Verfügung steht, wird NULL zurückgeliefert.
realloc	<code>void *realloc(void *buffer, size_t size)</code> Vergrößert oder verkleinert einen angeforderten Speicherblock. Liefert die neue Adresse zurück und gibt gegebenenfalls den alten Block wieder frei. Als Blockadresse darf auch NULL übergeben werden. Dann verhält sich realloc wie malloc. Wenn nicht genügend Speicher zur Verfügung steht, wird NULL zurückgeliefert.
free	<code>void free(void *buffer)</code> Gibt einen angeforderten Speicherblock wieder frei. Achtung, er darf hinterher nicht mehr verwendet werden.

b. Typumwandlung

Funktion	Beschreibung
<code>double atof(const char *string)</code>	Wandelt eine Zeichenkette in eine Fließkommazahl um.
<code>int atoi(const char *string)</code>	Wandelt eine Zeichenkette in eine Integerzahl um.
<code>int atol(const char *string)</code>	Wandelt eine Zeichenkette in eine long Integerzahl um.

c. Prozesskontrolle

Funktion	Beschreibung
<code>int atexit(void (*func)(void))</code>	Bestimmt eine Funktion, welche ausgeführt werden soll, sobald das Programm terminiert.
<code>void abort(void)</code>	Beendet das Programm mit einer Fehlermeldung.
<code>void exit(int code)</code>	Beendet das Programm und gibt den Ergebniscode an den aufrufenden Prozeß zurück.

d. Sonstige Funktionen

Funktion	Beschreibung
<code>getenv</code>	<code>char *getenv(const char *name)</code> Liefert den Wert einer Environmentvariablen zurück. Wenn der Wert nicht definiert ist, wird NULL zurückgeliefert.
<code>qsort</code>	<code>void qsort(void *buffer, size_t nElements, size_t size, int (*compare)(const void *elem1, const void *elem2))</code> Sortiert ein Array. Sie müssen die Adresse des Arrays, die Größe eines einzelnen Elements und die Anzahl der Elemente, die sortiert werden sollen übergeben. Darüber hinaus erwartet die Funktion die Adresse einer Vergleichsfunktion, nach der das Array sortiert wird. Das Array, kann auch dynamisch mit Hilfe von malloc angefordert sein.
<code>srand</code>	<code>void srand(unsigned start)</code> Initialisiert den Zufallszahlengenerator.
<code>rand</code>	<code>int rand(void)</code> Liefert eine zufällige Zahl zwischen 0 der Präprozessorkonstante RAND_MAX

e. Sonstige Definitionen

Funktion	Beschreibung
<code>NULL</code>	Präprozessorkonstante für einen Zeiger auf die Adresse NULL. Dieser Wert wird für gewöhnlich im Fehlerfall zurückgeliefert.
<code>size_t</code>	Eine Typdefinition, mit der die Größe eines Elements in Bytes gespeichert werden kann.
<code>EXIT_SUCCESS</code>	Eine Präprozessorkonstante, die für den Rückgabewert von main oder als Parameter für exit im Erfolgsfall verwendet werden kann.

Funktion	Beschreibung
EXIT_FAILURE	Eine Präprozessorkonstante, die für den Rückgabewert von main oder als Parameter für exit im Fehlerfall verwendet werden kann.

18.2. Zeichenfunktionen <ctype.h>

Die Zeichenfunktionen dienen dazu, einzelne Zeichen zu bearbeiten bzw. zu prüfen. Sie erwarten alle einen int-parameter und liefern auch einen int-wert zurück.

Funktion	Beschreibung
isascii	prüft, ob der Parameter im Bereich von 0 - 127 liegt (ASCII Codes)
isalnum	prüft, ob der Parameter ein Buchstabe ('a'-'z', 'A'-'Z') oder eine Ziffer ('0'-'9') ist
isalpha	prüft, ob der Parameter ein Buchstabe ('a'-'z', 'A'-'Z') ist.
islower	prüft, ob der Parameter ein Kleinbuchstabe ('a'-'z') ist.
isupper	prüft, ob der Parameter ein Großbuchstabe ('A'-'Z') ist.
isxdigit	prüft, ob der Parameter eine Hexziffer ('0'-'9','a'-'f','A'-'F') ist.
isdigit	prüft, ob der Parameter eine Dezimalziffer ('0'-'9') ist.
ispunct	prüft auf alle Arten von Zeichen, die keine, Leerzeichen, Steuerzeichen, Buchstaben oder Ziffern sind.
isgraph	prüft auf alle Arten von Zeichen, die keine Leerzeichen und keine Steuerzeichen sind.
isspace	prüft auf alle Arten von Leerzeichen (Tabulatoren, Zeilenvorschübe, Seitenvorschübe etc)
isprint	prüft auf alle Arten von Zeichen, die keine Steuerzeichen sind.
iscntrl	prüft auf alle Arten von Steuerzeichen (0-31, 127)
tolower	wandelt einen Großbuchstaben in einen Kleinbuchstaben um. Alle anderen Zeichen werden nicht verändert.
toupper	wandelt einen Kleinbuchstaben in einen Großbuchstaben um. Alle anderen Zeichen werden nicht verändert.

18.3. Nützliche Makrodefinitionen <stddef.h>

In stddef.h befinden sich Definitionen für verschiedene Typen, Konstanten und Macros.

Typen	ptrdiff_t	Dies ist ein vorzeichenbehafteter Ganzzahltyp (signed integer), der benutzt wird, wenn zwei Zeiger subtrahiert werden.
	size_t	Der Typ, der vom Operator sizeof verwendet wird.
Konstanten	NULL	Ein Makro, das den Nullwert eines Zeigers enthält. Dieser Wert wird z.B. verwendet, um anzuzeigen, dass der Zeiger nicht benutzt werden darf.

Makros	offsetof	Mit diesem Makro erhalten Sie den byteoffset eines Strukturelements. Das Makro erwartet als erstes Argument des Typ der Struktur und als zweites Argument den Namen des Strukturelements.
--------	----------	---

18.4. Wertebereiche <limits.h>

In <limits.h> befinden sich verschiedene Präprozessorkonstanten, die Sie berücksichtigen können, um einen Überlauf eines Datentyps zu verhindern.

Name	Beschreibung
BOOL_WIDTH (C23)	Die Breite von _Bool in Bits
BOOL_MAX (C29)	Der maximale Wert von _Bool
CHAR_BIT, CHAR_WIDTH ¹ (C23)	Die Breite von char in Bits
MB_LEN_MAX	Die maximale Breite von einem Multibytezeichen in Byte
CHAR_MIN	Der kleinste Wert eines char
CHAR_MAX	Der größte Wert eines char
SCHAR_WIDTH SHRT_WIDTH INT_WIDTH LONG_WIDTH LLONG_WIDTH (alle C23)	Die Breite von signed char, short, int, long und long long in Bits
SCHAR_MIN SHRT_MIN INT_MIN LONG_MIN LLONG_MIN (C23)	Der kleinste mögliche Wert von signed char, short, int, long und long long
SCHAR_MAX SHRT_MAX INT_MAX LONG_MAX LLONG_MAX (C23)	Der größte mögliche Wert von signed char, short, int, long und long long
UCHAR_WIDTH USHRT_WIDTH UINT_WIDTH ULONG_WIDTH ULLONG_WIDTH (alle C23)	Die Breite von unsigned char, unsigned short, unsigned int, unsigned long und unsigned long long in Bits

¹ Das war wohl ein Monk, der das eingeführt hat.

Name	Beschreibung
UCHAR_MAX USHRT_MAX UINT_MAX ULONG_MAX ULLONG_MAX(C23)	Der größte mögliche Wert von unsigned char, unsigned short, unsigned int, unsigned long und unsigned long long
BITINT_MAXWIDTH (C23)	Liefert die maximale Breite einer Variable in Bit.

18.5. Mathematische Funktionen <math.h>

In <math.h> finden Sie verschiedene Typen, Konstanten und Funktionen rund um Fließkommazahlen.

In C99 wurden zwei neue Typen eingeführt, float_t und double_t. Diese sind die effizientesten Typen, die groß genug sind, um ein float oder double zu speichern.

Die Konstanten

HUGE_VAL, HUGE_VALF (C99), HUGE_VALL (C99)	Die Konstanten repräsentieren einen Wert für eine zu große Zahl bei den Typen double, float und long double.
INFINITY (C99)	Repräsentiert einen Wert für $+\infty$ im Typ float
NAN (C99)	Entspricht Not A Number, das oft als Fehler zurückgeliefert wird.

Die folgenden Funktionen erwarten, wenn nichts anderes angegeben wurde, ein double als Argument und liefern ein double auch wieder zurück. Ab C99 gilt, bei den gleichnamigen Funktionen mit Suffix f wird statt double ein float erwartet und zurückgegeben, beim Suffix l wird der Typ long double verwendet.

Funktion	Beschreibung
fabs	Berechnet den Absolutewert des Arguments.
fmod	double fmod(double x, double y); Berechnet den Divisionsrest von x/y.
remainder (C99)	double remainder(double x, double y); Berechnet den Divisionsrest von x/y nach IEEE-Standard.
remquo (C99)	double remquo(double x, double y, int *quo); Berechnet wie remainder den Divisionsrest von x/y liefert aber noch zusätzliche Informationen in der Variable zurück, auf die quo zeigt.
fma (C99)	double fma(double x, double y, double z); Berechnet $x*y + z$.

Funktion	Beschreibung
fmax (C99)	<code>double fmax(double x, double y);</code> Gibt den größeren Wert von x und y zurück.
fmin (C99)	<code>double fmin(double x, double y);</code> Gibt den kleineren Wert von x und y zurück.
fdim (C99)	<code>double fdim(double x, double y);</code> Liefert die Differenz von x-y zurück, wenn x größer als y ist, ansonsten 0.
nan (C99)	<code>double nan(const char* arg);</code> Konvertiert eine implementationsspezifische Zeichenkette in ein nicht signalisierendes NaN.
exp	Berechnet e^x .
exp2 (C99)	Berechnet 2^x .
expm1 (C99)	Berechnet $e^x - 1$.
log	Berechnet den natürlichen Logarithmus $\ln x$.
log10	Berechnet den dekadischen Logarithmus $\log_{10} x$.
log2 (C99)	Berechnet den binären Logarithmus $\log_2 x$.
log1p (C99)	Berechnet den natürlichen Logarithmus $\ln(1+x)$.
pow	<code>double pow(double x, double y);</code> Berechnet die Potenz x^y .
sqrt	Berechnet die Quadratwurzel.
cbrt (C99)	Berechnet die Kubikwurzel.
hypot (C99)	<code>double hypot(double x, double y);</code> Berechnet die Hypotenuse aus zwei Katheten.
sin, cos, tan	Berechnet Sinus, Cosinus, Tangens. Der Winkel muss als Radialwinkel angegeben werden ($0-2\pi$).
asin, acos, atan	Berechnet $\sin^{-1}$, $\cos^{-1}$, $\tan^{-1}$. Der Winkel wird als Radialwinkel zurückgegeben.
atan2	<code>double atan2(double x, double y);</code> Berechnet $\tan^{-1}$ aus x/y. Dabei werden auch die Vorzeichen der Argumente berücksichtigt.
sinh, cosh, tanh	Berechnet den hyperbolischen Sinus, Cosinus und Tangens.
asinh, acosh, atanh	Berechnet die Umkehrung hyperbolischen Sinus, Cosinus und Tangens.
erf (C99)	Berechnet die Gaußsche-Fehlerfunktion.

Funktion	Beschreibung
erfc (C99)	Berechnet die komplementäre Gaußsche-Fehlerfunktion.
tgamma	Berechnet die Gammafunktion.
lgamma	Berechnet den natürlichen Logarithmus der Gammafunktion.
ceil	Liefert die kleinste ganze Zahl zurück, die größer wie das Argument ist.
floor	Liefert die größte ganze Zahl zurück, die kleiner wie das Argument ist.
trunc (C99)	Liefert das Argument ohne Nachkommastellen zurück.
round (C99)	Rundet des Argument zur nächstgelegenen ganzen Zahl.
lround (C99)	<code>long lround(double x);</code> Wie oben, liefert jedoch eine Ergebnis vom Typ long.
llround (C99)	<code>long long llround(double x);</code> Wie oben, liefert jedoch ein Ergebnis vom Typ long long.
rint (C99)	rint rundet eine übergebene Filekommanzahl zur nächsten ganzen Zahl gibt diese aber als Fließkommanahl zurück.
lrint (C99)	Wie rint gibt aber ein long zurück.
llrint (C99)	Wie rint gibt aber ein long long zurück.
nearbyint (C99)	Wie rint arbeitet aber ohne Fehlerbehandlung.
frexp	<code>double frexp(double x, int *exp);</code> Berechnet die Zerlegung des Argument so das gilt: $x = \text{frexp} * 2^{\text{exp}}$.
ldexp	<code>double ldexp(double x, int exp);</code> Berechnet $x * 2^{\text{exp}}$.
modf	<code>double modf(double x, double *i);</code> Zerlegt x in den ganzzahligen Teil und den Nachkommastellen. Beide Ergebnisse haben das gleiche Vorzeichen wie das Argument. Der ganzzahlige Teil wird in i gespeichert, die Nachkommastellen werden zurückgegeben. Aus 123,45 wird 123 und 0,45, aus -123,45 wird -123 und -0,45
scalbn (C99)	Arbeitet auf Binärssystemen wie ldexp. Ansonsten wird die jeweilige Basis des Fließkommanzahlrensystems verwendet.
ilogb (C99)	Gibt den Exponenten der angegebenen Zahl in einen Integer zurück
logb (C99)	Gibt den Exponenten der angegebenen Zahl in einer Fließkommazahl zurück

Funktion	Beschreibung
nextafter (C99)	<code>double nextafter(double from, double to);</code> Diese Funktion liefert den nächsten Wert von from, der sich von from unterscheidet und am Zahlenstrahl näher an to liegt.
copysign (C99)	<code>double copysign(double x, double y);</code> Überträgt das Voreichen von y auf den Wert von x und gibt das Ergebnis zurück.
isnan (C99)	Signalisiert, ob die abgegebene Zahl NaN ist.
isinf (C99)	Signalisiert, ob die abgegebene Zahl ∞ ist.
isnormal (C99)	Signalisiert, ob die abgegebene Zahl nicht 0, subnormal, ∞ oder NaN ist.

18.6. Andere Funktionen

Neben den bisher betrachteten Headerdateien gibt es noch einige Dateien, die wir hier aber nicht näher betrachten wollen.

Header	Beschreibung
<complex.h> (C99)	Behandlung komplexer Zahlen.
<fenv.h> (C99)	Verschiedene Konstanten und Funktionen für die Umgebung, die mit Fließkommazahlen rechnet.
<float.h>	Konstanten mit Limits der Fließkommazahlen.
<inttypes.h> (C99)	Formatdefinitionen für Integertypen bei printf, scanf etc.
<iso646.h> (C95)	Enthält Makros, um manche Operatoren mit alternativen Namen benutzen zu können. Ist eigentlich nur interessant für Programmierer, die vorher eine andere Programmiersprache benutzt haben.
<locale.h>	Enthält Funktionen und Makros für nationale Spracheinstellungen.
<setjmp.h>	Enthält Funktionen für einen funktionsübergreifenden Rücksprung.
<signal.h>	Enthält Funktionen und Konstanten für Signalisierungen zwischen mehreren Prozessen.
<stdbit.h> (C23)	Macros für Bitrepräsentationen der Datentypen.
<stdckdint.h> (C23)	Funktionen für Integerarithmetik. Die einen Überlaufanzeigen können.
<stdmchar.h> (C29)	Texttranscoding
<tgmath.h> (C99)	Macros für Typunabhängige Fließkommaoperationen.

Header	Beschreibung
<uchar.h> (C11)	Hilfsfunktionen für UTF-Zeichen
<wchar.h> (C95)	Hilfsfunktionen für wide strings
<wctype.h> (C95)	Hilfsfunktionen für Multibytezeichen.

19. C++ Standardbibliothek

19.1. Allgemeines (General Utilities)

Header	Beschreibung
<any>	Hiermit kann die Klasse std::any verwendet werden, die einen Wert aus verschiedenen Typen speichern kann.
<bit> (C++20) <bitset>	Hiermit können Sie Bitfelder bearbeiten, wenn die Bitoperatoren und die Bitsets des Sprachkerns ihnen nicht genügen.
<expected> (C++23)	Das Template std::expected kann entweder einen beliebigen Wert oder einen Fehlerstatus speichern.
<optional> (C++17)	Mit dem Template std::optional können Sie eine Variable deklarieren, die undefiniert sein kann, was Sie auch prüfen können. Das ist ähnlich wie bei Zeigern, die Sie auf NULL oder nullptr vergleichen können.
<tuple>	Mit dem Template std::tuple können Sie eine Aggregation aus mehreren Datentypen erzeugen.
<utility>	Dieser Header enthält verschiedene nützliche Komponenten zum Kopieren, Vergleichen oder die Vorläufer des Tuples.
<variant>	C++-Erweiterungen für eine union (S. 37).

19.2. Zeichenfunktionen

Header	Beschreibung
<cctype>	Dies ist die C++-Variante von <ctype.h>
<codecvt> (C++11 bis C++26)	Enthält Methoden für Unicodezeichen. Sollte in neuen Programmen nicht mehr eingesetzt werden,
<cuchar> (C++11)	Enthält Funktionen, um Multibytezeichen (wchar_t) nach UTF-8, -16 oder 32 umzuwandeln und umgekehrt.
<cwctype>	Dies ist die C++-Variante von <wctype.h> Hilfsfunktionen für Multibytezeichen.
<text_encoding> (C++26)	Hilfsfunktionen für Zeichenkodierung.

19.3. Zeichenketten

Neben <string> gibt es noch weitere Header zur Bearbeitung von Zeichenketten.

Header	Beschreibung
<cstring>	Dies ist die C++-Variante von <string.h>
<string_view> (C++17)	Das Klassentemplate basic_string_view ist ähnlich wie string nur dass die Zeichenkette nur zum Lesen gespeichert wird, dadurch ist eine schnellere Speicherverwaltung möglich.
<regex> (C++17)	Klassen und Methoden zu Bearbeitung von Regular Expressions.
<format> (C++20)	Klassen zum Formatieren verschiedene Textausgaben.

19.4. Mathematische Funktionen

Header	Beschreibung
<cfenv>	Dies ist die C++ Variante von <fenv.h>. Verschiedene Konstanten und Funktionen für die Umgebung, die mit Fließkommazahlen rechnet.
<cmath>	Dies ist die C++ Variante von <math.h>. Verschiedene mathematische Funktionen. (S. 200)
<complex>	Deklariert Klassen und Funktionen für komplexe Zahlen.
<linalg> (C++26)	Deklariert Klassen und Funktionen für lineare Algebra.
<numbers> (C++20)	Definiert verschiedene mathematischen Konstanten wie π oder die Eulersche Zahl e .
<numeric>	Verschiedene mathematische Funktionen wie zum Beispiel größter gemeinsamer Teiler oder kleinstes gemeinsames Vielfaches etc.
<random> (C++11)	Verschieden Algorithmen zur Zufallszahlengenerierung.
<simd> (C++26)	Verschieden Operationen und Klassen für die Parallelverarbeitung großer Datenmengen.
<valarray>	Verschiedene Klassen und Funktionen zur Verarbeitung von Arrays mit numerischen Werten.

19.5. Sprachunterstützung (Language Support)

Header	Beschreibung
<cfloat>	Die C++-Variante von <float.h> Konstanten mit Limits der Fließkommazahlen.
<compare> (C++20)	Vergleichsroutinen und Hilfsfunktionen für den Dreiwegevergleich.
<coroutine> (C++26)	Die Koroutine ist eine hohe Abstraktionsebene für Funktionen, die unterbrochen werden und/oder asynchron laufen können. Für Koroutinen wurden auch neue Schlüsselworte co_wait, co_return und co_yield seit C++20 eingeführt.

Header	Beschreibung
<code><csetjmp></code>	Ist die C++-Variante von <code><setjmp.h></code> und enthält Funktionen für einen funktionsübergreifenden Rücksprung.
<code><csignal></code>	Ist die C++-Variante von <code><signal.h></code> und enthält Funktionen und Konstanten für Signalisierungen zwischen mehreren Prozessen.
<code><cstdint></code>	Ist die C++-Variante von <code><stdint.h></code> und enthält nützliche Makrosdefinitionen (Seite 198).
<code><initializer_list></code> (C++11)	Enthält ein Template, damit auch Initialisierungslisten wie eine Arbeitsmenge (Range) behandelt werden kann.
<code><source_location></code> (C++20)	Sourcelocation verschiebt die Präprozessormakros <code>__FILE__</code> <code>__LINE__</code> in die C++-Welt, was eine bessere Integration erlaubt.
<code><typeindex></code> (C++11)	<code><typeindex></code> erlaubt es Werte vom Type <code>std::type_info</code> auch als Key für Maps und ähnliches zu benutzen.
<code><version></code> (C++20)	In diesem Header werden einige Testmakros definiert, mit denen Sie verschiedenen Informationen über die Bibliothek ermitteln können. Das ist besonders dann nützlich, wenn Sie Teile Ihrer Funktionen nicht übersetzen können, wenn ein bestimmter Standard nicht erfüllt ist.

19.6. Wertebereiche

`<climits>` ist die C++-Version von `<limits.h>`. Da sie nur Präprozessormakros enthält, unterscheiden Sie sich nicht, daher verweise ich auf das Kapitel für die Sprache C (S. 199).

`<limits>` enthält u.a. das Klassentemplate `numeric_limits`, das für jeden numerischen Typ statische Methoden und Konstanten enthält, mit denen verschiedene Metriken über den Typ ermittelt werden können. Als Argument für das Template müssen Sie nur den Typ angeben, der untersucht werden soll.

```
std::numeric_limits<int>
```

Name	Beschreibung
<code>is_specialized</code>	<code>static const bool is_specialized;</code> True, wenn die Angaben für den Typ verwendet werden können.
<code>is_signed</code>	<code>static const bool is_signed;</code> True, wenn der Typ negative Zahlen kennt.
<code>is_integer</code>	<code>static const bool is_integer;</code> True, wenn es ein Ganzzahltyp ist.

Name	Beschreibung
is_exact	<pre>static const bool is_exact;</pre> True, wenn der Typ jeden Wert ohne Abweichung speichern kann.
has_infinity	<pre>static const bool has_infinity;</pre> True, wenn der Typ den Wert für ∞ speichern kann.
has_quiet_NaN	<pre>static const bool has_quiet_NaN;</pre> True, wenn der Typ stille Mathematikfehler speichern kann.
has_signaling_NaN	<pre>static const bool has_signaling_NaN;</pre> True, wenn der Typ signalisierende Mathematikfehler speichern kann.
has_denorm	<pre>static const std::float_denorm_style has_denorm;</pre> Zeigt an, ob der Datentyp besonders kleine Zahlen speichern kann. float_denorm_style ist eine Enumeration mit den möglichen Werten denorm_indeterminate, denorm_absent und denorm_present.
has_denorm_loss	<pre>static const bool has_denorm_loss;</pre> True, wenn besonders kleine Zahlen, weniger Genau gespeichert werden können.
round_style	<pre>static const std::float_round_style round_style;</pre> Zeigt an, wie Zahlen gerundet werden. float_round_style ist eine Enumeration mit den möglichen Werten round_indeterminate, round_toward_zero, round_to_nearest, round_toward_infinity und round_toward_neg_infinity
is_iec559	<pre>static const bool is_iec559;</pre> True, wenn der Typ den Fließkommastandard IEE 559 bzw. IEEE 754 entspricht.
is_bounded	<pre>static const bool is_bounded;</pre> True, wenn der Typ nur eine begrenzte Anzahl von Werten annehmen kann. Da für unendlich viele Werte auch ein unendlich großer Computer erforderlich ist und dies in einem endlichen Universum nicht möglich ist, ist dieser Wert für jede Spezialisierung true oder der Compilerhersteller hat ein Wunder vollbracht.
is_modulo	<pre>static const bool is_modulo;</pre> True, wenn der Datentyp bei einem Überlauf einer Operation innerhalb des Zahlenbereichs definierte Ergebnisse bekommen.

Name	Beschreibung
digits	<code>static const int digits;</code> Gibt an, mit wie vielen Bits ein Wert ohne Vorzeichen und Exponent gespeichert wird.
digits10	<code>static const int digits10;</code> Gibt an, mit wie vielen Dezimalziffern ein Wert ohne Vorzeichen und Exponent maximal dargestellt wird.
max_digits10 (C++11)	<code>static const int max_digits10</code> Das ist die Anzahl der Ziffern, die notwendig ist, um einen Wert zu serialisieren, den man auch wieder de-serialisieren kann.
radix	<code>static const int radix;</code> Dies ist die Basis, des verwendeten Zahlensystem. Bei Ganzzahltypen ist dies für gewöhnlich 2.
min_exponent	<code>static const int min_exponent;</code> Der kleinste mögliche Exponent
min_exponent10	<code>static const int min_exponent10;</code> Der kleinste mögliche Exponent einer normalisierten Fließkommazahl
max_exponent	<code>static const int max_exponent;</code> Der größte mögliche Exponent
max_exponent10	<code>static const int max_exponent10;</code> Der größte mögliche Exponent einer normalisierten Fließkommazahl
traps	<code>static const bool traps;</code> True für Datentypen, deren Startwert bei bestimmten Operationen eine Ausnahme auslösen können.
tinyness_before	<code>static const bool tinyness_before;</code> True, wenn das System vor dem Runden auf Unterlauf (underflow) testet.
min	<code>static T min() throw();</code> Der kleinst mögliche Wert
max	<code>static T max() throw();</code> Der größtmögliche Wert
lowest (C++11)	<code>static constexpr T lowest() noexcept;</code> Der kleinstmögliche finite Wert. Für Fließkommazahlen im Standard gilt <code>lowest() = -max()</code>

Name	Beschreibung
epsilon	<pre>static T epsilon() throw();</pre> Liefert die kleinste Abweichung von 1.0, die auch verwendet werden kann. Ist nur nützlich für Fließkommazahlen.
round_error	<pre>static T round_error() throw();</pre> Liefert den größtmöglichen Rundungsfehler beim Runden zur nächsten ganzen Zahl.
infinity	<pre>static T infinity() throw();</pre> Liefert den Spezialwert für ∞.
quiet_NaN	<pre>static T quiet_NaN() throw();</pre> Liefert den Wert für einen stillen Mathematikfehler.
signaling_NaN	<pre>static T signaling_NaN() throw();</pre> Liefert den Wert für einen signalisierenden Mathematikfehler.
denorm_min	<pre>static T denorm_min() throw();</pre> Der kleinstmögliche Wert für von der Normabweichende Werte

19.7. Sonstiges

Header	Beschreibung
<algorithm>	In dieser Headerdatei befinden sie verschiedene Funktionen mit denen Sie ganze Listen oder Arrays von Objekten bearbeiten können. Sie finden dort u.a. Suchroutinen, Kopierfunktionen und Berechnungsmethoden. Alle Funktionen aufzulisten würde hier den Rahmen des Buchs sprengen.
<locale> <locale>	Diese beiden Header definieren bzw. deklarieren nützliche Makros und Funktionen für international Sprachanpassungen. Wobei <locale> die C++-Variante von <locale.h> ist.
<streambuf>	<streambuf> enthält Lowlevelfunktionen und -klassen zur Dateieinausgabe.
<strstream>	Strstream ist ein veralteter Header, der von den meisten Compilerhersteller vor Verabschiedung des ersten Standards 1998 eingeführt wurde, der aber schon im ersten Standard als veraltet markiert wurde und im C++26 entfernt wurde. Verwenden Sie stattdessen die Definitionen und Deklarationen in <sstream> (S. 108).
<cwchar>	Dies ist die C++-Variante von <wchar.h> mit Hilfsfunktionen für wide strings

<i>Header</i>	<i>Beschreibung</i>
<filesystem> (C++17)	Funktionen und Klassen zu Behandlung des Dateisystems
<print> (C++23)	Dieser Header enthält eine Typsicherer Variante von printf.
<spanstream> (C++23)	Dieser Header enthält ebenfalls Lowlevelfunktionen und -klassen zur Dateneinausgabe.
<syncstream> (C++20)	Header für synchronisierte Ausgabe.

20. Weiterführende Informationen

Bisher haben wir uns im Wesentlichen mit dem beschäftigt, was der Standard uns vorschreibt. Zwar gab es, wann immer es angebracht war, auch über den Standard hinausgehende Hinweise, dennoch sind einige wichtige Hinweise gar nicht behandelt worden oder zu kurz gekommen. Auch werde ich hier die eine oder andere Änderung beschreiben, die sonst in den bisherigen Kapiteln passen wollen und oft auch erst in spätere Standards hinzugefügt wurden.

20.1. Lebensdauer

Grundsätzlich gilt für die Lebensdauer von C++-Objekten dasselbe wie für normale Variablen. Allerdings gilt es verschiedenes zu beachten:

- Dann, wenn ein Objekt definiert wird, wird auch der entsprechende Konstruktor aufgerufen.
- Wenn der Block, in denen nicht statische Objekte definiert wurden, beendet wird, wird auch der Destruktor des Objekts aufgerufen.
- Bevor ein Klassenobjekt erzeugt wird, werden die Basisobjekte erzeugt.
- Klassenelementobjekte werden in der Reihenfolge erzeugt, in der sie in der Klasse deklariert wurden. Die Aufrufreihenfolge in der Initialisierungsliste im Klassenkonstruktor spielt hierbei keine Rolle. Wenn die Konstruktoren der Elemente also in einer bestimmten Reihenfolge aufgerufen werden sollen, müssen Sie dies bei der Deklaration beachten.
- Erst wenn die Konstruktoren aller Elementobjekte ausgeführt wurden, wird der Rumpf des Konstruktor der Klasse ausgeführt.
- Beim Löschen eines Objekts wird zuerst der Destruktor der Klasse aufgerufen, dann die Destrukturen aller Elemente in umgekehrter Reihenfolge und zum Schluss der Destruktor der Basisklassen.
- Temporäre, namenlose Objekte werden erst nach der Abarbeitung der Anweisung gelöscht, für die sie erzeugt wurden. Bei den Kontrollstrukturen bedeutet dies, dass die Objekte gelöscht werden, wenn das letzte kontrollierte Statement bearbeitet wurde.
- Destrukturen werden nur für Objekte aufgerufen, die vollständig erzeugt

wurden. Wenn der Konstruktor einer Klasse eine Exception wirft, wird dessen Destruktor nicht mehr aufgerufen. Die Basisklasse und Elemente, deren Konstruktoren bereits fertig sind, werden aber ganz normal gelöscht.

Das müssen Sie auf jeden Fall dann beachten, wenn Ihr Konstruktor Ressourcen anfordert und diese nicht mit eigenen Smartpointern oder ähnlichen RAII-Klassen verwaltet werden.

- Globale Objekte und statische Objekte außerhalb von Funktionen werden vor dem Aufruf von `main()` erzeugt. Sie werden gelöscht, nachdem `main()` beendet wurde oder wenn `exit()` aufgerufen wurde.
- Statische Objekte innerhalb von Funktionen, werden erzeugt, wenn Sie das erste Mal aufgerufen werden. Wenn Sie mehrere Threads verwenden, müssen Sie sich vor C++-11 selbst darum kümmern, dass dies nicht gleichzeitig passiert. Ab C++-11 garantiert Ihnen der Compiler für die notwendige Synchronisation zu sorgen.

20.2. Virtuelle Destruktoren

Wenn in einer Klasse eine beliebige Funktion `virtual` deklariert wurde, so sollten Sie immer einen virtuellen Destruktor definieren auch dann, wenn er einen leeren Funktionsrumpf erhält.

Der Grund liegt darin, dass man sehr häufig virtuelle Funktionen einsetzt, weil es irgendwo einen Teil Ihrer Applikation gibt, der nur mit Zeigern auf die jeweilige Basisklasse hantiert und die Objekte dynamisch mit dem Operator `new` erzeugt wurden. Wenn diese Objekte auch über einen Basiszeiger gelöscht werden, wird die Löschung nicht vollständig durchgeführt, weil nur der Destruktor der Basisklasse aufgerufen wird.

Weil solche Fehler schwer zu finden sind, sollten Sie diesen Ratschlag immer beherzigen. Der Nachteil eines virtuellen Destruktors, den Sie nicht brauchen, ist minimal.

```
// virtuald.cpp: demonstrate virtual destructors
#include <iostream>
struct Base {
    virtual void perform() = 0;
    virtual ~Base() {}
};

class Derived : public Base {
    void perform() {
        std::cout << "Performed" << std::endl;
    }
    ~Derived() {
        std::cout << "Destroyd" << std::endl;
    }
};
```

```
int main() {
    Base *ptr = new Derived;
    ptr->perform();
    delete ptr;
    return 0;
}
```

20.3. Zeiger auf Memberfunktionen

Einen Zeiger auf eine Memberfunktion einer Klasse können Sie sehr leicht deklarieren. Sie brauchen dazu nur den Rückgabetyt der Funktion, den Klassennamen und die Parameterliste.

```
ReturnType (ClassName::*identifier)( ParameterList );
```

Um einer Zeigervariablen die Adresse einer Funktion zuzuweisen, benötigen Sie den Namen der Funktion mit Klassennamen und den Adressoperator.

```
&ClassName::MemberFunction
```

Um eine Memberfunktion aufzurufen, benötigen Sie das Objekt für das der Member aufgerufen werden soll und den Zeiger.

```
ReturnType result = (object.*pointer)( ArgumentList );
```

Mit einem kleinen funktionfähigen Beispiel soll das eben besprochene verdeutlicht werden.

```
// memberpointer.cpp: example for pointer to member functions of a class
#include <iostream>
```

```
class MyClass {
,   int c;
    int (MyClass::*m_funcPtr)(int, int);
    int add( int a, int b ) {
        return a+b+c;
    }
    int multiply( int a, int b ) {
        return a*b*c;
    }

public:
    MyClass( int c ) : c(c), m_funcPtr(&MyClass::add) {}
    int perform( int a, int b ) {
        return (this->*m_funcPtr)(a,b);
    }
};
```

```
int main () {
    MyClass object( 4 );
    int result = object.perform( 5, 11 );
    std::cout << result;

    return 0;
}
```

20.4. Die Regel der goldenen Drei (Fünf)

Wenn eine Klasse einen Destruktor braucht, um Ressourcen freizugeben, sollten Sie immer einen Copykonstruktor definieren und den Operator = überladen. Der Copykonstruktor sollte eine neue Ressource anfordern und den Inhalt der Ressource des Quellobjekts dort hinein kopieren. Alternativ könnten Sie auch das Resourcehandle kopieren und einen Nutzungszähler inkrementieren und im Destruktor die Ressource erst dann löschen, wenn der Nutzungszähler wieder bei 0 steht. Wenn Sie die Ressource weder kopieren noch einen Nutzungszähler einrichten können, sollten Sie den Copykonstruktor und der Operator = privat deklarieren ohne ihn zu definieren. Dies bewirkt, dass Sie keine Kopie Ihres Resourcehandles unkontrolliert erzeugen können. Statt diese Funktionen privat zu deklarieren, können Sie ab C++11 die Funktionen auch löschen.

Für C++ ab 2011 wurde die Regel um den move constructor und die move zuweisung erweitert.

Generell gilt, wenn eine der drei (5) Elementfunktionen definiert wurde, sollten auch die anderen zwei (4) Elementfunktionen definiert werden.

- Destruktor
- Copy-Konstruktor
- Copy-Zuweisung
- Move-Konstruktor
- Move-Zuweisung

Die „Strafe“ für das Nichtbeachten dieser Regel folgt für gewöhnlich recht schnell.

20.5. C und C++ mischen

Wenn Sie die Programmiersprachen mischen wollen und beide in einem Projekt nutzen wollen, haben Sie ein paar Möglichkeiten.

Sie kopieren die gewünschten Funktionen, die in C geschrieben wurden, einfach in eine C++ Quelldatei. Die meisten Statements, die in C akzeptiert werden, werden auch in C++ akzeptiert. Gelegentlich werden Sie Anpassungen machen müssen.

```
int funktion( void ); <-> int funktion();
```

Eine Funktion, die keine Argumente erwartet, müssen Sie in C mit void markieren in C++ ist das nicht erforderlich. Da C++ sich aber nicht an dem void stört, habe ich tatsächlich lange Zeit noch das void geschrieben, da es ja sein könnte, dass ich die Funktion mal nach C portieren muss.

```
char *ptr = malloc( 128 ); <-> char *ptr = (char *)malloc( 128 ); <-> char  
*ptr = static_cast<char *>(malloc( 128 ));
```

In C kann ein void Zeiger jedem beliebigen Zeiger zugewiesen werden. Manchmal bekommen Sie zwar eine Warnung, aber übersetzt wird auf jeden Fall korrekt. In C++

+ muss der Typ konvertiert werden. Wenn der Code immer noch von einem C Compiler verstanden werden muss, können Sie auch den C-Cast operator verwenden. Wenn C kein Thema mehr ist, können Sie auch einen C++ cast operator verwenden.

Alternativ können Sie natürlich Ihre C-Quelldatei umbenennen, dann wird sie allerdings von anderen Projekten nicht mehr gefunden. oder Sie konfigurieren Ihr Projekt so, dass der C++-Compiler statt dem C-Compiler verwendet werden soll.

Die letzte Möglichkeit besteht darin dem Compiler schon im Header mitzuteilen, dass Sie eine Schnittstelle zwischen C und C++ benötigen. Dies geschieht mit dem Konstrukt `extern "C"`. Alle Definitionen und Deklarationen innerhalb eines `extern "C"` Blocks sind von oder für C und nicht C++. Diese Schnittstellenfunktionen können zwar auch von C++ aus verwendet werden, aber Sie müssen mit Einschränkungen leben. Funktionsüberladung ist nicht möglich und die Funktionen befinden sich auch im globalen Namespace. In einem Header könnte das mit Hilfe der bedingten Compilierung so ausschauen:

```
#ifdef __cplusplus
extern "C" {
#endif
    void meineCfunktion( void );
#ifdef __cplusplus
}
#endif
```

Wenn `meineCfunktion` eine C-Funktion ist, die Sie aus C++ aufrufen wollen, genügt das und Sie müssen außer die C-Quellen oder die Bibliothek in Ihr Projekt einzubinden nichts weiter tun. Wenn `meineCfunktion` eine C++-Funktion ist, die Sie aus C aufrufen wollen, müssen Sie auch die Definition in einen `extern "C"` Block setzen.

```
extern "C" {
    void meineCfunktion() {
        /// TODO: The definition
    }
}
```

oder

```
extern "C" void meineCfunktion() {
    /// TODO: The definition
}
```

Warum das so ist. Ein C-Compiler erzeugt für jede Funktion ein Symbol, das nur aus dem Namen der Funktion gebildet wird. C++-Compiler müssen das Symbol noch um die Informationen für den Namensraum und die erwarteten Argumente erweitern. Mit `extern „C“` weisen Sie den C++-Compiler an, das Symbol für die Funktion wie ein C-Compiler zu erzeugen. Es ist natürlich nicht sinnvoll, Funktionen mit `extern „C“` zu markieren, die Klassenobjekte als Argumente erwarten, da diese in C unbekannt sind.

20.6. Programmierguidelines

Zum besseren Lesen eines Quelltextes sollten Sie immer eine einheitliche Form und einheitliche Stile bewahren. Das ist immer dann besonders wichtig, wenn mehrere Entwickler an den gleichen Quelltexten arbeiten. Ich stelle einmal ein paar sinnvolle Regeln zusammen:

Position der geschweiften Klammern

Geöffnete Klammern sind immer in einer neuen Zeile oder vor dem Ende der vorherigen Zeile. Also zum entweder so

```
if( condition )
{
    statement1;
    statement2;
}
```

oder

```
if( condition ) {
    statement1;
    statement2;
}
```

Egal wie sich entscheiden, machen Sie es immer einheitlich. Es gibt für beide gute Gründe.

Tabulatoren

Quelltexte sollten immer entsprechend der Ebene eingerückt sein. Legen Sie einheitlich fest, wie viele Zeichen die Tabulatorbreite sein soll und ob ein Tabulatorzeichen im Code eingefügt werden soll oder die entsprechende Anzahl an Leerzeichen. Die Verwendung von Leereichen kann sinnvoll sein, wenn Sie sehr viele unterschiedliche Quelltexteditoren verwenden und eine einheitliche Konfiguration der Tabulatorgröße nicht gewährleistet ist. Tabulatoren haben dafür aber den Vorteil, dass sie weniger Platz beanspruchen.

Leerzeichen

Die Verwendung von Leereichen kann auch reglementiert sein. Eine einheitliche Verwendung ist z.B. nützlich, wenn Sie den Quelltext durchsuchen müssen. Folgende Überlegungen können gemacht werden.

- Zwischen Schlüsselwort und Klammer
`if ( ... )` oder `if( ... )`
- Zwischen Klammern und Bedingung
`if( ... )` oder `if(...)`
- Bei der Funktionsdefinition
`int funk ( int param )` oder `int funk(int param)`
- Bei den Funktionsaufrufen

```
res = funk ( arg ) oder res = funk(arg)
```

- Bei Operatoren

```
res = a + b oder res=a+b
```

Verwendung der geschweiften Klammern

Bei den Fällen, in denen die geschweiften Klammern entfallen dürfen, weil nach dem `if` nur ein Statement folgt, das der Bedingung folgt, können Sie festlegen, dass die geschweiften Klammern trotzdem geschrieben werden sollen. Dies hat zum Beispiel den Vorteil, dass Sie sie nicht vergessen können, wenn Sie kurzfristig zum Testen ein weiteres Statement einfügen wollen. Das kann im Eifer des Gefechts schnell passieren und die Folgen können recht unangenehm sein.

Verwendung von Zeilenvorschüben

Die Reglementierung der Verwendung von Zeilenvorschüben kann auch sinnvoll sein. Es ist vor allen angenehmer, wenn sie bei der Arbeit mit einem Debugger leichter Unterbrechungspunkte (Breakpoints) setzen können.

```
if( cond ) doSomething();
```

oder

```
if( cond )
    doSomething();
```

Blockgrößen, Modulgrößen

Die maximale Größe von zusammengesetzten statements (compound statement) sollte ebenso wie die maximale Größe einer Funktion oder einer Quelltextdatei reglementiert werden. Das ist wichtig, weil zu große Funktionen sind schwieriger zu überblicken und daher fehleranfälliger. Den Vogel abgeschossen hat hier eine Firma in Linz, deren C++-Dateien schon mal über 10000 Zeilen hatte und nicht genug sie haben auch noch mehrere C++-Dateien mittels `#include` zu einer einzigen Compilationunit zusammen gefasst. Ein Präprozessor define in einer Datei war dann auch plötzlich in einer anderen Datei bekannt, was dann einige Fehler zur Folge hatte. Machen Sie solche Fehler nicht, denn die digitale Schuld wird sonst irgendwann so groß, dass eine Weiterentwicklung nicht mehr sinnvoll ist, und die Entwicklung komplett neu anfangen werden muss.

Bennenschema

Als ich zum Programmieren angefangen habe, durfte ein Bezeichner maximal 2 Zeichen lang sein, wenn er länger war, gab es zwar keinen Fehler, es wurden aber nur die ersten zwei Zeichen betrachtet. Mein erster C-Compiler erlaubte wenigstens 8 Zeichen pro Bezeichner. In den Standards sind jetzt genug Zeichen gesichert erlaubt, dass aussagekräftige Namen kein Problem sind. Machen Sie auch davon gebraucht, so dass Sie auch noch nach Jahren wissen, wo zu eine Variable, ein Typ oder eine Funktion benutzt wird. Verwenden Sie auch ein einheitliches Schema, um mehrere Begriffe zu kombinieren.

```
MeinSuperTyp
```

oder

```
mein_super_typ
```

Variablennamen fangen bei mir immer mit einem Kleinbuchstaben, Typen mit einem Großbuchstaben. Globale Variable, wenn sie nicht vermieden werden können, erhalten den Präfix `g_`, statische Variablen den Präfix `s_`, Membervariablen von Klassen bekommen `m_`, Konstante Werte definiere ich immer in Großbuchstaben. Dabei unterscheide ich nicht zwischen Präprozessorkonstanten und Compilerkonstanten.

```
#define GLUECKSZAHl      13
static const int GLUECKSZAHl=13;
```

Sie können sich natürlich auch andere Schemata ausdenken. Das hängt ganz von Ihren Bedürfnissen an

Funktionsstruktur

Die Struktur einer Funktion sollte nach Möglichkeit immer der gleichen Struktur gehorchen. D.h. vor allen ein Eingang und ein Ausgang. Die gilt besonders für die Programmiersprache C. Da in C++ mit Destruktoren auch im Fehlerfall aufgeräumt werden kann, ist es nicht ganz so wichtig, erhöht aber trotzdem die Wartbarkeit enorm. Bei mir hat sich folgendes Schema bewährt:

```
int function( int param )
{
    if( ParamCheck() )
        return -1;
    fetchResources();
    doWork();
    releaseResources();    // in C++ this should be done by destructors
    return result;
}
```

Ausnahmesicherheit

Wenn Sie das Aufräumen den Destruktoren überlassen, z.B. durch Verwendung der smart pointer, sorgen Sie dafür, dass Ihre Funktionen auch dann Ihre Ressourcen aufräumen, wenn eine Ausnahme den Programmfluss unterbrochen hat. Wenn das nicht möglich ist, können Sie mit `try ... catch` Ausnahmen abfangen und die Ressourcen selbst wieder freigeben.

In neueren Standard ab C++14 können Sie mit einer kleinen Hilfsklasse sich behelfen:

```
// 1. Die Guard-Klasse (C++11/14 kompatibel)
template <typename F>
class Finaly
{
    F cleanup_func;
public:
    // Konstruktor übernimmt das Lambda per Move (hervorragend für die
    Performance)
    explicit Finaly(F&& f) : cleanup_func(std::move(f)) {}

    ~Finaly() { cleanup_func(); }

    // Kopieren und Zuweisen verbieten (Sicherheit geht vor)
    Finaly(const Finaly&) = delete;
    Finaly& operator=(const Finaly&) = delete;
};

// 2. Die Helferfunktion für C++14 (Erlaubt die automatische Typableitung)
template <typename F>
Finaly<F> finally(F&& f) {
    return Finaly<F>(std::move(f));
}

void WriteToHardware()
{
    Send2Port(start);

    // finally() erzeugt das Objekt, 'auto' fängt es ab.
    auto _ = finally( [] { Send2Port(end); } );

    if (!doYourJob()) {
        return; // Destruktor von '_' wird aufgerufen ->
               //Send2Port(end) läuft!
    }
}
```

20.7. Sonstiges

constexpr

Mit dem Schlüsselwort `constexpr` können Sie einen Ausdruck markieren, der schon zur compile time ausgewertet werden soll. Für die Initialisierung solcher Ausdrücke dürfen nur andere Ausdrücke, die ebenfalls so markiert wurden, oder Literale verwenden.

<type_traits>

In dieser Datei finden Sie verschiedene Hilfsklassen und -methoden, mit denen bei der Templateprogrammierung leichter die Templateargumente überprüft werden können.

21. Index

A abort.....197

acos.....	201	basic_string_view.....	205
acosh.....	201	begin.....	166, 171
ANSI.....	25	bfloat16_t.....	27
any.....	204	Big-endian.....	10
app.....	109	binary.....	110
append.....	131	binary_semaphore.....	194
argc.....	63	BITINT_MAXWIDTH.....	200
argv.....	63	body.....	38
array.....	164	bool.....	25
ASCII.....	25	BOOL_MAX.....	199
asctime.....	175	BOOL_WIDTH.....	199
asctime_s.....	175	boolalpha.....	111
asin.....	201	break.....	59
asinh.....	201	C	
assert.....	146		
assign.....	130, 166	c_str.....	132
async.....	191	call_once.....	183, 193
asynchron.....	179	calloc.....	156, 196
at.....	132, 166	capacity.....	166
atan.....	201	captures.....	163
atan2.....	201	case.....	58
atanh.....	201	Cast.....	134
ate.....	110	catch.....	150
atexit.....	197	cbegin.....	166, 171
atof.....	142, 197	cbrt.....	201
atoi.....	142, 197	ceil.....	202
atol.....	142, 197	cend.....	166, 171
Atomic.....	180	cerr.....	108
auto.....	43	char.....	25
auto_ptr.....	157	CHAR_BIT.....	199
B		CHAR_MAX.....	199
		CHAR_MIN.....	199
back.....	166	CHAR_WIDTH.....	199
back_insert_iterator.....	172	chrono.....	176
back_inserter.....	172	chrono_literals.....	177
bad.....	113	cin.....	108
bad_alloc.....	151	class.....	37, 63
bad_any_cast.....	151	clear.....	130, 167
bad_array_new_length.....	151	clearerr.....	104
bad_cast.....	151	clock.....	174
bad_exception.....	152	clock_t.....	174
bad_function_call.....	151	CLOCKS_PER_SEC.....	174
bad_optional_access.....	151	cnd_broadcast.....	184
bad_typeid.....	151	cnd_destroy.....	184
bad_variant_access.....	152	cnd_init.....	183
bad_weak_ptr.....	151	cnd_signal.....	184
basic_fstream.....	109	cnd_t.....	183
basic_iostream.....	109	cnd_timedwait.....	184

cnd_wait.....	184	double_t.....	200
co_return.....	205	Dreiwegevergleich.....	53
co_wait.....	205	duration.....	176
co_yield.....	205	dynamic_cast.....	50, 138
compare.....	132	E	
compound statement.....	56	else.....	56
concept.....	126	empty.....	131, 166
Condition Variables.....	179	end.....	166, 171
const.....	27	Endianness.....	10
const_cast.....	50, 134	endl.....	111
const_iterator.....	172	ends.....	111
const_reverse_iterator.....	172	enum.....	33
constexpr.....	218	eof.....	113
continue.....	60	epsilon.....	209
copy constructor.....	71	erase.....	131
copysign.....	203	erf.....	201
cos.....	201	erfc.....	202
cosh.....	201	errno.....	147
counting_semaphore.....	194	exception.....	151, 152
cout.....	108	Exceptions.....	150
crbegin.....	166, 171	exit.....	197
crend.....	166, 171	EXIT_FAILURE.....	198
ctime.....	175	EXIT_SUCCESS.....	197
ctime_s.....	175	exp.....	201
D		exp2.....	201
data.....	166	expected.....	204
days.....	177	explicit.....	72, 134
Deadlock.....	180	expm1.....	201
Debugger.....	18	extern.....	44
default.....	58	extern "C".....	214
defaultfloat.....	111	F	
defined.....	24	fabs.....	200
delete.....	72, 157	fail.....	113
den.....	161	false.....	25
denorm_absent.....	207	fclose.....	97
denorm_indeterminate.....	207	fdim.....	201
denorm_min.....	209	feof.....	104
denorm_present.....	207	ferror.....	103
deque.....	165	fflush.....	106
Destruktor.....	72	fgetc.....	101
detach.....	189	fgetpos.....	104
difftime.....	174	fgets.....	101
digits.....	208	FIFO.....	155
digits10.....	208	filesystem::filesystem_error.....	151
do-while.....	58	final.....	80
domain_error.....	151	find.....	131
double.....	25	fixed.....	111

float.....	25	good.....	113
float_denorm_style.....	207	Google Test.....	17
float_round_style.....	207	goto.....	60
float_t.....	200	gps_clock.....	177
float128_t.....	27	H	
float16_t.....	26	has_denorm.....	207
float32_t.....	27	has_denorm_loss.....	207
float64_t.....	27	has_infinity.....	207
floor.....	202	has_quiet_NaN.....	207
flush.....	111, 113	has_signaling_NaN.....	207
fma.....	200	head.....	154
fmax.....	201	Heap.....	28
fmin.....	201	hexfloat.....	111
fmod.....	200	high_resolution_clock.....	177
fopen.....	97	hours.....	177
for.....	57	HUGE_VAL.....	200
forward_list.....	165	HUGE_VALF.....	200
fpos_t.....	104	HUGE_VALL.....	200
fprintf.....	99	hypot.....	201
fputc.....	101	I	
fputs.....	101	id.....	188
fread.....	102	IDE.....	16
free.....	156, 196	identifiers.....	45
freopen.....	106	if.....	56
frexp.....	202	ifstream.....	108
front.....	166	ignore.....	112
front_insert_iterator.....	172	ilogb.....	202
front_inserter.....	172	in.....	110
fscanf.....	99	INFINITY.....	200, 209
fseek.....	104	insert.....	130
fsetpos.....	104	insert_iterator.....	172
fstream.....	108	inserter.....	172
ftell.....	104	int.....	25
future.....	191	INT_MAX.....	199
future_error.....	151	INT_MIN.....	199
fwrite.....	102	INT_WIDTH.....	199
G		int16_t.....	26
get.....	112	int32_t.....	26
get_id.....	189	int64_t.....	26
get_stop_source.....	190	int8_t.....	26
get_stop_token.....	190	Intelformat.....	10
get_token.....	190	intmax_t.....	93
getchar.....	87	invalid_argument.....	151
getenv.....	197	ios_base.....	109
getline.....	112	ios_base::failure.....	151f.
gets.....	85	is_bounded.....	207
gmtime.....	176	is_exact.....	207

is_iec559.....	207	localtime.....	176
is_integer.....	206	Lock.....	179, 191, 193
is_modulo.....	207	lock_guard.....	192
is_open.....	109	log.....	201
is_signed.....	206	log10.....	201
is_specialized.....	206	log1p.....	201
isalnum.....	198	log2.....	201
isalpha.....	198	logb.....	202
isascii.....	198	logic_error.....	151
isctrl.....	198	long.....	26
isdigit.....	198	LONG_MAX.....	199
isgraph.....	198	LONG_MIN.....	199
isinf.....	203	LONG_WIDTH.....	199
islower.....	198	lowest.....	208
isnan.....	203	lrint.....	202
isnormal.....	203	lround.....	202
isprint.....	198	ltoa.....	144
ispunct.....	198	LValue.....	27
isspace.....	198	M	
istream.....	108	main.....	62
istreamstream.....	108	malloc.....	156, 196
isupper.....	198	map.....	165
isxdigit.....	198	max.....	208
iterator.....	172	max_digits10.....	208
itoa.....	144	max_exponent.....	208
J		max_exponent10.....	208
join.....	189	MB_LEN_MAX.....	199
joinable.....	189	mega.....	161
jthread.....	189f.	microseconds.....	177
JUnit.....	17	milli.....	161
K		milliseconds.....	177
kilo.....	161	min.....	208
Konstruktor.....	69	min_exponent.....	208
Koroutine.....	205	min_exponent10.....	208
L		minutes.....	177
ldexp.....	202	modf.....	202
length_error.....	151	months.....	177
lgamma.....	202	Motorolaformat.....	10
LIFO.....	155	move constructor.....	72
list.....	165	mtx_destroy.....	183
literals.....	177	mtx_init.....	182
Little-endian.....	10	mtx_lock.....	182
LLONG_MAX.....	199	mtx_plain.....	182
LLONG_MIN.....	199	mtx_recursive.....	182
LLONG_WIDTH.....	199	mtx_t.....	182
llrint.....	202	mtx_timed.....	182
llround.....	202	mtx_timedlock.....	183

mtx_trylock.....	183	P	
mtx_unlock.....	183		packaged_task.....
multimap.....	165		parameter pack.....
multiset.....	165		peek.....
mutable.....	28		perror.....
Mutex.....	179, 191		PHP.....
N			Posix.....
namespace.....	46		pow.....
nan.....	200f.		printf.....
nanoseconds.....	177		priority_queue.....
NDEBUG.....	24, 146		promise.....
nearbyint.....	202		ptrdiff_t.....
new.....	157		push_back.....
nextafter.....	203		push_front.....
noexcept.....	153		put.....
nonboolalpha.....	111		putback.....
noreplace.....	110		putc.....
NULL.....	30, 197f.		putchar.....
nullptr.....	30		puts.....
num.....	161	Q	
numeric_limits.....	206	qsort.....	197
O		Queue.....	155, 165
offsetof.....	199	quiet_NaN.....	209
ofstream.....	108	R	
once_flag.....	183, 193	Race Condition.....	180
ONCE_FLAG_INIT.....	183	radix.....	208
operator.....	55	rand.....	197
operator !.....	113	range_error.....	151
operator ().....	162	ratio.....	160
operator []......	132, 166	rbegin.....	166, 171
operator +.....	133	read.....	112
operator +=.....	131	realloc.....	196
operator <<.....	133	recursive_mutex.....	192
operator =.....	130	recursive_timed_mutex.....	192
operator ==.....	133	regex_error.....	151
operator >>.....	133	register.....	44
operator bool.....	114	reinterpret_cast.....	50, 139
operator overloading.....	55	remainder.....	200
operator=.....	189	remquo.....	200
optional.....	204	rend.....	166, 171
ostream.....	108	replace.....	131
ostringstream.....	108, 145	request_stop.....	190
out.....	110	requires.....	126
out_of_range.....	151	reserve.....	167
Overflow.....	10	resize.....	166
overflow_error.....	151	reverse_iterator.....	172
override.....	80	rewind.....	104

rfind.....	132	sizeof.....	50, 198
rint.....	202	sprintf.....	144
round.....	202	sqrt.....	201
round_error.....	209	srand.....	197
round_indeterminate.....	207	sscanf.....	143
round_style.....	207	Stack.....	155, 165
round_to_nearest.....	207	Stapel.....	155
round_toward_infinity.....	207	static.....	44
round_toward_neg_infinity.....	207	static_cast.....	50, 135
round_toward_zero.....	207	stderr.....	105
runtime_error.....	151	stdin.....	105
RVALUE.....	31	stdout.....	105
S		steady_clock.....	177
scalbn.....	202	stod.....	144
scanf.....	91	stof.....	144
SCHAR_MAX.....	199	stoi.....	144
SCHAR_MIN.....	199	stol.....	144
SCHAR_WIDTH.....	199	stold.....	144
scientific.....	111	stoll.....	144
scoped_lock.....	192	stop_possible.....	190
seconds.....	177	stop_requested.....	190
SEEK_CUR.....	105	stop_source.....	190
SEEK_END.....	105	stop_token.....	190
SEEK_SET.....	105	stoul.....	144
seekg.....	113	stoull.....	144
seekp.....	113	strcat.....	127
Semaphore.....	179	strchr.....	128
set.....	165	strcmp.....	127
setbase.....	111	strcoll.....	128
setbuf.....	106	strcpy.....	127
setfil.....	111	strcspn.....	128
setprecision.....	111	strdup.....	127
setvbuf.....	106	strerror.....	148
setw.....	111	strftime.....	175
shared_future.....	191	string.....	129
shared_ptr.....	158	stringstream.....	108
short.....	26	strlen.....	128
SHRT_MAX.....	199	strlwr.....	129
SHRT_MIN.....	199	strncat.....	127
SHRT_WIDTH.....	199	strncmp.....	127
signaling_NaN.....	209	strncpy.....	127
signature.....	38	strpbrk.....	128
signed.....	26	strchr.....	128
sin.....	201	strspn.....	128
sinh.....	201	strstr.....	128
size.....	131, 166	strtod.....	142
size_t.....	50, 197f.	strtok.....	128

strtol.....	143	tm_isdst.....	174
strtoul.....	143	tm_mday.....	174
struct.....	36	tm_min.....	174
strupr.....	129	tm_mon.....	174
strxfrm.....	129	tm_sec.....	174
substr.....	132	tm_yday.....	174
swap.....	189	tm_year.....	174
switch.....	58	to_string.....	145
system_clock.....	177	to_wstring.....	145
system_error.....	151	tolower.....	198
T		toupper.....	198
tai_clock.....	177	traps.....	208
tail.....	154	true.....	25
tan.....	201	trunc.....	110, 202
tanh.....	201	try.....	150
tellg.....	112	try_lock.....	191, 193
tellp.....	113	try_lock_for.....	192
template.....	115	try_lock_until.....	192
templates.....	61	tss_create.....	185
tgamma.....	202	tss_delete.....	185
this.....	64	TSS_DTOR_ITERATIONS.....	184
thrd_busy.....	181	tss_dtor_t.....	184
thrd_create.....	181	tss_get.....	185
thrd_current.....	181	tss_set.....	185
thrd_detach.....	182	tss_t.....	184
thrd_equal.....	181	tuple.....	204
thrd_error.....	181	type modifier.....	26
thrd_exit.....	181	type_info.....	50
thrd_join.....	182	typeid.....	50
thrd_nomem.....	181	typename.....	116, 124
thrd_sleep.....	181	U	
thrd_start_t.....	181	Überlauf.....	10
thrd_success.....	180	UCHAR_MAX.....	200
thrd_t.....	180	UCHAR_WIDTH.....	199
thrd_timedout.....	181	UINT_MAX.....	200
thrd_yield.....	181	UINT_WIDTH.....	199
Thread.....	179, 188	uint16_t.....	26
Thread Local Storage.....	180	uint32_t.....	26
thread_local.....	45, 184	uint64_t.....	26
throw.....	150, 153	uint8_t.....	26
time.....	174	ULLONG_MAX.....	200
time_point.....	177	ULLONG_WIDTH.....	199
time_t.....	174	ULONG_MAX.....	200
timed_mutex.....	192	ULONG_WIDTH.....	199
tinyness_before.....	208	ultoa.....	144
tm.....	174f.	Underflow.....	10
tm_hour.....	174	underflow_error.....	151

unget.....	112	<bitset>.....	204
ungetc.....	104	<cassert>.....	146
union.....	37	<cctype>.....	204
unique_lock.....	192	<cerrno>.....	148
unique_ptr.....	158	<cfenv>.....	205
Unittest.....	17	<cfloat>.....	205
unlock.....	192	<charconv>.....	145
unsigned.....	26	<chrono>.....	176
Unterlauf.....	10	<citme>.....	174
USHRT_MAX.....	200	<climits>.....	206
USHRT_WIDTH.....	199	<locale>.....	209
utc_clock.....	177	<cmath>.....	205
UTF-8.....	25	<codecvt>.....	204
V		<compare>.....	205
va_arg.....	160	<complex.h>.....	203
va_end.....	160	<complex>.....	205
va_list.....	159	<concepts>.....	126
va_start.....	159	<contracts>.....	150
vector.....	164	<coroutine>.....	205
volatile.....	28	<csetjmp>.....	206
W		<csignal>.....	206
Warteschlange.....	155	<cstdarg>.....	159
wchar_t.....	25	<cstddef>.....	206
weeks.....	177	<cstdint>.....	26
what.....	152	<stdio>.....	107
while.....	58	<stdlib>.....	50
wostringstream.....	145	<cstring>.....	127, 205
write.....	113	<cuchar>.....	204
wstring.....	129	<wchar>.....	209
Y		<cwtype>.....	204
years.....	177	<debugging>.....	150
—		<deque>.....	165
__cplusplus.....	24, 214	<errno.h>.....	148
__DATE__.....	24	<exception>.....	151
__FILE__.....	24	<expected>.....	204
__func__.....	42	<fenv.h>.....	203
__LINE__.....	24	<filesystem>.....	210
__STDC__.....	24	<float.h>.....	203
__TIME__.....	24	<format>.....	205
<		<forward_list>.....	165
<algorithm>.....	209	<fstream>.....	108
<any>.....	204	<functional>.....	162
<array>.....	164	<future>.....	190
<assert.h>.....	146	<hazard_pointer>.....	194
<atomic>.....	194	<initializer_list>.....	206
<barrier>.....	194	<inttypes.h>.....	203
<bit>.....	204	<iomanip>.....	111

<ios>.....	111	<stack>.....	165
<iostream>.....	108	<stacktrace>.....	149
<iso646.h>.....	203	<stdarg.h>.....	159
<iterator>.....	172	<stdatomic.h>.....	188
<latch>.....	194	<stdbit.h>.....	203
<limits.h>.....	199	<stdckdint.h>.....	203
<limits>.....	206	<stddef.h>.....	25, 30
<linalg>.....	205	<stdexcept>.....	151
<list>.....	165	<stdfloat>.....	26
<locale.h>.....	203	<stdint.h>.....	26
<locale>.....	209	<stdio.h>.....	85
<map>.....	165	<stdlib.h>.....	50
<math.h>.....	200	<stdmchar.h>.....	203
<memory_resource>.....	159	<stop_token>.....	189
<memory>.....	157	<streambuf>.....	209
<mutex>.....	191	<string_view>.....	205
<new>.....	151, 159	<string.h>.....	127
<numbers>.....	205	<stringstream>.....	209
<numeric>.....	205	<syncstream>.....	210
<optional>.....	204	<system_error>.....	149
<ostream>.....	111	<text_encoding>.....	204
<print>.....	210	<tgmath.h>.....	203
<queue>.....	165	<thread>.....	188
<random>.....	205	<threads.h>.....	180
<ratio>.....	160	<time.h>.....	174
<rcu>.....	194	<tuple>.....	204
<regex>.....	205	<type_traits>.....	218
<scoped_allocator>.....	159	<typeid>.....	206
<semaphore>.....	194	<typeinfo>.....	50
<set>.....	165	<uchar.h>.....	204
<setjmp.h>.....	203	<utility>.....	204
<shared_mutex>.....	195	<valarray>.....	205
<signal.h>.....	203	<variant>.....	204
<simd>.....	205	<vector>.....	164
<source_location>.....	206	<version>.....	206
<spanstream>.....	210	<wchar.h>.....	204
<sstream>.....	108	<wctype.h>.....	204
